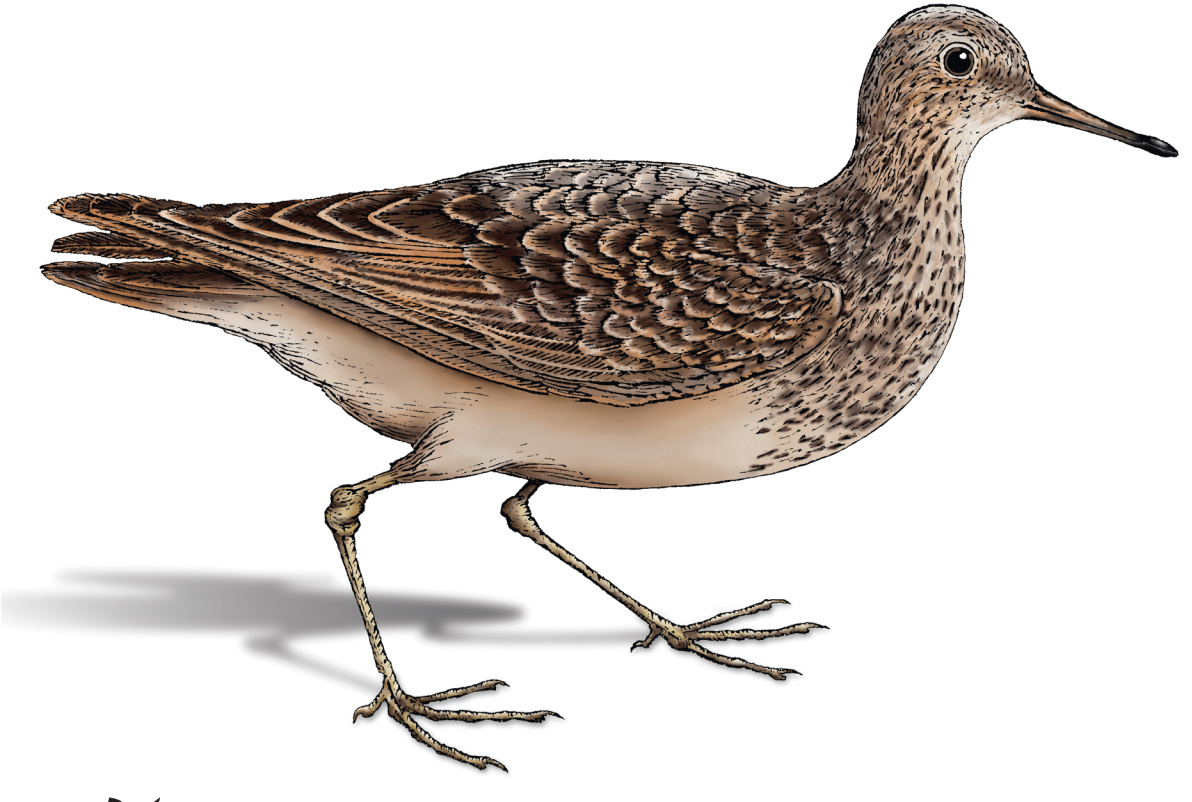
O'REILLY®

Spring Boot по-быстрому

Создаем облачные приложения на Java и Kotlin



Марк Хеклер

Spring Boot: Up and Running

Building Cloud Native Java and Kotlin Applications

Mark Heckler

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Spring Boot по-быстрому

Создаем облачные приложения на Java и Kotlin

Марк Хеклер

ПИТЕР®

Санкт-Петербург • Москва • Минск

2025

ББК 32.988.02-018
УДК 004.738.2
Х35

Хеклер Марк

Х35 Spring Boot по-быстрому. — СПб.: Питер, 2025. — 352 с.: ил. — (Серия «Бестселлеры O'Reilly»).

ISBN 978-5-4461-3942-2

Spring Boot, который скачивают более 75 миллионов раз в месяц, — наиболее широко используемый фреймворк Java. Его удобство и возможности совершили революцию в разработке приложений, от монолитных до микросервисов. Тем не менее простота Spring Boot может привести в замешательство. Что именно разработчику нужно изучить, чтобы сразу же выдавать результат? Это практическое руководство научит вас писать успешные приложения для критически важных задач.

Марк Хеклер из VMware, компании, создавшей Spring, проведет вас по всей архитектуре Spring Boot, охватив такие вопросы, как отладка, тестирование и развертывание. Если вы хотите быстро и эффективно разрабатывать нативные облачные приложения Java или Kotlin на базе Spring Boot с помощью реактивного программирования, создания API и доступа к разнообразным базам данных — эта книга для вас.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.988.02-018
УДК 004.738.2

Права на издание получены по соглашению с O'Reilly. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1492076988 англ.

Authorized Russian translation of the English edition of Spring Boot: Up and Running, ISBN 9781492076988 © 2021 Mark Heckler
This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

ISBN 978-5-4461-3942-2

© Перевод на русский язык ООО «Прогресс книга», 2022
© Издание на русском языке, оформление ООО «Прогресс книга», 2025
© Серия «Бестселлеры O'Reilly», 2022

Краткое содержание

Предисловие 11

Глава 1. Коротко о Spring Boot 15

Глава 2. Выбираем инструменты и приступаем к работе 22

Глава 3. Создаем первый Spring Boot REST API 41

Глава 4. Добавление в приложение Spring Boot доступа к базе данных 60

Глава 5. Настройка и контроль приложения Spring Boot 77

Глава 6. Займемся данными по-настоящему 107

Глава 7. Создание приложений с помощью Spring MVC 158

Глава 8. Реактивное программирование: Project Reactor и Spring WebFlux 189

Глава 9. Тестирование приложений Spring Boot для повышения их готовности к продакшену 218

Глава 10. Безопасность приложений Spring Boot 241

Глава 11. Развертывание приложений Spring Boot 287

Глава 12. Углубляемся в реактивное программирование 313

Об авторе 350

Об иллюстрации на обложке 351

Оглавление

Предисловие 11
Добро пожаловать 11
Условные обозначения 12
Использование примеров кода 13
Благодарности 13
От издательства 14

Глава 1. Коротко о Spring Boot 15
Три основополагающие возможности Spring Boot 15
Упрощение управления зависимостями с помощью стартовых пакетов 15
Упрощение развертывания с помощью исполняемых JAR-файлов 17
Автоконфигурация 19
Резюме 21

Глава 2. Выбираем инструменты и приступаем к работе 22
Maven или Gradle? 22
Apache Maven 22
Gradle 24
Выбор между Maven и Gradle 25
Java или Kotlin 26
Java 26
Kotlin 27
Выбор между Java и Kotlin 28
Выбираем версию Spring Boot 28
Spring Initializr 29
Прямиком из командной строки 33
Работа в интегрированных средах разработки 36
Прогулка по функции main() 38
Резюме 39

Глава 3. Создаем первый Spring Boot REST API 41
«Как» и «почему» API 41
Что такое REST и почему это важно 42
API в стиле HTTP-глаголов 43
Возвращаемся к Initializr 44
Создание простого класса предметной области 45
GET 47
Коротко об аннотации @RestController 47
POST 51
PUT 52
DELETE 52
И не только 53
Доверяй, но проверяй 55
Резюме 58

Глава 4. Добавление в приложение Spring Boot доступа к базе данных 60
Подготовка автоконфигурации для доступа к базе данных 60
Чего мы надеемся добиться 61
Добавление зависимости для базы данных 61
Добавление кода 63
Сохранение и извлечение данных 70
Наводим лоск 74
Резюме 75

Глава 5. Настройка и контроль приложения Spring Boot 77
Конфигурация приложения 78
@Value 79
@ConfigurationProperties 84
Возможные сторонние решения 89
Отчет об автоконфигурации 92
Actuator (актуатор) 94
Открываем доступ к Actuator 101
Лучше учитываем среду приложения с помощью Actuator 101
Регулировка уровня журналирования с помощью Actuator 104
Резюме 106

Глава 6. Займемся данными по-настоящему 107
Описание сущностей 108
Поддержка шаблонов 109
Поддержка репозиториев 109
@Before 110
Создание с помощью Redis сервиса на основе шаблона 110
Инициализация проекта 111
Разработка сервиса Redis 112
Преобразование из шаблона в репозиторий 121
Создание сервиса на основе репозитория с помощью Java Persistence API 125
Инициализация проекта 125
Разработка JPA-сервиса (MySQL) 126
Загрузка данных 131
Создание сервиса на основе репозитория с помощью документоориентированной базы данных NoSQL 136
Инициализация проекта 136
Разработка сервиса MongoDB 138
Создание сервиса на основе репозитория с помощью графовой базы данных NoSQL 144
Инициализация проекта 145
Разрабатываем сервис Neo4j 145
Резюме 156

Глава 7. Создание приложений с помощью Spring MVC 158
Что такое Spring MVC 158
Взаимодействия конечного пользователя с помощью шаблонизаторов 159
Инициализация проекта 160
Разработка приложения Aircraft Positions 161
Передача сообщений 167
Подключаем PlaneFinder 168
Расширяем приложение Aircraft Positions 173
Формирование диалогов с помощью WebSocket 178
Что такое WebSocket 179
Рефакторинг приложения Aircraft Positions 179
Резюме 187

Глава 8. Реактивное программирование: Project Reactor и Spring WebFlux ... 189

Введение в реактивное программирование ... 189
Манифест реактивных систем ... 190
Project Reactor ... 193
Tomcat и Netty ... 195
Реактивный доступ к данным ... 196
R2DBC и база данных H2 ... 196
Реактивный Thymeleaf ... 209
RSocket и полностью реактивное взаимодействие между процессами ... 210
Что такое RSocket ... 210
Применяем RSocket на практике ... 211
Резюме ... 216

Глава 9. Тестирование приложений Spring Boot для повышения их готовности к продакшену ... 218

Модульное тестирование ... 218
Знакомимся с аннотацией @SpringBootTest ... 219
Важнейшие модульные тесты для приложения Aircraft Positions ... 220
Рефакторинг кода для лучшего тестирования ... 226
Тестовые срезы ... 234
Резюме ... 240

Глава 10. Безопасность приложений Spring Boot ... 241

Аутентификация и авторизация ... 241
Аутентификация ... 242
Авторизация ... 243
Коротко о Spring Security ... 244
HTTP-брандмауэр ... 244
Цепочки фильтров безопасности ... 244
Заголовки запросов и ответов ... 245
Реализация аутентификации и авторизации на основе форм с помощью Spring Security ... 245
Добавление зависимостей Spring Security ... 246
Добавляем аутентификацию ... 252
Авторизация ... 259

Реализация OpenID Connect и OAuth2 для аутентификации и авторизации ... 268
Клиентское приложение Aircraft Positions ... 270
Сервер ресурсов PlaneFinder ... 277
Резюме ... 285

Глава 11. Развертывание приложений Spring Boot ... 287
Возвращаемся к исполняемым JAR-файлам Spring Boot ... 288
Сборка «полностью исполняемого» JAR-файла Spring Boot ... 289
Что это нам дает ... 295
Разобранные JAR-файлы ... 295
Развертывание приложений Spring Boot в контейнерах ... 302
Создание образа контейнера из IDE ... 304
Создание образа контейнера из командной строки ... 306
Проверяем наличие образа ... 307
Запуск контейнеризованного приложения ... 308
Утилиты для исследования образов контейнеров приложений Spring Boot ... 309
Pack ... 309
Dive ... 311
Резюме ... 312

Глава 12. Углубляемся в реактивное программирование ... 313
Когда следует использовать реактивное программирование ... 314
Тестирование реактивных приложений ... 315
Но сначала — рефакторинг ... 316
А теперь — тестирование ... 323
Диагностика и отладка реактивных приложений ... 332
Hooks.onOperatorDebug() ... 333
Контрольные точки ... 343
ReactorDebugAgent.init() ... 346
Резюме ... 348

Об авторе ... 350

Об иллюстрации на обложке ... 351

Предисловие

Добро пожаловать

> Воздушный змей взлетает против ветра,
> а не по ветру.
>
> *Джон Нил,*
> *эссе «Инициатива и настойчивость»*
> *(The Weekly Mirror)*

На сегодняшний день написано немало книг по Spring Boot. Хороших книг, созданных хорошими специалистами. Но любой автор сам выбирает, что включать в издание, что не включать, как представить отобранный материал, и принимает множество других решений, делающих каждую книгу уникальной. То, что одному автору кажется необязательным материалом, другой считает совершенно необходимым. Мы все разработчики, и наши мнения различаются.

Мое мнение таково: в уже существующих книгах недостает кое-каких деталей, которыми, как мне кажется, либо необходимо, либо очень полезно поделиться с разработчиками, только начинающими изучать Spring Boot. Список недостающих деталей непрерывно рос по мере того, как я общался с разработчиками со всего мира, находящимися на различных этапах путешествия в мир Spring Boot. Мы все учимся разному, в разное время и по-разному. Поэтому я и решил написать данную книгу.

Если вы новичок в Spring Boot или чувствуете, что нелишним будет углубить свои знания, — и, будем откровенны, когда это было лишним? — то эта книга специально для вас. В ней охватываются ключевые возможности Spring Boot, а также применение этих возможностей *на практике*.

Спасибо, что присоединились ко мне в этом путешествии! Приступим!

Условные обозначения

В книге применяются следующие условные обозначения.

Курсив

Курсивом выделены новые термины.

`Моноширинный шрифт`

Используется для листингов программ, а также внутри абзацев для обозначения таких элементов, как функции, базы данных, типы данных, переменные среды, операторы и ключевые слова, имена файлов и их расширения, команды и параметры командной строки.

`Моноширинный жирный шрифт`

Показывает команды или другой текст, который пользователь должен ввести самостоятельно.

`Моноширинный курсив`

Показывает текст, который должен быть заменен значениями, введенными пользователем, или значениями, определяемыми контекстом.

Шрифт без засечек

Используется для обозначения URL, адресов электронной почты, названий меню, параметров, кнопок, каталогов.



Этот рисунок указывает на общее примечание.



Этот рисунок указывает на предупреждение.



Этот рисунок указывает на совет или предложение.

Использование примеров кода

Дополнительные материалы (примеры кода, упражнения и т. д.) доступны для скачивания по адресу https://resources.oreilly.com/examples/0636920338727.

Относительно любых технических вопросов и проблем, возникших у вас при использовании примеров кода, пожалуйста, пишите по адресу электронной почты bookquestions@oreilly.com.

Эта книга создана, чтобы помочь вам в работе. В целом, если к ней прилагается какой-либо пример кода, можете использовать его в своих программах и документации. Обращаться к нам за разрешением нет необходимости, разве что вы копируете значительную часть кода. Например, написание программы, использующей несколько фрагментов кода из этой книги, не требует отдельного разрешения. Для продажи или распространения компакт-диска с примерами из книг O'Reilly, конечно, разрешение нужно.

Ответ на вопрос путем цитирования этой книги, в том числе примеров кода, не требует разрешения. Включение значительного количества кода примеров из нее в документацию вашего программного продукта требует разрешения.

Мы ценим, хотя и не требуем, ссылки на первоисточник. Ссылка на первоисточник включает название, автора, издательство и ISBN, например: «Хеклер М. Spring Boot по-быстрому. — СПб.: Питер, 2022, 978-5-4461-3942-2».

Благодарности

Нет предела моей благодарности всем, кто вдохновлял меня на написание этой книги и поддерживал во время работы над ней. Вы просто не представляете, как важно для меня то, что вы читали ее черновики и писали отзывы или просто тепло отозвались о ней в Twitter. Огромное вам спасибо!

Но есть несколько человек, сделавших эту книгу реальностью, а не просто оптимистическим планом по написанию.

Мой начальник, учитель и друг Таша Айзенберг (Tasha Isenberg). Таша, ты работала со мной, утрясая расписание, а когда ситуация стала совсем сложной, расчистила для меня дорогу для финишного рывка и помогла уложиться в основные сроки. Я очень благодарен за то, что у меня есть столь понимающий и энергичный заступник в VMware.

Доктор Дэвид Сайр (David Syer), основатель Spring Boot, Spring Cloud, Spring Batch, чей вклад в бесчисленные проекты Spring неоценим. Ваша проницатель-

ность была совершенно исключительной, а отзывы — невероятно содержательными, и никаких слов не хватит, чтобы выразить вам мою благодарность.

Грег Тернквист (Greg Turnquist), член команды Spring Data. Спасибо за критический взгляд и неприукрашенные отзывы: книга стала заметно лучше благодаря вашей бесценной новой точке зрения.

Мои редакторы Корбин Коллинз (Corbin Collins) и Сюзанна (Зан) Маккуэйд (Suzanne (Zan) McQuade). Ваша исключительная поддержка на всем пути от задумки книги до завершения вдохновляла меня на то, чтобы работать на пределе возможностей и как-то укладываться в, казалось бы, невозможные сроки. Я не мог желать большего.

Роб Романо (Rob Romano), Кэйтлин Геган (Caitlin Ghegan), Ким Сандовал (Kim Sandoval) и весь производственный отдел O'Reilly. Вы помогли мне завершить работу, придя на выручку в самый важный момент, на финишной прямой, буквально и фигурально выпустив эту книгу в мир.

Наконец, самое главное: моя прекрасная, любящая и чрезвычайно терпеливая жена Кэти. Сказать, что ты вдохновляешь меня и придаешь мне силы делать все, что я делаю, — величайшее преуменьшение. От всего сердца спасибо тебе за *все*.

От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

ГЛАВА 1

Коротко о Spring Boot

В этой главе мы обсудим три важнейшие возможности Spring Boot и расскажем, как они повышают производительность труда разработчиков.

Три основополагающие возможности Spring Boot

Три важнейшие возможности Spring Boot, на которых основывается все прочее: упрощение управления зависимостями, упрощение развертывания и автоконфигурация.

Упрощение управления зависимостями с помощью стартовых пакетов

Один из самых потрясающих аспектов Spring Boot — управление зависимостями — становится действительно... управляемым.

Если вы более или менее давно разрабатываете важное программное обеспечение, то почти наверняка сталкивались с несколькими проблемами, связанными с управлением зависимостями. Любая функциональность приложения обычно требует нескольких зависимостей «переднего края». Например, чтобы предоставить реализующий REST веб-API, необходимо обеспечить способ открытия конечных точек для доступа по HTTP, прослушивания запросов, привязки этих конечных точек к методам/функциям обработки запросов с последующим формированием и возвращением соответствующих ответов.

Практически всегда все основные зависимости включают множество других зависимостей для реализации обещанной ими функциональности. Развивая пример с реализацией REST API, мы могли бы ожидать увидеть набор зависимостей

(организованных в виде разумной, хотя и спорной структуры), включающих код для возврата ответов на запросы в конкретном формате, например JSON, XML, HTML; код для упаковки/распаковки объектов в требуемый (-е) формат (-ы); код для прослушивания и обработки запросов и возвращения ответов; код для декодирования сложных URI, применяемых при создании универсальных API; код поддержки различных протоколов связи и т. д.

Даже в этом относительно простом примере, вероятно, в файле сборки потребуется немало зависимостей. И это мы еще даже не думали, какую функциональность хотели бы включить в свое приложение, а говорили только про его внешние взаимодействия.

А теперь поговорим о версиях каждой из этих зависимостей.

Совместное использование библиотек требует определенной степени строгости, ведь зачастую одна версия конкретной зависимости может тестироваться (или вообще работать правильно) лишь с конкретной версией другой зависимости. И когда эти проблемы неизбежно возникают, то приводят к тому, что я называю лернейской гидрой зависимостей.

Как и ее тезка, лернейская гидра зависимостей может доставить немало проблем. И как и в случае с тезкой, никакой награды за поиск и искоренение программных ошибок, возникающих из-за расхождения версий, вас не ждет, только неуловимые окончательные диагнозы и *часы*, потраченные впустую на их поиск.

Знакомьтесь со Spring Boot и его стартовыми пакетами. Стартовые пакеты Spring Boot представляют собой спецификации (Bill of Materials, BOM), основанные на проверенном практикой предположении, что практически всегда создание конкретной функциональной возможности происходит практически одинаково.

В предыдущем примере при каждом создании API мы организовывали доступные для пользователей конечные точки, прослушивали на предмет запросов, обрабатывали запросы, отправляли и получали данные по сети посредством конкретного протокола и т. д. Этот паттерн проектирования/разработки/использования практически неизменен и представляет собой общепринятый подход, лишь с несколькими небольшими вариациями. И как и для прочих аналогичных паттернов, под рукой есть охватывающий его стартовый пакет Spring Boot.

Для получения всех этих связанных между собой элементов функциональности в *одной зависимости приложения* достаточно добавить один-единственный стартовый пакет, например `spring-boot-starter-web`. Все охватываемые им зависимости синхронизированы по версиям — протестированы на совместную работу, то есть проверено, что включенная в стартовый пакет версия библио-

теки А корректно работает с включенной версией библиотеки Б... и В... и Г, и т. д. Это резко сокращает список зависимостей и упрощает жизнь разработчика, практически исключая труднообнаруживаемые конфликты версий среди зависимостей, необходимых для реализации неотъемлемых возможностей приложения.

В тех редких случаях, когда необходимо включить в приложение функциональность, предоставляемую другой версией включенной зависимости, можно просто переопределить протестированную версию.



Если вам нужно переопределить используемую по умолчанию версию зависимости — переопределяйте... но тогда повысьте уровень тестирования, чтобы сократить возникающие риски.

При желании можно также исключать ненужные для конкретного приложения зависимости с учетом того же предостережения.

В целом идея стартовых пакетов Spring Boot существенно упрощает зависимости и сокращает усилия, необходимые для добавления целых их наборов в приложение, а также резко сокращает накладные расходы по их тестированию, поддержке и обновлению.

Упрощение развертывания с помощью исполняемых JAR-файлов

Давным-давно, в те времена, когда по земле бродили серверы приложений, развертывание приложений Java было непростой задачей.

Чтобы запустить в эксплуатацию приложения, скажем, с доступом к базе данных, подобно многим современным микросервисам и практически всем приложениям с монолитной архитектурой тогда и сейчас, — пришлось бы проделать следующее.

1. Установить и настроить сервер приложений.
2. Установить драйверы базы данных.
3. Создать соединение с базой данных.
4. Создать пул соединений.
5. Собрать и протестировать приложение.
6. Выполнить развертывание приложения и его обычно довольно многочисленных зависимостей на сервере приложений.

Обратите внимание, что этот список предполагает наличие администраторов, которые бы настроили машину/виртуальную машину, и что на каком-то этапе независимо от этого процесса была создана база данных.

Spring Boot перевернул большую часть этого неуклюжего процесса развертывания с ног на голову и схлопнул описанные шаги в один (может, два, если считать за шаг копирование/вставку с помощью команды `cf push` отдельного файла в нужное место).

Spring Boot не первоисточник так называемых über JAR-файлов[1], но он произвел в них переворот. Вместо того чтобы выделять каждый файл из JAR-файла приложения и всех JAR-файлов зависимостей с последующим объединением их в один целевой JAR-файл — этот процесс иногда называют *шейдингом* (shading), — разработчики Spring Boot посмотрели с совершенно новой точки зрения, задавшись вопросом: а нельзя ли создавать *вложенные* JAR-файлы с сохранением их желаемого и получающегося форматов?

Вложение JAR-файлов вместо их шейдинга позволяет устранить *огромное количество* потенциальных проблем, возникающих вследствие потенциальных конфликтов версий, когда JAR-файл зависимости А и JAR-файл зависимости Б используют различные версии В. Исчезают также возможные юридические проблемы, возникающие вследствие переупаковки программного обеспечения и объединения его с другим программным обеспечением под другой лицензией. Хранение всех JAR-файлов зависимостей в исходном формате явно позволяет избежать этих и других проблем.

Кроме того, при необходимости можно легко извлечь содержимое исполняемого JAR-файла Spring Boot. В некоторых случаях это может понадобиться, и далее в книге я расскажу почему. А пока просто знайте, что исполняемые JAR-файлы Spring Boot способны решить многие ваши проблемы.

Благодаря включению всех зависимостей в один JAR-файл Spring Boot, развертывание чрезвычайно упрощается. Вместо сбора и проверки всех развертываемых зависимостей плагин Spring Boot обеспечивает их упаковку в выходной JAR-файл. А при его наличии можно запускать приложение везде, где доступна виртуальная машина Java (JVM), с помощью команды вида `java -jar <SpringBootAppName.jar>`.

Это еще не все.

Достаточно установить нужное значение одного-единственного свойства в файле сборки, чтобы плагин сборки Spring Boot сделал этот один-единственный JAR-

[1] От нем. über — «сверх». — *Примеч. пер.*

файл полностью (само-)исполняемым. При наличии на машине JVM можно не набирать (описывать в сценарии) длинную строку `java -jar <SpringBootAppName.jar>`, а просто набрать команду `<SpringBootAppName.jar>` (конечно, заменив имя файла своим) — и дело в шляпе, все запущено и работает. Проще некуда.

Автоконфигурация

Новички в Spring Boot иногда смотрят на автоконфигурацию как на какое-то волшебство. Вероятно, автоконфигурация — величайший фактор повышения производительности работы программистов из предоставляемых Spring Boot. Я часто называю ее сверхспособностью разработчиков: Spring Boot обеспечивает *невероятную производительность*, учитывая мнения в популярных и повторяющихся сценариях использования.

Мнения в программном обеспечении? Чем они могут помочь?!

Если вы хоть сколько-нибудь долго работали программистом, то, без сомнения, заметили частое повторение определенных паттернов. Не точь-в-точь, конечно, но довольно близко — вероятно, в 80–90 % случаев проектирование, разработка и действия разработчика относятся к типичным сценариям использования.

Я уже упоминал эту повторяемость в программном обеспечении, именно она делает стартовые пакеты Spring Boot изумительно согласованными и удобными. Повторяемость также означает, что упомянутые действия, относящиеся к написанию кода для решения конкретной задачи, готовы к упрощению.

Возьмем для примера Spring Data — проект, связанный со Spring Boot и используемый им. Мы знаем, что для любого обращения к базе данных должно быть открыто какое-либо соединение с ней. Мы также знаем, что по выполнении приложением своих задач это соединение необходимо закрыть, чтобы избежать проблем. А в промежутке, вероятно, выполняются многочисленные обращения к базе данных посредством SQL-запросов — простых и сложных, только для чтения и с возможностью записи в базу данных, — и правильное создание этих SQL-запросов требует определенных усилий.

Теперь представьте себе, что все это можно существенно упростить. Автоматически открывать соединение при задании базы данных. Автоматически закрывать соединение по завершении работы приложения. Следовать простому и логичному соглашению по автоматическому созданию запросов, с минимальными затратами усилий разработчиком. Легкая настройка даже этого минимального кода под свои нужды, опять же в соответствии с простым соглашением, для создания упомянутых сложных SQL-запросов, заведомо согласованных и эффективных.

О подобном подходе к написанию кода иногда говорят: *соглашения важнее конфигурации* (convention over configuration), и если конкретное соглашение для вас внове, возможно, на первый взгляд оно покажется странноватым, но станет просто глотком свежего воздуха, если вам уже приходилось реализовывать схожую функциональность с написанием порой сотен повторяющихся, отупляющих строк кода создания/освобождения ресурсов/настройки простейших задач. Spring Boot (и большинство проектов Spring) следует мантре *«соглашения важнее конфигурации»*, гарантируя, что если следовать простым, четким и хорошо документированным соглашениям, код конфигурации будет минимальным или вообще окажется не нужен.

Кроме того, автоконфигурация наделяет разработчика сверхспособностями благодаря тому, что команда Spring подходит к конфигурации среды, придерживаясь принципа «разработчик прежде всего». Наша производительность как разработчиков наиболее высока, когда можно сосредоточиться на решаемой задаче, а не выполнять миллион рутинных операций настройки. Как же Spring Boot добивается этого?

Возьмем для примера другой связанный со Spring Boot проект — Spring Cloud Stream. При подключении к платформе обмена сообщениями наподобие RabbitMQ или Apache Kafka разработчик обычно должен указать определенные настройки для соединения с этой платформой и ее использования — имя хоста, порт, учетные данные и т. д. Концентрация внимания на нуждах разработчиков означает наличие настроек по умолчанию (на случай, если другие не указаны), создающих подходящие условия для разработчика, работающего локально: localhost, порт по умолчанию и т. д. Все это — хорошо продуманная среда выполнения, практически на 100 % совместимая со средами разработки, хотя в производственной среде это не так. В рабочей среде может понадобиться указать конкретные значения, так как среда платформы и хостинга очень изменчива.

Использование значений по умолчанию в совместно разрабатываемых проектах позволяет также существенно сократить время, необходимое для настройки среды разработки. Это чистая выгода как для вас, так и для вашей команды.

В некоторых случаях конкретные сценарии использования относятся не к упомянутым 80–90 % типичных сценариев, а к прочим 10–20 % вполне допустимых сценариев. В этих случаях можно выборочно переопределить автоконфигурацию или даже отключить ее полностью, хотя, конечно, тогда вы утратите все сверхспособности. Для изменения некоторых решений обычно требуется всего лишь задать нужные значения одного или нескольких свойств или предоставить один или несколько компонентов, реализующих нечто, для чего в Spring Boot в обычных условиях служит автоконфигурация. Другими словами, это очень просто сделать в тех редких случаях, когда действительно нужно. Автоконфи-

гурация — инструмент с широкими возможностями, незаметно и неустанно работающий ради упрощения жизни разработчика и невероятного повышения его производительности.

Резюме

Три важнейшие возможности Spring Boot, на которых основывается все прочее: упрощение управления зависимостями, упрощение развертывания и автоконфигурация. Их все можно настраивать под свои нужды, но делать это приходится редко. И все три делают очень многое, чтобы вы стали более производительным разработчиком. Spring Boot буквально окрыляет!

В следующей главе мы рассмотрим, какой замечательный выбор есть у приступающих к созданию приложений Spring Boot. Все варианты — отличные!

ГЛАВА 2

Выбираем инструменты и приступаем к работе

Как вы скоро увидите, начать создавать приложения Spring Boot очень просто. Сложнее всего выбрать какие-то из возможных вариантов инструментария.

В этой главе мы рассмотрим некоторые из превосходных возможностей, доступных разработчику при создании приложений Spring Boot: системы сборки, языки программирования, наборы программных средств, редакторы кода и многое другое.

Maven или Gradle?

Исторически у разработчиков приложений Java имелось несколько вариантов утилит сборки проектов. Некоторые со временем стали непопулярными, и не без причин, и теперь сообщество разработчиков сконцентрировалось на двух: Maven и Gradle. Spring Boot поддерживает обе с одинаковым успехом.

Apache Maven

Maven — популярный и надежный вариант системы автоматизации сборки. Она существует уже довольно давно, с 2002 года, став одним из основных проектов Apache Software Foundation в 2003-м. Используемый в ней декларативный подход был и остается концептуально проще альтернативных подходов, существовавших как тогда, так и сейчас: достаточно просто создать файл `pom.xml` в формате XML и указать в нем нужные зависимости и плагины. В команде `mvn` можно указать «фазу» завершения, которая выполняет желаемую задачу, например, компиляции, удаления результатов предыдущего выполнения, упаковки, выполнения приложения и т. д.:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
                xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
                xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.4.0</version>
    <relativePath/> <!-- Поиск родительского узла в репозитории -->
  </parent>
  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>demo</name>
  <description>Демонстрационный проект для Spring Boot</description>

  <properties>
    <java.version>11</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

Maven также создает (и предполагает) определенную структуру проекта по соглашению. Обычно не стоит слишком отклоняться от этой структуры, если вы не настроены сражаться с утилитой сборки, — это занятие заведомо нерациональное. Для абсолютного большинства проектов определяемая соглашением

структура Maven прекрасно подходит, так что менять ее обычно смысла не имеет. На рис. 2.1 приведено приложение Spring Boot с типичной структурой проекта Maven.



Рис. 2.1. Структура проекта Maven приложения Spring Boot



Подробнее об ожидаемой Maven структуре проекта можно прочитать во «Введении в стандартную организацию каталогов» (Introduction to the Standard Directory Layout) проекта Maven (https://oreil.ly/mavenprojintro).

Если же в какой-то момент вам покажется, что соглашения проектов Maven и/или подход с жесткой структурой слишком вас ограничивают, существует еще один прекрасный вариант.

Gradle

Gradle — еще одна популярная утилита для сборки проектов виртуальной машины Java (JVM). Выпущенная в 2008 году, Gradle генерирует минималистичный и гибкий файл сборки `build.gradle` на основе предметно-ориентированного языка (Domain Specific Language, DSL). Вот пример файла сборки для приложения Spring Boot:

```
plugins {
  id 'org.springframework.boot' version '2.4.0'
  id 'io.spring.dependency-management' version '1.0.10.RELEASE'
```

```
  id 'java'
}

group = 'com.example'
version = '0.0.1-SNAPSHOT'
sourceCompatibility = '11'

repositories {
  mavenCentral()
}

dependencies {
  implementation 'org.springframework.boot:spring-boot-starter'
  testImplementation 'org.springframework.boot:spring-boot-starter-test'
}

test {
  useJUnitPlatform()
}
```

Gradle дает возможность разработчику выбирать в качестве DSL языки программирования Groovy и Kotlin, а также предоставляет несколько других возможностей, сокращающих время сборки проектов, в частности:

- инкрементную компиляцию классов Java;
- обход компиляции для Java (если никаких изменений в код не внесено);
- специальный демон для компиляции проекта.

Выбор между Maven и Gradle

На этой стадии вам может показаться, что никакого выбора между утилитами сборки делать не надо. Почему бы просто не остановиться на Gradle?

Более жесткий декларативный, можно даже сказать, догматичный подход Maven значительно повышает единообразие процесса сборки для различных проектов и сред. Если следовать пути Maven, проблемы практически исключены и можно сосредоточиться на коде, не возясь со сборкой.

Как у любой системы сборки, связанной с программированием/написанием сценариев, у Gradle бывают проблемы с первыми выпусками новых версий языка. Но команда создателей Gradle реагирует очень быстро и решает эти проблемы весьма оперативно, однако если вы предпочитаете (или должны) сразу работать с предварительными выпусками версий языков, учитывайте этот нюанс.

Gradle, возможно, выполняет сборку несколько быстрее, а иногда и *существенно* быстрее, особенно в больших проектах. Тем не менее длительность сборки

аналогичных проектов Maven и Gradle на основе микросервисов вряд ли будет сильно различаться.

Для простых проектов, а также проектов с очень сложными требованиями к сборке гибкость Gradle может оказаться настоящим глотком свежего воздуха. Но дополнительная гибкость Gradle, особенно в упомянутых сложных проектах, означает и большее количество времени, потраченного на тонкую настройку, диагностику и устранение неполадок, когда все работает не так, как ожидалось. Бесплатный сыр — только в мышеловке.

Spring Boot поддерживает как Maven, так и Gradle, а если вы используете Initializr (его мы обсудим в одном из следующих разделов), проект и нужный файл сборки создаются автоматически, чтобы быстро приступить к работе. Короче говоря, попробуйте обе эти утилиты и выберите ту, что лучше подходит для вас. Spring Boot с радостью поддержит любой выбор.

Java или Kotlin

Хотя с JVM можно использовать множество разных языков, наиболее широко распространены два. Один — Java, первоначальный язык JVM, второй — относительно новый для этой сферы Kotlin. Оба — полноправные «граждане» Spring Boot.

Java

В зависимости от того, что считать официальной датой рождения проекта — общедоступный выпуск 1.0 или момент создания проекта, — Java существует уже 25 или 30 лет соответственно. Но никакого застоя нет и в помине. С сентября 2017 года цикл выпуска Java составляет 6 месяцев, в результате чего новые усовершенствованные возможности появляются чаще, чем раньше. База кода была очищена от мусора, убраны возможности, ставшие ненужными вследствие появления новых, а под влиянием сообщества разработчиков появились новые жизненно важные возможности. Java процветает, как никогда ранее.

Этот быстрый темп инноваций в сочетании с долговечностью и упором на обратную совместимость Java означает, что существует бесчисленное количество мелких фирм, занимающихся созданием и поддержкой критически важных приложений Java по всему миру. Многие из этих приложений используют Spring.

Java формирует железобетонный фундамент практически для всей базы кода Spring, а поэтому прекрасно подходит в качестве языка создания приложений

Spring Boot. Чтобы изучить код Spring, Spring Boot и всех связанных проектов, достаточно посетить веб-сайт GitHub, где он размещен, и просмотреть его там или клонировать проект для просмотра офлайн. Так как на Java написано множество доступных примеров кода, проектов и руководств «Для начинающих», поддержка написания приложений Spring Boot на языке Java организована лучше, чем любого другого сочетания инструментов на рынке.

Kotlin

Kotlin — относительно новый игрок на рынке. Созданный компанией JetBrains в 2010 году и открытый для всеобщего доступа в 2011-м, Kotlin облегчает использование Java. Kotlin с самого начала был спроектирован:

- *лаконичным* — на этом языке можно сообщить свои намерения компилятору (а равно и себе, и прочим программистам) при помощи минимального количества кода;
- *безопасным* — он устраняет ошибки, связанные с null-значениями, исключая возможность их использования *по умолчанию*, разве что разработчик осознанно переопределит поведение, чтобы это разрешить;
- *интероперабельным* — язык нацелен на органичное взаимодействие со всеми существующими библиотеками для JVM, Android и браузеров;
- *обладающим удобными утилитами* — сборка приложений Kotlin возможна как в многочисленных интегрированных средах разработки (IDE), так и из командной строки подобно приложениям Java.

Создатели Kotlin расширяют возможности языка с большой осторожностью, но все же довольно быстро. При проектировании им не требуется в первую очередь обеспечивать совместимость написанного на языке более чем за 25 лет, поэтому они быстро добавили очень удобные возможности, которые появятся в Java лишь многие версии спустя.

Kotlin — не только лаконичный, но и очень гибкий язык. Не углубляясь в нюансы, скажем только, что обеспечить такое лингвистическое изящество помогают несколько возможностей языка, в том числе функции расширения и инфиксная нотация. Мы обсудим их подробнее чуть позже, но Kotlin допускает следующие синтаксические конструкции:

```
infix fun Int.multiplyBy(x: Int): Int { ... }

// вызов функции с помощью инфиксной нотации
1 multiplyBy 2
```

```
// означает то же самое, что и
1.multiplyBy(2)
```

Легко представить, что возможность задавать собственный, более гибкий «язык в языке» может принести немалую пользу при проектировании API. Вдобавок Kotlin лаконичен, что делает написанные на нем приложения Spring Boot еще более короткими и удобочитаемыми по сравнению с аналогами на Java, причем ясность выражения намерений в них не утрачивается.

Kotlin был полноправной частью фреймворка Spring с момента выхода осенью 2017 года версии 5.0, распространения полной поддержки на Spring Boot (весна 2018-го) и прочих проектов компонентов позднее. Кроме того, документация Spring была расширена и теперь включает примеры как на Java, так и на Kotlin. Это значит, что можно писать целые приложения Spring Boot на Kotlin так же легко, как и на Java.

Выбор между Java и Kotlin

Самое замечательное, что выбирать на самом деле не надо. Kotlin компилируется в тот же байткод, что и Java, а поскольку можно создавать проекты Spring, включающие файлы исходного кода как на Java, так и на Kotlin и с легкостью вызывать оба компилятора, то можно использовать тот из них, который удобнее даже *в рамках одного проекта*. Как насчет того, чтобы поймать сразу двух зайцев?

Конечно, если вы предпочитаете один из этих языков или у вас есть иные личные либо профессиональные причины, то можете разрабатывать целые приложения на том или другом. Хорошо, когда есть выбор, правда?

Выбираем версию Spring Boot

Для находящихся в продакшене приложений всегда следует использовать актуальную версию Spring Boot со следующими временными и очень узкими исключениями.

- Вы сейчас работаете с более старой версией, но обновляете, повторно тестируете и развертываете свои приложения в таком порядке, что просто еще не дошли до конкретного приложения.
- Вы сейчас работаете с более старой версией из-за найденного конфликта или программной ошибки. Вы сообщили о ней команде создателей Spring и получили инструкции ждать обновления Spring Boot или соответствующей зависимости.

- Вам нужно использовать какие-либо возможности из мгновенной (snapshot), промежуточной (milestone) или предварительной (release candidate) необщедоступной версии, и вы готовы рисковать, применяя код, который пока еще не был объявлен общедоступным, то есть готовым для продакшена.



Мгновенная, промежуточная и предварительная версии перед публикацией всесторонне тестируются, так что их стабильность обеспечивается довольно строго. Однако до одобрения и публикации общедоступной версии всегда возможно внесение в API изменений, исправлений и т. д. Риски для приложения невелики, просто вы должны убедиться, что они приемлемы, прежде чем использовать любую предварительную версию программного обеспечения.

Spring Initializr

Существует множество способов создания приложений Spring Boot, но отправная точка у большинства одна — Spring Initializr (рис. 2.2).

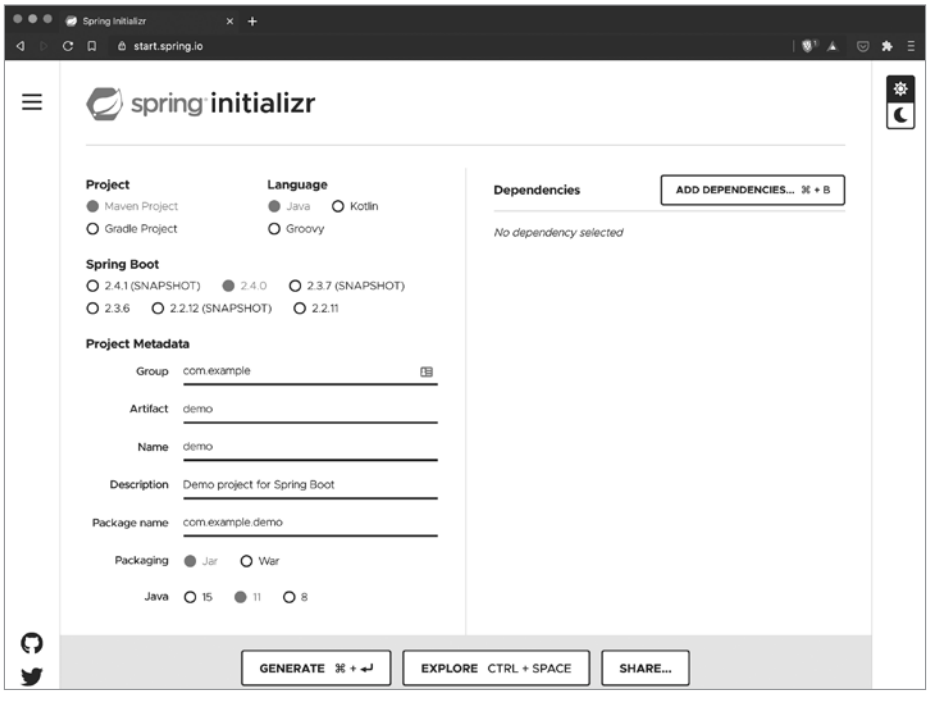


Рис. 2.2. Spring Initializr

Доступ к Spring Initializr, который иногда называют просто по URL, `start.spring.io`, можно получить из «мастеров» создания проектов большинства крупных IDE, из командной строки, но чаще всего из браузера. Последний предоставляет несколько удобных возможностей, пока что недоступных при прочих способах.

ИНСТАЛЛЯЦИЯ JAVA

Я предполагаю, что если вы дочитали до этого места, то уже установили на своей машине текущую версию набора инструментов разработчика Java (Java Development Kit, JDK), иногда называемого *стандартной версией платформы Java (Java Platform, Standard Edition)*.

Подробные инструкции по его установке выходят за рамки темы данной книги, но небольшие рекомендации не помешают, правда?

Я обнаружил, что простейший способ инсталляции на машине одного или нескольких JDK и управления ими — воспользоваться SDKMAN! (https://sdkman.io). Эта система управления пакетами также упрощает установку интерфейса командной строки Spring Boot, который вы будете использовать далее в этой книге, и многих других утилит, так что это чрезвычайно полезное вспомогательное ПО. Достаточно следовать инструкциям, которые вы получите по адресу https://sdkman.io/install, чтобы настроить все нужное.



SDKMAN! написан в виде скрипта bash (командная оболочка Unix/Linux), а потому работает нативным образом в macOS и Linux, а также в других операционных системах, основанных на Unix или Linux. SDKMAN! может работать и на Windows, но не естественным образом: для установки и запуска SDKMAN! в среде Windows необходимо сначала установить Windows Subsystem for Linux (WSL) (https://oreil.ly/WindowsSubL), Git Bash for Windows (https://oreil.ly/GitBashWin) и MinGW (http://www.mingw.org). См. подробности на упомянутой ранее установочной странице SDKMAN!.

Для установки нужной версии Java необходимо только просмотреть из среды SDKMAN! доступные варианты с помощью команды `sdk list java`, а затем выполнить `sdk install java <вставьте_сюда_нужную_версию_java>`. Есть много замечательных вариантов, но я рекомендую для начала выбрать версию с долгосрочной поддержкой (Long Term Support, LTS), скомпонованную AdoptOpenJDK в один пакет с Hotspot JVM, например `11.0.7.hs-adpt`.

Если вы по каким-то причинам не хотите использовать SDKMAN!, можете просто скачать и установить JDK непосредственно с сайта https://adoptopenjdk.net/. При этом у вас все будет работать, но обновляться в дальнейшем станет труднее, как и работать с несколькими JDK.

Для создания проекта Spring Boot наилучшим способом из возможных перейдите в браузере по адресу https://start.spring.io/. Здесь мы выберем несколько опций и приступим к созданию.

В начале работы со Spring Initializr необходимо выбрать систему сборки для проекта. Как уже упоминалось, есть два замечательных варианта, Maven и Gradle. Для этого примера возьмем Maven.

Далее выбираем Java в качестве языка проекта.

Как вы уже могли заметить, в Spring Initializr достаточно хорошие значения по умолчанию для представленных опций, чтобы можно было создать проект без какого-либо вмешательства пользователя. При открытии упомянутой страницы заранее были выбраны Maven и Java. А также текущая версия Spring Boot, которую необходимо выбрать как для этого проекта, так и для большинства других.

Пока что оставим значения в разделе Project Metadata неизменными, хотя в будущих проектах станем их менять.

Кроме того, пока что мы не включаем никаких зависимостей и благодаря этому можем сосредоточиться на создании проекта, ни на что не отвлекаясь.

Впрочем, перед генерацией проекта хотелось бы отметить несколько очень удобных возможностей Spring Initializr, хотя и с небольшой оговоркой.

Если хотите изучить метаданные своего проекта и подробности относительно зависимостей, основанные на выбранных опциях, перед его генерацией можете нажать кнопку Explore или открыть раздел Project Explorer страницы Spring Initializr с помощью горячих клавиш Ctrl+Пробел (рис. 2.3). После этого Initializr отобразит структуру проекта и файл сборки, который будет включен в сжатый (`.zip`) файл проекта, подготовленный для скачивания. При этом вы можете просмотреть структуру каталогов/пакета, файл свойств приложения (подробнее об этом далее) и свойства проекта и зависимостей, указываемые в файле сборки `pom.xml`, поскольку для этого проекта мы используем Maven.

Это быстрый и удобный способ проверки конфигурации проекта и зависимостей перед скачиванием, извлечением и загрузкой в свой IDE новенького пустого проекта.

Еще одна небольшая, но любимая многими разработчиками возможность Spring Initializr — темный режим (dark mode). Если установить переключатель Dark UI вверху страницы, как показано на рис. 2.4, сайт перейдет в темный режим и будет делать это по умолчанию при каждом заходе на страницу. Это незначительная возможность, но если вы предпочитаете работать в темном режиме,

использовать Initializr вам будет намного приятнее. Вы захотите возвращаться туда снова и снова!

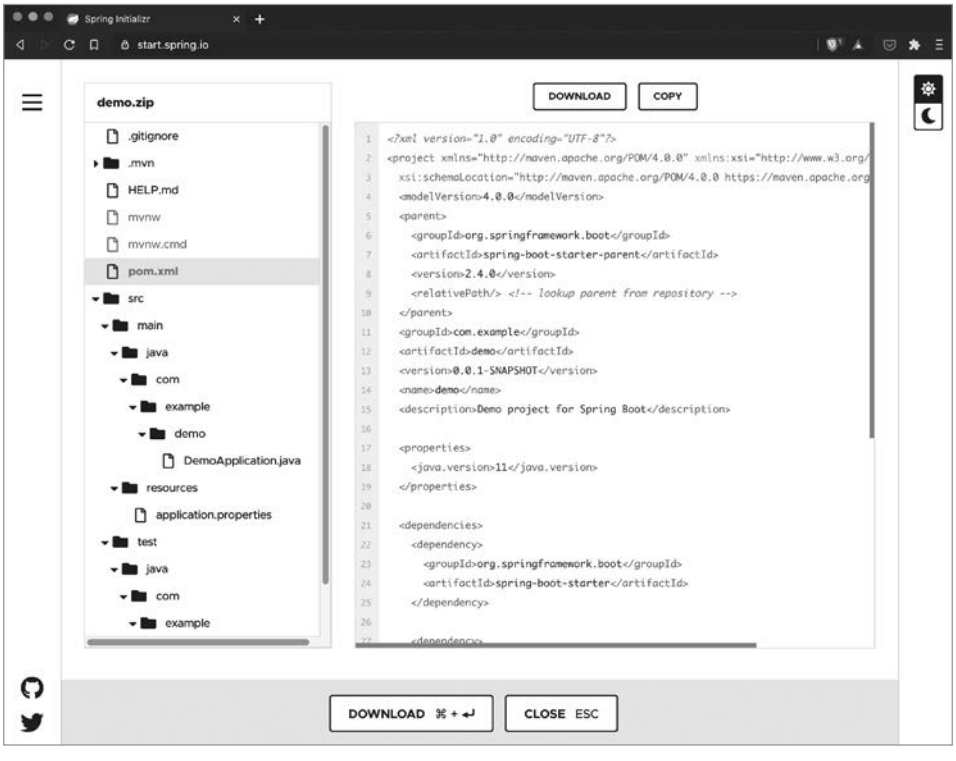


Рис. 2.3. Project Explorer на сайте Spring Initializr



Помимо основного класса приложения и его метода main, а также пустого файла теста, Spring Initializr не создает никакого кода — он генерирует проект под вашим чутким руководством. Небольшое, но очень важное различие: результаты генерации кода очень сильно варьируются и зачастую мешают, когда вы начинаете вносить изменения. Генерируя структуру проекта, в том числе файл сборки с зависимостями, Initializr обеспечивает трамплин для написания кода, необходимого для использования автоконфигурации Spring Boot. А автоконфигурация дает вам сверхспособности и ни в чем не ограничивает.

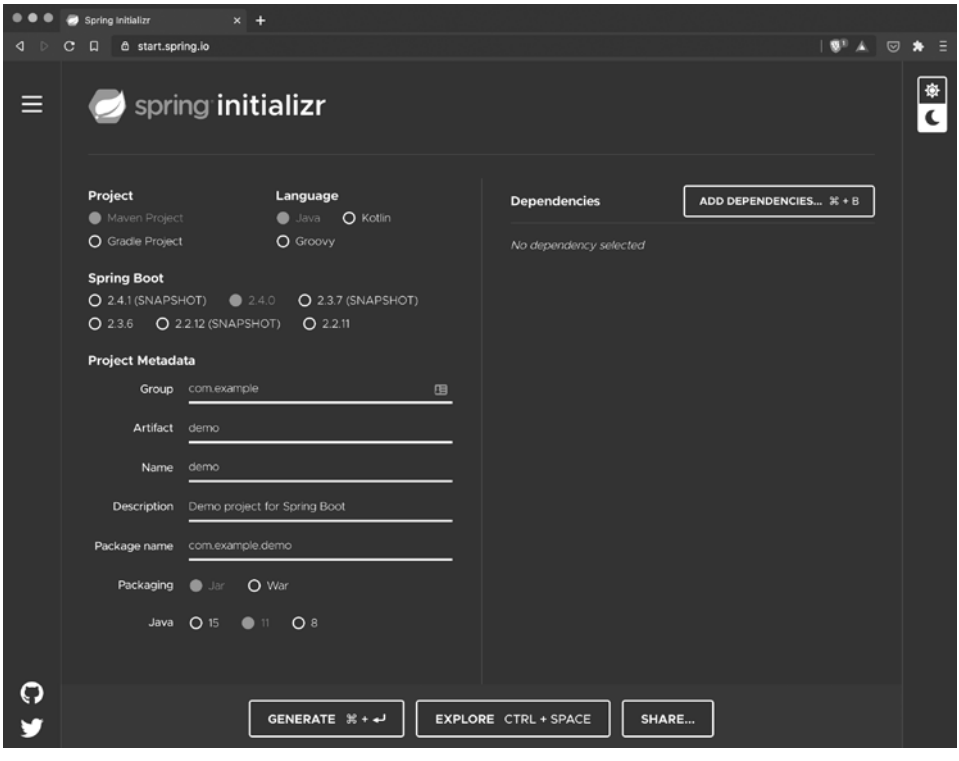


Рис. 2.4. Spring Initializr в темном режиме!

Далее нажмите кнопку Generate для генерации, компоновки и скачивания проекта и сохраните его в выбранный каталог на локальной машине. Затем перейдите к скачанному файлу `.zip` и разархивируйте его, чтобы начать разрабатывать приложение.

Прямиком из командной строки

Если вы предпочитаете работать с командной строкой или хотели бы написать сценарий для создания проекта, интерфейс командной строки Spring Boot как будто создан специально для вас. У CLI Spring Boot немало замечательных возможностей, но пока что мы сосредоточимся на создании нового проекта Boot.

УСТАНОВКА CLI SPRING BOOT

Наверное, простейший способ установки CLI Spring Boot, как и JDK, утилит Kotlin и пр., — с помощью SDKMAN!. Выполните в окне терминала:

```
sdk list
```

чтобы просмотреть все доступные для установки пакеты (на рис. 2.5 показана запись для CLI Spring Boot). Далее выполните:

```
sdk list springboot
```

чтобы просмотреть доступные версии CLI Spring Boot, и установите самую свежую (текущую) версию с помощью команды:

```
sdk install springboot
```

Если идентификатор конкретной версии не указан с помощью команды `sdk install <утилита> <идентификатор_версии>`, SDKMAN! обычно устанавливает последнюю рекомендуемую версию языка/утилиты, предназначенную для продакшена. Для различных поддерживаемых пакетов это означает разное: например, в случае Java будет установлена последняя версия с долгосрочной поддержкой (LTS), а не, возможно, доступная более свежая не-LTS-версия. А все потому, что каждые шести месяцев выпускается версия Java с новым номером и периодически одна из них объявляется LTS-версией. То есть может существовать одна или несколько более новых версий, официально поддерживаемых на протяжении только шесть месяцев каждая (для оценки функциональных возможностей, тестирования и даже развертывания в продакшене), в то время как конкретный LTS-выпуск полностью поддерживается в смысле обновлений и исправления ошибок.

Это замечание в какой-то мере общее для всех. Возможны различия для разных поставщиков JDK, хотя большинство из них практически не отклоняются от общепринятого стандарта. Подробностям этого вопроса посвящены целые конференции, но для наших целей они совершенно не важны.

```
--------------------------------------------------------------------------------
Spring Boot (2.4.0)                        http://projects.spring.io/spring-boot/

Spring Boot takes an opinionated view of building production-ready Spring
applications. It favors convention over configuration and is designed to get you
up and running as quickly as possible.

                                                          $ sdk install springboot
--------------------------------------------------------------------------------
```

Рис. 2.5. CLI Spring Boot на сайте SDKMAN!

После установки CLI Spring Boot можно создать такой же проект, как мы только что создали, с помощью следующей команды:

```
spring init
```

Для разархивирования архива проекта в каталог `demo` можно воспользоваться командой:

```
unzip demo.zip -d demo
```

Как, неужели все так просто? Если коротко, это настройки по умолчанию. CLI Spring использует те же настройки по умолчанию, что и Spring Initializr (Maven, Java и т. д.), благодаря чему можно указывать аргументы только для значений, которые необходимо изменить. Давайте намеренно укажем значения для нескольких из этих настроек по умолчанию (а заодно добавим удобный трюк для разархивирования проекта), просто чтобы продемонстрировать, как это происходит:

```
spring init -a demo -l java --build maven demo
```

Мы по-прежнему инициализируем проект с помощью CLI Spring, но теперь указываем следующие аргументы.

- `-a demo` или `--artifactId demo` позволяет указать идентификатор артефакта для проекта. В данном случае назовем его `demo`.
- `-l java` или `--language java` позволяет указать основной язык проекта — Java, Kotlin или Groovy[1].
- `--build` — флаг аргумента для системы сборки, допустимые значения — `maven` и `gradle`.
- `-x demo` указывает CLI на необходимость разархивировать возвращаемый Initializr файл проекта `.zip`. Обратите внимание, что флаг `-x` указывать не обязательно, текстовая метка без расширения (как в приведенной ранее команде) трактуется как каталог для разархивирования.



Подробную информацию обо всех этих опциях можно найти, выполнив команду spring help init из командной строки.

При указании зависимостей ситуация несколько усложняется. Как вы могли догадаться, сложно превзойти простоту их выбора из меню, предоставляемого Spring Initializr. Но гибкость CLI Spring очень удобна для быстрого запуска, написания сценариев и построения конвейеров.

[1] Spring Boot по-прежнему поддерживает Groovy, хотя он используется далеко не так широко, как Java или Kotlin.

Еще один нюанс: по умолчанию CLI использует Initializr для обеспечения функциональности создания проектов, так что проекты, созданные с помощью любого из этих механизмов (CLI или веб-страницы Initializr), идентичны. Такая согласованность совершенно необходима небольшой команде, напрямую использующей возможности Spring Initializr.

Впрочем, иногда компания строго контролирует, какие зависимости могут использовать разработчики при создании проектов. Честно говоря, такой подход меня удручает и представляется очень ограниченным, мешающим компании гибко реагировать на запросы пользователей или рыночную обстановку. Из-за этого сотрудникам подобных компаний бывает сложно довести любые поставленные задачи до завершения.

В таком случае можно создать собственный генератор проектов (даже клонировать репозиторий Spring Initializr), задействуя его непосредственно через получившуюся веб-страницу, или просто открыть доступ к части REST API и использовать ее из CLI Spring. Для этого добавьте следующий параметр в приведенную ранее команду (конечно, заменив URL своим):

```
--target https://вставьте.тут.свой.URL.org
```

Работа в интегрированных средах разработки

Каким бы способом вы ни создали проект Spring Boot, для того, чтобы получить полезное приложение, вам придется открыть его и написать какой-то код.

Существуют три основные интегрированные среды разработки (IDE) и множество текстовых редакторов, вполне достойно удовлетворяющих все нужды разработчиков. В числе этих IDE (список далеко не полон!): Apache NetBeans (https://netbeans.apache.org), Eclipse (https://www.eclipse.org) и IntelliJ IDEA (https://www.jetbrains.com/idea). Все три представляют собой ПО с открытым исходным кодом (OSS), и во многих случаях их можно получить бесплатно[1].

В этой книге, как и в своей повседневной деятельности, я использую в основном IntelliJ Ultimate Edition. Все варианты IDE хороши, дело скорее в личных

[1] Доступны два варианта: Community Edition (CE) и Ultimate Edition (UE). Community Edition поддерживает разработку приложений Java и Kotlin, но для полноценной поддержки Spring придется использовать Ultimate Edition. Определенные сценарии использования служат основанием для предоставления бесплатной лицензии на Ultimate Edition, но вы, конечно, можете приобрести ее. Кроме того, обе прекрасно поддерживают приложения Spring Boot.

предпочтениях (или политике либо предпочтениях компании), так что можете свободно применять все, что вам подходит и нравится. Большинство описанных приемов прекрасно переносятся между всеми основными вариантами IDE.

Существует также несколько редакторов, над которыми трудилось немало разработчиков. Некоторые, такие как Sublime Text (https://www.sublimetext.com), — платные приложения, они обрели горячих поклонников благодаря качеству и долгому сроку существования. Другие, появившиеся на рынке позднее, например Atom (https://atom.io), созданный GitHub и принадлежащий сейчас Microsoft, и Visual Studio Code (https://code.visualstudio.com) (сокращенно VSCode, создан Microsoft), быстро приобретают функциональность и верных сторонников.

В книге я иногда использую VSCode и его аналог VSCodium, собранный на основе той же базы кода, но с отключенной телеметрией/отслеживанием (https://vscodium.com). Для поддержки возможностей, которые большинство разработчиков ожидают или требуют от среды разработки, я добавил в VSCode/VSCodium следующие расширения.

- *Пакет расширения Spring Boot (Pivotal)* (https://oreil.ly/SBExtPack). Он включает еще несколько расширений, в том числе Spring Initializr Java Support, Spring Boot Tools и Spring Boot Dashboard, упрощающих создание и редактирование приложений Spring Boot и управление ими в VSCode соответственно.
- *Отладчик для языка Java (Microsoft)* (https://oreil.ly/DebuggerJava). Зависимость пакета Spring Boot Dashboard.
- *Привязки клавиш IntelliJ IDEA (Keisuke Kato)* (https://oreil.ly/IntellijIDEAKeys). Это расширение упрощает для меня переход между обеими IDE, поскольку в основном я использую IntelliJ.
- *Поддержку языка Java (Red Hat)* (https://oreil.ly/JavaLangSupport). Зависимость Spring Boot Tools.
- *Maven для языка Java (Microsoft)* (https://oreil.ly/MavenJava). Упрощает работу с проектами, основанными на Maven.

Существуют и другие расширения, полезные для работы с XML, Docker и прочими сопутствующими технологиями, но для наших целей необходимы лишь эти.

Вернемся к проекту Spring Boot. Далее вам нужно будет открыть его в выбранной вами IDE или текстовом редакторе. Для большинства примеров в книге будем использовать IntelliJ IDEA — созданную JetBrains IDE (написана на Java и Kotlin) с очень широкими возможностями. Если вы уже задали ассоциации своей IDE с файлами сборки проекта, дважды щелкните кнопкой мыши на

файле `pom.xml` в корневом каталоге проекта (с помощью Finder на компьютерах Mac, проводника Windows или одного из многочисленных диспетчеров файлов в Linux) и вы автоматически загрузите проект в свою IDE. Либо откройте проект из IDE или редактора так, как рекомендуют его разработчики.



Многие IDE и редакторы позволяют создавать сокращенные формы команд командной строки, с помощью которых можно запускать и загружать проект с помощью одной короткой команды. В качестве примеров можно назвать сокращенную команду idea IDE IntelliJ, code редактора VSCode/VSCodium и atom редактора Atom.

Прогулка по функции main()

Теперь, когда мы загрузили проект в IDE или в редакторе, можем взглянуть на небольшие различия проекта Spring Boot (рис. 2.6) и стандартного проекта приложения Java.

DemoApplication.java ×

src > main > java > com > example > demo > DemoApplication.java > {} com.example.demo

```
1   package com.example.demo;
2
3   import org.springframework.boot.SpringApplication;
4   import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6   @SpringBootApplication
7   public class DemoApplication {
8
        Run | Debug
9       public static void main(String[] args) {
10          SpringApplication.run(DemoApplication.class, args);
11      }
12
13  }
14
```

Рис. 2.6. Основной класс демонстрационного приложения Spring Boot

Стандартное приложение Java включает (по умолчанию) пустой метод `public static void main`. Выполняя приложение Java, JVM ищет этот метод в качестве начальной точки приложения, а в его отсутствие запуск приложения завершается неудачей и выдается сообщение об ошибке наподобие следующего:

```
Error:
Main method not found in class PlainJavaApp, please define the main method as:
  public static void main(String[] args)
or a JavaFX application class must extend javafx.application.Application
```

Конечно, можно поместить в метод `main` класса Java код для выполнения, именно это приложение Spring Boot и делает. При начальной загрузке Spring Boot проверяет среду, настраивает приложение, создает начальный контекст и запускает приложение. И все это благодаря одной аннотации верхнего уровня и одной строке кода, как показано на рис. 2.7.

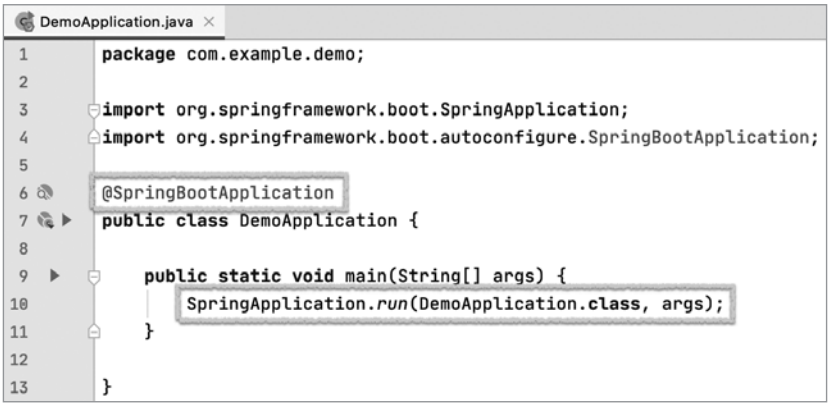


Рис. 2.7. Сущность приложения Spring Boot

В дальнейшем мы заглянем за кулисы этих механизмов. А пока достаточно сказать, что Boot намеренно и по умолчанию избавляет вас от многих утомительных нюансов настройки приложения во время его запуска, развязывая разработчику руки для написания осмысленного кода.

Резюме

В этой главе мы рассмотрели некоторые первоклассные варианты создания приложений Spring Boot. Предпочитаете ли вы производить сборку проектов с помощью Maven или Gradle, писать код на Java или Kotlin, создавать проекты в веб-интерфейсе Spring Initializr или его аналоге для командной строки CLI Spring Boot — все возможности и удобство Spring Boot без каких-либо компромиссов к вашим услугам. Работать с проектами Spring Boot можно также во множестве разнообразных IDE и текстовых редакторов с превосходной поддержкой Spring Boot.

Как было показано здесь и в главе 1, Spring Initializr делает все возможное для быстрого и удобного создания проектов. Spring Boot вносит значительный вклад на протяжении всего жизненного цикла разработки программного обеспечения благодаря следующим возможностям:

- упрощению управления зависимостями, играющему свою роль от создания проекта до разработки и сопровождения;
- автоконфигурации, резко сокращающей или даже исключающей шаблонный код, который в противном случае пришлось бы писать, прежде чем начать работать над проблемной областью;
- упрощению развертывания, в результате чего упаковка и развертывание становятся проще пареной репы.

И все эти возможности поддерживаются вне зависимости от выбранных вами по ходу дела системы сборки, языка программирования и набора программных средств. Удивительно гибкое и обладающее большими возможностями сочетание.

В следующей главе мы создадим свое первое приложение Spring Boot — приложение, предоставляющее REST API.

ГЛАВА 3

Создаем первый Spring Boot REST API

В этой главе я расскажу и покажу, как разработать простейшее работающее приложение с помощью Spring Boot. Поскольку большинство приложений предполагают доступ пользователей к серверным облачным ресурсам, обычно через пользовательский интерфейс клиентской части, программный интерфейс приложений (Application Programming Interface, API) — прекрасная отправная точка как для изучения, так и для применения на практике. Что ж, приступим.

«Как» и «почему» API

Эпоха монолитных приложений, отвечающих сразу за все, прошла.

Это не значит, что их больше нет или что их не будут создавать в дальнейшем. В силу различных обстоятельств монолитное приложение, предоставляющее «в одном флаконе» множество возможностей, все еще может пригодиться, особенно в следующих случаях:

- предметная область, а значит и ее границы, практически неизвестна;
- предоставляемые функциональные возможности тесно сцеплены, и скорость взаимодействий между модулями важнее гибкости;
- требования по масштабированию всех функциональных возможностей известны и согласованы между собой;
- функциональность стабильна, изменения происходят медленно или носят ограниченный характер, либо наблюдается и то и другое.

На все остальные случаи жизни существуют микросервисы.

Конечно, это крайне упрощенный, но полезный обзор ситуации. Благодаря разбиению функциональных возможностей на меньшие взаимосвязанные фрагменты их можно расцепить между собой, открывая дорогу для создания более гибких и надежных систем, которые удобнее сопровождать.

В любой распределенной системе — и не сомневайтесь, включающая микросервисы система относится как раз к этой категории — ключевым фактором является обмен информацией. Нет сервиса, который был бы как остров[1]. И хотя существуют многочисленные механизмы соединения приложений/микросервисов, чаще всего мы начинаем с эмуляции того, что пронизывает всю нашу повседневную жизнь, — интернета.

Интернет предназначен для обмена информацией. На самом деле создатели его предшественника, Сети Агентства перспективных исследовательских проектов (Advanced Research Projects Agency Network, ARPANET), предусмотрели необходимость поддержания обмена информацией между системами даже в случае серьезных сбоев. Логично предположить, что подход на основе HTTP, сходный с тем, что мы используем ежедневно, тоже позволит создавать, извлекать, обновлять и удалять различные ресурсы по сети.

И хотя я очень люблю историю, но не стану углубляться в особенности развития REST API, скажу только, что Рой Филдинг (Roy Fielding) изложил их принципы в своей диссертации 2000 года, основанной на *объектной модели HTTP* 1994 года.

Что такое REST и почему это важно

Как уже упоминалось, API — это спецификация/интерфейс, написанный нами, разработчиками, чтобы наш код мог использовать другой код: библиотеки, прочие приложения или сервисы. Но что означает аббревиатура *REST* в *REST API*?

REST представляет собой акроним фразы «передача состояния представления» (representational state transfer) — довольно загадочного способа заявить, что наше приложение взаимодействует с другим. Приложение А не ожидает от приложения Б сохранения своего состояния — текущего и накопленной информации о процессах — между вызовами связи. Приложение А передает представление нужной части своего состояния с каждым запросом к приложению Б. Сразу

[1] Аллюзия на начало проповеди Джона Донна, ставшее знаменитым как эпиграф к книге Эрнеста Хемингуэя «По ком звонит колокол». — *Примеч. пер.*

видно, почему это повышает живучесть и отказоустойчивость: состояние взаимодействий приложения Б с приложением А сохраняется в случае аварийного сбоя и перезапуска первого — приложение А может повторно выполнить запрос и продолжить с того же места, где произошел сбой.



Ресурсы, которые придерживаются этого общего принципа, часто называют приложениями/сервисами без сохранения состояния (stateless), поскольку каждый сервис поддерживает собственное состояние даже в ходе последовательности взаимодействий и не ждет от других сервисов/приложений, что они сделают это вместо него.

API в стиле HTTP-глаголов

А как же REST API, иногда называемый реализующим REST (RESTful API)?

Существует несколько стандартных HTTP-команд, описанных в рабочих предложениях (request for comments, RFC) IETF (Internet Engineering Task Force, инженерный совет Интернета). Небольшое их количество постоянно применяется для создания API, периодически используются еще несколько. REST API обычно создаются на основе следующих HTTP-команд:

- `POST`;
- `GET`;
- `PUT`;
- `PATCH`;
- `DELETE`.

Эти команды соответствуют типовым операциям над ресурсами: созданию (`POST`), чтению (`GET`), обновлению (`PUT` и `PATCH`) и удалению (`DELETE`).



Следует признать, что я несколько размыл границы команд, соотнеся PUT с обновлением ресурса, и чуть меньше — тем, что соотнес POST с созданием ресурса. Читатели, потерпите немного, я проясню все, когда мы будем обсуждать реализации.

Иногда применяются также следующие две команды:

- `OPTIONS`;
- `HEAD`.

Их можно использовать для извлечения параметров обмена информацией для пар «запрос — ответ» (`OPTIONS`) и заголовка ответа без его тела (`HEAD`).

Для целей этой книги и, конечно, большинства сценариев продакшена имеет смысл сосредоточиться на первой, чаще всего используемой группе. Для начала создадим элементарный микросервис, реализующий простейшее REST API.

Возвращаемся к Initializr

Начнем, как обычно, со Spring Initializr, как показано на рис. 3.1. Я изменил значения полей Group и Artifact в соответствии со своими предпочтениями (можете спокойно использовать обозначения, которые нравятся вам), указал Java 11 в разделе Options (необязательно, подойдет любая из перечисленных версий) и выбрал только зависимость Spring Web. Как указано в отображаемом описании, эта зависимость предоставляет несколько функциональных возможностей, в том числе возможность «создания веб-приложений, включая *реализующие REST*, с помощью Spring MVC» (курсив мой). Именно это нам и нужно.

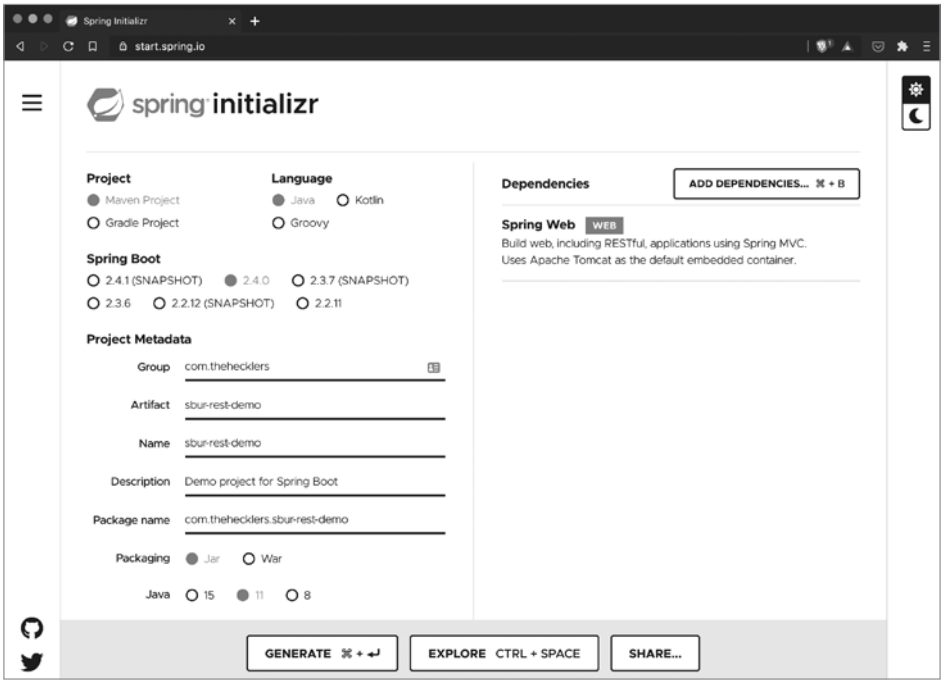


Рис. 3.1. Создание проекта Spring Boot для REST API

Сгенерировав проект в Initializr и сохранив полученный файл `.zip` на локальной машине, извлекаем из архива файлы проекта — обычно это можно сделать, дважды щелкнув кнопкой мыши на скачанном файле `sbur-rest-demo.zip` в Проводнике или с помощью команды `unzip` из окна командной оболочки/терминала. После этого открываем проект в своем любимом IDE или текстовом редакторе, в результате чего видим что-то вроде изображенного на рис. 3.2.

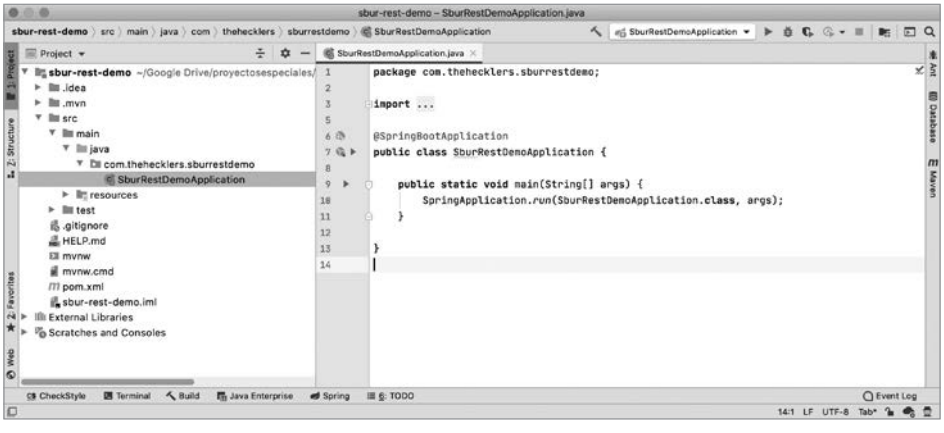


Рис. 3.2. Новый проект Spring Boot ждет нас

Создание простого класса предметной области

Для работы с ресурсами необходимо написать соответствующий им код. Начнем с создания очень простого класса предметной области, соответствующего нужному нам ресурсу.

Я страстный любитель кофе, как знают мои друзья, в число которых теперь входите и вы. Поэтому в качестве предметной области для этого примера я возьму кофе и создам класс, представляющий его конкретный вид.

Начнем с создания класса `Coffee`. Без него в этом примере не обойтись, ведь нам нужен какой-либо ресурс для демонстрации работы с ресурсами через REST API. Но простота или сложность предметной области для этого примера несущественны, так что можно взять попроще, чтобы сосредоточиться на конечной цели — итоговом REST API.

Как показано на рис. 3.3, класс `Coffee` имеет две переменные-члена:

- поле `id`, однозначно идентифицирующее конкретный вид кофе;
- поле `name` с названием кофе.

```
@SpringBootApplication
public class SburRestDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(SburRestDemoApplication.class, args);
    }

}

class Coffee {
    private final String id;
    private String name;

    public Coffee(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public Coffee(String name) {
        this(UUID.randomUUID().toString(), name);
    }

    public String getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

Рис. 3.3. Класс Coffee — класс нашей предметной области

Я объявил поле `id` как `final`, чтобы ему можно было присвоить значение только один раз и нельзя было модифицировать в дальнейшем, поэтому значение ему необходимо присваивать при создании экземпляра класса `Coffee` и метода-модификатора у него нет.

Я создал два конструктора: один принимает оба параметра, другой предоставляет уникальный идентификатор, если он не указан при создании экземпляра `Coffee`.

Далее я создал методы доступа (accessor) и изменения (mutator) — они же метод-сеттер и метод-геттер, если вы предпочитаете такую терминологию, — для поля

name, которое не объявлено final, то есть является изменяемым. Это спорное архитектурное решение, но для этого примера оно хорошо подходит.

С предметной областью разобрались. Пришло время для REST.

GET

Вероятно, чаще всего используемая из наиболее широко используемых HTTP-команд — GET. Так что приступим!

Коротко об аннотации @RestController

Если не залезать слишком глубоко в кроличью нору, то можно сказать, что фреймворк Spring MVC («модель — представление — контроллер») был создан для разделения ответственности относительно данных, их доставки и визуализации в предположении, что представления будут визуализироваться в виде веб-страницы, отображаемой сервером. Для связи всего этого служит аннотация @Controller.

@Controller — стереотип/псевдоним для аннотации @Component, означающий, что при запуске приложения из этого класса образуется компонент Spring, создаваемый и управляемый контейнером инверсии управления (inversion of control, IoC) в приложении. Классы, снабженные аннотацией @Controller, включают объект Model для передачи слою представления данных, соответствующих модели, и отображения (совместно с ViewResolver) приложением конкретного представления с помощью специальной технологии.



Spring поддерживает несколько технологий представлений и шаблонизаторов, они описываются в следующей главе.

Можно также указать классу, снабженному аннотацией Controller, возвращать ответ в формате нотации объектов JavaScript (JSON) или другом ориентированном на данные формате, например XML, посредством добавления к классу или методу аннотации @ResponseBody (по умолчанию JSON). В результате в возвращаемое методом значение входит *все тело* ответа на веб-запрос вместо лишь части Model.

Удобная аннотация @RestController сочетает в себе аннотации @Controller и @ResponseBody, упрощая тем самым код и более прозрачно выражая намерения

разработчика. Снабдив класс аннотацией `@RestController`, можно приступать к созданию REST API.

Попробуем GET

REST API работают с объектами — либо по отдельности, либо в виде группы связанных объектов. Возвращаясь к нашему сценарию с кофе: может понадобиться извлечь конкретный вид кофе, все виды кофе или все сильно обжаренные, все относящиеся к определенному диапазону идентификаторов либо со словом «Колумбийский» в названии, например. Чтобы учесть варианты извлечения одного или нескольких экземпляров класса, рекомендуется создавать в коде несколько методов.

Начнем с создания списка объектов класса `Coffee` для метода, возвращающего несколько объектов `Coffee`, как показано в следующем простейшем описании класса. Я определил переменную для хранения этой группы видов кофе в виде `List` объектов `Coffee`. В качестве высокоуровневого интерфейса для своего типа переменной-члена выбрал `List`, но на самом деле присваиваю для использования внутри класса `RestApiDemoController` пустой `ArrayList`:

```
@RestController
class RestApiDemoController {
  private List<Coffee> coffees = new ArrayList<>();
}
```



Рекомендуемая практика: выбирать самый верхний уровень типа (класс, интерфейс), способный удовлетворить потребности внутренних и внешних API. В некоторых случаях, как здесь, они не совпадают. Что касается внутреннего API, то List обеспечивает API уровня, позволяющего создать чистейшую реализацию, на основе моих критериев. Что же касается внешнего, то можно описать абстракцию даже еще более высокого уровня, как я вскоре продемонстрирую.

Никогда не помешает иметь немного данных для извлечения, чтобы убедиться, что все работает должным образом. В следующем коде я написал конструктор класса `RestApiDemoController`, добавив код для заполнения списка видов кофе при создании объекта:

```
@RestController
class RestApiDemoController {
  private List<Coffee> coffees = new ArrayList<>();

  public RestApiDemoController() {
    coffees.addAll(List.of(
```

```
        new Coffee("Café Cereza"),
        new Coffee("Café Ganador"),
        new Coffee("Café Lareño"),
        new Coffee("Café Três Pontas")
    ));
  }
}
```

Как показано в следующем коде, я создал в классе `RestApiDemoController` метод, возвращающий итерируемую группу видов кофе, представленную переменной экземпляра `coffees`. Я решил воспользоваться `Iterable<Coffee>`, поскольку желаемую функциональность для этого API легко обеспечит любой итерируемый тип.

Используем `@RequestMapping` для получения (`GET`) списка видов кофе:

```
@RestController
class RestApiDemoController {
  private List<Coffee> coffees = new ArrayList<>();

  public RestApiDemoController() {
    coffees.addAll(List.of(
        new Coffee("Café Cereza"),
        new Coffee("Café Ganador"),
        new Coffee("Café Lareño"),
        new Coffee("Café Três Pontas")
    ));
  }

  @RequestMapping(value = "/coffees", method = RequestMethod.GET)
  Iterable<Coffee> getCoffees() {
    return coffees;
  }
}
```

К аннотации `@RequestMapping` я добавил спецификацию пути `/coffees` и тип метода `RequestMethod.GET`, указывающий, что метод будет отвечать на запросы по пути `/coffees`, причем возможны только запросы типа `HTTP GET`. Этот метод отвечает за извлечение данных, но не за обновления какого-либо вида. Spring Boot с помощью включенных в Spring Web зависимостей Jackson автоматически выполняет маршалинг и демаршалинг объектов в JSON и другие форматы.

Можно еще больше упростить этот код с помощью другой удобной аннотации — `@GetMapping`, допускающей лишь запросы типа `GET`, что позволяет указать только путь (даже без `path =`, поскольку никакого разрешения конфликтов параметров не требуется), сокращая тем самым стереотипный код. Следующий код ясно

демонстрирует преимущества перехода на эту аннотацию в смысле удобочитаемости:

```
@GetMapping("/coffees")
Iterable<Coffee> getCoffees() {
    return coffees;
}
```

ПОЛЕЗНЫЕ УКАЗАНИЯ ОТНОСИТЕЛЬНО @REQUESTMAPPING

У аннотации `@RequestMapping` есть несколько специализированных версий:

- `@GetMapping`;
- `@PostMapping`;
- `@PutMapping`;
- `@PatchMapping`;
- `@DeleteMapping`.

Все эти аннотации можно применять на уровне как класса, так и метода, причем пути аддитивны. Например, если аннотировать `RestApiDemoController` и его метод `getCoffees()` так, как показано в коде, приложение отреагирует точно так же, как и в случае кода из двух предыдущих фрагментов:

```
@RestController
@RequestMapping("/")
class RestApiDemoController {
  private List<Coffee> coffees = new ArrayList<>();

  public RestApiDemoController() {
    coffees.addAll(List.of(
        new Coffee("Café Cereza"),
        new Coffee("Café Ganador"),
        new Coffee("Café Lareño"),
        new Coffee("Café Três Pontas")
    ));
  }

  @GetMapping("/coffees")
  Iterable<Coffee> getCoffees() {
    return coffees;
  }
}
```

Извлекать все виды кофе в наше импровизированное хранилище данных удобно, но этого недостаточно. Что делать, если нужно извлечь один конкретный вид кофе?

Извлечение одного элемента аналогично извлечению нескольких. Для этой цели добавим еще один метод, `getCoffeeById`, как показано в следующем фрагменте кода.

Часть `{id}` указанного пути представляет собой переменную URI (унифицированный идентификатор ресурса), ее значение передается методу `getCoffeeById` через параметр экземпляра `id` путем снабжения его аннотацией `@PathVariable`.

Проходя в цикле по списку видов кофе, метод возвращает заполненный объект `Optional<Coffee>` при обнаружении соответствия или пустой объект `Optional<Coffee>`, если запрошенный `id` отсутствует в нашей маленькой группе:

```
@GetMapping("/coffees/{id}")
Optional<Coffee> getCoffeeById(@PathVariable String id) {
    for (Coffee c: coffees) {
        if (c.getId().equals(id)) {
            return Optional.of(c);
        }
    }
    return Optional.empty();
}
```

POST

Рекомендуемый метод создания ресурсов — `HTTP POST`.



Метод POST передает описание ресурса, обычно в формате JSON, и требует у целевого сервиса создать этот ресурс по указанному URI.

Как показано в следующем фрагменте кода, `POST` довольно прост: сервис получает информацию об указанном кофе в виде объекта `Coffee` — спасибо Spring Boot за автоматический маршалинг — и добавляет его в наш список видов кофе. А затем возвращает объект `Coffee`, автоматически демаршалируемый Spring Boot в формат JSON (по умолчанию), запрашивающему сервису или приложению:

```
@PostMapping("/coffees")
Coffee postCoffee(@RequestBody Coffee coffee) {
    coffees.add(coffee);
    return coffee;
}
```

PUT

В общем случае запросы `PUT` применяются для модификации существующих ресурсов с известными URI.



Согласно документу IETF Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content («Протокол передачи гипертекста (HTTP/1.1): семантика и содержимое»), запросы PUT должны обновлять указанный ресурс при его наличии, если же ресурса еще не существует — создавать его.

Следующий код работает в полном соответствии со спецификацией: найти вид кофе с указанным идентификатором, если найден — обновить. Если в списке нет такого вида кофе — создает его:

```
@PutMapping("/coffees/{id}")
Coffee putCoffee(@PathVariable String id, @RequestBody Coffee coffee) {
    int coffeeIndex = -1;

    for (Coffee c: coffees) {
        if (c.getId().equals(id)) {
            coffeeIndex = coffees.indexOf(c);
            coffees.set(coffeeIndex, coffee);
        }
    }

    return (coffeeIndex == -1) ? postCoffee(coffee) : coffee;
}
```

DELETE

Для удаления ресурсов используются запросы `HTTP DELETE`. Как показано в следующем фрагменте кода, мы создали метод, принимающий идентификатор вида кофе в переменной `@PathVariable` и удаляющий соответствующий вид из нашего списка с помощью метода `removeIf` интерфейса `Collection`[1]. Метод `removeIf` получает предикат `Predicate`, так что можно передать лямбда-выражение, возвращающее `True` для вида кофе, который нужно удалить. Просто и эффективно:

```
@DeleteMapping("/coffees/{id}")
void deleteCoffee(@PathVariable String id) {
    coffees.removeIf(c -> c.getId().equals(id));
}
```

[1] Родительского для List. — *Примеч. пер.*

И не только

Хотя в этот сценарий можно внести еще немало улучшений, я сосредоточусь на двух нюансах: уменьшении повторов в коде и возврате кодов состояния HTTP там, где этого требует спецификация.

Для уменьшения количества повторов в коде я вынесу часть сопоставления URI, общую для всех методов класса `RestApiDemoController`, в аннотацию уровня класса `@RequestMapping` — `"/coffees"`. После этого можно удалить соответствующую часть из спецификаций сопоставления URI всех методов, тем самым несколько сокращая степень текстового шума, как демонстрирует следующий код:

```
@RestController
@RequestMapping("/coffees")
class RestApiDemoController {
  private List<Coffee> coffees = new ArrayList<>();

  public RestApiDemoController() {
    coffees.addAll(List.of(
        new Coffee("Café Cereza"),
        new Coffee("Café Ganador"),
        new Coffee("Café Lareño"),
        new Coffee("Café Três Pontas")
    ));
  }

  @GetMapping
  Iterable<Coffee> getCoffees() {
    return coffees;
  }

  @GetMapping("/{id}")
  Optional<Coffee> getCoffeeById(@PathVariable String id) {
    for (Coffee c: coffees) {
      if (c.getId().equals(id)) {
        return Optional.of(c);
      }
    }

    return Optional.empty();
  }

  @PostMapping
  Coffee postCoffee(@RequestBody Coffee coffee) {
    coffees.add(coffee);
    return coffee;
  }
```

```
  @PutMapping("/{id}")
  Coffee putCoffee(@PathVariable String id, @RequestBody Coffee coffee) {
    int coffeeIndex = -1;

    for (Coffee c: coffees) {
      if (c.getId().equals(id)) {
        coffeeIndex = coffees.indexOf(c);
        coffees.set(coffeeIndex, coffee);
      }
    }

    return (coffeeIndex == -1) ? postCoffee(coffee) : coffee;
  }

  @DeleteMapping("/{id}")
  void deleteCoffee(@PathVariable String id) {
    coffees.removeIf(c -> c.getId().equals(id));
  }
}
```

Далее мы заглянем в вышеупомянутый документ IETF и обнаружим, что хотя коды состояния HTTP для метода `GET` не оговорены и лишь предлагаются для методов `POST` и `DELETE`, для ответов метода `PUT` они обязательны. Для этого я модифицирую метод `putCoffee`, как показано в следующем фрагменте кода. Теперь метод `putCoffee` будет возвращать не только модифицированный или созданный объект `Coffee`, но и объект `ResponseEntity`, содержащий вышеупомянутый объект `Coffee` и код состояния HTTP: 201 (Создано), если соответствующий вид кофе отсутствовал в списке, и 200 (OK), если он уже существовал и был обновлен. Можно, конечно, сделать еще многое, но текущий код приложения удовлетворяет требованиям и предоставляет простые и чистые внутренние и внешние API:

```
@PutMapping("/{id}")
ResponseEntity<Coffee> putCoffee(@PathVariable String id,
        @RequestBody Coffee coffee) {
    int coffeeIndex = -1;

    for (Coffee c: coffees) {
        if (c.getId().equals(id)) {
            coffeeIndex = coffees.indexOf(c);
            coffees.set(coffeeIndex, coffee);
        }
    }

    return (coffeeIndex == -1) ?
            new ResponseEntity<>(postCoffee(coffee), HttpStatus.CREATED) :
            new ResponseEntity<>(coffee, HttpStatus.OK);
}
```

Доверяй, но проверяй

Код готов, осталось проверить наш API в действии.



Практически для всех связанных с HTTP задач я буду использовать клиент командной строки HTTPie (https://httpie.org). Иногда буду применять также curl (https://curl.haxx.se) и Postman (https://www.postman.com), но HTTPie — гибкий клиент с простым интерфейсом командной строки и превосходной функциональностью.

Как показано на рис. 3.4, я запрашиваю у конечной точки `coffees` все виды кофе, содержащиеся сейчас в нашем списке. HTTPie по умолчанию выполняет запрос `GET` и использует домен `localhost`, если не указано иное имя домена, тем самым уменьшая объем набираемого текста. Как и можно было ожидать, выводятся все четыре вида кофе, внесенные в список.

```
mheckler-a01 :: ~ » http :8080/coffees
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/json
Date: Thu, 19 Nov 2020 00:04:42 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

[
    {
        "id": "41ba3a26-b94c-4ab2-84ff-71a8ab63aad9",
        "name": "Café Cereza"
    },
    {
        "id": "686ed31a-0719-4907-b4ec-d79f41c8be2d",
        "name": "Café Ganador"
    },
    {
        "id": "f96da5f2-ede8-4862-aa81-ea4c3a5b626a",
        "name": "Café Lareño"
    },
    {
        "id": "11f1dcef-7808-4971-99fc-0cc1458baff2",
        "name": "Café Três Pontas"
    }
]
```

Рис. 3.4. Получаем полный список видов кофе с помощью GET

Далее копируем поле `id` одного из только что перечисленных видов кофе и вставляем его в другой запрос `GET`. Рисунок 3.5 демонстрирует полученный правильный ответ.

```
mheckler-a01 :: ~ » http :8080/coffees/41ba3a26-b94c-4ab2-84ff-71a8ab63aad9
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/json
Date: Thu, 19 Nov 2020 00:09:10 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "id": "41ba3a26-b94c-4ab2-84ff-71a8ab63aad9",
    "name": "Café Cereza"
}
```

Рис. 3.5. Получаем один из видов кофе с помощью GET

Выполнить запрос `POST` с помощью HTTPie можно очень просто: передать текстовый файл, содержащий JSON-представление объекта `Coffee` с полями `id` и `name`, и HTTPie поймет, что подразумевается операция `POST`. На рис. 3.6 показаны команда и ее результат.

```
mheckler-a01 :: ~/dev » http :8080/coffees < coffee.json
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/json
Date: Thu, 19 Nov 2020 00:10:48 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "id": "99999",
    "name": "Kaldi's Coffee"
}
```

Рис. 3.6. Добавляем в список новый вид кофе с помощью POST

Как уже упоминалось, команда `PUT` должна модифицировать существующий ресурс или добавлять новый, если запрашиваемый ресурс отсутствует. На рис. 3.7 я указал `id` только что добавленного вида кофе и передал команде еще один объект JSON с другим названием. В результате кофе с идентификатором 99999 теперь называется Caribou Coffee, а не Kaldi's Coffee, как раньше. Возвращаемый код состояния — 200 (OK), как и можно было ожидать.

На рис. 3.8 я выполнил аналогичный запрос `PUT`, только указал в URI несуществующий `id` кофе. И приложение послушно добавило его в соответствии с определенным IETF поведением, после чего вернуло код состояния HTTP 201 (Создано).

```
mheckler-a01 :: ~/dev » http PUT :8080/coffees/99999 < coffee2.json
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/json
Date: Thu, 19 Nov 2020 00:12:13 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "id": "99999",
    "name": "Caribou Coffee"
}
```

Рис. 3.7. Обновление уже существующего вида кофе с помощью PUT

```
mheckler-a01 :: ~/dev » http PUT :8080/coffees/88888 < coffee3.json
HTTP/1.1 201
Connection: keep-alive
Content-Type: application/json
Date: Thu, 19 Nov 2020 00:13:35 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "id": "88888",
    "name": "Mötor Oil Coffee"
}
```

Рис. 3.8. Добавление нового вида кофе с помощью PUT

Создание запроса типа `DELETE` с помощью HTTPie очень напоминает создание запроса `PUT`: необходимо указать HTTP команду и полный URI ресурса. На рис. 3.9 показан результат: код состояния HTTP 200 (OK), указывающий, что ресурс был успешно удален, без отображения какого-либо значения, поскольку ресурс более не существует.

```
mheckler-a01 :: ~/dev » http DELETE :8080/coffees/99999
HTTP/1.1 200
Connection: keep-alive
Content-Length: 0
Date: Thu, 19 Nov 2020 00:14:47 GMT
Keep-Alive: timeout=60
```

Рис. 3.9. Удаление вида кофе с помощью DELETE

Наконец, снова запросим полный список видов кофе и убедимся, что итоговое состояние такое, как ожидалось. Как демонстрирует рис. 3.10, в списке теперь есть новый вид кофе Mötor Oil Coffee. Проверка API успешна.

```
mheckler-a01 :: ~/dev » http :8080/coffees
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/json
Date: Thu, 19 Nov 2020 00:15:50 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

[
    {
        "id": "41ba3a26-b94c-4ab2-84ff-71a8ab63aad9",
        "name": "Café Cereza"
    },
    {
        "id": "686ed31a-0719-4907-b4ec-d79f41c8be2d",
        "name": "Café Ganador"
    },
    {
        "id": "f96da5f2-ede8-4862-aa81-ea4c3a5b626a",
        "name": "Café Lareño"
    },
    {
        "id": "11f1dcef-7808-4971-99fc-0cc1458baff2",
        "name": "Café Três Pontas"
    },
    {
        "id": "88888",
        "name": "Mötor Oil Coffee"
    }
]
```

Рис. 3.10. Получаем полный список видов кофе с помощью GET

Резюме

В этой главе было показано, как создать простейшее работоспособное приложение с помощью Spring Boot. Поскольку большинство приложений предполагают доступ пользователей к серверным облачным ресурсам, обычно через пользовательский интерфейс клиентской части, я показал, как создать и развить удобный REST API, единообразно обеспечивающий функциональность, необходимую для создания, чтения, модификации и удаления ресурсов, без которой невозможна никакая серьезная система.

Я рассмотрел и подробно объяснил аннотацию `@RequestMapping` и все ее многочисленные удобные специализированные версии, соответствующие описанным HTTP командам:

- `@GetMapping`;
- `@PostMapping`;
- `@PutMapping`;
- `@PatchMapping`;
- `@DeleteMapping`.

После создания методов, использующих эти аннотации и соответствующие им действия, я несколько переделал код, упростив его и обеспечив возврат кодов ответов HTTP там, где это необходимо. Проверка API подтвердила правильность его работы.

В следующей главе мы обсудим и продемонстрируем добавление доступа к базе данных в приложение Spring Boot, чтобы сделать его полезнее и подготовить к продакшену.

ГЛАВА 4

Добавление в приложение Spring Boot доступа к базе данных

Как обсуждалось в предыдущей главе, по различным очень веским причинам приложения часто предоставляют API без сохранения состояния. Впрочем, лишь очень немногие приложения таковы — то или иное состояние обычно для *каких-либо целей* сохраняется. Например, любой запрос к корзине заказов интернет-магазина вполне может включать ее состояние, но после размещения заказа его данные сохраняются. Способов осуществить это очень много, как и способов совместного использования или маршрутизации этих данных, но практически все системы более или менее значительного размера используют одну или несколько баз данных.

В этой главе я продемонстрирую, как добавить возможность доступа к базе данных в разработанное в предыдущей главе приложение Spring Boot. Эта глава задумывалась лишь как краткая вводная лекция в возможности Spring Boot по работе с данными, и в последующих главах мы изучим все это намного подробнее. Но во многих случаях изложенных тут основ более чем достаточно для создания полноценного решения. Приступим.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Пожалуйста, не забудьте для начала извлечь из репозитория код ветки chapter4begin.

Подготовка автоконфигурации для доступа к базе данных

Как показано ранее, Spring Boot стремится максимально упростить 80–90 % сценариев использования — паттерны кода и процессов, которые программи-

стам приходится повторять снова и снова. Как только паттерны определены, Spring Boot запускает автоматическую инициализацию нужных компонентов с разумными настройками по умолчанию. Чтобы настроить какую-либо функциональную возможность под себя, достаточно всего лишь указать одно или несколько значений свойств или написать адаптированную версию одного или нескольких компонентов. Как только средство автоконфигурации обнаружит изменения, то сразу же отключается и будет следовать указаниям разработчика. Доступ к базе данных — прекрасный пример этого.

Чего мы надеемся добиться

В приведенном ранее примере приложения я воспользовался `ArrayList` для хранения и поддержания в актуальном состоянии списка видов кофе. Для отдельного приложения этот подход довольно прост и ясен, но у него есть недостатки.

Во-первых, никакой отказоустойчивости. При сбое вашего приложения или платформы, на которой оно работает, все изменения, внесенные в список за время его работы — неважно, идет речь о секундах или месяцах, — будут утеряны.

Во-вторых, он плохо масштабируется. Второй и любой последующий запущенный экземпляр приложения будет работать с собственным отдельным списком видов кофе. Различные экземпляры не используют данные совместно, так что изменения, внесенные в список видов кофе одним экземпляром, — добавление новых видов кофе, удаление/обновление старых — не видны никому из работающих с другими экземплярами.

Очевидно, что это не лучший вариант.

В следующих главах рассмотрим несколько различных вариантов решения этих весьма реальных проблем. А пока подготовимся к их реализации.

Добавление зависимости для базы данных

Чтобы получить доступ к базе данных из приложения Spring Boot, необходимы:

- работающая база данных, запускаемая приложением или встроенная в него либо такая, к которой это приложение просто обращается;
- драйверы базы данных, обеспечивающие программный доступ к ней. Обычно их предоставляет поставщик базы данных;
- модуль Spring Data для доступа к нужной базе данных.

Некоторые модули Spring Data включают соответствующие драйверы базы данных в виде одной зависимости, которую можно выбрать в Spring Initializr. В других случаях, например, когда Spring использует API Java Persistence (JPA) для доступа к JPA-совместимым базам данных, необходимо выбрать зависимость Spring Data JPA *и* зависимость для конкретного драйвера целевой базы данных, например PostgreSQL.

В качестве первого шага на пути от конструкций в оперативной памяти к базе данных постоянного хранения я добавлю зависимости, а значит, и функциональные возможности, в файл сборки нашего проекта.

H2 — быстрая база данных, написанная целиком на Java[1], отличающаяся некоторыми интересными и полезными возможностями. Во-первых, она совместима с JPA, что позволяет подключаться к ней из приложения так же, как и к любой другой базе данных JPA, например Microsoft SQL, MySQL, Oracle или PostgreSQL. Во-вторых, в ней есть режимы работы в оперативной памяти и с использованием диска. Это открывает удобные возможности после преобразования размещенного в памяти `ArrayList` в базу данных: либо перевести H2 в режим постоянного хранения данных на диске, либо, поскольку речь идет о базе данных JPA, перейти на использование другой базы данных JPA. Любой из вариантов при этом намного упрощается.

Чтобы приложение могло взаимодействовать с базой данных H2, добавлю в раздел `<dependencies>` файла `pom.xml` нашего проекта следующие две зависимости:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```



Область видимости runtime зависимости для драйвера базы данных H2 означает его наличие по задаваемым classpath пути к классам в среде выполнения и тестирования, но не при компиляции. Такова рекомендуемая практика для библиотек, которые не требуются для компиляции.

[1] Нельзя не отметить некоторую внутреннюю противоречивость этого утверждения, ведь Java как интерпретируемый, а не компилируемый язык в принципе демонстрирует меньшее быстродействие, чем, скажем, C++. — *Примеч. пер.*

После сохранения модифицированного файла `pom.xml` и при необходимости повторного импорта/обновления зависимостей Maven вы получите полный доступ к функциональности добавленных зависимостей. Теперь нужно написать немного кода для ее использования.

Добавление кода

Поскольку у нас уже есть код для выполнения операций с видами кофе, необходимо провести небольшой рефакторинг при добавлении новых возможностей работы с базой данных. Полагаю, лучше всего начать с класса(ов) предметной области, в данном случае `Coffee`.

Аннотация @Entity

Как упоминалось ранее, H2 — JPA-совместимая база данных, так что мы добавим аннотации JPA, чтобы связать все воедино. Сам класс `Coffee` я снабжу аннотацией `@Entity` из API `javax.persistence`, указывающей, что `Coffee` — сохраняемая сущность, а к имеющейся переменной экземпляра `id` присоединю аннотацию `@Id` (также из `javax.persistence`), обозначая тем самым ее как поле идентификатора таблицы базы данных.



Если имя класса — в данном случае Coffee — не совпадает с желаемым названием таблицы базы данных, можно задать соответствующее аннотируемой сущности имя таблицы с помощью параметра name аннотации @Entity.

Хорошая IDE может сообщить разработчику, что в классе `Coffee` все еще чего-то не хватает. Например, IntelliJ подчеркнет название класса красным и выведет всплывающее окно с полезной информацией, если навести на него указатель мыши (рис. 4.1).

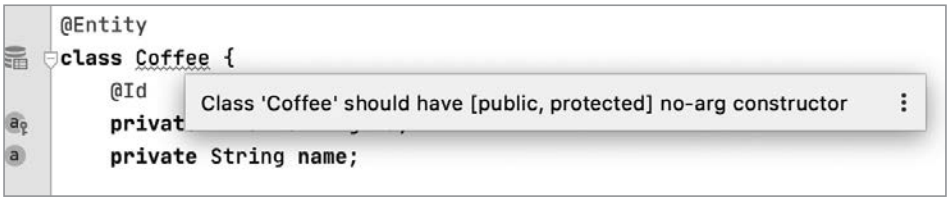


Рис. 4.1. В JPA-классе Coffee недостает конструктора

API Java Persistence требует использования конструктора без аргументов при создании объектов из строк таблицы базы данных, так что далее я его добавлю в класс. В результате получаем следующее предупреждение IDE (рис. 4.2): для конструктора без аргументов необходимо, чтобы все переменные экземпляра были изменяемыми, то есть не `final`.

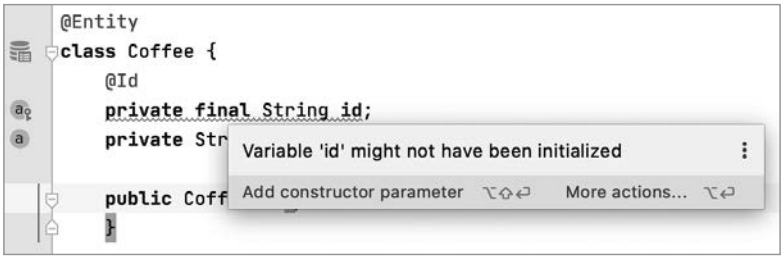


Рис. 4.2. При наличии конструктора без аргументов поле id не может быть терминальным

Удаление ключевого слова `final` из объявления переменной экземпляра `id` решает эту проблему. Но превращение поля `id` в изменяемое требует наличия в классе `Coffee` метода-модификатора для `id`, чтобы JPA мог присваивать ей значения, так что я добавлю также метод `setId()` (рис. 4.3).

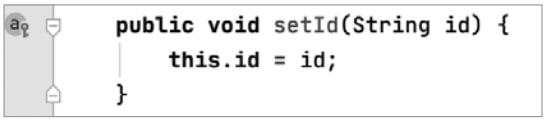


Рис. 4.3. Новый метод setId()

Репозиторий

Теперь, когда класс `Coffee` описывает допустимую сущность JPA, которую можно сохранять и извлекать, пора заняться соединением с базой данных.

Настройка и установление соединения с базой данных в экосистеме Java долго была довольно утомительной задачей. Как упоминалось в главе 1, размещение Java-приложения на сервере приложений требует от разработчиков выполнения нескольких довольно утомительных подготовительных действий. При взаимодействии с базой данных или обращении к хранилищу данных непосредственно из утилиты Java или клиентского приложения вам придется сделать дополнительные шаги с помощью API `PersistenceUnit`, `EntityManagerFactory` и `EntityManager` (и, возможно, объектов `DataSource`), открыть и закрыть соеди-

нение с базой данных и т. д. Прямо скажем, слишком много усилий для такого часто повторяющегося действия.

Spring Data вводит понятие репозиториев. `Repository` — описанный в Spring Data интерфейс, играющий роль удобной абстракции для различных баз данных. Существуют и другие механизмы доступа к базам данных из Spring Data, о которых мы поговорим в следующих главах, но, вероятно, в большинстве случаев наиболее удобны именно различные варианты `Repository`.

Сам `Repository` представляет собой простой заменитель для следующих типов данных:

- хранимого в базе данных объекта;
- поля уникального идентификатора объекта/первичного ключа.

Конечно, существует много других видов репозиториев, и большую часть их мы обсудим в главе 6. А пока что сосредоточимся на двух видах репозиториев, напрямую связанных с рассматриваемым примером: `CrudRepository` и `JpaRepository`.

Помните, раньше я упоминал, что при написании кода рекомендуется использовать интерфейс самого высокого уровня из числа подходящих для решения задачи? И хотя `JpaRepository` расширяет несколько интерфейсов и тем самым обеспечивает более широкую функциональность, `CrudRepository` охватывает все ключевые возможности CRUD и вполне достаточен для нашего пока что очень простого приложения.

Для того чтобы наше приложение могло пользоваться репозиторием, прежде всего необходимо определить специфичный для нашего приложения интерфейс `CoffeeRepository`, расширив интерфейс Spring Data `Repository`:

```
interface CoffeeRepository extends CrudRepository<Coffee, String> {}
```



Упомянутые тут два типа: тип хранимого объекта и тип его уникального идентификатора.

Таков простейший вариант создания репозитория в приложении Spring Boot. Можно, и порой очень удобно описывать запросы для репозитория, в одной из следующих глав я рассмотрю эту тему подробнее. Но самое захватывающее то, что Spring Boot автоматически находит драйвер базы данных (в данном случае H2) по пути классов, описанный в приложении интерфейс и класс JPA-сущности `Coffee` и *самостоятельно* создает прокси-компонент для базы данных. Не нужно

писать одну за другой строки практически идентичного стереотипного кода для каждого приложения, когда паттерны столь ясны и единообразны, так что у разработчика появляется свободное время для работы над новой нужной функциональностью.

Spring в действии

Пришло время применить репозиторий в деле. Мы пройдем этот путь шаг за шагом, как и в предыдущих главах, начиная со знакомства с функциональностью, а затем доводя программу до совершенства.

Начнем с автосвязывания/внедрения компонента репозитория с `RestApiDemoController`, чтобы контроллер мог обращаться к нему при получении запросов через внешний API (рис. 4.4).

```
@RestController
@RequestMapping("/coffees")
class RestApiDemoController {
    private final CoffeeRepository coffeeRepository;

    private List<Coffee> coffees = new ArrayList<>();

    public RestApiDemoController(CoffeeRepository coffeeRepository) {
        this.coffeeRepository = coffeeRepository;

        this.coffeeRepository.saveAll(List.of(
                new Coffee( name: "Café Cereza"),
                new Coffee( name: "Café Ganador"),
                new Coffee( name: "Café Lareño"),
                new Coffee( name: "Café Três Pontas")
        ));

        coffees.addAll(List.of(
                new Coffee( name: "Café Cereza"),
                new Coffee( name: "Café Ganador"),
                new Coffee( name: "Café Lareño"),
                new Coffee( name: "Café Três Pontas")
        ));
    }
```

Рис. 4.4. Автосвязывание репозитория с RestApiDemoController

Прежде всего объявим переменную-член класса:

```
private final CoffeeRepository coffeeRepository;
```

Далее добавим ее в качестве параметра конструктора:

```
public RestApiDemoController(CoffeeRepository coffeeRepository){}
```



До версии 4.3 фреймворка Spring приходилось во всех случаях добавлять аннотацию @Autowired перед методом, чтобы указать на необходимость автоматического связывания параметра, соответствующего компоненту Spring. Начиная же с 4.3 для класса с одним конструктором аннотация для автосвязываемых параметров не требуется, что экономит немало времени.

Когда репозиторий готов, я удаляю переменную-член `List<Coffee>` и меняю код изначального заполнения этого списка в конструкторе на сохранение этих же видов кофе в репозиторий (см. рис. 4.4).

Как видно из рис. 4.5, удаление переменной `coffees` сразу же делает все ссылки на нее неразрешимыми символами, так что следующая наша задача — заменить их соответствующими операциями обмена данными с репозиторием.



Рис. 4.5. Замена удаленной переменной экземпляра coffees

Удобнее всего начать с `getCoffees()` — простого метода без параметров для извлечения всех видов кофе. При использовании встроенного метода `findAll()` интерфейса `CrudRepository` можно даже не менять тип возвращаемого значения `getCoffees()`, поскольку он возвращает также тип `Iterable`. Достаточно просто вызвать метод `coffeeRepository.findAll()` и вернуть результат его выполнения, как показано здесь:

```
@GetMapping
Iterable<Coffee> getCoffees() {
    return coffeeRepository.findAll();
}
```

Рефакторинг метода `getCoffeeById()` демонстрирует, насколько проще может быть ваш код благодаря предоставляемой репозиториями функциональности. Больше не надо вручную искать в списке видов кофе нужный `id` — метод `findById()` интерфейса `CrudRepository` делает это за нас, как показано в следующем фрагменте кода. А поскольку метод `findById()` возвращает тип `Optional`, никаких изменений в сигнатуру нашего метода вносить не требуется:

```
@GetMapping("/{id}")
Optional<Coffee> getCoffeeById(@PathVariable String id) {
    return coffeeRepository.findById(id);
}
```

Модификация метода `postCoffee()` для использования репозитория также довольно проста, как показано здесь:

```
@PostMapping
Coffee postCoffee(@RequestBody Coffee coffee) {
    return coffeeRepository.save(coffee);
}
```

Метод `postCoffee()` тоже наглядно демонстрирует, как функциональность `CrudRepository` экономит время и сокращает код. Чтобы определить, новый это `Coffee` или уже существующий, и вернуть соответствующий код состояния HTTP вместе с сохраненным объектом `Coffee`, я воспользовался встроенным методом `existsById()` репозитория, как показано в следующем листинге:

```
@PutMapping("/{id}")
ResponseEntity<Coffee> putCoffee(@PathVariable String id,
                                 @RequestBody Coffee coffee) {

    return (!coffeeRepository.existsById(id))
            ? new ResponseEntity<>(coffeeRepository.save(coffee),
                  HttpStatus.CREATED)
            : new ResponseEntity<>(coffeeRepository.save(coffee), HttpStatus.OK);
}
```

Наконец, я модифицировал метод `deleteCoffee()`, воспользовавшись встроенным методом `deleteById()` интерфейса `CrudRepository`:

```
@DeleteMapping("/{id}")
void deleteCoffee(@PathVariable String id) {
    coffeeRepository.deleteById(id);
}
```

Применение компонента репозитория, созданного с помощью гибкого API `CrudRepository`, упрощает код `RestApiDemoController` и делает его намного более удобочитаемым и понятным, что демонстрирует следующий листинг:

```
@RestController
@RequestMapping("/coffees")
class RestApiDemoController {
    private final CoffeeRepository coffeeRepository;

    public RestApiDemoController(CoffeeRepository coffeeRepository) {
        this.coffeeRepository = coffeeRepository;

        this.coffeeRepository.saveAll(List.of(
                new Coffee("Café Cereza"),
                new Coffee("Café Ganador"),
                new Coffee("Café Lareño"),
                new Coffee("Café Três Pontas")
        ));
    }

    @GetMapping
    Iterable<Coffee> getCoffees() {
        return coffeeRepository.findAll();
    }

    @GetMapping("/{id}")
    Optional<Coffee> getCoffeeById(@PathVariable String id) {
        return coffeeRepository.findById(id);
    }

    @PostMapping
    Coffee postCoffee(@RequestBody Coffee coffee) {
        return coffeeRepository.save(coffee);
    }

    @PutMapping("/{id}")
    ResponseEntity<Coffee> putCoffee(@PathVariable String id,
                                     @RequestBody Coffee coffee) {

        return (!coffeeRepository.existsById(id))
                ? new ResponseEntity<>(coffeeRepository.save(coffee),
                HttpStatus.CREATED)
                : new ResponseEntity<>(coffeeRepository.save(coffee),
                  HttpStatus.OK);
    }

    @DeleteMapping("/{id}")
    void deleteCoffee(@PathVariable String id) {
        coffeeRepository.deleteById(id);
    }
}
```

Теперь осталось лишь убедиться, что наше приложение работает так, как ожидалось, и внешняя функциональность не изменилась.



Альтернативный, причем рекомендуемый, подход к тестированию функциональности — создание сначала модульных тестов в стиле разработки через тестирование (test-driven development, TDD). Я настоятельно рекомендую использовать его в среде реальной разработки программного обеспечения. Но оказалось, что когда нужно продемонстрировать и пояснить отдельные идеи разработки ПО, то чем меньше, тем лучше: если показывать ровно столько, сколько нужно, чтобы донести до читателя основные идеи, — полезный сигнал растет, а шум уменьшается, хотя последний и может пригодиться в дальнейшем. Поэтому я посвящу тестированию отдельную главу далее в книге.

Сохранение и извлечение данных

И снова обращаемся к API из командной строки с помощью HTTPie. Запрос к конечной точке `coffees` возвращает из базы данных H2 тех же четырех видов кофе, что и раньше (рис. 4.6).

```
mheckler-a01 :: ~ » http :8080/coffees
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/json
Date: Wed, 25 Nov 2020 21:08:48 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

[
    {
        "id": "ff3d96e0-236e-4157-8b45-9e9699276d6d",
        "name": "Café Cereza"
    },
    {
        "id": "d7a0f2a1-38f7-46ef-a884-8beb43e655cf",
        "name": "Café Ganador"
    },
    {
        "id": "d5458c8c-f480-47dc-9926-42fcb1f4051d",
        "name": "Café Lareño"
    },
    {
        "id": "1726fcdf-94f9-4f7b-9e60-e6e1b453f56f",
        "name": "Café Três Pontas"
    }
]
```

Рис. 4.6. Получаем полный список видов кофе с помощью GET

Копируем поле `id` одного из только что выведенных видов кофе и вставляем его в запрос `GET`, получая результат, приведенный на рис. 4.7.

```
mheckler-a01 :: ~ » http :8080/coffees/ff3d96e0-236e-4157-8b45-9e9699276d6d
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/json
Date: Wed, 25 Nov 2020 21:20:18 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "id": "ff3d96e0-236e-4157-8b45-9e9699276d6d",
    "name": "Café Cereza"
}
```

Рис. 4.7. Получаем один из видов кофе с помощью GET

На рис. 4.8 я добавлю в приложение и его базу данных новый вид кофе.

```
mheckler-a01 :: ~/dev » http :8080/coffees < coffee.json
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/json
Date: Wed, 25 Nov 2020 21:22:17 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "id": "99999",
    "name": "Kaldi's Coffee"
}
```

Рис. 4.8. Добавляем в список новый вид кофе с помощью POST

Как обсуждалось в предыдущей главе, команда `PUT` позволяет модифицировать уже существующий ресурс или добавить новый, если запрошенного ресурса не существует. На рис. 4.9 я указываю `id` только что добавленного вида кофе и передаю в команду объект JSON с изменениями в названии этого вида кофе. После обновления название вида кофе с `id` 99999 становится Caribou Coffee вместо Kaldi's Coffee, при этом возвращается код состояния 200 (OK), как и ожидалось.

```
mheckler-a01 :: ~/dev » http PUT :8080/coffees/99999 < coffee2.json
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/json
Date: Wed, 25 Nov 2020 21:24:04 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "id": "99999",
    "name": "Caribou Coffee"
}
```

Рис. 4.9. Обновление уже существующего вида кофе с помощью PUT

Далее я выполняю аналогичный запрос типа `PUT`, но указываю в URI несуществующий `id`. Приложение добавляет новый вид кофе в базу данных в соответствии с заданным IETF поведением и правильно возвращает код состояния HTTP 201 (Создано), как показано на рис. 4.10.

```
mheckler-a01 :: ~/dev » http PUT :8080/coffees/88888 < coffee3.json
HTTP/1.1 201
Connection: keep-alive
Content-Type: application/json
Date: Wed, 25 Nov 2020 21:25:28 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "id": "88888",
    "name": "Mötor Oil Coffee"
}
```

Рис. 4.10. Добавление нового вида кофе с помощью PUT

Наконец, проверяем удаление указанного вида кофе с помощью запроса типа `DELETE`, возвращающего только код состояния HTTP 200 (OK), указывающий, что ресурс был успешно удален, и более ничего, поскольку ресурса больше нет (рис. 4.11). Для проверки конечного состояния опять запрашиваем полный список видов кофе (рис. 4.12).

```
mheckler-a01 :: ~/dev » http DELETE :8080/coffees/99999
HTTP/1.1 200
Connection: keep-alive
Content-Length: 0
Date: Wed, 25 Nov 2020 21:26:55 GMT
Keep-Alive: timeout=60
```

Рис. 4.11. Удаление вида кофе с помощью DELETE

```
mheckler-a01 :: ~/dev » http :8080/coffees
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/json
Date: Wed, 25 Nov 2020 21:28:20 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

[
    {
        "id": "ff3d96e0-236e-4157-8b45-9e9699276d6d",
        "name": "Café Cereza"
    },
    {
        "id": "d7a0f2a1-38f7-46ef-a884-8beb43e655cf",
        "name": "Café Ganador"
    },
    {
        "id": "d5458c8c-f480-47dc-9926-42fcb1f4051d",
        "name": "Café Lareño"
    },
    {
        "id": "1726fcdf-94f9-4f7b-9e60-e6e1b453f56f",
        "name": "Café Três Pontas"
    },
    {
        "id": "88888",
        "name": "Mötor Oil Coffee"
    }
]
```

Рис. 4.12. Получаем с помощью GET все виды кофе, содержащиеся теперь в списке

И вновь видим один дополнительный вид кофе, ранее отсутствовавший в репозитории: Mötor Oil Coffee.

Наводим лоск

Как всегда, есть много областей, на которые не помешает взглянуть пристальнее, но я ограничусь двумя: извлечением начальной выборки данных в отдельный компонент и небольшим изменением порядка условий для большей ясности.

В предыдущей главе я заполнил в классе `RestApiDemoController` список видов кофе начальными значениями и до настоящего момента поддерживал эту структуру после преобразования в базу данных с доступом через репозиторий. Рекомендуемая практика — выделить эту функциональность в отдельный компонент, который можно быстро и легко отключать или включать.

Существует множество способов автоматического выполнения кода при запуске приложения, включая использование `CommandLineRunner` и `ApplicationRunner` и задание лямбда-выражения для достижения поставленной цели, в данном случае создания и сохранения демонстрационных данных. Но я предпочитаю использовать для этой цели класс `@Component` и метод `@PostConstruct` по следующим причинам.

- При автоматическом связывании компонента репозитория с помощью методов генерации компонентов `CommandLineRunner` и `ApplicationRunner` модульные тесты, имитирующие компонент репозитория внутри теста (как чаще всего происходит), обычно перестают работать.
- При имитации компонента репозитория или необходимости запуска приложения без создания демонстрационных данных можно легко и быстро имитировать компонент, отвечающий за заполнение данными, просто закомментировав его аннотацию `@Component`.

Я рекомендую создать класс `DataLoader`, аналогичный приведенному в следующем блоке кода. Выделение логики создания выборки данных, аналогичной методу `loadData()` класса `DataLoader`, и снабжение ее аннотацией `@PostConstruct` возвращает `RestApiDemoController` к его единственной задаче — предоставлению внешнего API и возлагает на `DataLoader` ответственность за *его* предполагаемые (и очевидные) задачи:

```
@Component
class DataLoader {
    private final CoffeeRepository coffeeRepository;
```

```
    public DataLoader(CoffeeRepository coffeeRepository) {
        this.coffeeRepository = coffeeRepository;
    }

    @PostConstruct
    private void loadData() {
        coffeeRepository.saveAll(List.of(
                new Coffee("Café Cereza"),
                new Coffee("Café Ganador"),
                new Coffee("Café Lareño"),
                new Coffee("Café Três Pontas")
        ));
    }
}
```

Еще один нюанс, требующий небольшой корректировки, — небольшое изменение булева условия тернарного оператора в методе `putCoffee()`. После рефакторинга этого метода для использования репозитория не остается никаких разумных причин вычислять сначала отрицательное условие, так что удаление оператора отрицания (`!`) из условия немного повышает ясность кода. Конечно, для сохранения изначальных исходов требуется поменять местами значения тернарного оператора для случаев `True` и `False`, что отражено в следующем коде:

```
@PutMapping("/{id}")
ResponseEntity<Coffee> putCoffee(@PathVariable String id,
                                 @RequestBody Coffee coffee) {

    return (coffeeRepository.existsById(id))
            ? new ResponseEntity<>(coffeeRepository.save(coffee),
                HttpStatus.OK)
            : new ResponseEntity<>(coffeeRepository.save(coffee),
                HttpStatus.CREATED);
}
```



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Полный код этой главы можно извлечь из репозитория кода по ветке chapter4end.

Резюме

В этой главе было показано, как добавить доступ к базе данных в созданное ранее приложение Spring Boot. И хотя эта глава должна была стать коротким введением в возможности работы с данными Spring Boot, в ней вы можете найти обзор:

- доступа к базе данных из Java;
- Java Persistence API (JPA);
- базы данных H2;
- Spring Data JPA;
- репозитория Spring Data;
- механизмов создания выборок данных с помощью репозиториев.

Далее мы намного подробнее обсудим доступ к базе данных Spring Boot, но материал этой главы формирует надежный фундамент и во многих случаях вполне самодостаточен.

В следующей главе я расскажу о полезных инструментах Spring Boot, с помощью которых можно получить четкую картину происходящего внутри приложения, когда оно не работает ожидаемым образом или нужно убедиться, что работает. И продемонстрирую их.

ГЛАВА 5

Настройка и контроль приложения Spring Boot

Любое приложение может начать работать не так, как ожидалось, причем в некоторых случаях решить проблему очень просто. Впрочем, если не считать редких удачных догадок, обычно приходится выискивать первопричину проблемы, чтобы по-настоящему от нее избавиться.

Отладка приложений Java или Kotlin, да и любых других, — важнейший навык, который каждый программист должен освоить в самом начале карьеры, а в дальнейшем постоянно оттачивать и расширять. Впрочем, далеко не все разработчики могут этим похвастать. Так что если вы еще не освоились с возможностями своего языка в области отладки и соответствующими утилитами, пожалуйста, как можно раньше выясните, какие варианты у вас есть. Это пригодится при разработке любых приложений и сэкономит колоссальное количество времени.

Но отладка кода — лишь один из уровней выяснения, идентификации и изоляции видов поведения, проявляющихся в ходе работы приложения. По мере превращения приложений в более динамические и распределенные разработчикам часто приходится:

- задавать и менять настройки приложений динамически;
- определять/подтверждать текущие настройки и их источники;
- осуществлять контроль и мониторинг среды выполнения приложения и индикаторов его состояния;
- временно изменять уровни журналирования работающих в текущий момент приложений для выявления первопричин различных проблем.

В этой главе показано, как использовать встроенные возможности Spring Boot по заданию настроек, его Autoconfiguration Report, а также Spring Boot Actuator

для гибкого и динамического создания, определения и изменения параметров среды приложения.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Пожалуйста, для начала извлеките из репозитория код ветки chapter5begin.

Конфигурация приложения

Ни одно приложение не является изолированным.

В большинстве случаев, когда я говорю эту фразу, то подразумеваю очевидное: практически всегда приложение не может выполнять свои функции, не взаимодействуя с другими приложениями/сервисами. Но у нее есть и другой смысл, столь же справедливый: никакое приложение не могло бы приносить пользу, не имея доступа к своей среде в той или иной форме. Статическое приложение без возможностей настройки оказалось бы негибким и неуклюжим.

Приложения Spring Boot представляют разработчикам множество замечательных механизмов для динамического задания и изменения настроек приложений, даже непосредственно во время их работы. Эти механизмы используют интерфейс `Environment` фреймворка Spring для управления параметрами конфигурации, получаемыми из всех источников, включая следующие.

- Глобальные настройки утилит разработчика Spring Boot (devtools) в каталоге `$HOME/.config/spring-boot`, при запущенных devtools.
- Аннотации `@TestPropertySource` для тестов.
- Атрибут `properties` для тестов, доступный в `@SpringBootTest` различных тестовых аннотаций для проверки различных срезов приложений.
- Аргументы командной строки.
- Свойства из `SPRING_APPLICATION_JSON` (JSON, встраиваемый в переменную среды или системное свойство). (Приведенные источники свойств перечислены в порядке убывания приоритета: свойства из источников, расположенных выше в списке, превалируют над аналогичными свойствами из источников, расположенных ниже[1].)
- Параметры инициализации `ServletConfig`.
- Параметры инициализации `ServletContext`.

[1] Порядок старшинства PropertySources Spring Boot (https://oreil.ly/OrderPredSB).

- Атрибуты JNDI из `java:comp/env`.
- Системные свойства Java (`System.getProperties()`).
- Переменные среды операционной системы.
- Класс `RandomValuePropertySource`, возвращающий случайные значения для свойств, начинающихся с `random.*`.
- Связанные с конкретным профилем свойства приложения, расположенные вне упакованного JAR-файла (варианты `application-{profile}.properties` и YAML).
- Связанные с конкретным профилем свойства приложения, упакованные внутрь JAR-файла (варианты `application-{profile}.properties` и YAML).
- Свойства приложения, расположенные вне упакованного JAR-файла (варианты `application.properties` и YAML).
- Свойства приложения, упакованные внутрь JAR-файла (варианты `application.properties` и YAML).
- Аннотации `@PropertySource` для классов `@Configuration`. Обратите внимание на то, что подобные источники свойств не добавляются в `Environment` до обновления контекста приложения, что слишком поздно для настройки некоторых свойств, читаемых до начала обновления, например `logging.*` и `spring.main.*`.
- Свойства по умолчанию, задаваемые с помощью `SpringApplication.setDefaultProperties`.

Все это очень полезно, но для сценариев кода в этой главе я буду использовать лишь несколько источников:

- аргументы командной строки;
- переменные среды операционной системы;
- свойства приложения, упакованные внутрь JAR-файла (варианты `application.properties` и YAML).

Начнем со свойств, описываемых в файле `application.properties` приложения, и постепенно будем подниматься по иерархии.

@Value

Аннотация `@Value`, вероятно, — наиболее простой подход к включению параметров конфигурации в код. Построенная на основе сопоставления с образцом

и языка выражений Spring (Spring Expression Language, SpEL), она проста, но обладает большими возможностями.

Начнем с описания отдельного свойства в файле `application.properties` (рис. 5.1).



Рис. 5.1. Описание свойства greeting-name в файле application.properties

Чтобы продемонстрировать это свойство в действии, я создам в приложении дополнительный класс с аннотацией `@RestController`, чтобы приветствовать пользователей приложения и выполнять связанные с этим задачи (рис. 5.2).

```
@RestController
@RequestMapping("/greeting")
class GreetingController {
    @Value("${greeting-name: Mirage}")
    private String name;

    @GetMapping
    String getGreeting() {
        return name;
    }
}
```

Рис. 5.2. Класс @RestController, призванный приветствовать пользователей

Отметим, что аннотация `@Value` применяется к переменной-члену `name` и принимает один параметр `value` типа `String`. Я определяю значение с помощью SpEL, поместив имя переменной в качестве вычисляемого выражения между разделителями `${` и `}`. Еще отмечу, что SpEL позволяет указывать после двоеточия значение по умолчанию — в данном примере `Mirage` — для случаев, когда переменная не описана в объекте `Environment` приложения.

Как показано на рис. 5.3, при запуске приложения и выполнении запроса к конечной точке `/greeting` оно отвечает: `Dakota`, как и ожидалось.

```
mheckler-a01 :: ~/dev » http :8080/greeting
HTTP/1.1 200
Connection: keep-alive
Content-Length: 6
Content-Type: text/plain;charset=UTF-8
Date: Fri, 27 Nov 2020 15:24:32 GMT
Keep-Alive: timeout=60

Dakota
```

Рис. 5.3. Приложение приветствует пользователя заданным значением свойства

Чтобы убедиться, что значение по умолчанию работает, я закомментировал следующую строку в файле `application.properties` с помощью `#`, как показано далее, и перезапустил приложение:

```
#greeting-name=Dakota
```

Теперь при запросе к конечной точке `/greeting` мы получаем ответ, показанный на рис. 5.4. Поскольку `greeting-name` больше не описано в каких-либо источниках для `Environment` приложения, вступает в действие значение по умолчанию `Mirage`, как и ожидалось.

```
mheckler-a01 :: ~/dev » http :8080/greeting
HTTP/1.1 200
Connection: keep-alive
Content-Length: 7
Content-Type: text/plain;charset=UTF-8
Date: Fri, 27 Nov 2020 15:28:28 GMT
Keep-Alive: timeout=60

Mirage
```

Рис. 5.4. Приложение приветствует пользователя значением по умолчанию

Использование `@Value` с собственными свойствами дает еще одну удобную возможность: значение одного свойства можно получить из значения другого или создать на его основе.

Для демонстрации вложенных свойств нам понадобятся по крайней мере два свойства. Я создал второе свойство `greeting-coffee` в файле `application.properties` (рис. 5.5).

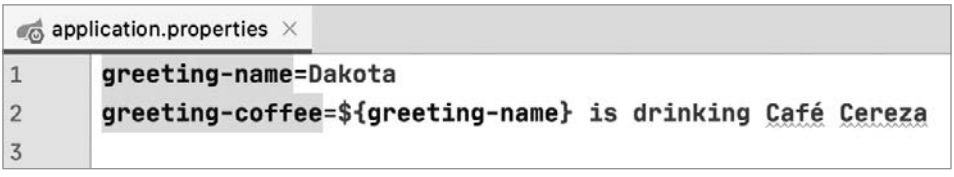


Рис. 5.5. Значение свойства, вставляемого в другое свойство

Далее добавил кое-какой код в класс `GreetingController` для «кофеизированного» приветствия и конечной точки, сделав запрос к которой мы увидим результат. Обратите внимание, что я указал также значение по умолчанию для `coffee` (рис. 5.6).

```
@RestController
@RequestMapping("/greeting")
class GreetingController {
    @Value("${greeting-name: Mirage}")
    private String name;

    @Value("${greeting-coffee: ${greeting-name} is drinking Café Ganador}")
    private String coffee;

    @GetMapping
    String getGreeting() {
        return name;
    }

    @GetMapping("/coffee")
    String getNameAndCoffee() {
        return coffee;
    }
}
```

Рис. 5.6. Добавляем «кофейное» приветствие в класс GreetingController

Чтобы убедиться, что результат соответствует ожиданиям, я перезапустил приложение и выполнил запрос к новой конечной точке `/greeting/coffee`, получив

показанный на рис. 5.7 результат. Обратите внимание: поскольку оба наших свойства описаны в файле `application.properties`, отображаемые значения соответствуют их описаниям.

```
mheckler-a01 :: ~/dev » http :8080/greeting/coffee
HTTP/1.1 200
Connection: keep-alive
Content-Length: 30
Content-Type: text/plain;charset=UTF-8
Date: Fri, 27 Nov 2020 15:36:51 GMT
Keep-Alive: timeout=60

Dakota is drinking Cafe Cereza
```

Рис. 5.7. Запрос к конечной точке /greeting/coffee

Как и у всего прочего в жизни вообще и разработке программного обеспечения в частности, у `@Value` есть ограничения. Поскольку мы задали значение по умолчанию для свойства `greeting-coffee`, то можем закомментировать его описание в файле `application.properties`, и аннотация `@Value` все равно корректно обработает значение по умолчанию свойства `greeting-coffee` с помощью переменной экземпляра `coffee` в классе `GreetingController`. Впрочем, если закомментировать описание как `greeting-name`, так и `greeting-coffee` в файле свойств, окажется, что ни в одном источнике `Environment` они фактически не описаны. Из-за этого при попытке приложения инициализировать компонент `GreetingController` с помощью ссылки на не описанное здесь свойство `greeting-name` внутри `greeting-coffee` будет выдана следующая ошибка:

```
org.springframework.beans.factory.BeanCreationException:
    Error creating bean with name 'greetingController':
        Injection of autowired dependencies failed; nested exception is
        java.lang.IllegalArgumentException:
            Could not resolve placeholder 'greeting-name' in value
            "greeting-coffee: ${greeting-name} is drinking Cafe Ganador"
```



Ради краткости и ясности полную трассу вызовов в стеке я не привожу.

Еще одно ограничение, относящееся к описанным в файле `application.properties` свойствам, используемым исключительно посредством `@Value`:

IDE не понимает, что они используются приложением, поскольку код на них ссылается только внутри ограниченных кавычками строковых переменных, а поэтому никакой прямой привязки к коду нет. Конечно, разработчик может визуально проверить правильность написания названий и применения свойств, но делать это придется исключительно вручную, из-за чего вероятность ошибок возрастает.

Как можно догадаться, гораздо лучший обходной вариант — типобезопасный механизм с возможностью проверки с помощью соответствующей утилиты.

@ConfigurationProperties

Команда создателей Spring, высоко ценя гибкость аннотации `@Value`, но и отдавая себе отчет в ее недостатках, создала аннотацию `@ConfigurationProperties`. С помощью `@ConfigurationProperties` разработчик может описывать свойства, группируя связанные между собой, и ссылаться на них либо использовать их типобезопасно и с возможностью проверки с помощью соответствующей утилиты.

Например, если в файле `application.properties` описано свойство, которое не используется в коде, разработчик увидит, что его название выделено, и поймет, что это свойство заведомо не используется. Аналогично, если свойство описано с типом `String`, но связано с переменной экземпляра другого типа, IDE укажет на рассогласование типов. Все это очень помогает выявлять простые, но часто встречающиеся ошибки.

Чтобы продемонстрировать использование `@ConfigurationProperties` на практике, я начну с описания POJO, инкапсулирующего нужные нам связанные между собой свойства, в данном случае вышеупомянутые `greeting-name` и `greeting-coffee`. Как показано в следующем коде, я создал для них класс `Greeting`:

```
class Greeting {
    private String name;
    private String coffee;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCoffee() {
        return coffee;
```

```
    }

    public void setCoffee(String coffee) {
        this.coffee = coffee;
    }
}
```

Чтобы зарегистрировать класс `Greeting` для управления свойствами конфигурации, я добавил аннотацию `@ConfigurationProperties`, показанную на рис. 5.8, и указал `prefix`, используемый для всех свойств из класса `Greeting`. Данная аннотация подготавливает класс к использованию только для свойств конфигурации. Кроме того, приложение должно знать, что свойства из снабженных подобной аннотацией классов необходимо включать в `Environment` приложения. Обратите внимание на выдаваемое полезное сообщение об ошибке.

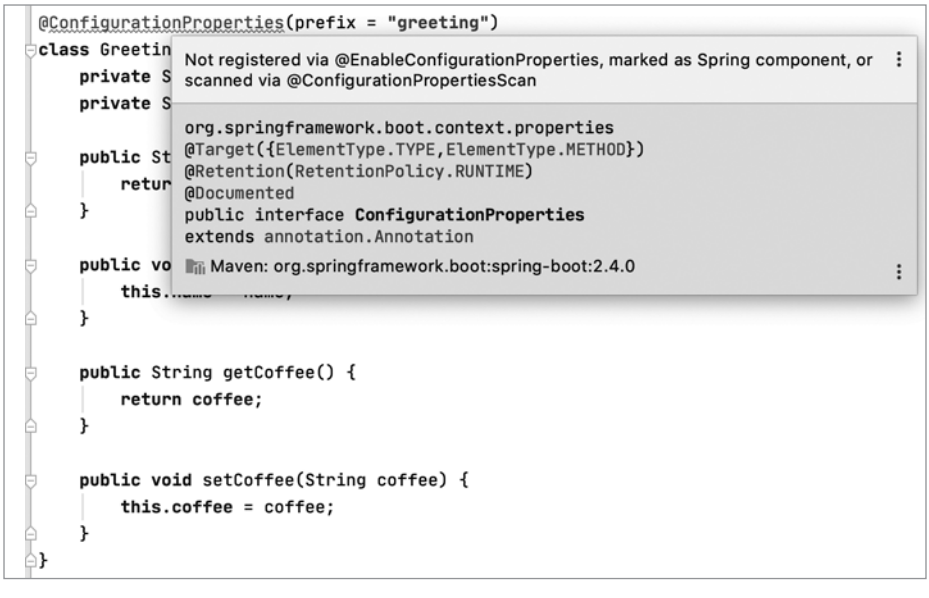


Рис. 5.8. Аннотация и сообщение об ошибке

Проще всего указать приложению на необходимость обработки классов, снабженных аннотацией `@ConfigurationProperties`, и добавления их свойств в объект `Environment` приложения. Это делается добавлением аннотации `@ConfigurationPropertiesScan` к основному классу приложения, как показано далее:

```
@SpringBootApplication
@ConfigurationPropertiesScan
```

```
public class SburRestDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(SburRestDemoApplication.class, args);
    }

}
```



Иногда не требуется, чтобы Boot искал снабженные аннотацией @ConfigurationProperties классы, в частности при необходимости подключения определенных снабженных аннотацией @ConfigurationProperties классов по условию или при создании собственных автоконфигураций. Во всех прочих случаях следует использовать @ConfigurationPropertiesScan для поиска и подключения классов с @ConfigurationProperties аналогично механизму поиска компонентов Spring Boot.

Чтобы сгенерировать метаданные с помощью процессора аннотаций и дать IDE возможность найти взаимосвязи между снабженными аннотацией `@ConfigurationProperties` классами и соответствующими свойствами, описанными в файле `application.properties`, я добавил в файл сборки `pom.xml` проекта следующую зависимость:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-configuration-processor</artifactId>
    <optional>true</optional>
</dependency>
```



Эту зависимость можно выбрать в Spring Initializr и добавить автоматически на этапе создания проекта.

После добавления зависимости процессора конфигурации в файл сборки необходимо обновить/повторно импортировать зависимости и собрать проект, чтобы их использовать. Для повторного импорта зависимостей я открыл меню Maven в IntelliJ и нажал кнопку Reimport слева вверху (рис. 5.9).



Если соответствующая возможность не отключена, IntelliJ отображает над измененным файлом pom.xml небольшую кнопку, позволяющую быстро произвести повторный импорт, не открывая меню Maven. Наложенную кнопку повторного импорта — маленькую букву m с круговыми стрелками внизу — можно видеть на рис. 5.9 слева над элементом <groupid> первой зависимости. По завершении повторного импорта она пропадает.

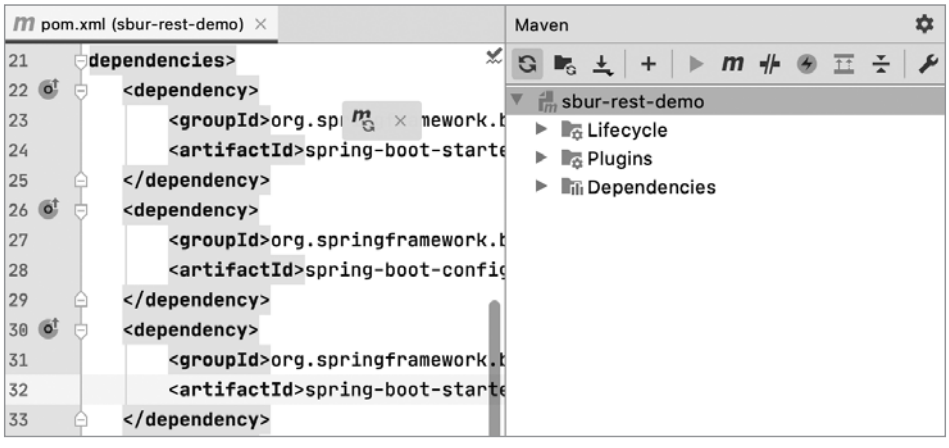


Рис. 5.9. Повторный импорт зависимостей проекта

После обновления зависимостей я произвел повторную сборку проекта из IDE для включения в него процессора конфигураций.

Теперь зададим значения для этих свойств. Возвратимся к файлу `application.properties`: когда я начал вводить название `greeting`, IDE, что очень удобно, отобразила соответствующие названия свойств (рис. 5.10).



Рис. 5.10. Полная поддержка IDE свойств для @ConfigurationProperties

Чтобы воспользоваться этими свойствами вместо тех, что мы применяли ранее, придется немного переделать код.

Можно полностью отказаться от имен собственных переменных-членов `name` и `coffee` класса `GreetingController` вместе с их аннотациями `@Value`, но вместо этого я создал переменную-член для компонента `Greeting`, который теперь отвечает за свойства `greeting.name` и `greeting.coffee`, и внедрил ее в `GreetingController`, как показано в следующем коде:

```
@RestController
@RequestMapping("/greeting")
class GreetingController {
    private final Greeting greeting;

    public GreetingController(Greeting greeting) {
        this.greeting = greeting;
    }

    @GetMapping
    String getGreeting() {
        return greeting.getName();
    }

    @GetMapping("/coffee")
    String getNameAndCoffee() {
        return greeting.getCoffee();
    }
}
```

Если запустить приложение и выполнить запросы к конечным точкам `greeting` и `greeting/coffee`, мы получим результаты, отображенные на рис. 5.11.

```
mheckler-a01 :: ~/dev » http :8080/greeting
HTTP/1.1 200
Connection: keep-alive
Content-Length: 6
Content-Type: text/plain;charset=UTF-8
Date: Fri, 27 Nov 2020 16:37:52 GMT
Keep-Alive: timeout=60

Dakota

mheckler-a01 :: ~/dev » http :8080/greeting/coffee
HTTP/1.1 200
Connection: keep-alive
Content-Length: 30
Content-Type: text/plain;charset=UTF-8
Date: Fri, 27 Nov 2020 16:37:57 GMT
Keep-Alive: timeout=60

Dakota is drinking Cafe Cereza
```

Рис. 5.11. Извлечение свойств Greeting

Свойства под управлением компонента с аннотацией `@ConfigurationProperties` по-прежнему получают значения от `Environment` и всех его потенциальных источников. Единственное существенное отличие от свойств на основе `@Value` — невозможность задавать значение по умолчанию для аннотируемой переменной экземпляра. Это не такая уж большая жертва, как кажется на первый взгляд, поскольку разумные значения по умолчанию для приложения обычно задаются в файле `application.properties`. При необходимости задать другие значения свойств, относящиеся к конкретным средам развертывания, их можно внести в `Environment` приложения через другие источники, например переменные среды или параметры командной строки. Короче говоря, `ConfigurationProperties` попросту обеспечивает соблюдение рекомендуемой практики задания значений свойств по умолчанию.

Возможные сторонние решения

Дальнейшее расширение и без того впечатляющей функциональности аннотации `ConfigurationProperties` — возможность оборачивать сторонние компоненты и включать их свойства в объект `Environment` приложения. Чтобы продемонстрировать это, я создал объект POJO для имитации включаемого в приложение компонента. Обратите внимание на то, что в типичных сценариях использования, когда эта возможность приносит наибольшую пользу, имеет смысл добавить в проект внешнюю зависимость и обратиться к документации проекта, чтобы найти класс для создания компонента Spring вместо создания его вручную, как сделал я.

В следующем листинге я создал имитацию стороннего компонента `Droid` с двумя свойствами, `id` и `description`, и соответствующими методами доступа и изменения:

```
class Droid {
    private String id, description;

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getDescription() {
        return description;
    }
```

```
    public void setDescription(String description) {
        this.description = description;
    }
}
```

Следующий шаг укладывается в общую картину аналогично настоящему стороннему компоненту — создание экземпляра компонента как компонента Spring. Компоненты Spring можно получать из описанных объектов POJO несколькими способами, но наиболее подходящий для нашего конкретного сценария — создать внутри снабженного аннотацией `@Configuration` класса метод, имеющий аннотацию `@Bean`, либо непосредственно, либо с помощью метааннотации.

Одна из метааннотаций, включающих в свое определение `@Configuration`, — `@SpringBootApplication`, указанная нами для класса `main` приложения. Именно поэтому разработчики нередко помещают туда методы создания компонентов.



В IntelliJ и большинстве других IDE и продвинутых текстовых редакторов с хорошей поддержкой Spring можно разворачивать метааннотации Spring, изучая вложенные внутрь них аннотации. В IntelliJ развернуть аннотацию можно, нажав Cmd и щелкнув левой кнопкой мыши (на MacOS) (в Windows — Ctrl+Щелчок левой кнопкой мыши). Аннотация @SpringBootApplication включает аннотацию @SpringBootConfiguration, в свою очередь, включающую @Configuration, так что лишь два шага отделяют нас от Кевина Бейкона[1].

В следующем листинге показан метод создания компонента, необходимая аннотация `@ConfigurationProperties` и параметр `prefix`, указывающий, что нужно включить свойства `Droid` в `Environment`, на верхнем уровне группировки свойств `droid`:

```
@SpringBootApplication
@ConfigurationPropertiesScan
public class SburRestDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(SburRestDemoApplication.class, args);
    }

    @Bean
    @ConfigurationProperties(prefix = "droid")
    Droid createDroid() {
```

[1] Автор намекает на шуточную игру, в которой участники должны не более чем за шесть переходов найти связь загаданного голливудского актера и Кевина Бейкона. — *Примеч. пер.*

```
        return new Droid();
    }
}
```

Как и ранее, необходимо собрать проект заново, чтобы процессор конфигурации смог обнаружить свойства, предоставляемые этим новым источником свойств конфигурации. После сборки мы возвращаемся к файлу `application.properties` и видим, что оба свойства `droid` теперь отображаются полностью, с информацией о типе (рис. 5.12).

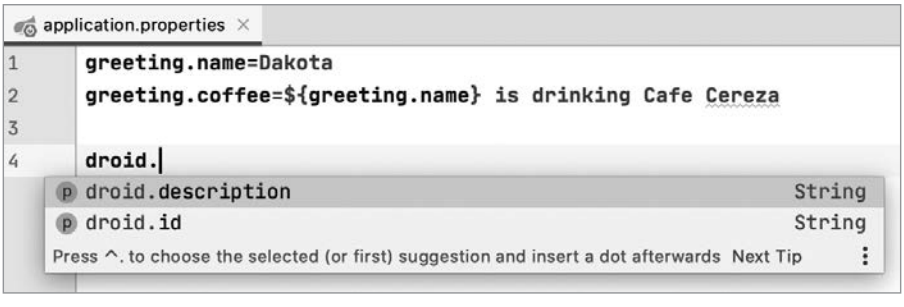


Рис. 5.12. В файле application.properties теперь видны свойства droid и информация о типе

Присваиваем `droid.id` и `droid.description` какие-то значения в качестве значений по умолчанию (рис. 5.13). Рекомендуется делать это для всех свойств `Environment`, даже полученных со стороны.

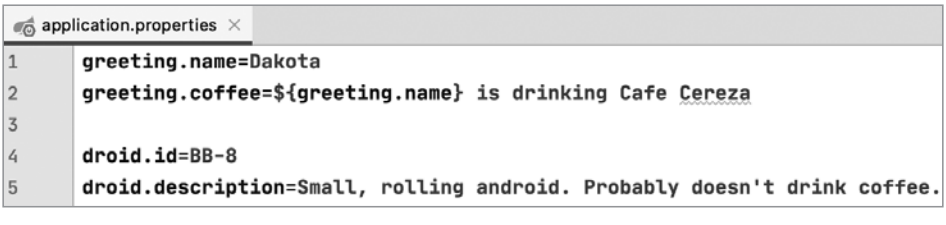


Рис. 5.13. Свойства droid со значениями по умолчанию, присвоенными в application.properties

Чтобы проверить, что все касающееся свойств `Droid` работает как надо, я создал очень простой класс с аннотацией `@RestController`, включающий один метод `@GetMapping`:

```
@RestController
@RequestMapping("/droid")
class DroidController {
```

```
    private final Droid droid;

    public DroidController(Droid droid) {
        this.droid = droid;
    }

    @GetMapping
    Droid getDroid() {
        return droid;
    }
}
```

После сборки и запуска проекта выполняем запрос к новой конечной точке `/droid` и убеждаемся, что приложение реагирует должным образом (рис. 5.14).

```
mheckler-a01 :: ~/dev » http :8080/droid
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/json
Date: Fri, 27 Nov 2020 17:29:29 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "description": "Small, rolling android. Probably doesn't drink coffee.",
    "id": "BB-8"
}
```

Рис. 5.14. Запрос к конечной точке /droid на получение свойств из нее

Отчет об автоконфигурации

Как говорилось ранее, Spring Boot посредством автоконфигурации выполняет вместо разработчика *массу* действий: подготавливает компоненты для реализации функциональности приложения, ориентируясь на выбранные возможности, зависимости и имеющийся код. Также ранее упоминалась возможность переопределения любой части автоконфигурации, если это нужно для реализации функциональности так, как требует конкретный сценарий использования. Но откуда мы знаем, какие компоненты созданы, а какие — нет и какие условия привели к такому исходу?

Благодаря гибкости JVM сгенерировать отчет об автоконфигурации очень просто. Для этого достаточно воспользоваться флагом `debug` одним из следующих способов.

- Выполнить JAR-файл приложения с указанием опции `--debug` — `java -jar bootapplication.jar –debug`.
- Выполнить JAR-файл приложения с указанием соответствующего параметра JVM — `java -Ddebug=true -jar bootapplication.jar`.
- Добавить `debug=true` в файл `application.properties` приложения.
- Выполнить команду `export DEBUG=true` в командной оболочке (Linux или Mac) или добавить ее в среду Windows, после чего выполнить `java -jar / bootapplication.jar`.



Все обсуждавшиеся ранее способы добавления утвердительного значения для debug в Environment приложения дадут одинаковые результаты. Приведенные здесь способы просто используются чаще всего.

Раздел отчета об автоконфигурации, перечисляющий позитивные соответствия условиям, оцененным как `True` и приведшим к выполнению соответствующего действия, озаглавлен *Positive matches*. Я скопировал сюда заголовок этого раздела вместе с одним примером позитивного соответствия и вытекающим из него действием автоконфигурации:

```
============================
CONDITIONS EVALUATION REPORT
============================
Positive matches:
-----------------
   DataSourceAutoConfiguration matched:
      - @ConditionalOnClass found required classes 'javax.sql.DataSource',
      'org.springframework.jdbc.datasource.embedded.EmbeddedDatabaseType'
      (OnClassCondition)
```

Это конкретное соответствие условию демонстрирует, что должно произойти, хотя никогда не помешает проверить следующее:

- JPA и H2 включены в число зависимостей приложения;
- JPA работает с SQL-источниками данных;
- H2 — встраиваемая база данных;
- были найдены классы, поддерживающие встраиваемые SQL-источники данных.

В результате вызывается `DataSourceAutoConfiguration`.

Аналогично, в разделе *Negative matches* («Негативные соответствия условиям») приводятся действия, которые автоконфигурация Spring Boot не предприняла, и указано почему:

```
Negative matches:
-----------------
   ActiveMQAutoConfiguration:
      Did not match:
         - @ConditionalOnClass did not find required class
          'javax.jms.ConnectionFactory' (OnClassCondition)
```

В данном случае `ActiveMQAutoConfiguration` не выполнялась, поскольку при запуске приложение не нашло класс JMS `ConnectionFactory`.

Еще один полезный кусочек — раздел *Unconditional classes* — перечень классов, создаваемых без проверки каких-либо условий. Далее приведен один, особенно интересный с учетом предыдущего раздела:

```
Unconditional classes:
----------------------
    org.springframework.boot.autoconfigure.context
     .ConfigurationPropertiesAutoConfiguration
```

Как видите, экземпляр `ConfigurationPropertiesAutoConfiguration` создается всегда и предназначен для управления всеми объектами `ConfigurationProperties`, которые были созданы в приложении Spring Boot и на которые ссылается его код. Это неотъемлемая часть любого приложения Spring Boot.

Actuator (актуатор)

Актуатор (сущ.) — исполнительный элемент, механическое устройство для перемещения чего-либо или управления чем-либо.

Изначальная версия Spring Boot Actuator стала общедоступной версией (General Availability, GA) в 2014 году и была высоко оценена, поскольку давала возможность глубже анализировать производственные приложения Spring Boot. Позволяя выполнять мониторинг и управлять работающими приложениями через конечные точки HTTP или расширения Java для управления (Java Management Extensions, JMX), Actuator охватывал и открывал для пользователей все возможности Spring Boot уровня продакшена.

Полностью переработанный для версии 2.0 Spring Boot модуль Actuator сейчас представляет данные многих ведущих систем мониторинга через единообраз-

ный интерфейс с помощью инструментальной библиотеки Micrometer подобно тому, как SLF4J работает с различными механизмами ведения журнала. Это значительно расширяет спектр того, что можно интегрировать, отслеживать и отображать с помощью Actuator в любом приложении Spring Boot.

Чтобы воспользоваться Actuator, я добавил еще одну зависимость в раздел зависимостей файла `pom.xml` текущего проекта. Как показано в следующем фрагменте кода, зависимость `spring-boot-starter-actuator` предоставляет все необходимые возможности, включая для этого сам модуль Actuator, библиотеку Micrometer и средства автоконфигурации, что позволяет ей встраиваться в приложения Spring Boot практически без усилий:

```
<dependencies>
    ... (для краткости прочие зависимости опущены)
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
</dependencies>
```

Обновив/импортировав зависимости еще раз, я перезапустил приложение. Когда приложение запущено, обратившись к основной конечной точке Actuator, можно посмотреть, какую информацию он предоставляет по умолчанию. И вновь я для этого воспользовался HTTPie (рис. 5.15).



Вся информация актуатора сгруппирована по умолчанию в одной конечной точке /actuator, но такое поведение можно менять с помощью настроек.

На первый взгляд не так уж много информации, чтобы оправдать все овации и толпы сторонников модуля Actuator. Но подобная лаконичность — умышленная.

Actuator имеет доступ к большому объему информации о работающем приложении и может предоставить ее пользователям. Информации, очень полезной для разработчиков, эксплуатационного персонала, но также для нечистоплотных людей, угрожающих безопасности приложения. Следуя политике «безопасность по умолчанию» Spring Security, автоконфигурация модуля Actuator предоставляет пользователям доступ к очень немногим ответам из разделов `health` и `info` — на самом деле по умолчанию из раздела `info` вообще ничего не доступно. Так что изначально можно видеть практически только общее состояние приложения.

```
mheckler-a01 :: ~/dev » http :8080/actuator
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/vnd.spring-boot.actuator.v3+json
Date: Fri, 27 Nov 2020 17:34:29 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "_links": {
        "health": {
            "href": "http://localhost:8080/actuator/health",
            "templated": false
        },
        "health-path": {
            "href": "http://localhost:8080/actuator/health/{*path}",
            "templated": true
        },
        "info": {
            "href": "http://localhost:8080/actuator/info",
            "templated": false
        },
        "self": {
            "href": "http://localhost:8080/actuator",
            "templated": false
        }
    }
}
```

Рис. 5.15. Обращение к конечной точке Actuator, конфигурация по умолчанию

Как и для большинства других возможностей Spring, можно создавать весьма хитроумные механизмы для управления доступом к различным потокам данных Actuator. Существуют также быстрые, согласованные и простые механизмы, которые мы сейчас и рассмотрим.

Можно легко настраивать Actuator, задавая с помощью свойств множества включаемых/исключаемых конечных точек. Ради простоты мы будем указывать включаемые, для чего добавим в файл `application.properties` следующее:

```
management.endpoints.web.exposure.include=env, info, health
```

В этом примере я указал приложению и модулю Actuator открыть для доступа только конечные точки `/actuator/env`, `/actuator/info` и `/actuator/health`, а также все их дочерние конечные точки.

Рисунок 5.16 демонстрирует, что результат, полученный после перезапуска приложения и выполнения запроса к конечной точке `/actuator`, соответствует нашим ожиданиям.

```
mheckler-a01 :: ~/dev » http :8080/actuator
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/vnd.spring-boot.actuator.v3+json
Date: Fri, 27 Nov 2020 17:38:30 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "_links": {
        "env": {
            "href": "http://localhost:8080/actuator/env",
            "templated": false
        },
        "env-toMatch": {
            "href": "http://localhost:8080/actuator/env/{toMatch}",
            "templated": true
        },
        "health": {
            "href": "http://localhost:8080/actuator/health",
            "templated": false
        },
        "health-path": {
            "href": "http://localhost:8080/actuator/health/{*path}",
            "templated": true
        },
        "info": {
            "href": "http://localhost:8080/actuator/info",
            "templated": false
        },
        "self": {
            "href": "http://localhost:8080/actuator",
            "templated": false
        }
    }
}
```

Рис. 5.16. Обращение к Actuator после указания включаемых конечных точек

Чтобы продемонстрировать все возможности Actuator по умолчанию, можно пойти еще дальше и полностью отключить безопасность *только для целей демон-*

страции. Для этого укажите подстановочный знак в вышеупомянутой настройке из файла `application.properties`:

```
management.endpoints.web.exposure.include=*
```



Еще раз подчеркнем этот важнейший момент: механизмы безопасности для конфиденциальных данных следует отключать лишь для демонстрации или проверки. **Никогда не отключайте механизмы безопасности для приложений в продакшене.**

С целью проверки при запуске приложения Actuator скрупулезно выводит количество конечных точек, открываемых в текущий момент, а также корневой путь для доступа к ним, как показано в приведенном далее фрагменте отчета, выводимого при запуске. Эта информация служит удобным напоминанием или предостережением, позволяющим перед развертыванием приложения быстро визуально проверить, что открыто не больше конечных точек, чем нужно:

```
INFO 22115 --- [           main] o.s.b.a.e.web.EndpointLinksResolver      :
    Exposing 13 endpoint(s) beneath base path '/actuator'
```

Чтобы просмотреть полный список всех доступных через Actuator конечных точек, выполните запрос к указанному корневому пути Actuator:

```
mheckler-a01 :: ~/dev " http :8080/actuator
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/vnd.spring-boot.actuator.v3+json
Date: Fri, 27 Nov 2020 17:43:27 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked
{
    "_links": {
        "beans": {
            "href": "http://localhost:8080/actuator/beans",
            "templated": false
        },
        "caches": {
            "href": "http://localhost:8080/actuator/caches",
            "templated": false
        },
        "caches-cache": {
            "href": "http://localhost:8080/actuator/caches/{cache}",
            "templated": true
        },
        "conditions": {
            "href": "http://localhost:8080/actuator/conditions",
```

```
    "templated": false
},
"configprops": {
    "href": "http://localhost:8080/actuator/configprops",
    "templated": false
},
"env": {
    "href": "http://localhost:8080/actuator/env",
    "templated": false
},
"env-toMatch": {
    "href": "http://localhost:8080/actuator/env/{toMatch}",
    "templated": true
},
"health": {
    "href": "http://localhost:8080/actuator/health",
    "templated": false
},
"health-path": {
    "href": "http://localhost:8080/actuator/health/{*path}",
    "templated": true
},
"heapdump": {
    "href": "http://localhost:8080/actuator/heapdump",
    "templated": false
},
"info": {
    "href": "http://localhost:8080/actuator/info",
    "templated": false
},
"loggers": {
    "href": "http://localhost:8080/actuator/loggers",
    "templated": false
},
"loggers-name": {
    "href": "http://localhost:8080/actuator/loggers/{name}",
    "templated": true
},
"mappings": {
    "href": "http://localhost:8080/actuator/mappings",
    "templated": false
},
"metrics": {
    "href": "http://localhost:8080/actuator/metrics",
    "templated": false
},
"metrics-requiredMetricName": {
    "href": "http://localhost:8080/actuator/metrics/{requiredMetricName}",
    "templated": true
},
```

```
        "scheduledtasks": {
            "href": "http://localhost:8080/actuator/scheduledtasks",
            "templated": false
        },
        "self": {
            "href": "http://localhost:8080/actuator",
            "templated": false
        },
        "threaddump": {
            "href": "http://localhost:8080/actuator/threaddump",
            "templated": false
        }
    }
}
```

Список конечных точек Actuator хорошо демонстрирует спектр собранной и доступной для изучения информации, но и добросовестным пользователям, и злоумышленникам особенно интересны следующие:

- `/actuator/beans` — все создаваемые приложением компоненты Spring;
- `/actuator/conditions` — выполняющиеся и невыполняющиеся условия создания компонентов Spring, аналогично обсуждавшемуся ранее отчету о проверенных условиях;
- `/actuator/configprops` — все доступные приложению свойства `Environment`;
- `/actuator/env` — множество аспектов среды работы приложения, особенно удобно здесь смотреть источники отдельных значений `configprop`;
- `/actuator/health` — информация о состоянии приложения (основная или расширенная в зависимости от настроек);
- `/actuator/heapdump` — инициирует дамп «кучи» для целей отладки и/или анализа;
- `/actuator/loggers` — уровни журналирования для отдельных компонентов;
- `/actuator/mappings` — все сопоставления конечных точек и сопутствующая информация;
- `/actuator/metrics` — фиксируемые приложением показатели;
- `/actuator/threaddump` — инициирует дамп потоков выполнения для целей отладки и/или анализа.

Эти и все прочие предварительно настроенные конечные точки Actuator всегда под рукой, и при необходимости обращаться к ним для получения информации очень просто. Если говорить о среде приложения, то даже в числе этих конечных точек есть «первые среди равных».

Открываем доступ к Actuator

Как уже упоминалось, уровень безопасности Actuator по умолчанию состоит в открытии для пользователей лишь очень ограниченной информации из разделов `health` и `info`. Фактически конечная точка `/actuator/health` предоставляет по умолчанию лишь очень краткие сведения о состоянии приложения в стиле «Работает»/«Не работает».

Для большинства приложений, впрочем, существуют зависимости, для которых Actuator отслеживает дополнительную информацию о состоянии, просто не открывает к ней доступ, если не получил на это разрешения. Чтобы показать расширенную информацию о состоянии приложения для заранее настроенных зависимостей, я добавил в файл `application.properties` следующее свойство:

```
management.endpoint.health.show-details=always
```



Существует три возможных значения свойства-индикатора состояния приложения show-details: never (по умолчанию), when_authorized и always. Для этого примера я выбрал значение always, просто чтобы продемонстрировать все возможности, но для любых приложений в продакшене следует выбирать варианты never или when_authorized, чтобы ограничить видимость расширенной информации о работоспособности приложения.

После перезапуска приложения при обращении к конечной точке `/actuator/health` к общей сводке его состояния добавляется информация о состоянии его основных компонентов (рис. 5.17).

Лучше учитываем среду приложения с помощью Actuator

Одна из вечных болезней разработчиков, и вашего покорного слуги в том числе, — они всегда считают, что знают все о текущей среде или состоянии приложения, когда его поведение не соответствует ожидаемому. Ничего удивительного, особенно когда речь идет о написанном собственноручно коде. Относительно быстрый, но от этого ничуть не менее полезный первый шаг — *проверить все допущения*. Вы знаете, что это за ценность? Или просто абсолютно уверены, что знаете?

Проверяли ли вы?

```
mheckler-a01 :: ~/dev » http :8080/actuator/health
HTTP/1.1 200
Connection: keep-alive
Content-Type: application/vnd.spring-boot.actuator.v3+json
Date: Fri, 27 Nov 2020 17:47:32 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "components": {
        "db": {
            "details": {
                "database": "H2",
                "validationQuery": "isValid()"
            },
            "status": "UP"
        },
        "diskSpace": {
            "details": {
                "exists": true,
                "free": 133346631680,
                "threshold": 10485760,
                "total": 499963174912
            },
            "status": "UP"
        },
        "ping": {
            "status": "UP"
        }
    },
    "status": "UP"
}
```

Рис. 5.17. Расширенная информация о состоянии приложения

Это совершенно необходимая отправная точка, особенно для кода, поведение которого зависит от входных данных. Actuator помогает упростить эту задачу. При запросе к конечной точке `/actuator/env` приложения возвращается вся информация о среде. Далее приведена часть результата такого запроса, отображающая лишь текущий набор свойств приложения:

```
{
    "name": "Config resource 'classpath:/application.properties' via location
     'optional:classpath:/'",
    "properties": {
        "droid.description": {
```

```
            "origin": "class path resource [application.properties] - 5:19",
            "value": "Small, rolling android. Probably doesn't drink coffee."
        },
        "droid.id": {
            "origin": "class path resource [application.properties] - 4:10",
            "value": "BB-8"
        },
        "greeting.coffee": {
            "origin": "class path resource [application.properties] - 2:17",
            "value": "Dakota is drinking Cafe Cereza"
        },
        "greeting.name": {
            "origin": "class path resource [application.properties] - 1:15",
            "value": "Dakota"
        },
        "management.endpoint.health.show-details": {
            "origin": "class path resource [application.properties] - 8:41",
            "value": "always"
        },
        "management.endpoints.web.exposure.include": {
            "origin": "class path resource [application.properties] - 7:43",
            "value": "*"
        }
    }
}
```

Actuator показывает не только текущие значения всех описанных свойств, но также их источники вплоть до номера строки и столбца, в которых описано значение. Но что произойдет, если одно или несколько из этих значений будут переопределены в другом источнике, например во внешней переменной среды или в аргументе командной строки при запуске приложения?

Для демонстрации типичного сценария, реализуемого при продакшене, я выполнил в командной строке каталога приложения команду `mvn clean package`, после чего запустил приложение с помощью команды

```
java -jar target/sbur-rest-demo-0.0.1-SNAPSHOT.jar --greeting.name=Sertanejo
```

Выполняя новый запрос к конечной точке `/actuator/env`, видим, что появился новый раздел для аргументов командной строки, включающий одну запись для `greeting.name`:

```
{
    "name": "commandLineArgs",
    "properties": {
        "greeting.name": {
            "value": "Sertanejo"
        }
    }
}
```

В соответствии с приведенной ранее информацией о старшинстве входных данных `Environment` аргументы командной строки должны обладать приоритетом перед набором значений из файла `application.properties`. Запрос к конечной точке `/greeting` возвращает Sertanejo, как и ожидалось, и ответ на запрос к конечной точке `/greeting/coffee` также содержит значение, переопределенное с помощью SpEL: `Sertanejo is drinking Cafe Cereza`.

Поиск ошибочного поведения, управляемого данными, намного упростился благодаря Spring Boot Actuator.

Регулировка уровня журналирования с помощью Actuator

Как и во многих других вопросах, связанных с разработкой и развертыванием программного обеспечения, выбор уровня журналирования для приложений в продакшене означает определенный компромисс. Чем больше результатов журналирования, тем больше операций системного уровня и тем больше нужно места для хранения. Кроме того, собираются как относящиеся, так и не относящиеся к делу данные. В результате найти причины трудноуловимых проблем становится труднее.

Actuator в рамках миссии по обеспечению функциональных возможностей Boot, готовых к продакшену, решает и эту проблему, позволяя разработчикам задавать общий уровень журналирования (например, "INFO") для всех компонентов и временно менять его при возникновении критической проблемы... и все это прямо в находящемся в продакшене приложении Spring Boot. Actuator помогает задавать и переопределять уровни журналирования с помощью простого запроса `POST` к соответствующей конечной точке. Например, на рис. 5.18 показан уровень журналирования по умолчанию для `org.springframework.data.web`.

Особенно любопытно то, что, поскольку уровень журналирования для этого компонента не был задан в настройках, используется уровень по умолчанию — `"INFO"`. И опять Spring Boot обеспечивает разумное значение по умолчанию, если конкретное значение не задано.

Если вы получили оповещение о проблеме с запущенным приложением и хотели бы повысить уровень журналирования для конкретного компонента с целью диагностики и решения проблемы, достаточно с помощью `POST` отправить новое значение параметра `configuredLevel` в формате JSON в конечную точку `/actuator/loggers`, как показано далее:

```
echo '{"configuredLevel": "TRACE"}'
  | http :8080/actuator/loggers/org.springframework.data.web
```

```
mheckler-a01 :: ~/dev » http :8080/actuator/loggers/org.springframework.data.web

HTTP/1.1 200
Connection: keep-alive
Content-Disposition: inline;filename=f.txt
Content-Type: application/vnd.spring-boot.actuator.v3+json
Date: Fri, 27 Nov 2020 18:01:15 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "configuredLevel": null,
    "effectiveLevel": "INFO"
}
```

Рис. 5.18. Уровень журналирования по умолчанию для org.springframework.data.web

Запрос уровня журналирования для `org.springframework.data.web` теперь подтверждает, что он установлен в `"TRACE"` и обеспечит подробную диагностическую информацию о приложении (рис. 5.19).

```
mheckler-a01 :: ~/dev » http :8080/actuator/loggers/org.springframework.data.web

HTTP/1.1 200
Connection: keep-alive
Content-Disposition: inline;filename=f.txt
Content-Type: application/vnd.spring-boot.actuator.v3+json
Date: Fri, 27 Nov 2020 18:05:34 GMT
Keep-Alive: timeout=60
Transfer-Encoding: chunked

{
    "configuredLevel": "TRACE",
    "effectiveLevel": "TRACE"
}
```

Рис. 5.19. Новый уровень журналирования "TRACE" для org.springframework.data.web



"TRACE" может помочь в поиске причин трудноуловимой проблемы, но это довольно тяжелый уровень журналирования, фиксирующий даже более подробную информацию, чем "DEBUG". В продакшене он может обеспечить необходимую информацию, но не следует забывать о нагрузке на приложение.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Полный код для этой главы находится в ветке chapter5end в репозитории кода.

Резюме

Для разработчика критически важны удобные утилиты для выяснения, идентификации и изоляции видов поведения, проявляющихся в ходе работы приложения. По мере превращения приложений в более динамические и распределенные разработчикам часто приходится делать следующее:

- задавать и менять настройки приложений динамически;
- определять или подтверждать текущие настройки и их источники;
- осуществлять контроль и мониторинг среды выполнения приложения и индикаторов его состояния;
- временно корректировать уровни журналирования работающих в текущий момент приложений для выявления первопричин различных проблем.

В этой главе показано, как использовать встроенные возможности Spring Boot по заданию настроек, его Autoconfiguration Report, а также Spring Boot Actuator для гибкого динамического создания, определения и модификации параметров среды приложения.

В следующей главе мы подробнее обсудим данные: как описать для них хранилище и методы извлечения на основе различных промышленных стандартов и ведущих движков баз данных, а также проектов и возможностей Spring Data, позволяющих применять их наиболее простыми и полнофункциональными способами.

ГЛАВА 6

Займемся данными по-настоящему

Данные — непростая и очень обширная тема: их структура и взаимосвязи с другими данными; варианты обработки, хранения и извлечения; различные стандарты; поставщики и движки баз данных и многое другое. Возможно, данные — самый сложный аспект разработки, с которым разработчики сталкиваются и в самом начале карьеры, и позднее при изучении любого нового набора инструментов.

Дело в том, что без данных в какой-либо форме практически все приложения теряют смысл. Очень немногие приложения приносят хоть какую-то пользу без хранения, извлечения или установления взаимосвязей данных.

Будучи основой практически всего ценного, что приносят приложения, *данные* становятся причиной множества инноваций, вносимых поставщиками баз данных и платформ. Но во многих случаях сложность никуда не девается — в конце концов, это весьма глубокая и обширная тема.

И тут на сцену выходит Spring Data (https://spring.io/projects/spring-data). Заявленная цель Spring Data: «Обеспечить привычную и единообразную модель программирования на основе Spring для доступа к данным с сохранением при этом всех характеристик используемого хранилища данных». Вне зависимости от движка базы данных или платформы, цель Spring Data — сделать для разработчика доступ к данным наиболее простым и полнофункциональным.

В этой главе описывается хранение и извлечение данных на основе различных промышленных стандартов и ведущих баз данных, а также проекты и возможности Spring Data, позволяющие использовать их наиболее простым и полнофункциональным способом — посредством Spring Boot.

Описание сущностей

Практически во всех случаях, когда мы имеем дело с данными, применяется какая-либо сущность предметной области: счет-фактура ли это, автомобиль или что-то иное, — редко используют данные в виде набора несвязанных свойств. То, что мы считаем полезными данными, неизбежно представляет собой взаимосвязанные совокупности элементов, которые все вместе составляют нечто осмысленное. Автомобиль — в виде данных или реальный — полезен лишь тогда, когда обладает всеми нужными уникальными атрибутами.

Spring Data предоставляет несколько различных механизмов и вариантов доступа к данным для приложений Spring Boot на самых разных уровнях абстракции. Вне зависимости от уровня абстракции, выбранного разработчиком для конкретного сценария использования, первый шаг — описание классов предметной области, которые будут использоваться для обработки соответствующих данных.

И хотя полноценное обсуждение предметно-ориентированного проектирования (Domain-Driven Design, DDD) выходит за рамки темы книги, мы воспользуемся некоторыми его идеями в качестве основы для описания нужных классов предметной области в примерах приложений в этой и следующих главах. Полное описание DDD можно найти, например, в посвященном этой теме фундаментальном труде Эрика Эванса (Eric Evans) *Domain-Driven Design: Tackling Complexity in the Heart of Software* (https://oreil.ly/DomainDrivDes)[1].

Если объяснять в общих словах, *класс предметной области* (domain class) инкапсулирует основную сущность предметной области, обладающую значимостью независимо от других данных. Это не означает, что она не связана с другими сущностями предметной области, а говорит лишь о том, что она самодостаточна и имеет смысл как отдельная единица даже без привязки к другим сущностям.

Для создания класса предметной области в Spring с помощью Java можно создать класс с переменными-членами, соответствующими конструкторами, методами доступа и изменения, методами `equals()`/`hashCode()`/`toString()` и многими другими. Можно также воспользоваться Lombok вместе с Java или классами данных в Kotlin, чтобы создавать классы предметной области для представления, хранения и извлечения данных. Все вышеупомянутое я проделаю в этой главе, чтобы продемонстрировать, насколько просто работать с предметной областью с помощью Spring Boot и Spring Data. Чем больше вариантов, тем лучше.

---

[1] *Эванс Э.* Предметно-ориентированное проектирование (DDD): структуризация сложных программных систем. — М.: Вильямс, 2010.

Определив для примеров из этой главы класс предметной области, я выберу базу данных и уровень абстракции в соответствии с целями использования данных и предоставляемых API для базы данных. В экосистеме Spring это сводится обычно к одному из двух вариантов (с небольшими вариациями): шаблоны или репозитории.

Поддержка шаблонов

Spring Data, чтобы предоставить набор связных абстракций довольно высокого уровня, описывает интерфейсы типа `Operations` для большинства различных источников данных. Эти интерфейсы `Operations`, например `MongoOperations`, `RedisOperations` и `CassandraOperations`, задают базовый набор операций, которые можно применять напрямую для достижения максимальной гибкости или создавать на их основе абстракции более высокого уровня. Классы `Template` обеспечивают прямые реализации интерфейсов `Operations`.

Шаблоны можно считать своего рода интерфейсами поставщиков сервисов (Service Provider Interface, SPI). Они обладают очень большими возможностями и допускают непосредственное использование, но каждый раз требуют повтора множества шагов для реализации наиболее распространенных сценариев использования, встречающихся разработчикам. Для сценариев, в которых доступ к данным имеет типичные закономерности, возможно, больше подойдут репозитории. А лучше всего то, что в основе репозиториев лежат шаблоны, так что вы ничего не потеряете от перехода к абстракции более высокого уровня.

Поддержка репозиториев

В Spring Data определен интерфейс `Repository`, базовый для всех остальных типов интерфейсов репозиториев Spring Data, например `JPARepository` и `MongoRepository`, предоставляющих ориентированные на JPA и Mongo возможности соответственно, и различные многофункциональные интерфейсы наподобие `CrudRepository`, `ReactiveCrudRepository` и `PagingAndSortingRepository`. В этих многофункциональных интерфейсах описано много полезных высокоуровневых операций, например `findAll()`, `findById()`, `count()`, `delete()`, `deleteAll()` и др.

Репозитории определены как для блокирующих, так и для неблокирующих взаимодействий. Кроме того, репозитории Spring Data поддерживают создание запросов в стиле «соглашения важнее конфигурации» и даже точных операторов

запросов. Использование репозиториев Spring Data со Spring Boot упрощает реализацию сложных взаимодействий с базами данных практически до уровня тривиальной задачи.

Я продемонстрирую все эти возможности далее в книге. В этой главе собираюсь охватить ключевые элементы нескольких вариантов баз данных путем сочетания различных деталей реализации: Lombok, Kotlin и др. Таким образом я создам широкий и прочный фундамент для последующих глав.

@Before

Как бы сильно я ни любил кофе и ни полагался на него при разработке приложений, изучать рассматриваемые в остальной части книги идеи будет удобнее на более универсальной предметной области. Я не только разработчик программного обеспечения, но и пилот, поэтому полагаю, что все более сложный и управляемый данными мир авиации в избытке предоставит интересные ситуации и захватывающие данные для исследования, когда мы будем углубляться в механизмы Spring Boot в многочисленных сценариях использования.

Чтобы работать с данными, прежде всего нужны *данные*. Я разработал небольшой реализующий REST веб-сервис `PlaneFinder` (доступен в прилагаемом к книге репозитории кода) в качестве API-шлюза для запросов о воздушных судах, находящихся в пределах досягаемости маленького устройства на моем столе, и их местоположении. Это устройство получает данные автоматического наблюдения — передачи данных (Automatic Dependent Surveillance Broadcast, ADS-B) от самолетов, находящихся в пределах определенного расстояния, и передает их онлайн-сервису PlaneFinder.net (https://planefinder.net). Также оно предоставляет API HTTP, данные от которого мой шлюз потребляет, упрощает и открывает для использования расположенными далее по конвейеру сервисами наподобие представленных в этой главе.

Далее вас ожидает больше подробностей, а пока что создадим несколько сервисов, подключаемых к базе данных.

Создание с помощью Redis сервиса на основе шаблона

Redis — база данных, используемая обычно в качестве хранилища данных в оперативной памяти для обмена информацией о состоянии между различными

экземплярами сервиса, кэширования и в качестве брокера сообщений между сервисами. Подобно многим основным базам данных, Redis способна на большее, но в этой главе мы воспользуемся ею лишь для хранения и извлечения из памяти информации о воздушных судах, получаемой нашим сервисом от вышеупомянутого сервиса `PlaneFinder`.

Инициализация проекта

Для начала обратимся к Spring Initializr. Я выбрал следующие опции:

- проект Maven;
- Java;
- текущая стабильная версия Spring Boot;
- упаковка — JAR;
- Java — 11.

И зависимости:

- Spring Reactive Web (`spring-boot-starter-webflux`);
- Spring Data Redis (Access+Driver) (`spring-boot-starter-data-redis`);
- Lombok (`lombok`).

ПРИМЕЧАНИЯ ОБ ОПЦИЯХ ПРОЕКТА

В скобках после названия меню Initializr указаны идентификаторы артефактов для приведенных ранее возможностей/библиотек. У первых двух — общий идентификатор группы — `org.springframework.boot`, а у Lombok — `org.projectlombok`.

И хотя я не выбирал умышленно какие-либо неблокирующие реактивные функциональные возможности для приложений этой главы, я включил зависимость для Spring Reactive Web вместо Spring Web, чтобы получить доступ к WebClient — предпочтительному клиенту как для блокирующих, так и для неблокирующих взаимодействий приложений, основанных на Spring Boot 2.0 и более поздних версий, с сервисами. С точки зрения разработчика, создающего простейший веб-сервис, код не меняется, какую бы из этих зависимостей я ни включил: в примерах этой главы код, аннотации и свойства совершенно одинаковы для обоих. Я укажу все различия по мере того, как в следующих главах два этих пути начнут расходиться.

Далее сгенерируем проект, сохраним его на локальной машине, разархивируем и откроем в IDE.

Разработка сервиса Redis

Начнем с предметной области.

В настоящее время API-шлюз `PlaneFinder` предоставляет одну конечную точку REST:

```
http://localhost:7634/aircraft
```

Любой локальный сервис может обратиться с запросом к этой конечной точке и получить ответ в виде JSON со списком всех воздушных судов, находящихся в пределах досягаемости приемника, в следующем формате (с типичными образцами данных):

```
[
    {
        "id": 108,
        "callsign": "AMF4263",
        "squawk": "4136",
        "reg": "N49UC",
        "flightno": "",
        "route": "LAN-DFW",
        "type": "B190",
        "category": "A1",
        "altitude": 20000,
        "heading": 235,
        "speed": 248,
        "lat": 38.865905,
        "lon": -90.429382,
        "barometer": 0,
        "vert_rate": 0,
        "selected_altitude": 0,
        "polar_distance": 12.99378,
        "polar_bearing": 345.393951,
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-11-11T21:44:04Z",
        "pos_update_time": "2020-11-11T21:44:03Z",
        "bds40_seen_time": null
    },
    {<другое воздушное судно в пределах досягаемости, те же поля, что и выше>},
    {<последнее воздушное судно, находящееся сейчас в пределах досягаемости,
    те же поля, что и выше>}
]
```

Определение класса предметной области

Для ввода и обработки отчетов о воздушных судах я создал класс `Aircraft` следующего вида:

```
package com.thehecklers.sburredis;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;

import java.time.Instant;

@Data
@NoArgsConstructor
@AllArgsConstructor
@JsonIgnoreProperties(ignoreUnknown = true)
public class Aircraft {
    @Id
    private Long id;
    private String callsign, squawk, reg, flightno, route, type, category;
    private int altitude, heading, speed;
    @JsonProperty("vert_rate")
    private int vertRate;
    @JsonProperty("selected_altitude")
    private int selectedAltitude;
    private double lat, lon, barometer;
    @JsonProperty("polar_distance")
    private double polarDistance;
    @JsonProperty("polar_bearing")
    private double polarBearing;
    @JsonProperty("is_adsb")
    private boolean isADSB;
    @JsonProperty("is_on_ground")
    private boolean isOnGround;
    @JsonProperty("last_seen_time")
    private Instant lastSeenTime;
    @JsonProperty("pos_update_time")
    private Instant posUpdateTime;
    @JsonProperty("bds40_seen_time")
    private Instant bds40SeenTime;

    public String getLastSeenTime() {
        return lastSeenTime.toString();
    }

    public void setLastSeenTime(String lastSeenTime) {
        if (null != lastSeenTime) {
            this.lastSeenTime = Instant.parse(lastSeenTime);
        } else {
            this.lastSeenTime = Instant.ofEpochSecond(0);
        }
```

```
    }

    public String getPosUpdateTime() {
        return posUpdateTime.toString();
    }

    public void setPosUpdateTime(String posUpdateTime) {
        if (null != posUpdateTime) {
            this.posUpdateTime = Instant.parse(posUpdateTime);
        } else {
            this.posUpdateTime = Instant.ofEpochSecond(0);
        }
    }

    public String getBds40SeenTime() {
        return bds40SeenTime.toString();
    }

    public void setBds40SeenTime(String bds40SeenTime) {
        if (null != bds40SeenTime) {
            this.bds40SeenTime = Instant.parse(bds40SeenTime);
        } else {
            this.bds40SeenTime = Instant.ofEpochSecond(0);
        }
    }
}
```

Этот класс предметной области включает несколько полезных аннотаций, упрощающих код и/или повышающих его гибкость. В число аннотаций уровня класса входят следующие:

- `@Data` — указывает Lombok создать метод-геттер, метод-сеттер, `equals()`, `hashCode()` и `toString()`, то есть так называемый класс данных;
- `@NoArgsConstructor` — позволяет создать конструктор без параметров, то есть не требующий аргументов;
- `@AllArgsConstructor` — позволяет создать конструктор с параметром для каждой переменной-члена, то есть требующий указания аргумента для каждого из них;
- `@JsonIgnoreProperties(ignoreUnknown = true)` — указывает механизмам десериализации Jackson игнорировать поля в JSON-ответах, для которых не существует соответствующей переменной экземпляра.

Аннотации уровня полей позволяют точнее контролировать поведение. В числе примеров аннотаций уровня полей — две аннотации этого класса:

- `@Id` — помечает аннотированную переменную-член класса как содержащую уникальный идентификатор записи базы данных;
- `@JsonProperty("vert_rate")` — связывает переменную-член класса с полем JSON с другим именем.

Вам может быть интересно, зачем я создал явным образом методы доступа и изменения для трех переменных экземпляра типа `Instant`, если аннотация `@Data` обеспечивает создание методов-геттеров и методов-сеттеров для всех переменных экземпляра. В случае этих трех переменных необходимо выполнить синтаксический разбор JSON-значения и преобразовать его из `String` в составной тип данных с помощью вызова метода `Instant::parse`. Если значения вообще нет (`null`), нужно следовать другой логике, чтобы не передать случайно `null` методу `parse()` и чтобы присвоить осмысленное подстановочное значение соответствующей переменной-члену с помощью метода-сеттера. Кроме того, сериализацию значения типа `Instant` лучше всего выполнять путем преобразования в `String` — отсюда и необходимость в явных методах-геттерах.

Теперь, описав класс предметной области, мы можем создать и настроить механизм доступа к базе данных Redis.

Добавляем поддержку шаблонов

Spring Boot предоставляет базовую функциональность `RedisTemplate` посредством автоконфигурации, и чтобы выполнять операции над значениями `String` с помощью Redis, нужно совсем немного работы и кода. Работа же со сложными объектами предметной области требует чуть более обширной конфигурации.

Класс `RedisTemplate` расширяет класс `RedisAccessor` и реализует интерфейс `RedisOperations`. Для нашего приложения особенно интересен `RedisOperations`, поскольку в нем описана необходимая для взаимодействия с Redis функциональность.

Разработчики должны стараться писать код для интерфейсов, а не для реализаций, поскольку это позволяет использовать наиболее подходящую реализацию для решения поставленной задачи без изменения кода или API или излишних либо ненужных нарушений принципа DRY (Don't Repeat Yourself — «Не повторяйся»). Если интерфейс реализован полностью, то любая конкретная реализация работает не хуже остальных.

В следующем листинге я создаю компонент типа `RedisOperations`, возвращающий `RedisTemplate` в качестве конкретной реализации компонента. Чтобы

настроить его должным образом для работы с входящим[1] `Aircraft`, я сделал следующее.

1. Создал `Serializer` для преобразования между объектами и записями JSON. Поскольку для маршалинга/демаршалинга (сериализации/десериализации) значений JSON применяется Jackson, уже включенный в веб-приложения Spring Boot, я создал `Jackson2JsonRedisSerializer` для объектов типа `Aircraft`.
2. Создал объект `RedisTemplate`, принимающий ключи типа `String` и значения типа `Aircraft`, для работы с получаемым на входе объектом `Aircraft` с идентификаторами типа `String`. А также подключил компонент `RedisConnectionFactory`, автоматически связываемый с единственным параметром метода создания этого компонента, `RedisConnectionFactory factory`, к объекту `template`, что позволяет ему создавать и получать соединение с базой данных Redis.
3. Передал сериализатор `Jackson2JsonRedisSerializer<Aircraft>` объекту `template` в качестве сериализатора по умолчанию. У `RedisTemplate` есть несколько сериализаторов, в качестве которых может использоваться сериализатор по умолчанию, если им не был присвоен какой-то конкретный, что очень удобно.
4. Создал еще один сериализатор для ключей, чтобы шаблон не пытался использовать сериализатор по умолчанию, ожидающий объекты типа `Aircraft`, для преобразования в значения ключей типа `String` и обратно. Для этого отлично подходит `StringRedisSerializer`.
5. Наконец, возвратил созданный и настроенный объект `RedisTemplate` в качестве компонента, который должен применяться, когда внутри приложения требуется какая-нибудь реализация компонента `RedisOperations`:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.core.RedisOperations;
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.data.redis.serializer.Jackson2JsonRedisSerializer;
import org.springframework.data.redis.serializer.StringRedisSerializer;

@SpringBootApplication
public class SburRedisApplication {
    @Bean
    public RedisOperations<String, Aircraft>
```

[1] В оригинале игра слов — inbound может означать и «входящий», и «прилетающий». — *Примеч. пер.*

```
        redisOperations(RedisConnectionFactory factory) {
            Jackson2JsonRedisSerializer<Aircraft> serializer =
                    new Jackson2JsonRedisSerializer<>(Aircraft.class);

            RedisTemplate<String, Aircraft> template = new RedisTemplate<>();
            template.setConnectionFactory(factory);
            template.setDefaultSerializer(serializer);
            template.setKeySerializer(new StringRedisSerializer());

            return template;
        }

        public static void main(String[] args) {
            SpringApplication.run(SburRedisApplication.class, args);
        }
    }
```

Собираем все вместе

Мы подготовили фундамент для доступа к базе данных Redis с помощью шаблона, пришло время пожинать плоды. Как показано в следующем листинге, я создал класс Spring Boot, снабженный аннотацией `@Component`, для опроса конечной точки `PlaneFinder` и обработки записей `Aircraft`, полученных благодаря поддержке шаблонов Redis.

Для инициализации компонента `PlaneFinderPoller` и подготовки его к работе я создал объект `WebClient`, указывающий на целевую конечную точку, предоставляемую внешним сервисом `PlaneFinder`, и присвоил его переменной экземпляра. В настоящее время сервис `PlaneFinder` работает на моей локальной машине и прослушивает на порте 7634.

Компоненту `PlaneFinderPoller` для работы необходим доступ к двум другим компонентам: `RedisConnectionFactory` (предоставляется средствами автоконфигурации Spring Boot, поскольку одна из зависимостей приложения — Redis) и реализации интерфейса `RedisOperations` — созданному ранее `RedisTemplate`. Оба присваиваются описанным должным образом переменным-членам посредством внедрения зависимости через конструктор (с автоматическим связыванием):

```
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.core.RedisOperations;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;

@EnableScheduling
@Component
class PlaneFinderPoller {
```

```
    private WebClient client =
            WebClient.create("http://localhost:7634/aircraft");

    private final RedisConnectionFactory connectionFactory;
    private final RedisOperations<String, Aircraft> redisOperations;

    PlaneFinderPoller(RedisConnectionFactory connectionFactory,
                      RedisOperations<String, Aircraft> redisOperations) {
        this.connectionFactory = connectionFactory;
        this.redisOperations = redisOperations;
    }
}
```

Далее я создал метод, в котором и выполняется основная работа. Для реализации опроса по заданному расписанию воспользовался аннотацией `@EnableScheduling`, размещенной ранее на уровне класса, и снабдил созданный мною метод `pollPlanes()` аннотацией `@Scheduled`, передав в нее параметр `fixedDelay=1000`, указывающий, что опрос выполняется каждые 1000 мс (один раз в секунду). Остальная часть этого метода состоит лишь из трех декларативных операторов: для очистки ранее сохраненных объектов `Aircraft`, извлечения и сохранения текущих местоположений самолетов и вывода отчета о последней захваченной информации.

Что касается первой из упомянутых задач: я воспользовался автоматически связываемым объектом `Connectionfactory` для получения соединения с базой данных, через которое выполнил команду сервера по очистке всех имеющихся ключей — `flushDb()`.

Второй оператор вызывает `PlaneFinder` с помощью объекта `WebClient` и извлекает набор воздушных судов в пределах досягаемости вместе с информацией об их местоположении в данный момент. Тело ответа преобразуется в `Flux` объектов `Aircraft`, фильтруется для исключения объектов `Aircraft` без регистрационных номеров, преобразуется в `Stream` объектов `Aircraft` и сохраняется в базу данных Redis. Сохранение каждого допустимого объекта `Aircraft` производится присваиванием паре «ключ — значение» регистрационного номера `Aircraft` и самого объекта `Aircraft` соответственно с помощью предназначенных для манипулирования значениями данных операций Redis.



Flux — реактивный тип данных, он описан в следующих главах, а пока что можете считать его просто набором объектов, поставляемых без блокирования.

Последний оператор в `pollPlanes()` снова использует несколько операций над значениями, определенных в Redis, для извлечения всех ключей (с помощью

подстановочного знака *), извлечения соответствующего каждому из ключей значения Aircraft и последующего вывода на экран. Вот законченная версия метода pollPlanes():

```
@Scheduled(fixedRate = 1000)
private void pollPlanes() {
    connectionFactory.getConnection().serverCommands().flushDb();

    client.get()
            .retrieve()
            .bodyToFlux(Aircraft.class)
            .filter(plane -> !plane.getReg().isEmpty())
            .toStream()
            .forEach(ac -> redisOperations.opsForValue().set(ac.getReg(), ac));

    redisOperations.opsForValue()
            .getOperations()
            .keys("*")
            .forEach(ac ->
                System.out.println(redisOperations.opsForValue().get(ac)));
}
```

Окончательная (на данный момент) версия класса PlaneFinderPoller приведена в следующем листинге:

```
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.core.RedisOperations;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;

@EnableScheduling
@Component
class PlaneFinderPoller {
    private WebClient client =
            WebClient.create("http://localhost:7634/aircraft");

    private final RedisConnectionFactory connectionFactory;
    private final RedisOperations<String, Aircraft> redisOperations;

    PlaneFinderPoller(RedisConnectionFactory connectionFactory,
                      RedisOperations<String, Aircraft> redisOperations) {
        this.connectionFactory = connectionFactory;
        this.redisOperations = redisOperations;
    }

    @Scheduled(fixedRate = 1000)
    private void pollPlanes() {
```

```
        connectionFactory.getConnection().serverCommands().flushDb();

        client.get()
                .retrieve()
                .bodyToFlux(Aircraft.class)
                .filter(plane -> !plane.getReg().isEmpty())
                .toStream()
                .forEach(ac ->
                    redisOperations.opsForValue().set(ac.getReg(), ac));

        redisOperations.opsForValue()
                .getOperations()
                .keys("*")
                .forEach(ac ->
                    System.out.println(redisOperations.opsForValue().get(ac)));
    }
}
```

Механизмы опроса полностью реализованы, и мы можем запустить приложение и посмотреть, что получилось.

Результаты

С предварительно запущенным на моей машине сервисом `PlaneFinder` я запустил приложение `sbur-redis` для получения, сохранения и извлечения информации из Redis, а также отображения на экране результатов каждого из опросов сервиса `PlaneFinder`. Далее приведен пример полученных результатов, отредактированный для краткости и немного отформатированный для удобства чтения:

```
Aircraft(id=1, callsign=EDV5015, squawk=3656, reg=N324PQ, flightno=DL5015,
route=ATL-OMA-ATL, type=CRJ9, category=A3, altitude=35000, heading=168,
speed=485, vertRate=-64, selectedAltitude=0, lat=38.061808, lon=-90.280629,
barometer=0.0, polarDistance=53.679699, polarBearing=184.333345, isADSB=true,
isOnGround=false, lastSeenTime=2020-11-27T18:34:14Z,
posUpdateTime=2020-11-27T18:34:11Z, bds40SeenTime=1970-01-01T00:00:00Z)

Aircraft(id=4, callsign=AAL500, squawk=2666, reg=N839AW, flightno=AA500,
route=PHX-IND, type=A319, category=A3, altitude=36975, heading=82, speed=477,
vertRate=0, selectedAltitude=36992, lat=38.746399, lon=-90.277644,
barometer=1012.8, polarDistance=13.281347, polarBearing=200.308663, isADSB=true,
isOnGround=false, lastSeenTime=2020-11-27T18:34:50Z,
posUpdateTime=2020-11-27T18:34:50Z, bds40SeenTime=2020-11-27T18:34:50Z)

Aircraft(id=15, callsign=null, squawk=4166, reg=N404AN, flightno=AA685,
route=PHX-DCA, type=A21N, category=A3, altitude=39000, heading=86, speed=495,
vertRate=0, selectedAltitude=39008, lat=39.701611, lon=-90.479309,
barometer=1013.6, polarDistance=47.113195, polarBearing=341.51817, isADSB=true,
isOnGround=false, lastSeenTime=2020-11-27T18:34:50Z,
posUpdateTime=2020-11-27T18:34:50Z, bds40SeenTime=2020-11-27T18:34:50Z)
```

Поддержка шаблонов Spring Data позволяет работать с базами данных через очень гибкий низкоуровневый API. Впрочем, если вы стремитесь к наиболее беспрепятственной работе и максимальной производительности и повторяемости, лучше воспользоваться поддержкой репозиториев. Далее я покажу, как перейти от использования шаблонов для взаимодействия с Redis к использованию репозитория Spring Data. Наличие выбора — замечательная вещь.

Преобразование из шаблона в репозиторий

Прежде чем использовать репозиторий, необходимо его определить, и автоконфигурация Spring Boot сильно облегчает эту задачу. Я создал следующий интерфейс репозитория, расширив класс `CrudRepository` Spring Data и указав тип хранимых объектов вместе с ключами — `Aircraft` и `Long` в данном случае:

```
public interface AircraftRepository extends CrudRepository<Aircraft, Long> {}
```

Как объяснялось в главе 4, Spring Boot находит драйвер базы данных Redis по пути к классам приложения и замечает, что мы расширяем интерфейс репозитория Spring Data, после чего автоматически создает прокси для базы данных, причем для создания его экземпляра не требуется никакого дополнительного кода. Вот и все, у приложения теперь есть доступ к компоненту `AircraftRepository`. Давайте подключим его и начнем использовать.

Возвращаясь к классу `PlaneFinderPoller`, мы можем теперь заменить низкоуровневые ссылки на интерфейс `RedisOperations` и задействующие его операции на `AircraftRepository`.

Прежде всего удалим переменную-член класса `RedisOperations`:

```
private final RedisOperations<String, Aircraft> redisOperations;
```

И заменим ее на репозиторий `AircraftRepository` для автоматического связывания:

```
private final AircraftRepository repository;
```

Далее путем внедрения зависимости через конструктор заменяем автоматически связываемый компонент `RedisOperations` на `AircraftRepository`, модифицируя также присваивание внутри конструктора, так что в итоге он выглядит так:

```
public PlaneFinderPoller(RedisConnectionFactory connectionFactory,
                         AircraftRepository repository) {
```

```
    this.connectionFactory = connectionFactory;
    this.repository = repository;
}
```

Следующий шаг — рефакторинг метода `pollPlanes()` для замены операции на основе шаблона операциями на основе репозитория.

Проще всего изменить последнюю строку первого оператора. Упрощаем лямбда-выражение еще больше с помощью ссылки на метод:

```
client.get()
        .retrieve()
        .bodyToFlux(Aircraft.class)
        .filter(plane -> !plane.getReg().isEmpty())
        .toStream()
        .forEach(repository::save);
```

Второй оператор упрощается еще сильнее, тоже благодаря ссылке на метод:

```
repository.findAll().forEach(System.out::println);
```

Класс `PlaneFinderPoller`, который теперь использует репозиторий, состоит из следующего кода:

```
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;

@EnableScheduling
@Component
class PlaneFinderPoller {
    private WebClient client =
            WebClient.create("http://localhost:7634/aircraft");

    private final RedisConnectionFactory connectionFactory;
    private final AircraftRepository repository;

    PlaneFinderPoller(RedisConnectionFactory connectionFactory,
                      AircraftRepository repository) {
        this.connectionFactory = connectionFactory;
        this.repository = repository;
    }

    @Scheduled(fixedRate = 1000)
    private void pollPlanes() {
        connectionFactory.getConnection().serverCommands().flushDb();
```

```
        client.get()
                .retrieve()
                .bodyToFlux(Aircraft.class)
                .filter(plane -> !plane.getReg().isEmpty())
                .toStream()
                .forEach(repository::save);

        repository.findAll().forEach(System.out::println);
    }
}
```

Поскольку реализующий интерфейс `RedisOperations` компонент больше не нужен, можно удалить его определение `@Bean` из основного класса приложения, оставив только `SburRedisApplication`, как демонстрирует следующий код:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SburRedisApplication {

    public static void main(String[] args) {
        SpringApplication.run(SburRedisApplication.class, args);
    }
}
```

Чтобы полностью обеспечить поддержку репозитория Redis в нашем приложении, осталось сделать одну мелочь и изящно сократить объем кода. Я добавил аннотацию `@RedisHash` в сущность `Aircraft`, помечая таким образом этот `Aircraft` как корневой объект агрегата для сохранения в хеше Redis, аналогично тому, что аннотация `@Entity` делает для объектов JPA. Затем удалил прежде необходимые для переменных экземпляра типа `Instant` явные объявления методов доступа и изменения, поскольку доступные благодаря поддержке репозиториев Spring Data преобразователи легко справляются с преобразованиями сложных типов данных. Новый, упрощенный класс `Aircraft` выглядит следующим образом:

```
import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.redis.core.RedisHash;

import java.time.Instant;

@Data
@NoArgsConstructor
```

```
@AllArgsConstructor
@RedisHash
@JsonIgnoreProperties(ignoreUnknown = true)
public class Aircraft {
    @Id
    private Long id;
    private String callsign, squawk, reg, flightno, route, type, category;
    private int altitude, heading, speed;
    @JsonProperty("vert_rate")
    private int vertRate;
    @JsonProperty("selected_altitude")
    private int selectedAltitude;
    private double lat, lon, barometer;
    @JsonProperty("polar_distance")
    private double polarDistance;
    @JsonProperty("polar_bearing")
    private double polarBearing;
    @JsonProperty("is_adsb")
    private boolean isADSB;
    @JsonProperty("is_on_ground")
    private boolean isOnGround;
    @JsonProperty("last_seen_time")
    private Instant lastSeenTime;
    @JsonProperty("pos_update_time")
    private Instant posUpdateTime;
    @JsonProperty("bds40_seen_time")
    private Instant bds40SeenTime;
}
```

После этих изменений и перезапуска сервиса выводимые результаты не отличаются от полученных с использованием шаблонов, но требуется намного меньше кода и необходимых формальностей. Вот пример результатов, опять же отредактированный ради краткости и отформатированный для удобства чтения:

```
Aircraft(id=59, callsign=KAP20, squawk=4615, reg=N678JG, flightno=,
route=STL-IRK, type=C402, category=A1, altitude=3825, heading=0, speed=143,
vertRate=768, selectedAltitude=0, lat=38.881034, lon=-90.261475, barometer=0.0,
polarDistance=5.915421, polarBearing=222.434158, isADSB=true, isOnGround=false,
lastSeenTime=2020-11-27T18:47:31Z, posUpdateTime=2020-11-27T18:47:31Z,
bds40SeenTime=1970-01-01T00:00:00Z)

Aircraft(id=60, callsign=SWA442, squawk=5657, reg=N928WN, flightno=WN442,
route=CMH-DCA-BNA-STL-PHX-BUR-OAK, type=B737, category=A3, altitude=8250,
heading=322, speed=266, vertRate=-1344, selectedAltitude=0, lat=38.604034,
lon=-90.357593, barometer=0.0, polarDistance=22.602864, polarBearing=201.283,
isADSB=true, isOnGround=false, lastSeenTime=2020-11-27T18:47:25Z,
posUpdateTime=2020-11-27T18:47:24Z, bds40SeenTime=1970-01-01T00:00:00Z)

Aircraft(id=61, callsign=null, squawk=null, reg=N702QS, flightno=,
route=SNA-RIC, type=CL35, category=, altitude=43000, heading=90, speed=500,
```

```
vertRate=0, selectedAltitude=0, lat=39.587997, lon=-90.921299, barometer=0.0,
polarDistance=51.544552, polarBearing=316.694343, isADSB=true, isOnGround=false,
lastSeenTime=2020-11-27T18:47:19Z, posUpdateTime=2020-11-27T18:47:19Z,
bds40SeenTime=1970-01-01T00:00:00Z)
```

При необходимости получить прямой доступ к низкоуровневым возможностям, предоставляемым шаблонами Spring Data, придется воспользоваться поддержкой доступа к базам данных на основе шаблонов. Но практически для всех распространенных сценариев использования, в которых Spring Data дает доступ к нужной базе данных на основе репозитория, стоит именно с него и начинать — и, скорее всего, на нем и остановиться.

Создание сервиса на основе репозитория с помощью Java Persistence API

Одна из самых сильных сторон экосистемы Spring — единообразие: любой изученный подход позволяет успешно решать задачи с другими компонентами. Показательный пример — доступ к базе данных.

Spring Boot и Spring Data поддерживают репозитории для нескольких баз данных: JPA-совместимых баз, многочисленных хранилищ данных NoSQL различных типов, а также хранилищ в оперативной памяти и/или постоянных хранилищ данных. Spring сглаживает для разработчиков шероховатости перехода от одной базы данных к другой вне зависимости от того, идет ли речь об отдельном приложении или обширной системе приложений.

Чтобы продемонстрировать некоторые гибкие возможности создания ориентированных на данные приложений Spring Boot, в следующих разделах я выделю несколько различных поддерживаемых Spring Boot подходов, где Boot и Spring Data используются для упрощения относящейся к взаимодействию с базой данных части различных, хотя и сходных сервисов. Первый на очереди — JPA, и для этого примера я воспользуюсь Lombok, чтобы сократить объем кода и повысить удобочитаемость.

Инициализация проекта

И снова обратимся к Spring Initializr. На этот раз я выбрал следующие опции:

- проект Maven;
- Java;

- текущая стабильная версия Spring Boot;
- упаковка — JAR;
- Java — 11.

И зависимости:

- Spring Reactive Web (`spring-boot-starter-webflux`);
- Spring Data JPA (`spring-boot-starter-data-jpa`);
- MySQL Driver (`mysql-connector-java`);
- Lombok (`lombok`).

Далее генерируем проект, сохраняем его на локальной машине, разархивируем и открываем в IDE.



Как и предыдущий проект Redis и большинство других примеров из этой главы, любой ориентированный на данные сервис должен уметь обращаться к работающей базе данных. Сценарии Docker для создания и запуска подходящих контейнеризованных движков баз данных вы найдете в прилагаемых к этой книге репозиториях кода.

Разработка JPA-сервиса (MySQL)

Если сравнить пример из главы 4, созданный на основе JPA и базы данных H2, и предыдущий пример на основе репозитория Redis, то мы увидим, что сервис на основе JPA, использующий MariaDB/MySQL, ясно демонстрирует, каким образом Spring благодаря согласованности повышает производительность разработчиков.

Описание класса предметной области

Как и для всех проектов этой главы, я создам класс предметной области `Aircraft` в качестве основного объекта данных. Все проекты будут вращаться около единой темы, с небольшими различиями. Вот структура ориентированного на JPA класса предметной области `Aircraft`:

```
import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import javax.persistence.Entity;
```

```
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
import java.time.Instant;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Aircraft {
    @Id
    @GeneratedValue
    private Long id;

    private String callsign, squawk, reg, flightno, route, type, category;

    private int altitude, heading, speed;
    @JsonProperty("vert_rate")
    private int vertRate;
    @JsonProperty("selected_altitude")
    private int selectedAltitude;

    private double lat, lon, barometer;
    @JsonProperty("polar_distance")
    private double polarDistance;
    @JsonProperty("polar_bearing")
    private double polarBearing;

    @JsonProperty("is_adsb")
    private boolean isADSB;
    @JsonProperty("is_on_ground")
    private boolean isOnGround;

    @JsonProperty("last_seen_time")
    private Instant lastSeenTime;
    @JsonProperty("pos_update_time")
    private Instant posUpdateTime;
    @JsonProperty("bds40_seen_time")
    private Instant bds40SeenTime;
}
```

Стоит отметить несколько особенностей этой версии класса `Aircraft` по сравнению с предыдущими и последующими версиями.

Все аннотации `@Entity`, `@Id` и `@GeneratedValue` импортируются из пакета `javax.persistence`. Наверное, вы помните, что в версии для Redis и некоторых других аннотация `@Id` была взята из `org.springframework.data.annotation`.

Аннотации уровня класса аналогичны тем, которые мы встречали в примере, где использовалась поддержка репозиториев Redis, лишь `@RedisHash` заменена на

JPA-аннотацию `@Entity`. Чтобы освежить в памяти неизменившиеся аннотации, пожалуйста, обратитесь к предыдущему разделу.

Аннотации уровня полей также не изменились, если не считать добавления `@GeneratedValue`. Как ясно из ее названия, аннотация `@GeneratedValue` означает, что идентификатор генерируется базовым движком базы данных. При желании или необходимости разработчик может указать дополнительные параметры генерации ключа, но для наших целей достаточно самой аннотации.

Как и при использовании поддержки Spring Data-репозиториев для Redis, нет необходимости в описании явным образом методов доступа и изменения для переменных-членов типа `Instant`, благодаря чему код класса `Aircraft` остается весьма лаконичным.

Создание интерфейса репозитория

Далее я определяю необходимый интерфейс репозитория, расширяя `CrudRepository` из Spring Data и указывая тип хранимого объекта и его ключа — `Aircraft` и `Long` в данном случае:

```
public interface AircraftRepository extends CrudRepository<Aircraft, Long> {}
```



Как Redis, так и JPA базы данных прекрасно работают с уникальными значениями ключей/идентификаторами типа Long, так что никаких отличий от описания из предыдущего примера с Redis нет.

Собираем все вместе

Пришло время создать компонент для опроса `PlaneFinder` и настроить его для доступа к базе данных.

Опрос сервиса PlaneFinder. Я снова создал класс с аннотацией `@Component` Spring Boot для опроса текущих данных о местоположении и обработки получаемых им записей `Aircraft`.

Как и в предыдущем примере, я создал объект `WebClient`, указывающий на целевую конечную точку, предоставляемую сервисом `PlaneFinder` через порт 7634, и присвоил его переменной экземпляра.

Как можно было ожидать от столь близкой реализации, код очень похож на конечный вид кода для репозитория Redis. Отметим несколько отличий подхода для этого примера.

Вместо того чтобы создавать конструктор для получения автоматически связываемого компонента `AircraftRepository` вручную, я запросил у Lombok — посредством его статического генератора кода — конструктор со всеми нужными переменными-членами. Lombok определяет, какие аргументы требуются, с помощью двух аннотаций: `@RequiredArgsConstructor` для класса и `@NonNull` для переменных-членов, обозначенных как требующие инициализации. При указании для переменной экземпляра `AircraftRepository` аннотации `@NonNull` Lombok создает конструктор с `AircraftRepository` в качестве параметра, а затем Spring Boot автоматически связывает уже существующий компонент репозитория для использования внутри компонента `PlaneFinderPoller`.



Стоит ли удалять ли все хранимые записи из базы данных при каждом опросе, в значительной степени зависит от требований, частоты опроса и механизма хранения. Например, затраты на очистку базы данных в оперативной памяти перед каждым опросом сильно отличаются от затрат на удаление всех записей из таблицы базы, размещенной в облаке. Если опрос будет выполняться часто, это тоже повысит затраты. Альтернатив много, пожалуйста, тщательно взвешивайте, какую выбрать.

Возвращаясь к нюансам оставшегося кода `PlaneFinderPoller`: пожалуйста, просмотрите еще раз соответствующий раздел из обсуждения поддержки репозиториев Redis. Переработанный для полноценного использования поддержки Spring Data JPA полный код класса `PlaneFinderPoller` приведен в следующем листинге:

```
import lombok.NonNull;
import lombok.RequiredArgsConstructor;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;

@EnableScheduling
@Component
@RequiredArgsConstructor
class PlaneFinderPoller {
    @NonNull
    private final AircraftRepository repository;
    private WebClient client =
            WebClient.create("http://localhost:7634/aircraft");

    @Scheduled(fixedRate = 1000)
    private void pollPlanes() {
        repository.deleteAll();
```

```
        client.get()
                .retrieve()
                .bodyToFlux(Aircraft.class)
                .filter(plane -> !plane.getReg().isEmpty())
                .toStream()
                .forEach(repository::save);

        repository.findAll().forEach(System.out::println);
    }
}
```

Подключение к MariaDB/MySQL. Spring Boot автоматически задает конфигурацию среды приложения на основе всей доступной среде выполнения информации. Это один из источников его непревзойденной гибкости. А поскольку Spring Boot и Spring Data поддерживают множество JPA-совместимых баз данных, необходимо предоставить Boot кое-какую важную информацию для беспроблемного соединения с выбранной для конкретного приложения базой данных. Для запущенного в моей среде сервиса эти свойства включают:

```
spring.datasource.platform=mysql
spring.datasource.url=jdbc:mysql://${MYSQL_HOST:localhost}:3306/mark
spring.datasource.username=mark
spring.datasource.password=sbux
```



Как название базы данных, так и имя пользователя базы в приведенном примере — mark. Замените источник данных, имя пользователя и пароль на соответствующие вашей среде.

Результаты

Не отключая сервис `PlaneFinder`, я запустил приложение `sbur-jpa` для получения, сохранения и извлечения информации из MariaDB, а также отображения на экране результатов каждого из опросов сервиса `PlaneFinder`. Далее приведен пример полученных результатов, отредактированный для краткости и немного отформатированный для удобства чтения:

```
Aircraft(id=106, callsign=null, squawk=null, reg=N7816B, flightno=WN2117,
route=SJC-STL-BWI-FLL, type=B737, category=, altitude=4400, heading=87,
speed=233, vertRate=2048, selectedAltitude=15008, lat=0.0, lon=0.0,
barometer=1017.6, polarDistance=0.0, polarBearing=0.0, isADSB=false,
isOnGround=false, lastSeenTime=2020-11-27T18:59:10Z,
posUpdateTime=2020-11-27T18:59:17Z, bds40SeenTime=2020-11-27T18:59:10Z)

Aircraft(id=107, callsign=null, squawk=null, reg=N963WN, flightno=WN851,
route=LAS-DAL-STL-CMH, type=B737, category=, altitude=27200, heading=80,
```

```
speed=429, vertRate=2112, selectedAltitude=0, lat=0.0, lon=0.0, barometer=0.0,
polarDistance=0.0, polarBearing=0.0, isADSB=false, isOnGround=false,
lastSeenTime=2020-11-27T18:58:45Z, posUpdateTime=2020-11-27T18:59:17Z,
bds40SeenTime=2020-11-27T18:59:17Z)

Aircraft(id=108, callsign=null, squawk=null, reg=N8563Z, flightno=WN1386,
route=DEN-IAD, type=B738, category=, altitude=39000, heading=94, speed=500,
vertRate=0, selectedAltitude=39008, lat=0.0, lon=0.0, barometer=1013.6,
polarDistance=0.0, polarBearing=0.0, isADSB=false, isOnGround=false,
lastSeenTime=2020-11-27T18:59:10Z, posUpdateTime=2020-11-27T18:59:17Z,
bds40SeenTime=2020-11-27T18:59:10Z)
```

Сервис запрашивает, захватывает и отображает позиции воздушных судов так, как и должен.

Загрузка данных

До сих пор в этой главе мы сосредоточивались на взаимодействии с базой данных в смысле передачи данных в приложение. А что, если у приложения появляются данные — примеры данных, тестовые данные или собственно данные для наполнения базы, которые необходимо сохранить?

Spring Boot включает несколько различных механизмов инициализации и заполнения базы данных. Я рассмотрю два, по моему мнению, наиболее удобных подхода.

- Использовать сценарии на языках определения данных (Data Definition Language, DDL) и манипулирования данными (Data Manipulation Language, DML) для инициализации и заполнения базы.
- Позволить Spring Boot (с помощью Hibernate) автоматически создать структуру таблиц на основе определенных классов с аннотацией `@Entity` и заполнять через bean-компонент репозитория.

У каждого из этих подходов к описанию данных и наполнению ими базы данных есть свои достоинства и недостатки.

Скрипты для конкретных API или базы данных

Spring Boot ищет по обычным корневым путям к классам файлы, названия которых удовлетворяют следующим шаблонам:

- `schema.sql`;
- `data.sql`;

- `schema-${`*платформа*`}.sql`;
- `data-${`*платформа*`}.sql`.

В последние два названия подставляется задаваемое разработчиком свойство приложения `spring.datasource.platform`. В числе допустимых значений: `h2`, `mysql`, `postgresql` и прочие базы данных Spring Data JPA, а сочетание свойства `spring.datasource.platform` и соответствующих файлов `.sql` позволяет разработчику полноценно использовать синтаксис этой конкретной базы данных.

Создание и заполнение базы данных с помощью сценариев. Чтобы было проще использовать сценарии для создания и заполнения базы данных MariaDB или MySQL, я создал в каталоге `resources` проекта `sbur-jpa` два файла: `schema-mysql.sql` и `data-mysql.sql`.

Для создания схемы таблицы `aircraft` я добавил в `schema-mysql.sql` следующий DDL:

```
DROP TABLE IF EXISTS aircraft;
CREATE TABLE aircraft (id BIGINT not null primary key, callsign VARCHAR(7),
squawk VARCHAR(4), reg VARCHAR(6), flightno VARCHAR(10), route VARCHAR(25),
type VARCHAR(4), category VARCHAR(2),
altitude INT, heading INT, speed INT, vert_rate INT, selected_altitude INT,
lat DOUBLE, lon DOUBLE, barometer DOUBLE,
polar_distance DOUBLE, polar_bearing DOUBLE,
isadsb BOOLEAN, is_on_ground BOOLEAN,
last_seen_time TIMESTAMP, pos_update_time TIMESTAMP, bds40seen_time TIMESTAMP);
```

А чтобы внести в таблицу `aircraft` одну строку примера данных, добавил в `data-mysql.sql` следующий DML:

```
INSERT INTO aircraft (id, callsign, squawk, reg, flightno, route, type,
category, altitude, heading, speed, vert_rate, selected_altitude, lat, lon,
barometer, polar_distance, polar_bearing, isadsb, is_on_ground,
last_seen_time, pos_update_time, bds40seen_time)
VALUES (81, 'AAL608', '1451', 'N754UW', 'AA608', 'IND-PHX', 'A319', 'A3', 36000,
255, 423, 0, 36000, 39.150284, -90.684795, 1012.8, 26.575562, 295.501994,
true, false, '2020-11-27 21:29:35', '2020-11-27 21:29:34',
'2020-11-27 21:29:27');
```

По умолчанию Spring Boot автоматически создает структуру таблиц на основе любых классов, снабженных аннотацией `@Entity`. Переопределить подобное поведение очень легко с помощью следующих значений свойств из файла `application.properties`:

```
spring.datasource.initialization-mode=always
spring.jpa.hibernate.ddl-auto=none
```

Значение `always` свойства `spring.datasource.initialization-mode` означает, что приложение будет использовать внешнюю (невстраиваемую) базу данных и должно инициализировать ее при каждом запуске приложения. Значение `none` свойства `spring.jpa.hibernate.ddl-auto` отключает автоматическое создание Spring Boot-таблиц на основе снабженных аннотацией `@Entity` классов.

Чтобы проверить, что таблица `aircraft` была создана и заполнена с помощью предыдущих сценариев, я внес в класс `PlaneFinderPoller` следующие изменения:

- закомментировал оператор `repository.deleteAll();` в `pollPlanes()`, чтобы избежать удаления записи, добавляемой в `data-mysql.sql`;
- закомментировал оператор `client.get()...`, также в `pollPlanes()`. В результате для упрощения проверки никакие дополнительные записи не извлекаются и не создаются в результате опроса внешнего сервиса `PlaneFinder`.

Теперь при перезапуске сервиса `sbur-jpa` мы получим следующие результаты (значения полей `id` могут отличаться). Они отредактированы для краткости и немного отформатированы для удобства чтения:

```
Aircraft(id=81, callsign=AAL608, squawk=1451, reg=N754UW, flightno=AA608,
route=IND-PHX, type=A319, category=A3, altitude=36000, heading=255, speed=423,
vertRate=0, selectedAltitude=36000, lat=39.150284, lon=-90.684795,
barometer=1012.8, polarDistance=26.575562, polarBearing=295.501994, isADSB=true,
isOnGround=false, lastSeenTime=2020-11-27T21:29:35Z,
posUpdateTime=2020-11-27T21:29:34Z, bds40SeenTime=2020-11-27T21:29:27Z)
```



Единственная сохраненная запись — указанная в файле data-mysql.sql.

У этого метода создания и заполнения таблицы, как и у любого другого подхода к чему угодно, есть свои достоинства и недостатки. В числе преимуществ:

- возможность непосредственного использования SQL-сценариев, как DDL, так и DML, в том числе уже существующих, и/или накопленного опыта работы с SQL;
- возможность использования особенностей синтаксиса языка SQL конкретной базы данных.

Недостатки не слишком серьезны, но их также следует учитывать.

- Разумеется, SQL-файлы можно применять только для поддерживающих язык SQL реляционных баз данных.
- Сценарии могут использовать особенности синтаксиса SQL конкретной базы данных, а значит, при переходе на другую базу данных может потребоваться их модификация.
- Для переопределения поведения Boot по умолчанию необходимо задать несколько свойств приложения, а точнее два.

Заполнение базы данных с помощью репозитория приложения

Есть и другой способ, очень гибкий и с богатыми возможностями: создание структуры таблиц (если их еще не существует) с использованием поведения Spring Boot по умолчанию и заполнение их образцами данных при поддержке репозиториев приложением.

Чтобы восстановить поведение Spring Boot по умолчанию — создание таблицы `aircraft` на основе снабженного аннотацией `@Entity` JPA-класса, я закомментировал два только что добавленных в файл `application.properties` свойства:

```
#spring.datasource.initialization-mode=always
#spring.jpa.hibernate.ddl-auto=none
```

Без них Spring Boot не будет искать и выполнять `data-mysql.sql` или другие сценарии начального заполнения данными.

Далее я создал класс с именем, отражающим его назначение, — `DataLoader`. Снабдил его аннотациями уровня класса `@Component` (чтобы Spring создал компонент `DataLoader`) и `@AllArgsConstructor` (чтобы Lombok создал конструктор, включающий по параметру для каждой переменной экземпляра). А затем добавил еще одну переменную-член для компонента `AircraftRepository`, который Spring Boot автоматически связывает посредством внедрения зависимости через конструктор:

```
private final AircraftRepository repository;
```

Метод `loadData()` для очистки таблицы `aircraft` и ее наполнения данными:

```
@PostConstruct
private void loadData() {
    repository.deleteAll();
```

```
    repository.save(new Aircraft(81L,
            "AAL608", "1451", "N754UW", "AA608", "IND-PHX", "A319", "A3",
            36000, 255, 423, 0, 36000,
            39.150284, -90.684795, 1012.8, 26.575562, 295.501994,
            true, false,
            Instant.parse("2020-11-27T21:29:35Z"),
            Instant.parse("2020-11-27T21:29:34Z"),
            Instant.parse("2020-11-27T21:29:27Z")));
}
```

Вот и все. Правда. Теперь при перезапуске сервиса `sbur-jpa` мы получаем следующие результаты (значения полей `id` могут отличаться). Они отредактированы для краткости и немного отформатированы для удобства чтения:

```
Aircraft(id=110, callsign=AAL608, squawk=1451, reg=N754UW, flightno=AA608,
route=IND-PHX, type=A319, category=A3, altitude=36000, heading=255, speed=423,
vertRate=0, selectedAltitude=36000, lat=39.150284, lon=-90.684795,
barometer=1012.8, polarDistance=26.575562, polarBearing=295.501994, isADSB=true,
isOnGround=false, lastSeenTime=2020-11-27T21:29:35Z,
posUpdateTime=2020-11-27T21:29:34Z, bds40SeenTime=2020-11-27T21:29:27Z)
```



Единственная сохраненная запись — та, что описана в упомянутом ранее классе DataLoader, с одним небольшим отличием: поскольку поле id генерируется базой данных (это определено в спецификации класса предметной области Aircraft), движок базы данных заменяет передаваемое значение id при сохранении записи.

Преимущества такого подхода весьма существенны.

- Полная независимость от используемой базы данных.
- Весь относящийся к конкретной базе данных код или аннотации находятся внутри приложения — для поддержки доступа к базе данных.
- Для отключения достаточно просто закомментировать аннотацию `@Component` класса `DataLoader`.

Другие механизмы

Рассмотренные варианты инициализации и наполнения базы данных предоставляют большие возможности и очень широко применяются, но существуют и другие, включая использование поддержки Hibernate для файла `import.sql` (аналогично описанному ранее подходу JPA), использование внешних импортов, а также FlywayDB и др. Обсуждение других вариантов выходит за рамки темы книги, мы оставим это читателю в качестве упражнения.

Создание сервиса на основе репозитория с помощью документоориентированной базы данных NoSQL

Как упоминалось ранее, есть несколько способов повысить производительность разработчика при создании приложений с помощью Spring Boot. Один из них — воспользоваться Kotlin в качестве основного языка приложения, для того чтобы сделать код более лаконичным.

Всестороннее обсуждение Kotlin выходит далеко за рамки темы данной книги, существует немало других изданий, решающих эту задачу. К счастью, хотя Kotlin несомненно отличается от языка Java во многих важных пунктах, но и общего у них немало, так что нескольких пояснений в местах расхождений со «стилем Java» будет достаточно, чтобы разработчик смог легко адаптироваться к его особенностям. Я попробую дать все нужные пояснения по ходу изложения, а дополнительную информацию можно найти в посвященных языку Kotlin монографиях.

В этом примере мы воспользуемся MongoDB. Вероятно, это наиболее известная документоориентированная база данных. MongoDB не без причин широко применяется и очень популярна: она прекрасно работает и в целом облегчает разработчикам хранение, обработку и извлечение данных разнообразных и нередко довольно запутанных форм. Команда создателей MongoDB постоянно стремится улучшить набор возможностей, безопасность и API: MongoDB — одна из первых баз данных, у которой появились реактивные драйверы, в результате чего вся отрасль перешла на неблокирующий доступ вплоть до уровня базы данных.

Инициализация проекта

Как вы можете догадаться, сначала мы снова обратимся к Spring Initializr. Для этого проекта я выбрал опции (показаны также на рис. 6.1), несколько отличающиеся от заданных прежде:

- проект Gradle;
- Kotlin;
- текущая стабильная версия Spring Boot;
- упаковка — JAR;
- Java — 11.

И зависимости:

- Spring Reactive Web (`spring-boot-starter-webflux`);
- Spring Data MongoDB (`spring-boot-starter-data-mongodb`);
- Embedded MongoDB Database (`de.flapdoodle.embed.mongo`).

Далее сгенерируем проект, сохраним его на локальной машине, разархивируем и откроем в IDE.

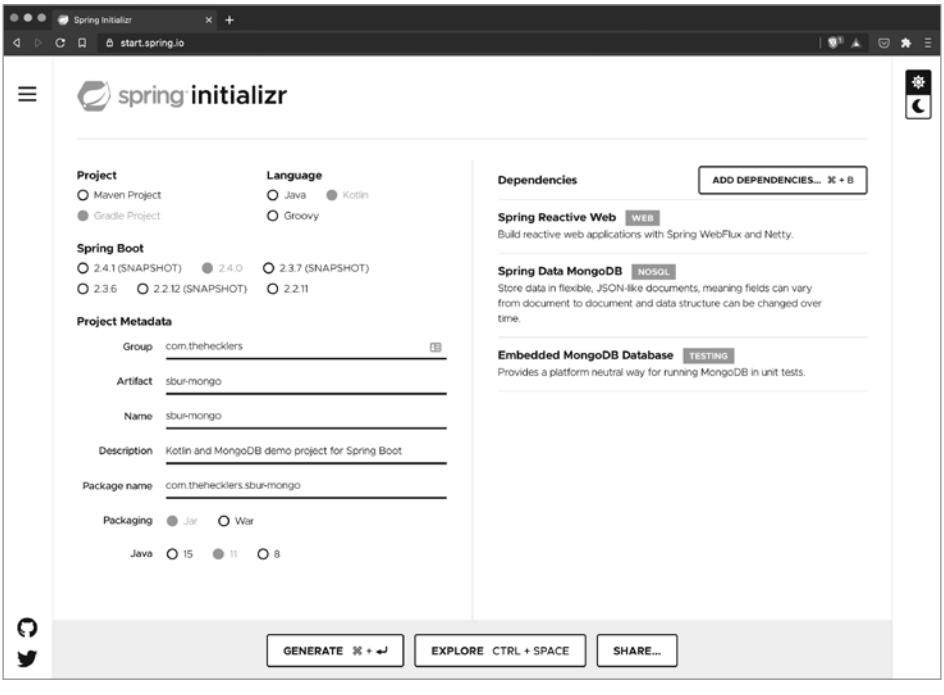


Рис. 6.1. Создание приложения Kotlin с помощью Spring Boot Initializr

Следует особо отметить несколько нюансов, связанных с выбранными мной вариантами: Gradle взят в качестве системы сборки этого проекта не случайно — если выбрать Gradle вместе с Kotlin в проекте Spring Boot, то файл сборки Gradle будет использовать DSL Kotlin, поддерживаемый командой Gradle наравне с DSL Groovy. Обратите внимание на то, что итоговый файл сборки называется `build.gradle.kts` — расширение `.kts` означает скрипт Kotlin, а не, наверное, привычный вам файл Gradle `build.gradle`. Maven также прекрасно подходит в качестве системы сборки для приложений Spring Boot + Kotlin, но как

декларативная система сборки на основе XML не использует непосредственно Kotlin или какой-либо другой язык.

Я воспользовался для этого приложения наличием стартового пакета Spring Boot для встраиваемой базы данных MongoDB. Поскольку встраиваемый экземпляр MongoDB предназначен исключительно для тестирования, не советую задействовать его для продакшена. Тем не менее он прекрасно подходит для демонстрации работы Spring Boot и Spring Data с MongoDB и с точки зрения разработчика обладает всеми возможностями развернутой на локальной машине базы данных, не требуя дополнительной установки и/или запуска контейнеризованного экземпляра MongoDB. Единственное, что нужно для использования встраиваемой базы данных из не предназначенного для тестирования кода, — заменить одну строку в файле `build.gradle.kts`, вот эту:

```
testImplementation("de.flapdoodle.embed:de.flapdoodle.embed.mongo")
```

на эту:

```
implementation("de.flapdoodle.embed:de.flapdoodle.embed.mongo")
```

Теперь мы готовы к созданию сервиса.

Разработка сервиса MongoDB

Как и в предыдущих примерах, сервис на основе MongoDB предлагает очень схожий подход, даже при том, что в качестве основного языка используется Kotlin, а не Java.

Описание класса предметной области

В качестве основного объекта данных этого проекта я создал класс предметной области `Aircraft` на языке Kotlin. Вот структура нового класса предметной области `Aircraft` и несколько последующих примечаний:

```
import com.fasterxml.jackson.annotation.JsonIgnoreProperties
import com.fasterxml.jackson.annotation.JsonProperty
import org.springframework.data.annotation.Id
import org.springframework.data.mongodb.core.mapping.Document
import java.time.Instant

@Document
@JsonIgnoreProperties(ignoreUnknown = true)
```

```
data class Aircraft(
    @Id val id: String,
    val callsign: String? = "",
    val squawk: String? = "",
    val reg: String? = "",
    val flightno: String? = "",
    val route: String? = "",
    val type: String? = "",
    val category: String? = "",
    val altitude: Int? = 0,
    val heading: Int? = 0,
    val speed: Int? = 0,
    @JsonProperty("vert_rate") val vertRate: Int? = 0,
    @JsonProperty("selected_altitude")
    val selectedAltitude: Int? = 0,
    val lat: Double? = 0.0,
    val lon: Double? = 0.0,
    val barometer: Double? = 0.0,
    @JsonProperty("polar_distance")
    val polarDistance: Double? = 0.0,
    @JsonProperty("polar_bearing")
    val polarBearing: Double? = 0.0,
    @JsonProperty("is_adsb")
    val isADSB: Boolean? = false,
    @JsonProperty("is_on_ground")
    val isOnGround: Boolean? = false,
    @JsonProperty("last_seen_time")
    val lastSeenTime: Instant? = Instant.ofEpochSecond(0),
    @JsonProperty("pos_update_time")
    val posUpdateTime: Instant? = Instant.ofEpochSecond(0),
    @JsonProperty("bds40_seen_time")
    val bds40SeenTime: Instant? = Instant.ofEpochSecond(0)
)
```

Прежде всего, стоит отметить отсутствие тут фигурных скобок, то есть у этого класса отсутствует тело. Это непривычно для тех, кому Kotlin внове, но если поместить в тело класса (интерфейса) нечего, фигурные скобки никакой пользы не принесут. Поэтому Kotlin их и не требует.

Второй интересный нюанс — множество операций присваивания в круглых скобках, стоящих непосредственно после имени класса. Для чего они служат?

Основной конструктор класса языка Kotlin часто записывают подобным образом — в заголовке класса, сразу за его названием. Вот пример полной формальной записи:

```
class Aircraft constructor(<параметр_1>,<параметр_2>,...,<параметр_n>)
```

Как часто бывает в языке Kotlin, четко различимые повторяющиеся шаблоны можно сжать. Удаление ключевого слова `constructor` перед списком параметров не позволяет перепутать его с какой-либо другой конструкцией языка, так что оно необязательно.

Внутри конструктора перечислены параметры. Параметр, перед которым указано ключевое слово `var` (в случае допускающих многократное присваивание изменяемых переменных) или `val` (в случае однократно присваиваемых значений, аналогичных терминальным (`final`) переменным языка Java), становится свойством. Свойство Kotlin по функциональности примерно эквивалентно переменной-члену языка Java, ее методу доступа и (при объявлении с ключевым словом `var`) методу изменения, вместе взятым.

Значения с типами, содержащими знак вопроса (`?`), например `Double?`, — необязательные, соответствующие параметры конструктора можно опустить. В таком случае параметру присваивается значение по умолчанию, указанное после знака равенства (`=`).

Параметры и свойства методов языка Kotlin, в том числе конструкторы, могут включать аннотации, как и их аналоги из языка Java. `@Id` и `@JsonProperty` выполняют те же функции, что и в приведенных ранее примерах на языке Java.

Относительно аннотаций уровня класса: `@Document` указывает MongoDB, что необходимо сохранять каждый объект типа `Aircraft` в виде отдельного документа в базе данных. Как и ранее, `@JsonIgnoreProperties(ignoreUnknown = true)` просто придает сервису `sbur_mongo` немного гибкости: если в какой-то момент в поток данных от расположенного выше по конвейеру сервиса `PlaneFinder` будут добавлены дополнительные поля, они станут игнорироваться и `sbur_mongo` продолжит работать без каких-либо проблем.

Последнее замечание касается предшествующего определению класса слова `data`. Распространенная практика — создавать классы предметной области, играющие в основном роль сегментов данных, которыми нужно манипулировать и/или которые следует передавать между процессами.

Подобная практика настолько широко распространена, что существует несколько способов создания так называемых классов данных. Например, функциональность `@Data` существует в Lombok уже многие годы.

Kotlin включил эту возможность в сам язык и добавил ключевое слово `data`, указывающее, что класс данных автоматически наследует от всех объявленных в основном конструкторе свойств:

- функции `equals()` и `hashCode()` (в языке Java — методы, в Kotlin — функции);

- `toString()`;
- по одной функции `componentN()` для каждого свойства в порядке их объявления;
- функцию `copy()`.

У классов данных Kotlin есть определенные требования и ограничения, но вполне обоснованные и минимальные. Подробности можно найти в документации Kotlin по классам данных (https://kotlinlang.org/docs/reference/data-classes.html#data-classes).



Еще одно интересное отличие — тип поля/свойства id местоположения каждого из воздушных судов. В Redis и JPA использовался тип Long, а в MongoDB для уникальных идентификаторов документов применяется тип String. Это не слишком важно, просто нужно иметь этот нюанс в виду.

Создание интерфейса репозитория

Далее я опишу необходимый интерфейс репозитория, расширяющий интерфейс `CrudRepository` Spring Data, и укажу тип хранимого объекта и его уникального идентификатора — `Aircraft` и `String`, как упоминалось ранее.

```
interface AircraftRepository: CrudRepository<Aircraft, String>
```

В этом лаконичном описании интерфейса интересны две вещи.

1. В языке Kotlin из-за отсутствия тела интерфейса не нужны фигурные скобки. Если ваша IDE добавляет их при создании этого интерфейса, можете спокойно их удалить.
2. В зависимости от контекста Kotlin использует двоеточие (`:`) для указания типа `val` или `var` или, как в данном случае, указания того, что интерфейс расширяет или реализует другой. В рассматриваемом случае мы описываем интерфейс `AircraftRepository`, расширяющий интерфейс `CrudRepository`.



Существует интерфейс MongoRepository, расширяющий как интерфейс PagingAndSortingRepository (он, в свою очередь, расширяет CrudRepository), так и QueryByExampleExecutor, который можно использовать вместо CrudRepository, как здесь. Но если никаких дополнительных возможностей не требуется, стоит писать интерфейс как можно более высокого уровня, удовлетворяющий всем требованиям. В данном случае интерфейса CrudRepository вполне достаточно.

Собираем все вместе

Следующий этап — создание компонента для периодического опроса сервиса `PlaneFinder`.

Опрос PlaneFinder. Как и в предыдущих примерах, я создал класс компонента `PlaneFinderPoller` Spring Boot для опроса на предмет данных о текущем местоположении `Aircraft` и обработки всех полученных записей:

```
import org.springframework.scheduling.annotation.EnableScheduling
import org.springframework.scheduling.annotation.Scheduled
import org.springframework.stereotype.Component
import org.springframework.web.reactive.function.client.WebClient
import org.springframework.web.reactive.function.client.bodyToFlux

@Component
@EnableScheduling
class PlaneFinderPoller(private val repository: AircraftRepository) {
    private val client =
        WebClient.create("http://localhost:7634/aircraft")

    @Scheduled(fixedRate = 1000)
    private fun pollPlanes() {
        repository.deleteAll()

        client.get()
            .retrieve()
            .bodyToFlux<Aircraft>()
            .filter { !it.reg.isNullOrEmpty() }
            .toStream()
            .forEach { repository.save(it) }

        println("--- All aircraft ---")
        repository.findAll().forEach { println(it) }
    }
}
```

В заголовке я создал основной конструктор с параметром `AircraftRepository`. Spring Boot автоматически связывает существующий компонент `AircraftRepository` в компонент `PlaneFinderPoller`, причем я пометил его ключевыми словами `private val`, чтобы обеспечить следующее:

- невозможность повторного присваивания ему в дальнейшем;
- его недоступность извне в качестве одного из свойств компонента `PlaneFinderPoller`, ведь репозиторий уже доступен для всего приложения.

Далее я создал объект `WebClient`, указывающий на целевую конечную точку, предоставляемую сервисом `PlaneFinder` через порт 7634, и присвоил его переменной экземпляра.

Также я снабдил данный класс аннотацией `@Component`, чтобы Spring Boot создал при запуске приложения соответствующий компонент, и аннотацией `@EnableScheduling` для включения периодических опросов с помощью следующей аннотированной функции.

И наконец, я создал функцию для удаления всех существующих данных `Aircraft`, опроса конечной точки `PlaneFinder` посредством клиентского свойства `WebClient`, преобразования и сохранения извлеченных данных о местоположении воздушных судов в MongoDB, а также их отображения. Аннотация `@Scheduled(fixedRate = 1000)` приводит к выполнению аннотированной функции каждые 1000 мс (каждую секунду).

Стоит отметить еще три интересных нюанса функции `pollPlanes()`, связанных с лямбда-выражениями Kotlin.

Во-первых, если лямбда-выражение — последний из параметров функции, то скобки можно опустить, так как они не добавляют дополнительной ясности или смысла. Естественно, в эту категорию попадают и функции, у которых только один параметр — лямбда-выражение. Благодаря этому уменьшается количество символов в порой перегруженных строках кода.

Во-вторых, если у самого лямбда-выражения только один параметр, разработчик может указать его явным образом, но не обязан это делать. Kotlin неявно распознает одиночный параметр лямбда-выражения и позволяет ссылаться на него по имени `it`, что тоже упрощает применение лямбда-выражений, как демонстрирует следующий параметр-лямбда `forEach()`:

```
forEach { repository.save(it) }
```

И в-третьих, функция `isNullOrEmpty()` интерфейса `CharSequence` — прекрасное универсальное средство для обработки `String`. Она сначала выполняет проверку на `null`, а затем, если значение оказалось не `null`, проверяет, не нулевая ли у него длина, то есть не пустое ли оно. Во многих случаях разработчик может обрабатывать свойство лишь тогда, когда в нем содержится настоящее значение, и эта функция выполняет обе проверки в один прием. При наличии значения в свойстве `reg` объекта `Aircraft` отчет о местоположении этого воздушного судна передается далее, отчеты же о местоположении воздушных судов, у которых отсутствуют регистрационные данные, отфильтровываются.

Все оставшиеся отчеты о местоположении воздушных судов передаются в репозитории для сохранения, после чего мы запрашиваем оттуда все сохраненные документы и отображаем результаты.

Результаты

Не отключая сервис `PlaneFinder`, я запустил сервис `sbur-mongo` для получения, сохранения и извлечения информации из встраиваемого экземпляра MongoDB, а также отображения на экране результатов каждого из опросов сервиса `PlaneFinder`. Далее приведен пример полученных результатов, отредактированный для краткости и немного отформатированный для удобства чтения:

```
Aircraft(id=95, callsign=N88846, squawk=4710, reg=N88846, flightno=, route=,
type=P46T, category=A1, altitude=18000, heading=234, speed=238, vertRate=-64,
selectedAltitude=0, lat=39.157288, lon=-90.844992, barometer=0.0,
polarDistance=33.5716, polarBearing=290.454061, isADSB=true, isOnGround=false,
lastSeenTime=2020-11-27T20:16:57Z, posUpdateTime=2020-11-27T20:16:57Z,
bds40SeenTime=1970-01-01T00:00:00Z)

Aircraft(id=96, callsign=MVJ710, squawk=1750, reg=N710MV, flightno=,
route=IAD-TEX, type=GLF4, category=A2, altitude=18050, heading=66, speed=362,
vertRate=2432, selectedAltitude=23008, lat=38.627655, lon=-90.008897,
barometer=0.0, polarDistance=20.976944, polarBearing=158.35465, isADSB=true,
isOnGround=false, lastSeenTime=2020-11-27T20:16:57Z,
posUpdateTime=2020-11-27T20:16:57Z, bds40SeenTime=2020-11-27T20:16:56Z)

Aircraft(id=97, callsign=SWA1121, squawk=6225, reg=N8654B, flightno=WN1121,
route=MDW-DAL-PHX, type=B738, category=A3, altitude=40000, heading=236,
speed=398, vertRate=0, selectedAltitude=40000, lat=39.58548, lon=-90.049259,
barometer=1013.6, polarDistance=38.411587, polarBearing=8.70042, isADSB=true,
isOnGround=false, lastSeenTime=2020-11-27T20:16:57Z,
posUpdateTime=2020-11-27T20:16:55Z, bds40SeenTime=2020-11-27T20:16:54Z)
```

Как и можно было ожидать, наш сервис без проблем запрашивает, захватывает и отображает местоположение воздушных судов, а благодаря Spring Boot, Kotlin и MongoDB это практически не требует усилий.

Создание сервиса на основе репозитория с помощью графовой базы данных NoSQL

У графовых баз иной подход к данным, в частности, к их взаимосвязям. На рынке есть несколько графовых баз данных, но лидер среди них во всех смыслах — Neo4j.

И хотя теория графов и архитектура графовых баз данных не относятся к предмету данной книги, демонстрация работы с графовой базой с помощью Spring Boot и Spring Data как раз входит в сферу ее рассмотрения. В этом разделе мы покажем, как легко подключиться к Spring Data Neo4j в приложении Spring Boot и работать с данными с ее помощью.

Инициализация проекта

И снова обращаемся к Spring Initializr. На этот раз я выбрал следующие опции:

- проект Gradle;
- Java;
- текущая стабильная версия Spring Boot;
- упаковка — JAR;
- Java — 11.

И зависимости:

- Spring Reactive Web (`spring-boot-starter-webflux`);
- Spring Data Neo4j (`spring-boot-starter-data-neo4j`).

Далее сгенерируем проект, сохраним его на локальной машине, разархивируем и откроем в IDE.

Я выбрал Gradle в качестве системы сборки этого проекта исключительно для демонстрации того, что при создании приложений Spring Boot Java с помощью Gradle сгенерированный файл сборки будет использовать DSL Groovy. Но вы можете применить и Maven.



Как и в большинстве примеров из этой главы, экземпляр Neo4j запущен в контейнере на локальной машине и готов реагировать на запросы приложения.

Разрабатываем сервис Neo4j

Как и в предыдущих примерах, благодаря Spring Boot и Spring Data работа с базами данных Neo4j практически не отличается от работы с другими типами хранилищ данных. Из приложений Spring Boot легко получить доступ к ши-

роким возможностям графового хранилища данных, а предварительных работ требуется намного меньше.

Определение класса предметной области

И снова мы начнем с определения предметной области `Aircraft`. Без зависимости Lombok я создам его с обычным обширным списком конструкторов, методов доступа и изменения и вспомогательных методов:

```
import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import com.fasterxml.jackson.annotation.JsonProperty;
import org.springframework.data.neo4j.core.schema.GeneratedValue;
import org.springframework.data.neo4j.core.schema.Id;
import org.springframework.data.neo4j.core.schema.Node;

@Node
@JsonIgnoreProperties(ignoreUnknown = true)
public class Aircraft {
    @Id
    @GeneratedValue
    private Long neoId;

    private Long id;
    private String callsign, squawk, reg, flightno, route, type, category;

    private int altitude, heading, speed;
    @JsonProperty("vert_rate")
    private int vertRate;
    @JsonProperty("selected_altitude")
    private int selectedAltitude;

    private double lat, lon, barometer;
    @JsonProperty("polar_distance")
    private double polarDistance;
    @JsonProperty("polar_bearing")
    private double polarBearing;

    @JsonProperty("is_adsb")
    private boolean isADSB;
    @JsonProperty("is_on_ground")
    private boolean isOnGround;

    @JsonProperty("last_seen_time")
    private Instant lastSeenTime;
    @JsonProperty("pos_update_time")
    private Instant posUpdateTime;
    @JsonProperty("bds40_seen_time")
    private Instant bds40SeenTime;
```

```
public Aircraft() {
}

public Aircraft(Long id,
                String callsign, String squawk, String reg, String flightno,
                String route, String type, String category,
                int altitude, int heading, int speed,
                int vertRate, int selectedAltitude,
                double lat, double lon, double barometer,
                double polarDistance, double polarBearing,
                boolean isADSB, boolean isOnGround,
                Instant lastSeenTime,
                Instant posUpdateTime,
                Instant bds40SeenTime) {
    this.id = id;
    this.callsign = callsign;
    this.squawk = squawk;
    this.reg = reg;
    this.flightno = flightno;
    this.route = route;
    this.type = type;
    this.category = category;
    this.altitude = altitude;
    this.heading = heading;
    this.speed = speed;
    this.vertRate = vertRate;
    this.selectedAltitude = selectedAltitude;
    this.lat = lat;
    this.lon = lon;
    this.barometer = barometer;
    this.polarDistance = polarDistance;
    this.polarBearing = polarBearing;
    this.isADSB = isADSB;
    this.isOnGround = isOnGround;
    this.lastSeenTime = lastSeenTime;
    this.posUpdateTime = posUpdateTime;
    this.bds40SeenTime = bds40SeenTime;
}

public Long getNeoId() {
    return neoId;
}

public void setNeoId(Long neoId) {
    this.neoId = neoId;
}

public Long getId() {
    return id;
}
```

```
public void setId(Long id) {
    this.id = id;
}

public String getCallsign() {
    return callsign;
}

public void setCallsign(String callsign) {
    this.callsign = callsign;
}

public String getSquawk() {
    return squawk;
}

public void setSquawk(String squawk) {
    this.squawk = squawk;
}

public String getReg() {
    return reg;
}

public void setReg(String reg) {
    this.reg = reg;
}

public String getFlightno() {
    return flightno;
}

public void setFlightno(String flightno) {
    this.flightno = flightno;
}

public String getRoute() {
    return route;
}

public void setRoute(String route) {
    this.route = route;
}

public String getType() {
    return type;
}

public void setType(String type) {
```

```
    this.type = type;
}

public String getCategory() {
    return category;
}

public void setCategory(String category) {
    this.category = category;
}

public int getAltitude() {
    return altitude;
}

public void setAltitude(int altitude) {
    this.altitude = altitude;
}

public int getHeading() {
    return heading;
}

public void setHeading(int heading) {
    this.heading = heading;
}

public int getSpeed() {
    return speed;
}

public void setSpeed(int speed) {
    this.speed = speed;
}

public int getVertRate() {
    return vertRate;
}

public void setVertRate(int vertRate) {
    this.vertRate = vertRate;
}

public int getSelectedAltitude() {
    return selectedAltitude;
}

public void setSelectedAltitude(int selectedAltitude) {
    this.selectedAltitude = selectedAltitude;
```

```
}

public double getLat() {
    return lat;
}

public void setLat(double lat) {
    this.lat = lat;
}

public double getLon() {
    return lon;
}

public void setLon(double lon) {
    this.lon = lon;
}

public double getBarometer() {
    return barometer;
}

public void setBarometer(double barometer) {
    this.barometer = barometer;
}

public double getPolarDistance() {
    return polarDistance;
}

public void setPolarDistance(double polarDistance) {
    this.polarDistance = polarDistance;
}

public double getPolarBearing() {
    return polarBearing;
}

public void setPolarBearing(double polarBearing) {
    this.polarBearing = polarBearing;
}

public boolean isADSB() {
    return isADSB;
}

public void setADSB(boolean ADSB) {
    isADSB = ADSB;
}
```

```
public boolean isOnGround() {
    return isOnGround;
}

public void setOnGround(boolean onGround) {
    isOnGround = onGround;
}

public Instant getLastSeenTime() {
    return lastSeenTime;
}

public void setLastSeenTime(Instant lastSeenTime) {
    this.lastSeenTime = lastSeenTime;
}

public Instant getPosUpdateTime() {
    return posUpdateTime;
}

public void setPosUpdateTime(Instant posUpdateTime) {
    this.posUpdateTime = posUpdateTime;
}

public Instant getBds40SeenTime() {
    return bds40SeenTime;
}

public void setBds40SeenTime(Instant bds40SeenTime) {
    this.bds40SeenTime = bds40SeenTime;
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Aircraft aircraft = (Aircraft) o;
    return altitude == aircraft.altitude &&
            heading == aircraft.heading &&
            speed == aircraft.speed &&
            vertRate == aircraft.vertRate &&
            selectedAltitude == aircraft.selectedAltitude &&
            Double.compare(aircraft.lat, lat) == 0 &&
            Double.compare(aircraft.lon, lon) == 0 &&
            Double.compare(aircraft.barometer, barometer) == 0 &&
            Double.compare(aircraft.polarDistance, polarDistance) == 0 &&
            Double.compare(aircraft.polarBearing, polarBearing) == 0 &&
            isADSB == aircraft.isADSB &&
            isOnGround == aircraft.isOnGround &&
```

```
                Objects.equals(neoId, aircraft.neoId) &&
                Objects.equals(id, aircraft.id) &&
                Objects.equals(callsign, aircraft.callsign) &&
                Objects.equals(squawk, aircraft.squawk) &&
                Objects.equals(reg, aircraft.reg) &&
                Objects.equals(flightno, aircraft.flightno) &&
                Objects.equals(route, aircraft.route) &&
                Objects.equals(type, aircraft.type) &&
                Objects.equals(category, aircraft.category) &&
                Objects.equals(lastSeenTime, aircraft.lastSeenTime) &&
                Objects.equals(posUpdateTime, aircraft.posUpdateTime) &&
                Objects.equals(bds40SeenTime, aircraft.bds40SeenTime);
    }

    @Override
    public int hashCode() {
        return Objects.hash(neoId, id, callsign, squawk, reg, flightno, route,
                type, category, altitude, heading, speed, vertRate,
                selectedAltitude,  lat, lon, barometer, polarDistance,
                polarBearing, isADSB, isOnGround, lastSeenTime, posUpdateTime,
                bds40SeenTime);
    }

    @Override
    public String toString() {
        return "Aircraft{" +
                "neoId=" + neoId +
                ", id=" + id +
                ", callsign=’" + callsign + '\'' +
                ", squawk=’" + squawk + '\'' +
                ", reg=’" + reg + '\'' +
                ", flightno=’" + flightno + '\'' +
                ", route=’" + route + '\'' +
                ", type=’" + type + '\'' +
                ", category=’" + category + '\'' +
                ", altitude=" + altitude +
                ", heading=" + heading +
                ", speed=" + speed +
                ", vertRate=" + vertRate +
                ", selectedAltitude=" + selectedAltitude +
                ", lat=" + lat +
                ", lon=" + lon +
                ", barometer=" + barometer +
                ", polarDistance=" + polarDistance +
                ", polarBearing=" + polarBearing +
                ", isADSB=" + isADSB +
                ", isOnGround=" + isOnGround +
                ", lastSeenTime=" + lastSeenTime +
```

```
                ", posUpdateTime=" + posUpdateTime +
                ", bds40SeenTime=" + bds40SeenTime +
                '}';
    }
}
```

Да, код на языке Java бывает очень длинным. Честно говоря, это не такая уж большая проблема в случае классов предметной области, поскольку основное место занимают методы доступа и изменения, а их может сгенерировать IDE, да и особого сопровождения они не требуют, ведь меняются редко. Тем не менее это *изрядное* количество стереотипного кода, почему многие разработчики и предпочитают такие решения, как Lombok или Kotlin, пусть лишь для создания классов предметной области на Kotlin для Java-приложений.



Neo4j требуется генерируемый базой данных уникальный идентификатор, даже если сохраняемая сущность уже содержит уникальный идентификатор. Чтобы удовлетворить это требование, я добавил параметр/переменную-член neoId и снабдил ее аннотациями @Id и @GeneratedValue, чтобы Neo4j правильно связала эту переменную-член класса со сгенерированным внутренним значением.

Далее я добавил две аннотации уровня класса:

- `@Node` — указывает, что каждый экземпляр этой `record` является экземпляром узла Neo4j;
- `@JsonIgnoreProperties(ignoreUnknown = true)` — игнорирование новых полей, возможно, добавленных в поток данных конечной точкой сервиса `PlaneFinder`.

Обратите внимание на то, что, подобно `@Id` и `@GeneratedValue`, аннотация `@Node` взята из пакета `org.springframework.data.neo4j.core.schema`, предназначенного для приложений Spring Data, основанных на Neo4j.

На этом описание предметной области нашего сервиса завершено.

Создание интерфейса репозитория

Далее я снова опишу необходимый интерфейс репозитория, расширяющий интерфейс `CrudRepository` Spring Data, и укажу тип хранимого объекта и его ключа — `Aircraft` и `Long` в данном случае:

```
public interface AircraftRepository extends CrudRepository<Aircraft, Long> {}
```



Подобно предыдущему проекту на основе MongoDB, существует интерфейс Neo4jRepository, расширяющий интерфейс PagingAndSortingRepository (он, в свою очередь, расширяет CrudRepository), который можно использовать вместо CrudRepository, но поскольку CrudRepository — интерфейс самого высокого уровня из числа удовлетворяющих всем требованиям, то его я и беру в качестве базового для AircraftRepository.

Собираем все вместе

Следующий этап — создание компонента для опроса сервиса `PlaneFinder` и настройка его доступа к базе данных Neo4j.

Опрос PlaneFinder. И снова я создал класс Spring Boot с аннотацией `@Component` для опроса данных о текущем местоположении воздушных судов и обработки всех полученных записей `Aircraft`.

Подобно прочим проектам на основе Java в этой главе, я создал объект `WebClient`, указывающий на целевую конечную точку, предоставляемую сервисом `PlaneFinder` через порт 7634, и присвоил его переменной-члену.

Поскольку зависимости Lombok нет, я создал конструктор для получения автоматически связываемого компонента `AircraftRepository`.

Как показано в следующем полном листинге класса `PlaneFinderPoller`, метод `pollPlanes()` выглядит практически идентично прочим примерам из-за абстракций, задействованных благодаря поддержке репозиториев. Все прочие нюансы оставшегося кода класса `PlaneFinderPoller` вы можете найти в соответствующих частях предыдущих разделов:

```
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;

@EnableScheduling
@Component
public class PlaneFinderPoller {
    private WebClient client =
            WebClient.create("http://localhost:7634/aircraft");
    private final AircraftRepository repository;

    public PlaneFinderPoller(AircraftRepository repository) {
        this.repository = repository;
    }
```

```
    @Scheduled(fixedRate = 1000)
    private void pollPlanes() {
        repository.deleteAll();

        client.get()
                .retrieve()
                .bodyToFlux(Aircraft.class)
                .filter(plane -> !plane.getReg().isEmpty())
                .toStream()
                .forEach(repository::save);

        System.out.println("--- All aircraft ---");
        repository.findAll().forEach(System.out::println);
    }
}
```

Соединение с Neo4j. Как и в случае с MariaDB/MySQL, необходимо указать Spring Boot кое-какую информацию для беспроблемного соединения с базой данных Neo4j. В моей среде для работы этого сервиса необходимо указать следующие свойства:

```
spring.neo4j.authentication.username=neo4j
spring.neo4j.authentication.password=mkheck
```



Замените имя пользователя и пароль на соответствующие вашей среде.

Результаты

Не отключая сервис `PlaneFinder`, я запустил сервис `sbur-neo` для получения, сохранения и извлечения информации, а также отображения на экране результатов каждого из опросов сервиса `PlaneFinder` с Neo4j в качестве базы данных. Далее приведен пример полученных результатов, отредактированный для краткости и немного отформатированный для удобства чтения:

```
Aircraft(neoId=64, id=223, callsign='GJS4401', squawk='1355', reg='N542GJ',
flightno='UA4401', route='LIT-ORD', type='CRJ7', category='A2', altitude=37000,
heading=24, speed=476, vertRate=128, selectedAltitude=36992, lat=39.463961,
lon=-90.549927, barometer=1012.8, polarDistance=35.299257,
polarBearing=329.354686, isADSB=true, isOnGround=false,
lastSeenTime=2020-11-27T20:42:54Z, posUpdateTime=2020-11-27T20:42:53Z,
bds40SeenTime=2020-11-27T20:42:51Z)
```

```
Aircraft(neoId=65, id=224, callsign='N8680B', squawk='1200', reg='N8680B',
flightno='', route='', type='C172', category='A1', altitude=3100, heading=114,
speed=97, vertRate=64, selectedAltitude=0, lat=38.923955, lon=-90.195618,
barometer=0.0, polarDistance=1.986086, polarBearing=208.977102, isADSB=true,
isOnGround=false, lastSeenTime=2020-11-27T20:42:54Z,
posUpdateTime=2020-11-27T20:42:54Z, bds40SeenTime=null)

Aircraft(neoId=66, id=225, callsign='AAL1087', squawk='1712', reg='N181UW',
flightno='AA1087', route='CLT-STL-CLT', type='A321', category='A3',
altitude=7850, heading=278, speed=278, vertRate=-320, selectedAltitude=4992,
lat=38.801559, lon=-90.226474, barometer=0.0, polarDistance=9.385111,
polarBearing=194.034005, isADSB=true, isOnGround=false,
lastSeenTime=2020-11-27T20:42:54Z, posUpdateTime=2020-11-27T20:42:53Z,
bds40SeenTime=2020-11-27T20:42:53Z)
```

Сервис работает быстро и эффективно, используя Spring Boot и Neo4j для извлечения, захвата и отображения местоположения воздушных судов по мере получения отчетов о них.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Полный код для этой главы находится в ветке chapter6end в репозитории кода.

Резюме

Данные — непростая тема, к которой относятся бесчисленные переменные и ограничения, включая структуры данных, отношения, соответствующие стандарты, поставщики, механизмы и многое другое. И все же без данных в какомлибо виде большинство приложений практически не имеют смысла.

Данные, в которых заключается почти вся ценность приложений, привлекли немало инноваций от поставщиков баз данных и платформ. Однако во многих случаях сложность не уменьшается, и разработчикам приходится справляться с ней, чтобы реализовать потенциал данных.

Заявленная цель Spring Data — обеспечить привычную и единообразную модель программирования на основе Spring для доступа к данным с сохранением при этом всех характеристик используемого хранилища данных. Вне зависимости от движка базы данных или платформы, цель Spring Data — сделать доступ к данным для разработчика максимально простым и полнофункциональным.

Эта глава демонстрирует упрощение хранения и извлечения данных с помощью различных вариантов баз данных, а также проектов и возможностей Spring Data, позволяющих использовать их наиболее простым и полнофункциональным способом — с помощью Spring Boot.

В следующей главе я покажу, как создавать важнейшие приложения с помощью REST-взаимодействия Spring MVC, платформ обмена сообщениями и прочих механизмов обмена информацией. Также в ней вас ждет введение в поддержку языка шаблонов. И если эта глава была сосредоточена на фундаменте приложения — данных, то в главе 7 мы рассмотрим внешние взаимодействия приложений.

ГЛАВА 7

Создание приложений с помощью Spring MVC

Из этой главы вы узнаете, как с помощью Spring MVC создавать приложения Spring Boot, использующие REST-взаимодействия, платформы обмена сообщениями и другие механизмы связи. Также она расскажет о поддержке языка шаблонов. И хотя взаимодействия между сервисами уже описывались в ходе изучения в предыдущей главе множества возможностей Spring Boot по обработке данных, в этой главе фокус внимания смещается с самого приложения на окружающий его мир — его взаимодействие с другими приложениями и/или сервисами, а также конечными пользователями.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Пожалуйста, для начала извлеките из репозитория кода ветку chapter7begin.

Что такое Spring MVC

Как и многое другое в данной сфере, термин *Spring MVC* многозначен. Под Spring MVC может пониматься любое из следующих понятий.

- Реализация каким-либо образом паттерна «модель — представление — контроллер» в приложении Spring.
- Создание приложения на основе компонентов Spring MVC, например интерфейса `Model`, классов `@Controller` и различных технологий представлений.
- Разработка блокирующих/нереактивных приложений с помощью Spring.

В зависимости от контекста Spring MVC можно считать как подходом, так и реализацией. Его можно применять как в Spring Boot, так и вне его. Обсуждение применения паттерна MVC с помощью Spring, как и использование Spring MVC вне Spring Boot, выходит за рамки темы данной книги. Я сосредоточусь на реализации двух последних из перечисленных идей с помощью Spring Boot.

Взаимодействия конечного пользователя с помощью шаблонизаторов

Хотя приложения Spring Boot выполняют немало сложных прикладных задач, Spring Boot допускает и непосредственные взаимодействия конечных пользователей. И хотя Spring Boot все еще поддерживает давно устоявшиеся стандарты, например Java Server Pages (JSP), для старых приложений, большинство современных приложений либо использует более функциональные технологии представлений, поддерживаемые непрерывно развивающимися шаблонизаторами, либо переводит разработку клиентской части на HTML и JavaScript. Можно даже сочетать эти два варианта, используя сильные стороны каждого.

Spring Boot прекрасно работает с интерфейсами HTML и JavaScript, как я покажу далее в этой главе. А пока рассмотрим подробнее шаблонизаторы.

Шаблонизаторы позволяют приложению, работающему на стороне сервера, генерировать конечные страницы для отображения и выполнения в браузере конечного пользователя. Эти технологии представлений могут различаться подходами, но в общем случае предоставляют следующие возможности:

- язык шаблонов и/или набор тегов, описывающих входные данные шаблонизатора, используемые обработчиком шаблонов для формирования нужных результатов;
- распознаватель представлений, определяющий, какое представление/шаблон использовать для воплощения запрашиваемого ресурса.

Помимо прочих, реже используемых вариантов, Spring Boot поддерживает такие технологии представлений, как Thymeleaf (https://www.thymeleaf.org), FreeMarker (https://freemarker.apache.org), Groovy Markup (http://groovy-lang.org/templating.html) и Mustache (https://mustache.github.io). Вероятно, по нескольким причинам

наиболее широко применяется Thymeleaf, прекрасно поддерживающий приложения Spring MVC и Spring WebFlux.

В Thymeleaf используются естественные шаблоны — файлы, включающие элементы кода, которые, тем не менее, можно открыть и просмотреть в любом стандартном браузере. Возможность просматривать файлы шаблонов как HTML позволяет разработчикам и архитекторам создавать и развивать шаблоны Thymeleaf без запуска каких-либо серверных процессов. Все взаимодействия кода, для которых требуются соответствующие серверные элементы, помечаются как ориентированные на Thymeleaf, отсутствующие части просто не отображаются.

Давайте на основе предыдущих наработок создадим с помощью Spring Boot, Spring MVC и Thymeleaf простое веб-приложение, предоставляющее конечному пользователю интерфейс, который позволяет запрашивать у сервиса PlaneFinder текущее местоположение воздушных судов и отображать результаты. Изначально оно будет представлять собой всего лишь простейшую пробную версию, которую мы станем развивать в последующих главах.

Инициализация проекта

Для начала обратимся к Spring Initializr. Там я выбрал следующие опции:

- проект Maven;
- Java;
- текущая стабильная версия Spring Boot;
- упаковка — JAR;
- Java — 11.

И зависимости:

- Spring Web (`spring-boot-starter-web`);
- Spring Reactive Web (`spring-boot-starter-webflux`);
- Thymeleaf (`spring-boot-starter-thymeleaf`);
- Spring Data JPA (`spring-boot-starter-data-jpa`);
- H2 Database (`h2`);
- Lombok (`lombok`).

Далее генерируем проект, сохраняем его на локальной машине, разархивируем и открываем в IDE.

Разработка приложения Aircraft Positions

Поскольку это приложение учитывает только текущее состояние — местоположения воздушных судов в момент запроса, а не исторические данные, для него подойдет база данных в оперативной памяти. Конечно, вместо нее можно воспользоваться каким-либо `Iterable`, но для нашего сценария вполне достаточно репозиториев Spring Data и базы данных H2, поддерживаемых Spring Boot, и остается пространство для дальнейшего расширения приложения.

Описание класса предметной области

Как и для всех прочих проектов, взаимодействующих с `PlaneFinder`, я создам класс предметной области `Aircraft` в качестве основного средоточия данных. Вот структура предметной области `Aircraft` для приложения `Aircraft Positions`:

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Aircraft {
    @Id
    private Long id;
    private String callsign, squawk, reg, flightno, route, type, category;

    private int altitude, heading, speed;
    @JsonProperty("vert_rate")
    private int vertRate;
    @JsonProperty("selected_altitude")
    private int selectedAltitude;

    private double lat, lon, barometer;
    @JsonProperty("polar_distance")
    private double polarDistance;
    @JsonProperty("polar_bearing")
    private double polarBearing;

    @JsonProperty("is_adsb")
    private boolean isADSB;
    @JsonProperty("is_on_ground")
    private boolean isOnGround;

    @JsonProperty("last_seen_time")
    private Instant lastSeenTime;
    @JsonProperty("pos_update_time")
    private Instant posUpdateTime;
    @JsonProperty("bds40_seen_time")
    private Instant bds40SeenTime;
}
```

Мы описали этот класс предметной области с помощью JPA и H2 в качестве нижележащей JPA-совместимой базы данных, а также воспользовались Lombok для создания класса данных с конструкторами — без аргументов и со всеми аргументами, по одному для каждой переменной-члена.

Создание интерфейса репозитория

Далее я опишу необходимый интерфейс репозитория, расширяя `CrudRepository` из Spring Data и указывая тип хранимого объекта и его ключи — `Aircraft` и `Long` в данном случае:

```
public interface AircraftRepository extends CrudRepository<Aircraft, Long> {}
```

Объекты Model и Controller

Данные модели были описаны в классе предметной области `Aircraft`, пора включить их в `Model` и предоставить пользователям доступ к ним посредством `Controller`.

Как обсуждалось в главе 3, `@RestController` — удобная форма записи, сочетающая `@Controller` и `@ResponseBody` в одной наглядной аннотации, возвращающей форматированный ответ в нотации объектов JavaScript (JavaScript Object Notation, JSON) или каком-либо другом формате, ориентированном на данные. В результате *все тело* ответа на веб-запрос возвращается методом в виде значения Object/Iterable, а не как часть `Model`. `@RestController` позволяет создать API — узкоспециализированный, но очень распространенный сценарий использования.

Теперь наша цель — создать приложение, включающее интерфейс пользователя, а это возможно благодаря `@Controller`. В классе `@Controller` каждый метод, снабженный аннотацией `@RequestMapping` или одной из ее специализированных версий, будет возвращать значение типа `String`, соответствующее названию файла шаблона без расширения. Например, расширение файлов Thymeleaf — `.html`, так что метод `@GetMapping` класса `@Controller` вернет строковое значение `"myfavoritepage"`, и шаблонизатор Thymeleaf будет использовать шаблон `myfavoritepage.html` для генерации страницы и возврата ее браузеру пользователя.



Шаблоны технологий представления по умолчанию размещаются в каталоге src/main/resources/templates проекта. Именно там шаблонизатор будет их искать, если не переопределить поведение с помощью свойств приложения или программным способом.

Возвращаясь к контроллеру, я создал следующий класс `PositionController`:

```
@RequiredArgsConstructor
@Controller
public class PositionController {
    @NonNull
    private final AircraftRepository repository;
    private WebClient client =
            WebClient.create("http://localhost:7634/aircraft");

    @GetMapping("/aircraft")
    public String getCurrentAircraftPositions(Model model) {
        repository.deleteAll();

        client.get()
                .retrieve()
                .bodyToFlux(Aircraft.class)
                .filter(plane -> !plane.getReg().isEmpty())
                .toStream()
                .forEach(repository::save);

        model.addAttribute("currentPositions", repository.findAll());
        return "positions";
    }
}
```

Этот контроллер очень похож на предыдущие варианты, но имеет несколько важных отличий. Во-первых, конечно, вместо `@RestController` здесь используется обсуждавшаяся ранее аннотация `@Controller`. Во-вторых, у метода `getCurrentAircraftPositions()` появился автоматически связываемый параметр `Model model` — компонент модели, используемый шаблонизатором для предоставления доступа к компонентам приложения — их данным и операциям — после добавления этих компонентов в `Model` в качестве атрибутов. И в-третьих, метод возвращает `String` вместо типа класса и включает настоящий оператор возврата с названием шаблона без расширения `.html`.



В сложных предметных областях/приложениях я предпочитаю дополнительно разделять ответственность, создавая отдельные классы @Service и @Controller. В этом примере всего один метод, обращающийся к одному репозиторию, так что я поместил внутрь контроллера всю функциональность для заполнения исходных данных, заполнения Model и ее передачи соответствующему View.

Создание требуемых файлов представления

В качестве основы для этой и следующих глав я создал один файл с обычным кодом HTML и один файл шаблона.

Поскольку мы собираемся показывать всем посетителям страницу с простым HTML и поскольку для нее не требуется поддержки шаблонов, я поместил файл `index.html` непосредственно в каталог `src/main/resources/static` проекта:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Retrieve Aircraft Position Report</title>
</head>
<body>
    <p><a href="/aircraft">Click here</a>
        to retrieve current aircraft positions in range of receiver.</p>
</body>
</html>
```

ПРИМЕЧАНИЯ ОТНОСИТЕЛЬНО ФАЙЛА INDEX.HTML

По умолчанию приложения Spring Boot ищут статические страницы по пути к классам, в подкаталогах `static` и `public`. Чтобы они попали туда во время сборки, их необходимо поместить в один из этих двух каталогов в каталоге `src/main/resources` внутри проекта.

Особенно интересна в контексте нашего приложения гиперссылка `href "/aircraft"`. Она соответствует аннотации `@GetMapping` метода `PositionController getCurrentAircraftPositions()` и указывает на открываемую им для доступа конечную точку — еще один пример внутренней интеграции Spring Boot различных компонентов в приложении. При щелчке на надписи Click here на отображаемой запущенным приложением странице выполняется метод `getCurrentAircraftPositions()` и возвращает `"positions"`, в результате чего `ViewResolver` генерирует и возвращает следующую страницу, основанную на шаблоне `positions.html`.

И последнее примечание: если Spring Boot находит файл `index.html` в одном из просматриваемых каталогов, то автоматически загружает его для пользователя при обращении из браузера или другого агента пользователя по адресу `host:port` приложения, не требуя от разработчика каких-либо дополнительных настроек.

Для динамического контента я создал файл шаблона, добавив в обычный во всех прочих отношениях HTML-файл пространство имен XML для тегов Thymeleaf, которые в дальнейшем играют роль инструкций по внедрению контента для шаблонизатора Thymeleaf, как показано в следующем файле `positions.html`. Чтобы отметить этот файл как файл шаблона для дальнейшей обработки шаблонизатором, я поместил его в каталог `src/main/resources/templates` проекта:

```
<!DOCTYPE HTML>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Position Report</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8"/>
</head>
<body>
<div class="positionlist" th:unless="${#lists.isEmpty(currentPositions)}">

    <h2>Current Aircraft Positions</h2>

    <table>
        <thead>
        <tr>
            <th>Call Sign</th>
            <th>Squawk</th>
            <th>AC Reg</th>
            <th>Flight #</th>
            <th>Route</th>
            <th>AC Type</th>
            <th>Altitude</th>
            <th>Heading</th>
            <th>Speed</th>
            <th>Vert Rate</th>
            <th>Latitude</th>
            <th>Longitude</th>
            <th>Last Seen</th>
            <th></th>
        </tr>
        </thead>
        <tbody>
        <tr th:each="ac : ${currentPositions}">
            <td th:text="${ac.callsign}"></td>
            <td th:text="${ac.squawk}"></td>
            <td th:text="${ac.reg}"></td>
            <td th:text="${ac.flightno}"></td>
            <td th:text="${ac.route}"></td>
            <td th:text="${ac.type}"></td>
            <td th:text="${ac.altitude}"></td>
            <td th:text="${ac.heading}"></td>
            <td th:text="${ac.speed}"></td>
            <td th:text="${ac.vertRate}"></td>
            <td th:text="${ac.lat}"></td>
            <td th:text="${ac.lon}"></td>
            <td th:text="${ac.lastSeenTime}"></td>
        </tr>
        </tbody>
    </table>
</div>
</body>
</html>
```

Для страницы Aircraft Position Report (отчет о местоположении воздушных судов) я сократил отображаемую информацию до нескольких избранных элементов, особенно важных и интересных. Следует особо отметить несколько мест в шаблоне Thymeleaf `positions.html`.

Вначале, как уже упоминалось, я добавил в пространство имен XML теги Thymeleaf с префиксом `th` с помощью следующей строки кода:

```
<html lang="en" xmlns:th="http://www.thymeleaf.org">
```

При определении раздела (`division`), предназначенного для отображения текущего местоположения воздушных судов, я указал, что показывать раздел `positionList` нужно только при наличии данных, а если элемент `currentPositions` внутри `Model` пуст, весь раздел просто опускается:

```
<div class="positionlist" th:unless="${#lists.isEmpty(currentPositions)}">
```

Наконец, я определяю таблицу с помощью стандартных тегов таблиц HTML — для самой таблицы, строки заголовка и ее содержимого. Чтобы вывести тело таблицы, я воспользовался оператором `each` Thymeleaf для прохода в цикле по всем элементам `currentPositions` и тегом `text` Thymeleaf для заполнения столбцов в каждой из строк. При этом я ссылался на свойства объектов местоположения посредством синтаксиса динамических выражений `"${ object.property}"`. Теперь все готово для проверки приложения.

Результаты

Не отключая сервис `PlaneFinder`, я запустил приложение `Aircraft Positions` из IDE. После этого открыл вкладку в браузере, ввел `localhost:8080` в адресной строке и нажал Enter. На рис. 7.1 приведена полученная в результате страница.



Рис. 7.1. Очень простая целевая страница приложения Aircraft Positions

На этой странице я выбрал ссылку Click here, чтобы перейти на страницу Aircraft Position Report (рис. 7.2).

При обновлении страницы выполняется повторный запрос к сервису PlaneFinder и в отчет включаются текущие данные.

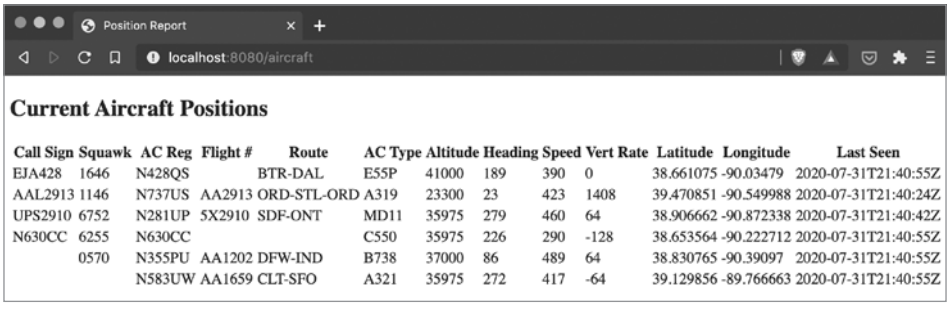


Рис. 7.2. Страница отчета о местоположениях воздушных судов

Трюк с обновлением

Возможность запрашивать список воздушных судов, находящихся в настоящий момент в области досягаемости, и данных об их точном местоположении очень удобна. Но необходимость вручную обновлять страницу сильно утомляет, к тому же есть риск пропустить важные данные. Чтобы включить функцию регулярного обновления шаблона Aircraft Position Report, просто добавьте в раздел body страницы функцию JavaScript следующего вида, указав частоту обновления в миллисекундах:

```
<script type="text/javascript">
    window.onload = setupRefresh;

    function setupRefresh() {
        setTimeout("refreshPage();", 5000); // частота обновления в миллисекундах
    }

    function refreshPage() {
        window.location = location.href;
    }
</script>
```

Шаблонизатор Thymeleaf передаст этот код в сгенерированную страницу в неизменном виде, а браузер пользователя будет выполнять сценарий с указанной частотой. Не самое изящное решение, но для простых сценариев использования вполне подходящее.

Передача сообщений

Для более серьезных сценариев использования могут потребоваться более сложные решения. Хотя предыдущий код и обеспечивает динамические обнов-

ления, отражающие текущие данные о местоположении воздушных судов, но его потенциальная проблема — значительный объем передаваемой информации, обусловленный периодическим выполнением запросов актуальных данных. Объем сетевого трафика при непрерывных запросах и получении обновлений несколькими клиентами может оказаться существенным.

Для реализации более сложных сценариев использования, где учитываются сетевые требования, может оказаться полезным перейти от модели вытягивания (pull model) к модели выталкивания (push model) или какому-либо их сочетанию.



В этом и следующем разделах обсудим два различных шага по направлению к модели выталкивания, сделав которые мы в конце концов придем к модели чистого выталкивания данных из сервиса PlaneFinder наружу. Сценарии использования будут указывать на условия, благоприятствующие одному из этих подходов либо какому-то совершенно иному, или диктовать их. А я продолжу исследования и буду демонстрировать альтернативные варианты в последующих главах, так что оставайтесь с нами!

Платформы обмена сообщениями предназначены для эффективного получения сообщений, их маршрутизации и доставки от одного приложения другому. В число их примеров входят RabbitMQ (https://www.rabbitmq.com), Apache Kafka (https://kafka.apache.org) и еще множество продуктов как с открытым исходным кодом, так и коммерческих. Spring Boot и экосистема Spring предлагают несколько различных вариантов для работы с конвейерами сообщений, но мой несомненный фаворит — Spring Cloud Stream.

Spring Cloud Stream поднимает уровень абстракции для разработчиков, предоставляя в то же время доступ к уникальным атрибутам поддерживаемых платформ через свойства приложения, компоненты и непосредственную конфигурацию. `Binder` (адаптер привязки) формирует связь между драйверами потоковой платформы и Spring Cloud Stream (SCSt), позволяя разработчикам сосредоточиваться на основных задачах — отправке, маршрутизации и получении сообщений, не отличающихся принципиально, независимо от лежащего в их основе подключения.

Подключаем PlaneFinder

Первое, что надо сделать, — переделать сервис `PlaneFinder` для публикации сообщений (в дальнейшем их потребляет приложение `Aircraft Positions` и другие подходящие) с помощью Spring Cloud Stream.

Необходимые зависимости

Я добавил в файл сборки Maven `pom.xml` сервиса `PlaneFinder` следующие зависимости:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-stream</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-stream-binder-kafka</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-stream-binder-rabbit</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
</dependency>
```

Прежде всего следует отметить вторую из перечисленных зависимостей — `spring-cloud-stream`. Это зависимость кода для Spring Cloud Stream, но одной ее недостаточно. Как уже упоминалось, SCSt необходимы адаптеры привязки, чтобы его замечательные абстракции могли без проблем работать с различными потоковыми платформами. Об этом напоминает даже описание зависимости Spring Cloud Stream в Spring Initializr: «Framework for building highly scalable event-driven microservices connected with shared messaging systems (requires a binder, e. g., Apache Kafka, RabbitMQ, or Solace PubSub+)[1]».

Чтобы Spring Cloud Stream мог работать с платформой обмена сообщениями, необходим драйвер платформы обмена сообщениями и работающий с ней адаптер привязки. В предыдущий пример было включено сочетание «адаптер привязки + драйвер для RabbitMQ *и* Apache Kafka».

[1] Фреймворк для создания высокомасштабируемых событийно-управляемых микросервисов, работающих с совместно используемыми системами обмена сообщениями (требует какого-либо адаптера привязки, например Apache Kafka, RabbitMQ или Solace PubSub+).



Если включено только одно сочетание «адаптер привязки + драйвер, например RabbitMQ», автоконфигурация Spring Boot сможет однозначно определить, что приложение должно поддерживать обмен сообщениями с экземпляром(-ами) RabbitMQ и соответствующей точкой обмена (exchange) и очередью (queue), а также создать соответствующие вспомогательные компоненты. От разработчика не потребуется каких-либо дополнительных усилий. Если же включить несколько наборов «адаптер привязки + драйвер», то придется указать, какой использовать, но зато можно будет динамически переключаться между всеми включенными платформами во время выполнения, не меняя протестированное и развернутое приложение. Это чрезвычайно удобная и перспективная возможность.

Необходимо внести в файл `pom.xml` еще два дополнения. Во-первых, указать версию Spring Cloud уровня проекта, добавив в раздел `<properties></properties>` строку

```
<spring-cloud.version>2020.0.0-M5</spring-cloud.version>
```

Во-вторых, дать указания относительно спецификации зависимостей (Bill of Materials, BOM) Spring Cloud, из которой система сборки может определить версии любых используемых в проекте компонентов Spring Cloud — в данном случае Spring Cloud Stream:

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```



Версии компонентов проектов Spring часто обновляются. Простейший способ определить правильные синхронизированные версии, проверенные для работы с текущей версией Spring Boot, — воспользоваться Spring Initializr. Если выбрать нужные зависимости и нажать кнопку Explore CTRL+SPACE, будут отображены соответствующие элементы и версии.

Обновив зависимости проекта, можем перейти к обсуждению кода.

Поставка данных о местоположении воздушных судов

Существующая структура сервиса `PlaneFinder` и чистый функциональный подход Spring Cloud Stream позволяют опубликовать данные о текущем местоположении воздушных судов в RabbitMQ, которыми смогут пользоваться другие приложения, с помощью всего одного маленького класса:

```
@AllArgsConstructor
@Configuration
public class PositionReporter {
    private final PlaneFinderService pfService;

    @Bean
    Supplier<Iterable<Aircraft>> reportPositions() {
        return () -> {
            try {
                return pfService.getAircraft();
            } catch (IOException e) {
                e.printStackTrace();
            }
            return List.of();
        };
    }
}
```

Поскольку в результате каждого из опросов радиолокатора с помощью сервиса `PlaneFinder` формируется список местоположений воздушных судов, находящихся в настоящий момент в пределах досягаемости, сервис `PlaneFinder` создает сообщение, состоящее из нескольких воздушных судов, в `Iterable<Aircraft>` с помощью вызова метода `getAircraft()` сервиса `PlaneFinder`. Автоконфигурация Spring Boot основывается на мнении, что по умолчанию `Supplier` вызывается каждую секунду (можно переопределить через соответствующее свойство приложения), и некоторых обязательных/необязательных свойствах приложения, запуская тем самым весь механизм в действие.

Свойства приложения

Обязательным является лишь одно свойство приложения, хотя и остальные могут пригодиться. Вот содержимое обновленного файла `application.properties` сервиса `PlaneFinder`:

```
server.port=7634

spring.cloud.stream.bindings.reportPositions-out-0.destination=aircraftpositions
spring.cloud.stream.bindings.reportPositions-out-0.binder=rabbit
```

РАЗМЫШЛЕНИЯ ОБ АРХИТЕКТУРЕ ПРИЛОЖЕНИЙ

Во-первых, с технической точки зрения нужен лишь метод создания компонента `reportPositions()`, а не весь класс `PositionReporter`. Поскольку основной класс приложения снабжен аннотацией `@SpringBootApplication` — метааннотацией, включающей в себя аннотацию `@Configuration`, можно просто поместить метод `reportPositions()` внутрь основного класса приложения, `PlanefinderApplication`. Но лично я предпочитаю помещать методы `@Bean` внутрь соответствующих классов `@Configuration`, особенно когда создается множество компонентов.

Во-вторых, основанный на аннотациях унаследованный API Spring Cloud Stream до сих пор полностью поддерживается, но в этой книге я сосредоточусь исключительно на более новом функциональном API. Spring Cloud Stream основывается на ясном фундаменте Spring Cloud Function, который, в свою очередь, базируется на стандартных концепциях/интерфейсах Java: `Supplier<T>`, `Function<T, R>` и `Consumer<T>`. Это позволяет убрать из SCSt слегка дырявую абстракцию Spring Integration и ввести на ее место базовые конструкции языка, а также, как можно догадаться, предоставляет некоторые новые возможности.

Короче говоря, приложения могут поставлять сообщения (`Supplier<T>`), преобразовывать сообщения (`Function<T, R>`) одного вида в другой или потреблять сообщения (`Consumer<T>`). Предоставлять связующие конвейеры может любая из поддерживаемых потоковых платформ.

В настоящий момент Spring Cloud Stream поддерживает следующие платформы:

- RabbitMQ;
- Apache Kafka;
- Kafka Streams;
- Amazon Kinesis;
- Google Pub/Sub (поддерживается партнером);
- Solace PubSub+ (поддерживается партнером);
- Azure Event Hubs (поддерживается партнером);
- Apache RocketMQ (поддерживается партнером).

Свойство `server.port` осталось с первой версии, оно указывает, что наше приложение должно выполнять прослушивание на порте 7634.

Для полноценной работы функционального API Spring Cloud Stream в качестве отправной точки требуется лишь минимальная настройка свойств. У `Supplier` есть лишь выходные каналы, так как он производит только сообщения. А у `Consumer` есть только входные каналы, поскольку он только потребляет сообщения. У `Function` — и входные, и выходные, так как он используется для преобразования одних сообщений в другие.

Названия каналов всех адаптеров привязки состоят из названия метода компонента интерфейса, суффикса `in` или `out` и номера канала от `0` до `7`. После конкатенации названия канала в виде `<метод>-<in|out>-n` можно определить для него свойства адаптера привязки.

Для нашего сценария требуется лишь одно свойство — `destination`, да и то лишь для удобства. В данном примере при указании названия `destination` RabbitMQ создает `exchange` с названием `aircraftpositions`.

А поскольку мы включили в число зависимостей проекта адаптеры привязки и драйверы как для RabbitMQ, так и для Kafka, необходимо указать, какой именно адаптер привязки использовать. Для этого примера я выбрал `rabbit`.

Все нужные и желаемые свойства приложения определены, и сервис `PlaneFinder` готов к ежесекундной публикации в RabbitMQ текущего местоположения воздушных судов, которое будут использовать другие приложения.

Расширяем приложение Aircraft Positions

Добиться того, чтобы приложение `Aircraft Positions` потребляло сообщения из конвейера RabbitMQ с помощью Spring Cloud Stream, несложно. Чтобы заменить регулярные HTTP-запросы ориентированной на сообщения архитектурой, требуется лишь несколько изменений внутренних механизмов.

Необходимые зависимости

Как и в случае с сервисом `PlaneFinder`, я добавлю в файл `pom.xml` приложения `Aircraft Positions` следующие зависимости:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-stream</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-stream-binder-kafka</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-stream-binder-rabbit</artifactId>
```

```
</dependency>
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
</dependency>
```



Как уже упоминалось, на будущее я включил в число зависимостей проекта адаптеры привязки и драйверы как для RabbitMQ, так и для Kafka, но для текущего сценария использования требуется лишь набор для RabbitMQ — spring-boot-starter-amqp и spring-cloud-stream-binder-rabbit, чтобы Spring Cloud Stream (spring-cloud-stream) мог задействовать RabbitMQ.

Еще я добавил две дополнительные записи в файл `pom.xml`. Во-первых, включил в раздел `<properties></properties>` (рядом с `java.version`) следующее:

```
<spring-cloud.version>2020.0.0-M5</spring-cloud.version>
```

А во-вторых, информацию BOM Spring Cloud:

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

Обновляем зависимости приложения и можем переходить к следующему этапу.

Потребление данных о местоположении воздушных судов

Для извлечения и сохранения сообщений с информацией о текущем местоположении воздушных судов необходим всего один маленький дополнительный класс:

```
@AllArgsConstructor
@Configuration
public class PositionRetriever {
    private final AircraftRepository repo;
```

```
    @Bean
    Consumer<List<Aircraft>> retrieveAircraftPositions() {
        return acList -> {
            repo.deleteAll();

            repo.saveAll(acList);

            repo.findAll().forEach(System.out::println);
        };
    }
}
```

Подобно аналогичному классу `PositionReporter` в сервисе `PlaneFinder`, класс `PositionRetriever` снабжен аннотацией `@Configuration` и представляет собой компонент для использования со Spring Cloud Stream — в данном случае потребитель (`Consumer`) сообщений, каждое из которых содержит `List` с одним или несколькими объектами `Aircraft`. При поступлении каждого входящего сообщения компонент `Consumer` удаляет все местоположения из хранилища данных в оперативной памяти, сохраняет все входящие записи о местоположении, после чего выводит их все на консоль для проверки. Обратите внимание на то, что последний оператор, выводящий местоположения на консоль, необязателен, я включил его только для проверки на время разработки приложения.

Свойства приложения

Чтобы предоставить приложению немногие оставшиеся фрагменты информации, необходимые для подключения к входящему потоку сообщений, я добавил в файл `application.properties` следующие записи:

```
spring.cloud.stream.bindings.retrieveAircraftPositions-in-0.destination=
   aircraftpositions
spring.cloud.stream.bindings.retrieveAircraftPositions-in-0.group=
   aircraftpositions
spring.cloud.stream.bindings.retrieveAircraftPositions-in-0.binder=
   rabbit
```

Как и в случае с сервисом `PlaneFinder`, канал определяется путем конкатенации следующих элементов, разделяемых черточкой (`-`):

- имени компонента, в данном случае `Consumer<T>`;
- `in`, поскольку потребители лишь потребляют сообщения, а значит, у них есть только входные каналы;
- числа от `0` до `7` включительно, так что входных каналов может быть до 8.

Значения свойств `destination` и `binder` совпадают с соответствующими значениями `PlaneFinder`, так как приложение `Aircraft Positions` должно указывать в качестве входного канала на то же `destination`, которое использовалось в `PlaneFinder` в качестве выходного. И потому что платформа обмена сообщениями должна быть у обоих приложений одинаковой — в данном случае RabbitMQ. Однако свойство `group` новое для нас.

Задать группу можно для любого типа потребителя, включая приемную часть `Function<T, R>`, но это необязательно. На самом деле включение или исключение свойства `group` — отправная точка одного из паттернов маршрутизации.

Если для потребляющего сообщения приложения не задана группа, адаптер привязки RabbitMQ создает случайное уникальное имя, помещая его и потребителя в очередь автоудаления в экземпляре или кластере RabbitMQ. В результате каждая сгенерированная очередь обслуживается одним, и только одним потребителем. Почему это так важно?

При поступлении сообщения в точку обмена RabbitMQ его копия автоматически перенаправляется всем очередям, распределенным по умолчанию на эту точку обмена. Если у одной точки обмена нескольких очередей, одно и то же сообщение отправляется каждой очереди в соответствии с так называемым *паттерном разветвления по выходу* (fan-out pattern). Это возможность полезна для случаев, когда необходимо доставить каждое сообщение в несколько точек назначения, чтобы удовлетворить различные требования.

Если же приложение указывает группу потребителей, к которой оно принадлежит, то имя группы используется для обозначения базовой очереди в RabbitMQ. Если у нескольких приложений задано одинаковое значение свойства `group`, то есть они подключаются к одной очереди, значит, все вместе они реализуют паттерн конкурирующих потребителей, при котором каждое поступающее в назначенную очередь сообщение обрабатывается лишь одним из потребителей. Это позволяет приспосабливаться к различным объемам сообщений путем масштабирования числа потребителей.



Для еще более точной и гибкой маршрутизации можно воспользоваться секционированием и ключами маршрутизации.

Указание свойства `group` для нашего приложения дает возможность масштабировать его на случай, если для обработки потока поступающих сообщений понадобится несколько его экземпляров.

Controller

Поскольку компонент `Consumer` автоматически проверяет наличие новых сообщений и обрабатывает их, объем класса `PositionController` и его метода `getCurrentAircraftPositions()` существенно сокращается.

Можно удалить все ссылки на `WebClient`, поскольку получение списка текущих местоположений воздушных судов теперь сводится к извлечению того, что содержится в репозитории в данный момент. Упрощенный вариант нашего класса выглядит так:

```
@RequiredArgsConstructor
@Controller
public class PositionController {
    @NonNull
    private final AircraftRepository repository;

    @GetMapping("/aircraft")
    public String getCurrentAircraftPositions(Model model) {
        model.addAttribute("currentPositions", repository.findAll());
        return "positions";
    }
}
```

На этом все модификации как генератора сообщений (приложение `PlaneFinder`), так и их потребителя (приложение `Aircraft Positions`) завершены.



Для того чтобы использовать какую-либо внешнюю платформу обмена сообщениями, она должна быть запущена и доступна для приложений. Я запустил локальный экземпляр RabbitMQ с помощью Docker. Скрипты для быстрого создания и запуска/останова имеются в прилагаемых к этой книге репозиториях.

Результаты

После проверки доступности RabbitMQ можно запустить приложения и удостовериться, что все работает как должно.

И хотя это необязательно, я хотел бы запустить сначала потребляющее сообщения приложение, чтобы оно было готово к поступлению сообщений. В данном случае это означает запуск приложения `Aircraft Positions` из моей IDE.

Далее запускаю новое усовершенствованное приложение `PlaneFinder`. В результате поток сообщений начинает поступать в приложение `Aircraft Positions`,

как демонстрирует его консоль. Замечательно, мы можем дойти по этому пути вплоть до конечного пользователя.

Возвращаясь в браузер и заходя на страницу localhost:8080, мы снова видим целевую страницу. Щелкнув на имеющейся на ней кнопке Click here, попадаем на страницу Positions Report. Как и раньше, страница Positions Report обновляется автоматически и отображает текущее местоположение воздушных судов, но теперь эти данные передаются независимо от `PlaneFinder` «за кулисами» в `Aircraft Positions`, для чего больше не требуется сначала выполнять HTTP-запрос. И наша архитектура становится на один шаг ближе к событийно-управляемой системе.

Формирование диалогов с помощью WebSocket

Первая версия распределенной системы для запроса и отображения текущего местоположения воздушных судов была полностью ориентирована на вытягивание данных (pull-based). Пользователь запрашивал (или повторно запрашивал при обновлении) актуальные данные о местоположении из браузера, который передавал запрос в приложение `Aircraft Positions`, а оно, в свою очередь, перенаправляло этот запрос приложению `PlaneFinder`. Ответы передавались обратно по цепочке.

В предыдущем разделе этой главы мы заменили среднюю часть нашей распределенной системы событийно-управляемой архитектурой. Теперь `PlaneFinder`, получая данные о местоположении от радиолокатора, выталкивает их в конвейер потоковой платформы, где их использует приложение `Aircraft Positions`. В основе «последней мили» (километра, если вам так больше нравится), тем не менее, остается вытягивание, а обновления запрашиваются путем обновления страницы браузера, вручную или автоматически.

Стандартная семантика «запрос — ответ» великолепно работает во множестве сценариев использования, но отвечающей серверной стороне в них не хватает возможности инициировать передачу данных запрашивающей стороне вне зависимости от каких-либо запросов. Существует множество обходных путей и хитроумных решений для такого сценария использования, все со своими достоинствами и недостатками, один из наиболее универсальных вариантов — WebSocket.

Что такое WebSocket

По существу, WebSocket — полнодуплексный протокол обмена информацией, связывающий две системы через одно TCP-соединение. После установления соединения WebSocket каждая из участвующих сторон может инициировать передачу данных другой стороне, а специально выделенный сервер приложений может поддерживать многочисленные клиентские соединения, что позволяет создавать системы с низкими накладными расходами для широковещательной передачи и диалогового взаимодействия. Соединения WebSocket формируются на основе стандартных HTTP-соединений с помощью HTTP-заголовка upgrade, и по завершении процедуры установления связи соединение переключается с протокола HTTP на WebSocket.

IETF стандартизировал WebSocket в 2011 году, и сейчас его поддерживают все основные браузеры и языки программирования. По сравнению с HTTP-запросами и ответами у WebSocket накладные расходы очень малы: операции передачи информации не должны идентифицироваться и согласовывать условия обмена информацией при каждой передаче, что сокращает накладные расходы на кадрирование WebSocket до нескольких байтов. Полнодуплексность, способность сервера обрабатывать несколько открытых соединений и низкие накладные расходы сделали WebSocket полезным инструментом, который пригодится любому разработчику.

Рефакторинг приложения Aircraft Positions

Хотя я говорю о приложении `Aircraft Positions` как о чем-то целостном, проект aircraft-positions включает как серверное приложение Spring Boot + Java, так и функциональность клиентской части на HTML + JavaScript. Во время разработки обе его части работают в одной среде, обычно на машине разработчика. И хотя они собираются, тестируются и развертываются в производственной среде как единое целое, выполнение приложения в продакшене делится следующим образом.

- Код Spring + Java для серверной части работает в облаке, при необходимости включая шаблонизатор, генерирующий итоговые веб-страницы для отображения конечному пользователю.
- HTML + JavaScript клиентской части — статический и/или генерируемый динамически контент — отображается и выполняется в браузере конечного пользователя, где бы он ни находился.

В этом разделе я не менял существующую функциональность, а лишь добавил в систему возможность автоматически отображать местоположение воздушных судов по мере поступления данных в режиме реального времени. Благодаря соединению WebSocket между клиентским и прикладным приложениями последнее может легко отправлять обновления в браузер конечного пользователя и выполнять обновление отображения автоматически, не запуская его вручную.

Дополнительные зависимости

Для включения функциональности WebSocket в приложение `Aircraft Positions` достаточно добавить в его файл `pom.xml` всего одну зависимость:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
```

Обновили зависимости приложения — и можем переходить к следующему шагу.

Обработка соединений и сообщений WebSocket

Spring предлагает несколько различных подходов к настройке и использованию WebSocket, но я рекомендую накатанный путь непосредственной реализации на основе интерфейса `WebSocketHandler`. Благодаря часто возникающей необходимости обмена текстовой, то есть недвоичной, информацией был создан даже класс `TextWebSocketHandler`, на котором и будет основан наш код:

```
@RequiredArgsConstructor
@Component
public class WebSocketHandler extends TextWebSocketHandler {
    private final List<WebSocketSession> sessionList = new ArrayList<>();
    @NonNull
    private final AircraftRepository repository;

    public List<WebSocketSession> getSessionList() {
        return sessionList;
    }

    @Override
    public void afterConnectionEstablished(WebSocketSession session)
            throws Exception {
        sessionList.add(session);
        System.out.println("Connection established from " + session.toString() +
```

```
            " @ " + Instant.now().toString());
    }

    @Override
    protected void handleTextMessage(WebSocketSession session,
            TextMessage message) throws Exception {
        try {
            System.out.println("Message received: '" +
                message + "', from " + session.toString());

            for (WebSocketSession sessionInList : sessionList) {
                if (sessionInList != session) {
                    sessionInList.sendMessage(message);
                    System.out.println("--> Sending message '"
                        + message + "' to " + sessionInList.toString());
                }
            }
        } catch (Exception e) {
                System.out.println("Exception handling message: " +
            e.getLocalizedMessage());
        }
    }

    @Override
    public void afterConnectionClosed(WebSocketSession session,
            CloseStatus status) throws Exception {
        sessionList.remove(session);
        System.out.println("Connection closed by " + session.toString() +
            " @ " + Instant.now().toString());
    }
}
```

В приведенном ранее коде реализованы два метода интерфейса `WebSocketHandler`, `afterConnectionEstablished` и `afterConnectionClosed`, для ведения списка активных `WebSocketSession`, а также журналирования подключений и отключений. Еще я реализовал метод `handleTextMessage` для транслирования всех входящих сообщений во все прочие активные сеансы. Один этот класс обеспечивает функциональность WebSocket для серверной части, готовой к началу работы при получении местоположения воздушных судов от `PlaneFinder` через RabbitMQ.

Передача местоположения воздушных судов на соединения WebSocket

Предыдущая версия класса `PositionRetriever` потребляла полученные через сообщения RabbitMQ списки местоположения воздушных судов и сохраняла

их в базе данных H2 в оперативной памяти. Взяв ее за основу, я заменил предназначенный для подтверждения вызов `System.out::println` вызовом нового метода `sendPositions()`, задача которого заключается в отправке свежего списка местоположения воздушных судов с помощью только что добавленного компонента `@Autowired WebSocketHandler` всем подключенным по WebSocket клиентам:

```
@AllArgsConstructor
@Configuration
public class PositionRetriever {
    private final AircraftRepository repository;
    private final WebSocketHandler handler;

    @Bean
    Consumer<List<Aircraft>> retrieveAircraftPositions() {
        return acList -> {
            repository.deleteAll();

            repository.saveAll(acList);

            sendPositions();
        };
    }

    private void sendPositions() {
        if (repository.count() > 0) {
            for (WebSocketSession sessionInList : handler.getSessionList()) {
                try {
                    sessionInList.sendMessage(
                        new TextMessage(repository.findAll().toString())
                    );
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        }
    }
}
```

Теперь, когда WebSocket уже настроен должным образом и серверная часть может транслировать местоположение воздушных судов всем подключенным WebSocket-клиентам сразу по получении нового списка, необходимо научить серверную часть приложения прослушивать и принимать запросы на соединения. Для этого мы зарегистрируем созданный ранее `WebSocketHandler` через интерфейс `WebSocketConfigurer` и снабдим новый класс `@Configuration`

аннотацией `@EnableWebSocket`, чтобы наше приложение обрабатывало запросы WebSocket:

```
@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {
    private final WebSocketHandler handler;

    WebSocketConfig(WebSocketHandler handler) {
        this.handler = handler;
    }

    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(handler, "/ws");
    }
}
```

В методе `registerWebSocketHandlers(WebSocketHandlerRegistry registry)` я связал созданный ранее компонент `WebSocketHandler` с конечной точкой `ws://<имя_хоста:порт_хоста>/ws`. Приложение будет прослушивать в ней HTTP-запросы с заголовками upgrade WebSocket и выполнять нужные действия при их получении.



Если ваше приложение работает с протоколом HTTPS, вместо ws:// необходимо использовать wss:// (WebSocket Secure).

WebSocket в серверной части, WebSocket в клиентской части

Завершив работу в серверной части, начинаем пожинать плоды в функциональности клиентской части.

В качестве простого примера того, как благодаря WebSocket приложение прикладной части может выталкивать обновления без запроса со стороны пользователей и их браузеров, я создал следующий HTTP-файл с одним тегом `div` и `label`, а также несколькими строками кода на JavaScript и поместил его в каталог `src/main/resources/static` рядом с уже существующим файлом `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>Aircraft Position Report (Live Updates)</title>
    <script>
        var socket = new WebSocket('ws://' + window.location.host + '/ws');

        socket.onopen = function () {
            console.log(
              'WebSocket connection is open for business, bienvenidos!');
        };

        socket.onmessage = function (message) {
            var text = "";
            var arrAC = message.data.split("Aircraft");
            var ac = "";

            for (i = 1; i < arrAC.length; i++) {
                ac = (arrAC[i].endsWith(", "))
                    ? arrAC[i].substring(0, arrAC[i].length - 2)
                    : arrAC[i]

                text += "Aircraft" + ac + "\n\n";
            }

            document.getElementById("positions").innerText = text;
        };

        socket.onclose = function () {
            console.log('WebSocket connection closed, hasta la próxima!');
        };
    </script>
</head>
<body>
<h1>Current Aircraft Positions</h1>
<div style="border-style: solid; border-width: 2px; margin-top: 15px;
        margin-bottom: 15px; margin-left: 15px; margin-right: 15px;">
    <label id="positions"></label>
</div>
</body>
</html>
```

Как ни коротка эта веб-страница, она могла бы быть еще короче. Функции `socket.onopen` и `socket.onclose` предназначены для журналирования, их можно опустить, а `socket.onmessage` разработчик с хорошими навыками в JavaScript и желанием заняться этим практически наверняка мог бы переделать. Несколько ключевых нюансов:

- определение тегов `div` и `label` внизу HTML;
- переменная `socket`, устанавливающая WebSocket-соединение и используемая для ссылки на WebSocket-соединение;
- функция `socket.onmessage`, производящая разбор списка местоположений воздушных судов и присваивающая результат в другом формате полю `innerText` HTML-элемента с меткой `positions`.

После сборки проекта заново и его запуска, конечно, можно будет обращаться к странице `wspositions.html` непосредственно из браузера. Однако невозможность обратиться к странице и ее функциональности, не зная ее точного адреса и вводя его вручную в адресной строке браузера, — далеко не лучший способ создания приложений для реального использования, причем совершенно не помогающий подготовить почву для дальнейшего расширения примера в следующих главах.

Не будем пока ничего усложнять и добавим еще одну строку в существующий файл `index.html`, чтобы пользователь мог перейти на управляемую WebSocket страницу `wspositions.html` в дополнение к уже существующей:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Retrieve Aircraft Position Report</title>
</head>
<body>
    <p><a href="/aircraft">Click here</a> to retrieve current aircraft positions
        in range of receiver.</p>
    <p><a href="/wspositions.html">Click here</a> to retrieve a livestream of
        current aircraft positions in range of receiver.</p>
</body>
</html>
```

Реализация функциональности в клиентской части завершена, пора протестировать WebSocket.

Результаты

Я запустил из IDE приложение `Aircraft Positions` и `PlaneFinder`. Открыв окно браузера, обратился к приложению клиентской части по адресу `localhost:8080` (рис. 7.3).



Рис. 7.3. Целевая страница приложения Aircraft Positions, теперь с выбором из двух вариантов

Выбираем на все еще далекой от совершенства целевой странице второй вариант — Click here to retrieve a livestream of current aircraft positions in range of receiver, после чего генерируется страница `wspositions.html`, и результаты аналогичны показанным на рис. 7.4.

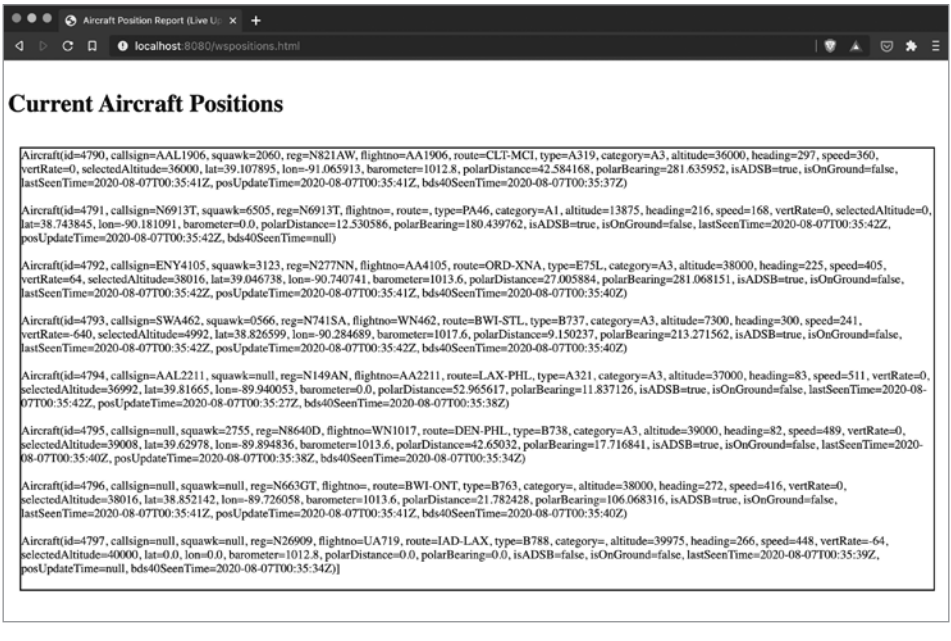


Рис. 7.4. Отчет Aircraft Position с динамическими обновлениями через WebSocket

Совершенно не сложно преобразовать приведенный формат записей базы данных в JSON для динамического наполнения таблицы результатами, получаемыми через WebSocket от приложения прикладной части. Примеры есть в прилагаемых к этой книге репозиториях кода.



Вполне можно собирать и запускать приложения PlaneFinder и Aircraft Positions из командной строки. Иногда я так и делаю, однако в большинстве циклов сборки/запуска намного быстрее запускать приложения и выполнять отладку непосредственно из IDE.

Резюме

Практически все приложения должны так или иначе взаимодействовать с конечными пользователями или другими приложениями, чтобы приносить реальную пользу, а это требует удобных и эффективных средств взаимодействия.

В этой главе рассказывалось о технологиях представлений — языках шаблонов или тегах наподобие Thymeleaf, а также шаблонизаторах для их обработки — и о том, как Spring Boot с их помощью создает и поставляет функциональность в браузер конечного пользователя. Вы также узнали, как Spring Boot обрабатывает статический контент наподобие стандартного HTML вместе с JavaScript, который можно поставлять непосредственно, без обработки шаблонизатором. Первая версия проекта из этой главы демонстрирует примеры приложения на основе Thymeleaf, извлекающего и отображающего местоположение воздушных судов, находящихся в пределах досягаемости в момент запроса, и модель чистого вытягивания данных.

Далее в главе было показано, как использовать возможности платформ обмена сообщениями из Spring Boot с помощью Spring Cloud Stream и RabbitMQ. Мы переработали приложение `PlaneFinder` так, чтобы выталкивать список текущего местоположения воздушных судов, получаемый радиолокатором. А приложение `Aircraft Positions` было модифицировано так, чтобы принимать новые списки местоположения воздушных судов по мере их поступления через конвейер RabbitMQ. Таким образом, модель чистого извлечения между двумя приложениями приходит на замену модели на основе выталкивания, превращая функциональность прикладной части приложения `Aircraft Positions` в событийно-управляемую. Функциональность клиентской части по-прежнему требует обновления страницы (вручную или зашитого в коде) для отображения обновленных результатов пользователю.

Наконец, реализация WebSocket-соединения и кода обработчика в компонентах серверной и клиентской частей приложения `Aircraft Positions` позволяет серверной части приложения Spring + Java выталкивать обновления местоположения воздушных судов *по мере их получения* через конвейер RabbitMQ из `PlaneFinder`. Обновления местоположения показываются динамически на простой HTML + JavaScript-странице и не требуют запросов на обновление

от конечного пользователя или его браузера, демонстрируя двунаправленную сущность WebSocket, отсутствие необходимости в паттерне «запрос — ответ» (или каком-либо обходном пути) и низкие накладные расходы на обмен информацией.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Полный код для этой главы находится в ветке chapter7end в репозитории кода.

В следующей главе я познакомлю вас с реактивным программированием и расскажу, как Spring выполняет разработку и продвижение многочисленных утилит и технологий, делающих его одним из наилучших решений для множества сценариев использования. Точнее говоря, продемонстрирую применение Spring Boot и Project Reactor для доступа к базам данных, интеграции реактивных типов данных с технологиями представлений наподобие Thymeleaf и поднятия межпроцессного взаимодействия на совершенно новый уровень.

ГЛАВА 8

Реактивное программирование: Project Reactor и Spring WebFlux

Эта глава рассказывает о реактивном программировании, его истоках и причинах возникновения, а также демонстрирует, что Spring лидирует в разработке и совершенствовании многочисленных инструментов и технологий, делающих его одним из наилучших решений для множества сценариев использования. Если точнее, я продемонстрирую применение Spring Boot и Project Reactor для доступа к SQL- и NoSQL-базам данных, интеграции реактивных типов данных с технологиями представлений наподобие Thymeleaf и поднятия межпроцессного взаимодействия на совершенно новый уровень с помощью RSocket.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Пожалуйста, для начала извлеките из репозитория кода ветку chapter8begin.

Введение в реактивное программирование

Хотя полноценное описание реактивного программирования может занять — и занимает, и будет занимать — целую книгу, очень важно понимать, почему вообще эта идея так важна.

В обычном сервисе для обработки каждого запроса создается поток выполнения. Каждый из них требует ресурсов, а поэтому приложение может запустить ограниченное количество потоков выполнения. В качестве несколько упрощенного примера: если приложение может обслуживать 200 потоков выполнения, то способно принимать запросы не более чем от 200 отдельных клиентов одновременно,

всем прочим попыткам соединения с сервисом придется ждать освобождения потока выполнения.

Быстродействие при 200 подключенных клиентах может быть как удовлетворительным, так и неудовлетворительным в зависимости от множества факторов. Бесспорно лишь то, что время отклика для клиентского приложения, выполняющего параллельный запрос номер 201 или более, может оказаться гораздо хуже вследствие блокировки его сервисом, ожидающим доступного потока выполнения. Подобный резкий сбой масштабирования может превратиться из не имеющей особого значения мелочи в серьезную проблему, причем безо всякого предупреждения и наличия простого решения с обходными путями наподобие «привлечь дополнительные экземпляры приложения для решения проблемы». Реактивное программирование было создано для преодоления этого кризиса масштабируемости.

Манифест реактивных систем

Манифест реактивных систем (https://www.reactivemanifesto.org/ru) гласит, что реактивные системы:

- отзывчивые;
- отказоустойчивые;
- гибкие;
- основаны на обмене сообщениями.

Если кратко, перечисленные четыре основные отличительные характеристики реактивных систем, вместе взятые (на макроуровне), приводят к максимально высокодоступной, масштабируемой и быстродействующей системе, требующей для эффективного выполнения конкретной задачи минимально возможного количества ресурсов.

Если рассматривать уровень системы, то есть несколько приложений/сервисов, работающих совместно для реализации разнообразных сценариев использования, можно заметить, что большинство трудных задач требуют обмена информацией между приложениями: ответа одного приложения другому, доступности приложения или сервиса при поступлении запроса, возможности масштабирования сервиса или подстройки по требованию, оповещения одним сервисом других заинтересованных сервисов о наличии обновленной либо ставшей доступной информации и т. д. Решение потенциальных проблем взаимодействия между приложениями может существенно смягчить и/или устранить упомянутые проблемы масштабирования.

Осознание того, что обмен информацией является крупнейшим потенциальным источником проблем, но в то же время обеспечивает максимальные возможности их решения, привело к возникновению инициативы Reactive Streams (реактивные потоки данных, `www.reactive-streams.org`). В центре внимания инициативы Reactive Streams (RS) — взаимодействие между сервисами — потоками, если хотите. RS включает четыре основных элемента:

- интерфейс прикладного программирования (API);
- спецификацию;
- примеры реализаций;
- набор для проверки совместимости технологий (Technology Compatibility Kit, TCK).

API состоит всего из четырех интерфейсов:

- `Publisher` — создатель;
- `Subscriber` — потребитель;
- `Subscription` — договоренность между `Publisher` и `Subscriber`;
- `Processor` — включает в себя как `Publisher`, так и `Subscriber` для получения, преобразования и отправки данных.

Подобная лаконичность и простота играют ключевую роль, как и то, что API состоит исключительно из *интерфейсов*, а не из реализаций, позволяя получить разнообразные совместимые реализации для различных платформ, языков программирования и моделей программирования.

В текстовой спецификации подробно описывается ожидаемое и/или нужное поведение реализаций API, например:

```
If a Publisher fails it MUST signal an onError.
```

Примеры реализаций очень полезны для разработчиков, они служат эталонным кодом при создании конкретных реализаций RS.

Вероятно, важнейшая его составляющая — Technology Compatibility Kit. TCK позволяет создателям реализаций проверять и демонстрировать уровень совместимости с их или чьей-то чужой реализацией RS и все существующие недочеты. Знание — сила, и выявление чего-либо, не работающего в полном соответствии со спецификацией, ускоряет разрешение проблем, а также служит предостережением для пользователей библиотеки до тех пор, пока недочет не будет устранен.

ПРИМЕЧАНИЯ ОТНОСИТЕЛЬНО РЕАКТИВНЫХ ПОТОКОВ ДАННЫХ, АСИНХРОННОСТИ И КОНТРОЛЯ ОБРАТНОГО ПОТОКА ДАННЫХ

В основе Reactive Streams лежит асинхронный обмен данными и их обработка, как ясно говорится в заявлении о цели в самом первом абзаце информационного сайта Reactive Streams (http://www.reactive-streams.org): «Reactive Streams — инициатива, направленная на создание стандарта асинхронной потоковой обработки с неблокирующим контролем обратного потока данных. Она охватывает как работы в сфере сред выполнения (JVM и JavaScript), так и сетевые протоколы».

Хотя и рискуя излишне упростить картину, можно рассматривать различные идеи и компоненты, составляющие Reactive Streams, следующим образом.

Асинхронность достигается, когда приложение не останавливает обработку в целом, выполняя какое-либо задание. Например, когда сервис А запрашивает информацию у сервиса Б, то не останавливает обработку всех последующих инструкций до получения ответа и драгоценные компьютерные ресурсы не простаивают (а значит, не расходуются впустую) в ожидании ответа от Б. Вместо этого сервис А продолжает работу с другими заданиями, пока не получит уведомление о поступлении ответа.

В отличие от синхронной обработки, при которой задания выполняются последовательно (каждое задание выполняется лишь после завершения предыдущего), асинхронная обработка может включать запуск задания и переход к следующему, если предыдущее можно выполнять в фоновом режиме (или ожидать уведомления о готовности/завершении), так что задания могут выполняться параллельно. Это позволяет более полно задействовать ресурсы, расходуя время CPU, которое было бы потрачено на простой или почти бездействие, на практическую работу вместо ожидания, и в некоторых случаях может повысить быстродействие.

Принимая на вооружение асинхронную модель какого-либо вида, не следует автоматически надеяться на рост производительности. При прямом взаимодействии всего двух сервисов с одной точкой обмена практически невозможно добиться большей производительности, чем у блокирующей синхронной модели обмена информацией и обработки. Продемонстрировать это легко: если у сервиса Б только один клиент, сервис А, причем последний выполняет одновременно только один запрос и блокирует всю прочую деятельность, выделяя все ресурсы на ожидание ответа от сервиса Б, то такая модель выделенной обработки и соединения обеспечивает наилучшее быстродействие для взаимодействия двух приложений при прочих равных условиях. Подобный сценарий и аналогичные ему встречаются чрезвычайно редко, но вполне возможны.

Накладные расходы при асинхронной обработке минимальны благодаря таким механизмам реализации, как цикл ожидания события, в котором сервис «прослушивает» ответы на ожидающие выполнения запросы. В результате быстродействие в сценариях с очень ограниченным взаимодействием между приложениями может оказаться чуть ниже, чем у точки с синхронными обменом информацией и обработкой. Однако эта ситуация быстро меняется по мере роста количества соединений и исчерпания числа потоков выполнения. В отличие от синхронной обработки, при асинхронной ресурсы не выделяются и не простаивают, они переназначаются и используются для других задач, что повышает как полноту использования ресурсов, так и масштабируемость приложения.

Reactive Streams добавляет к простой асинхронной обработке еще и неблокирующий контроль обратного потока данных, делая тем самым обмен информацией между приложениями более устойчивым к ошибкам.

Без контроля обратного потока данных сервис А, запрашивая информацию от сервиса Б, никак не защищается от содержащего слишком много данных ответа. Например, если сервис Б возвращает миллион объектов/записей, то сервис А послушно попытается принять и обработать их все, но вероятнее всего, не выдержит такой нагрузки. Если у сервиса А недостаточно вычислительных и сетевых ресурсов для обработки этого колоссального потока информации, вполне возможен и даже вероятен фатальный сбой приложения. Асинхронность на это никак не влияет, ведь ресурсы приложения полностью поглощаются в попытке справиться с лавиной информации. Тут-то и проявляется вся польза от контроля обратного потока данных.

Неблокирующий контроль обратного потока данных означает просто, что сервис А может оповестить сервис Б о своих возможностях по обработке ответов. Вместо того чтобы говорить: «Дай мне все», сервис А запрашивает у сервиса Б определенное количество объектов, обрабатывает их и запрашивает еще, если готов и имеет возможность их обработать. Пришедший из сферы гидроаэродинамики термин «контроль обратного потока данных» (backpressure) означает такой контроль потока источником, когда в трубопроводе создается давление в обратную потоку вещества сторону. В Reactive Streams благодаря контролю обратного потока данных сервис А может управлять скоростью поступления ответов, подстраивая его в режиме реального времени при смене обстоятельств.

Хотя было создано немало обходных путей различной степени сложности, успешности и сферы применимости для реализации контроля обратного потока данных в нереактивных системах, поощряемая Reactive Streams декларативная модель программирования позволяет разработчику прозрачно и беспроблемно интегрировать асинхронность и контроль обратного потока данных.

Project Reactor

Доступно несколько реализаций Reactive Streams для JVM, и Project Reactor — одна из самых активно развивающихся, продвинутых и производительных. Project Reactor взят на вооружение и формирует основной фундамент многочисленных важнейших проектов, реализуемых по всему миру, включая библиотеки, API и приложения, разработанные и развернутые как маленькими организациями, так и гигантами сферы информационных технологий. Вдобавок к впечатляющим объемам разработки и массовости применения Reactor служит фундаментом для реактивной веб-функциональности Spring WebFlux — реактивного доступа Spring Data к нескольким базам данных, коммерческим и с открытым исходным кодом, позволяя создавать насквозь реактивные конвейеры, как вертикальные (сверху вниз) по стеку, так и горизонтальные. Это всеобъемлющее решение.

В чем его важность?

Сверху вниз по стеку, от конечного пользователя к вычислительным ресурсам самого низкого уровня, каждое взаимодействие может потенциально стать камнем преткновения. Если взаимодействия между браузером пользователя и серверной частью приложения неблокирующие, но приложению приходится ждать результатов блокирующего взаимодействия с базой данных, система в целом оказывается блокирующей. То же самое относится и к взаимодействиям между приложениями: что выиграет пользователь, если его браузер обменивается информацией с сервисом А серверной части, который ждет (блокирующим образом) ответа от сервиса Б? Вероятно, очень немного, а то и вовсе ничего.

Разработчики обычно понимают, насколько широкие возможности открывает для них переход на Reactive Streams. Впрочем, их уравновешивает необходимость перемены самого образа мыслей в сочетании с относительной новизной идей и инструментов реактивного (по сравнению с императивным) программирования, к которым программистам необходимо приспособиться. Да и работы у них прибавляется, по крайней мере в краткосрочной перспективе. Но решение все равно очевидно, ведь преимущества Reactive Streams в смысле масштабируемости и широты применения явно превосходят усилия, которые необходимо затратить.

Реализация Reactive Streams в Project Reactor — простая и понятная, в основе ее лежат хорошо знакомые разработчикам Java и Spring идеи. Как и API потоков Java 8, Reactor лучше всего применять посредством декларативных, цепочечных операторов, зачастую с лямбда-выражениями. Такой код по сравнению с более процедурным, императивным кодом сначала кажется непривычным, но затем становится ясно его изящество. Разработчики, которым знаком API Stream, значительно быстрее адаптируются и начинают ценить Reactor.

Reactor уточняет идеи интерфейса `Publisher` Reactive Streams, по ходу дела вводя конструкции, схожие с характерными для императивного Java. Вместо использования универсального интерфейса `Publisher` везде, где нужен реактивный поток данных (можете считать его своего рода динамическим `Iterable` по требованию), Project Reactor описывает два типа `Publisher`:

- `Mono:` : генерирует один элемент или ни одного;
- `Flux:` : генерирует от 0 до *n* элементов — заданное или неограниченное количество.

Это прекрасно согласуется с императивными конструкциями. Например, в стандартном языке Java метод может возвращать объект типа `T` или `Iterable<T>`. При использовании Reactor тот же метод мог бы возвращать `Mono<T>` или

`Flux<T>` — один объект или множество либо при императивном коде `Publisher` этих объектов.

Reactor также прекрасно согласуется с решениями Spring. В зависимости от сценария преобразование блокирующего кода в неблокирующий требует всего лишь замены одной из зависимостей проекта и возвращаемых значений нескольких методов, как было показано ранее. Примеры из этой главы демонстрируют, как именно это сделать, а также как масштабировать простое реактивное приложение — вверх, вниз и в горизонтальном направлении — до реактивной системы, включая реактивный доступ к базе данных, чтобы добиться от него максимума.

Tomcat и Netty

В императивном мире Spring Boot для веб-приложений по умолчанию используется контейнер сервлетов Tomcat, хотя даже на этом уровне у разработчиков есть альтернативные варианты наподобие Jetty и Undertow, которыми его можно просто заменить. Впрочем, весьма логично использовать Tomcat по умолчанию как хорошо себя зарекомендовавшее, проверенное и быстродействующее решение, а создатели Spring внесли и продолжают вносить немалый вклад в доработку и развитие кодовой базы Tomcat. Это превосходный контейнер сервлетов для приложений Boot.

Тем не менее многочисленные версии спецификаций сервлетов были по своей сути синхронными, без каких-либо асинхронных возможностей. Для решения этой проблемы в Servlet 3.0 появилась асинхронная обработка запросов, но поддерживался лишь обычный блокирующий ввод/вывод. В версии 3.1 спецификации появился и неблокирующий ввод/вывод, благодаря чему она подходит для асинхронных, а следовательно, и реактивных приложений.

Реактивный аналог пакета Spring WebMVC, или просто Spring MVC, называется Spring WebFlux. Он основан на Reactor и использует Netty в качестве сетевого механизма по умолчанию, подобно тому как Spring MVC использует Tomcat для прослушивания запросов и их обслуживания. Netty — проверенный и производительный асинхронный движок, а создатели Spring сделали немало для интеграции Netty с Reactor и поддержания Netty в актуальном состоянии в смысле самых современных возможностей и быстродействия.

Как и в случае с Tomcat, существуют альтернативные варианты. С приложениями Spring WebFlux можно при необходимости использовать любой совместимый с Servlet 3.1 движок. Но Netty неслучайно занимает лидирующие позиции и является оптимальным вариантом для большинства сценариев использования.

Реактивный доступ к данным

Как упоминалось ранее, конечная цель масштабируемости и оптимальной пропускной способности — полностью реактивная реализация от начала до конца, которая на самом низком уровне опирается на доступ к базе данных.

Многие годы усилия прилагались к проектированию баз данных так, чтобы минимизировать конкуренцию за ресурсы и факторы, снижающие быстродействие системы. Но даже эта впечатляющая работа не избавила многие движки баз данных и драйверы от проблемных областей, среди них средства выполнения операций без блокировки запрашивающих приложений и запутанных механизмов контроля прямого и обратного потока данных.

Для снятия этих ограничений предпринимались попытки использовать методики страничной организации, но это не идеальные решения. Императивная модель со страничной организацией обычно требует генерации запросов со своим диапазоном и/или ограничениями для каждой страницы. Это означает необходимость каждый раз нового запроса и ответа вместо продолжения старого, что возможно при использовании `Flux`. Аналогия: черпать по чашке воды из раковины (императивный подход), вместо того чтобы просто открыть кран и наполнить чашку. В реактивном сценарии вода готова литься, в отличие от императивного подхода типа «набери и принеси».

R2DBC и база данных H2

В нынешней нашей версии PlaneFinder для сохранения (в экземпляре H2, хранящем данные в оперативной памяти) местоположения находящихся в пределах досягаемости воздушных судов, полученного от локального радиолокатора, использовались Java Persistence API (JPA) и база данных H2. JPA основан на императивной спецификации, поэтому по своей сути является блокирующим. Из-за острой необходимости в неблокирующих реактивных средствах взаимодействия с SQL-базами данных несколько ведущих компаний отрасли и известных специалистов объединили усилия ради создания и развития проекта Reactive Relational Database Connectivity (реактивная связь с базами данных, R2DBC).

Подобно JPA, R2DBC — открытая спецификация. Ее и предоставляемый ею интерфейс поставщика сервиса (Service Provider Interface, SPI) могут использовать поставщики или другие стороны, заинтересованные в создании драйверов для реляционных баз данных и клиентских библиотек для разработчиков. В отличие от JPA, R2DBC основана на реализации Reactive Streams в Project Reactor и является полностью реактивной и неблокирующей.

Модификация приложения PlaneFinder

Как и в большинстве сложных систем, невозможно одновременно контролировать все аспекты и узлы всей распределенной системы. Так же как и в большинстве сложных систем, чем полнее используется парадигма, тем больше отдача. Этот путь к реактивности начнем как можно ближе к начальной точке цепочки обмена информацией — с сервиса PlaneFinder.

Рефакторинг сервиса PlaneFinder — перевод его на использование типов данных Reactive Streams `Publisher`, например, `Mono` и `Flux` — лишь первый шаг. Мы воспользуемся существующей базой данных H2, но чтобы она стала реактивной, нам понадобится удалить зависимость проекта JPA, заменив его библиотеками R2DBC. Модифицируем файл сборки Maven `pom.xml` следующим образом:

```
<!--  Закомментируйте или удалите вот это -->
<!--<dependency>-->
<!--    <groupId>org.springframework.boot</groupId>-->
<!--  <artifactId>spring-boot-starter-data-jpa</artifactId>-->
<!--</dependency>-->

<!--  И добавьте вот это                  -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-r2dbc</artifactId>
</dependency>

<!--  Добавьте и это тоже            -->
<dependency>
    <groupId>io.r2dbc</groupId>
    <artifactId>r2dbc-h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

Необходимо модифицировать интерфейс `PlaneRepository`, чтобы он расширял интерфейс `ReactiveCrudRepository`, а не его блокирующий аналог `CrudRepository`. Вот это простое изменение:

```
public interface PlaneRepository
    extends ReactiveCrudRepository<Aircraft, String> {}
```

Это изменение интерфейса `PlaneRepository` естественным образом ведет к следующему пункту нашего пути — классу `PlaneFinderService`, в котором метод `getAircraft()` возвращает результат `PlaneRepository::saveAll`, когда воздушное судно найдено, и результат метода `saveSamplePositions()` в противном случае. Заменяем возвращаемое значение, блокирующий `Iterable<Aircraft>`, на правильный `Flux<Aircraft>` для методов `getAircraft()` и `saveSamplePositions()`:

```
public Flux<Aircraft> getAircraft() {
    ...
}

private Flux<Aircraft> saveSamplePositions() {
    ...
}
```

Поскольку метод `getCurrentAircraft()` класса `PlaneController` вызывает `PlaneFinderService::getAircraft`, то теперь возвращает `Flux<Aircraft>`. Так что надо внести изменения и в сигнатуру `PlaneController::getCurrentAircraft`:

```
public Flux<Aircraft> getCurrentAircraft() throws IOException {
    ...
}
```

Использование H2 с JPA — довольно зрелая технология, соответствующие спецификации, а также API и библиотеки разрабатываются в течение уже примерно десяти лет. R2DBC — относительно недавняя разработка, и хотя ее поддержка быстро расширяется, некоторые возможности, поддерживаемые JPA Spring Data для H2, еще только предстоит реализовать. Никаких особых затруднений это не вызывает, просто учитывайте это, если решите использовать реляционную базу данных, в данном случае H2, реактивно.

В настоящее время для использования H2 с R2DBC необходимо создать и настроить компонент `ConnectionFactoryInitializer`. Настройка состоит всего из двух шагов:

- установки фабрики соединений для автоматически сконфигурированного компонента `ConnectionFactory`, внедряемого в виде параметра;
- настройки средства заполнения базы данных для выполнения одного или нескольких сценариев инициализации или повторной инициализации базы данных нужным образом.

Напомним, что при использовании Spring Data JPA с H2 соответствующая таблица в базе данных H2 создается при помощи соответствующего класса `@Entity`. Этот шаг выполняется вручную с помощью стандартного сценария на DDL (язык описания данных) SQL при использовании H2 с R2DBC:

```
DROP TABLE IF EXISTS aircraft;

CREATE TABLE aircraft (id BIGINT auto_increment primary key,
callsign VARCHAR(7), squawk VARCHAR(4), reg VARCHAR(8), flightno VARCHAR(10),
route VARCHAR(30), type VARCHAR(4), category VARCHAR(2),
altitude INT, heading INT, speed INT, vert_rate INT, selected_altitude INT,
```

```
lat DOUBLE, lon DOUBLE, barometer DOUBLE, polar_distance DOUBLE,
polar_bearing DOUBLE, is_adsb BOOLEAN, is_on_ground BOOLEAN,
last_seen_time TIMESTAMP, pos_update_time TIMESTAMP, bds40_seen_time TIMESTAMP);
```



Это дополнительный не уникальный шаг. Он требуется во многих SQL-базах данных в ходе работы с Spring Data JPA. База H2 была исключением из правила.

Далее на очереди — код класса `DbConxInit` (Database Connection Initializer, класс инициализации соединения с базой данных). Необходимый метод создания компонента, первый в нем, — `initializer()`, который генерирует нужный компонент `ConnectionFactoryInitializer`. Второй метод генерирует компонент `CommandLineRunner`, выполняемый после задания конфигурации класса. `CommandLineRunner` — функциональный интерфейс с единственным абстрактным методом `run()`. Поэтому в качестве его реализации я указал лямбда-выражение, наполняющее содержимым `PlaneRepository`, включающий один объект `Aircraft`, а затем выводящее его на экран. В приведенном коде аннотация `@Bean` метода `init()` закомментирована, так что он не вызывается, компонент `CommandLineRunner` не создается и пример записи не сохраняется:

```
import io.r2dbc.spi.ConnectionFactory;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.ClassPathResource;
import org.springframework.r2dbc.connection.init.ConnectionFactoryInitializer;
import org.springframework.r2dbc.connection.init.ResourceDatabasePopulator;

@Configuration
public class DbConxInit {
    @Bean
    public ConnectionFactoryInitializer
            initializer(@Qualifier("connectionFactory")
            ConnectionFactory connectionFactory) {
        ConnectionFactoryInitializer initializer =
            new ConnectionFactoryInitializer();
        initializer.setConnectionFactory(connectionFactory);
        initializer.setDatabasePopulator(
            new ResourceDatabasePopulator(new ClassPathResource("schema.sql"))
        );
        return initializer;
    }

//    @Bean // Раскомментируйте аннотацию @Bean для добавления примера данных
    public CommandLineRunner init(PlaneRepository repo) {
```

```
        return args -> {
            repo.save(new Aircraft("SAL001", "N12345", "SAL001", "LJ",
                    30000, 30, 300,
                    38.7209228, -90.4107416))
                .thenMany(repo.findAll())
                    .subscribe(System.out::println);
        };
    }
}
```

Лямбда-выражение `CommandLineRunner` заслуживает определенных пояснений. Оно обладает типичной для лямбда-выражений структурой `x -> { <выполняемый код> }`, но код внутри отличается несколькими характерными для Reactive Streams особенностями.

Сначала объявлена операция `repo::save`, сохраняющая указанное содержимое — в данном случае новый объект `Aircraft` — и возвращающая `Mono<Aircraft>`. Можно просто подписаться (`subscribe()`) на этот результат и вывести его в журнал/на экран для проверки. Но не помешает обзавестись привычкой сохранять все нужные примеры данных, а затем запрашивать у репозитория список всех записей, чтобы окончательно проверить итоговое состояние таблицы на текущий момент. В результате этого должны быть выведены все записи.

Напомним, впрочем, что реактивный код — не блокирующий, так что возникает вопрос: как убедиться, прежде чем начинать обработку, что все предыдущие операции завершились? В данном случае как убедиться, что все записи сохранены, прежде чем пытаться их извлечь? В Project Reactor существуют операторы, которые ожидают сигнала о завершении и лишь затем переходят к следующей функции в цепочке. Оператор `then()` ожидает на входе `Mono`, а затем еще один объект `Mono` для дальнейшего выполнения. Показанный в предыдущем примере оператор `thenMany()` ждет завершения всех `Publisher`, расположенных ранее в конвейере, после чего выполняет новый `Flux`. В методе `init`, создающем компонент `CommandLineRunner`, `repo.findAll()` выдает `Flux<Aircraft>`, делая как раз то, что нужно.

Наконец, я подписался на `Flux<Aircraft>`, получаемый от `repo.findAll()`, и вывожу результаты в консоль. Заносить результаты в журнал не обязательно, и на самом деле обычный `subscribe()` прекрасно справляется с запуском потока данных. Но зачем нужно подписываться?

За несколькими исключениями, все классы `Publisher` Reactive Streams — так называемые холодные издатели (cold publisher), то есть в отсутствие подписчиков они не выполняют никакой работы и не потребляют ресурсы. Благодаря этому эффективность работы, а значит и масштабируемость, максимальна,

что логично, но для новичков в реактивном программировании становится подводным камнем. Если вы не возвращаете объект `Publisher` вызывающему коду для подписки и дальнейшего использования там, не забудьте добавить вызов `subscribe()`, чтобы активировать его или приводящую к нему цепочку операций.

ДЕКЛАРАТИВНЫЙ ПОДХОД

Нереактивный код я часто называю *блокирующим*, что в большинстве случаев логично, ведь такой код, за несколькими заметными исключениями, выполняется последовательно. Выполнение строки кода начинается по завершении выполнения предыдущей. Реактивный же код — не блокирующий, если не вызывает блокирующий код (это мы обсудим в одной из следующих глав), в результате чего последовательные строки кода никак не разграничивают выполняемые инструкции. Это обстоятельство несколько смущает разработчиков, особенно тех, кто ранее сталкивался с последовательно выполняемым кодом или до сих пор занимается им бóльшую часть времени, то есть практически всех нас.

Бóльшая часть блокирующего кода — императивный код, в котором мы определяем, как сделать что-либо. Возьмем для примера цикл `for`, в котором делаем следующее:

- объявляем переменную и присваиваем ей начальное значение;
- проверяем, не выходит ли она за внешнюю границу;
- выполняем какие-то инструкции;
- модифицируем значение переменной;
- повторяем цикл, начиная с проверки значения.

В блокирующем коде есть очень полезные декларативные конструкции. Вероятно, наиболее известным и любимым всеми примером является API Java Stream — декларативные лакомства, придающие блюду остроту, но все вместе составляющие лишь довольно небольшую его долю. С реактивным программированием дело обстоит иначе.

Наверное, из-за названия Reactive Streams вызывает ассоциации с интерфейсом Stream Java. И хотя они никак не связаны, декларативный подход пакета `java.util.Stream` годится и для Reactive Streams: объявление итогового результата при помощи цепочки функций, работающих путем передачи неизменяемых результатов от одной функции к следующей, что структурирует реактивный код как визуально, так и логически.

Наконец, необходимо внести некоторые изменения в класс предметной области `Aircraft` вследствие различий JPA и R2DBC и их кода поддержки H2. Аннотация `@Entity` JPA больше не нужна, как и аннотация `@GeneratedValue` для соответствующей первичному ключу переменной экземпляра `id`. Удаление их обеих,

а также соответствующих операторов импорта — единственные изменения, которые нужны при переводе PlaneFinder с JPA на R2DBC с использованием H2.

На случай, если понадобятся примеры данных, для показанного ранее компонента `CommandLineRunner` и его вызова конструктора с ограниченным числом полей я добавил соответствующий дополнительный конструктор в класс `Aircraft`. Обратите внимание на то, что это необходимо лишь в случае, если нужно создать экземпляр `Aircraft` без указания всех параметров, требуемых конструктором Lombok, основанным на аннотации `@AllArgsConstructor`. Видите, из конструктора с ограниченным числом аргументов вызывается полноценный конструктор с полным набором аргументов:

```
    public Aircraft(String callsign, String reg, String flightno, String type,
                    int altitude, int heading, int speed,
                    double lat, double lon) {
        this(null, callsign, "sqwk", reg, flightno, "route", type, "ct",
                altitude, heading, speed, 0, 0,
                lat, lon, 0D, 0D, 0D,
                false, true,
                Instant.now(), Instant.now(), Instant.now());
    }
```

Теперь пришло время проверить нашу работу.

После запуска приложения PlaneFinder из IDE возвращаемся к HTTPie в окне терминала для тестирования модифицированного кода:

```
mheckler-a01 :: OReilly/code " http -b :7634/aircraft
[
    {
        "altitude": 37000,
        "barometer": 0.0,
        "bds40_seen_time": null,
        "callsign": "EDV5123",
        "category": "A3",
        "flightno": "DL5123",
        "heading": 131,
        "id": 1,
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-09-19T21:40:56Z",
        "lat": 38.461505,
        "lon": -89.896606,
        "polar_bearing": 156.187542,
        "polar_distance": 32.208164,
        "pos_update_time": "2020-09-19T21:40:56Z",
        "reg": "N582CA",
```

```
    "route": "DSM-ATL",
    "selected_altitude": 0,
    "speed": 474,
    "squawk": "3644",
    "type": "CRJ9",
    "vert_rate": -64
},
{
    "altitude": 38000,
    "barometer": 0.0,
    "bds40_seen_time": null,
    "callsign": null,
    "category": "A4",
    "flightno": "FX3711",
    "heading": 260,
    "id": 2,
    "is_adsb": true,
    "is_on_ground": false,
    "last_seen_time": "2020-09-19T21:40:57Z",
    "lat": 39.348558,
    "lon": -90.330383,
    "polar_bearing": 342.006425,
    "polar_distance": 24.839372,
    "pos_update_time": "2020-09-19T21:39:50Z",
    "reg": "N924FD",
    "route": "IND-PHX",
    "selected_altitude": 0,
    "speed": 424,
    "squawk": null,
    "type": "B752",
    "vert_rate": 0
},
{
    "altitude": 35000,
    "barometer": 1012.8,
    "bds40_seen_time": "2020-09-19T21:41:11Z",
    "callsign": "JIA5304",
    "category": "A3",
    "flightno": "AA5304",
    "heading": 112,
    "id": 3,
    "is_adsb": true,
    "is_on_ground": false,
    "last_seen_time": "2020-09-19T21:41:12Z",
    "lat": 38.759811,
    "lon": -90.173632,
    "polar_bearing": 179.833023,
    "polar_distance": 11.568717,
    "pos_update_time": "2020-09-19T21:41:11Z",
    "reg": "N563NN",
```

```
        "route": "CLT-RAP-CLT",
        "selected_altitude": 35008,
        "speed": 521,
        "squawk": "6506",
        "type": "CRJ9",
        "vert_rate": 0
    }
]
```

Убедившись, что переделанное реактивное приложение PlaneFinder работает так, как ожидалось, можем обратить внимание на приложение Aircraft Positions.

Модифицируем приложение Aircraft Positions

В настоящее время в проекте `aircraft-positions` используются Spring Data JPA и H2, как и в PlaneFinder, когда оно было блокирующим приложением. И хотя можно переделать приложение Aircraft Positions, чтобы применять R2DBC и H2 подобно тому, как это сейчас делает PlaneFinder, такой рефакторинг проекта `aircraft-positions` дает нам прекрасную возможность изучить другие варианты реактивных баз данных.

MongoDB нередко оказывается в первых рядах по части инноваций в сфере баз данных, и действительно, она — один из первых поставщиков баз данных какого-либо вида, разработавший полностью реактивные драйверы для своей базы данных. Разработка приложений с помощью Spring Data и MongoDB происходит очень гладко, отражая зрелость поддержки MongoDB реактивных потоков данных. MongoDB идеально подходит для реактивного рефакторинга Aircraft Positions.

Необходимо внести кое-какие изменения в файл сборки, в данном случае `pom.xml`. Сначала убираем ненужные зависимости для Spring MVC, Spring Data JPA и H2:

- `spring-boot-starter-web`;
- `spring-boot-starter-data-jpa`;
- `h2`.

Далее добавляем следующие зависимости для будущей реактивной версии:

- `spring-boot-starter-data-mongodb-reactive`;
- `de.flapdoodle.embed.mongo`;
- `reactor-test`.



Зависимость spring-boot-starter-webflux уже была добавлена для WebClient, так что добавлять ее не нужно.

Как и в главе 6, для этого примера я задействую встроенную MongoDB. Поскольку встроенная MongoDB обычно применяется лишь для тестирования, то в зависимости обычно указывается область видимости `test`, а поскольку мы используем ее во время реальной работы приложения, то опустим либо удалим этот квалификатор из файла сборки. Обновленные зависимости файла сборки `pom.xml` Maven выглядят следующим образом:

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-mongodb-reactive</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webflux</artifactId>
    </dependency>

    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
        <exclusions>
            <exclusion>
                <groupId>org.junit.vintage</groupId>
                <artifactId>junit-vintage-engine</artifactId>
            </exclusion>
        </exclusions>
    </dependency>
    <dependency>
        <groupId>de.flapdoodle.embed</groupId>
        <artifactId>de.flapdoodle.embed.mongo</artifactId>
    </dependency>
    <dependency>
```

```
        <groupId>io.projectreactor</groupId>
        <artifactId>reactor-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Обновляем зависимости с помощью командной строки или IDE — и мы готовы к рефакторингу.

Вновь начнем с очень простого изменения интерфейса `AircraftRepository`, который теперь будет расширять интерфейс `ReactiveCrudRepository`, а не блокирующий `CrudRepository`:

```
public interface AircraftRepository extends ReactiveCrudRepository<Aircraft,
Long> {}
```

Модифицировать `PositionController` несложно, поскольку `WebClient` уже обменивается информацией с помощью типов `Publisher` Reactive Streams. Я определил локальную переменную `Flux<Aircraft> aircraftFlux`, после чего связал цепочкой нужные декларативные операции для очистки репозитория от ранее извлеченных данных о местоположении воздушных судов, извлечения новых местоположений, преобразования их в экземпляры класса `Aircraft`, фильтрации местоположений без указанного регистрационного номера воздушного судна и сохранения их в репозиторий встраиваемой базы данных MongoDB. Затем добавил переменную `aircraftFlux` в `Model` для использования в интерфейсном веб-API и вернул название шаблона Thymeleaf для представления:

```
@RequiredArgsConstructor
@Controller
public class PositionController {
    @NonNull
    private final AircraftRepository repository;
    private WebClient client
        = WebClient.create("http://localhost:7634/aircraft");

    @GetMapping("/aircraft")
    public String getCurrentAircraftPositions(Model model) {
        Flux<Aircraft> aircraftFlux = repository.deleteAll()
                .thenMany(client.get()
                        .retrieve()
                        .bodyToFlux(Aircraft.class)
                        .filter(plane -> !plane.getReg().isEmpty())
                        .flatMap(repository::save));

        model.addAttribute("currentPositions", aircraftFlux);
        return "positions";
    }
}
```

Наконец, необходимо внести несколько мелких изменений в сам класс предметной области `Aircraft`. Аннотация уровня класса `@Entity` — специфика JPA, соответствующая аннотация MongoDB — `@Document`, она указывает, что экземпляры класса необходимо сохранять в базе данных в виде документов. Кроме того, использовавшаяся ранее аннотация `@Id` ссылается на `javax.persistence.Id`, который исчезает после исключения зависимости JPA. При замене `import javax.persistence.Id;` на `import org.springframework.data.annotation.Id;` контекст идентификатора таблицы для использования с MongoDB сохраняется. Покажем файл класса полностью:

```
import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import java.time.Instant;

@Document
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Aircraft {
    @Id
    private Long id;
    private String callsign, squawk, reg, flightno, route, type, category;

    private int altitude, heading, speed;
    @JsonProperty("vert_rate")
    private int vertRate;
    @JsonProperty("selected_altitude")
    private int selectedAltitude;

    private double lat, lon, barometer;
    @JsonProperty("polar_distance")
    private double polarDistance;
    @JsonProperty("polar_bearing")
    private double polarBearing;

    @JsonProperty("is_adsb")
    private boolean isADSB;
    @JsonProperty("is_on_ground")
    private boolean isOnGround;

    @JsonProperty("last_seen_time")
    private Instant lastSeenTime;
```

```
    @JsonProperty("pos_update_time")
    private Instant posUpdateTime;
    @JsonProperty("bds40_seen_time")
    private Instant bds40SeenTime;
}
```

Запустив оба приложения, PlaneFinder и Aircraft Positions, возвращаемся на вкладку браузера, вводим в адресной строке `http://localhost:8080` и получаем в результате приведенную на рис. 8.1 страницу.



Рис. 8.1. Целевая страница приложения Aircraft Positions, index.html

При выборе ссылки Click here загружается страница отчета `Aircraft Positions` (рис. 8.2).

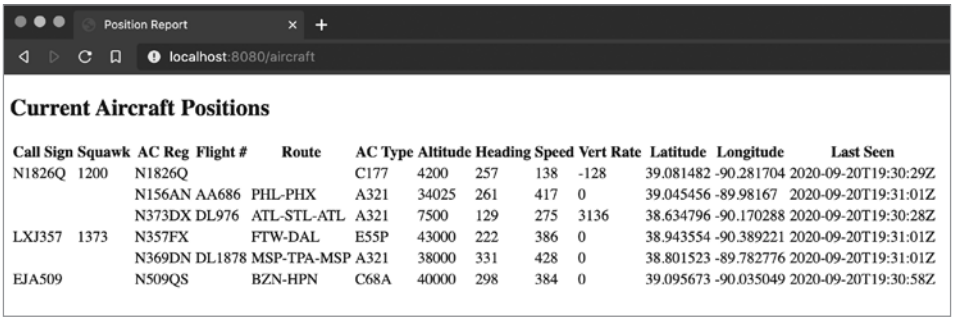


Рис. 8.2. Страница отчета приложения Aircraft Positions

При каждом из периодических обновлений страница повторно запрашивает PlaneFinder и, как и прежде, актуализирует по требованию отчет текущими данными с одним ключевым отличием: местоположения воздушных судов, передаваемые в шаблон Thymeleaf `positions.html` для отображения, теперь представляют собой не полностью сформированный блокирующий объ-

ект `List`, а `Publisher` Reactive Streams, конкретно — типа `Flux`. В следующем разделе мы обсудим это подробнее, а пока важно понимать, что для такого согласования содержимого разработчику не требуется прилагать никаких усилий.

Реактивный Thymeleaf

Как упоминалось в главе 7, абсолютное большинство веб-приложений клиентской части сейчас создается с помощью HTML и JavaScript. Это не отменяет существования немалого числа реальных приложений, использующих технологии представлений или шаблонизации для достижения поставленных целей. Это также не означает, что названные технологии перестали обеспечивать простое и эффективное удовлетворение определенных требований.

Thymeleaf поддерживает RS на трех уровнях, позволяя разработчикам выбирать тот, который лучше всего удовлетворяет их требованиям. Как уже упоминалось, можно изменить серверную обработку в прикладной части так, чтобы использовать Reactive Streams и чтобы Reactor подавал в Thymeleaf значения, поставляемые каким-либо `Publisher` — `Mono` или `Flux`, вместо `Object<T>` и `Iterable<T>`. В результате этого клиентская часть не становится реактивной, но если главная задача — преобразовать логику серверной части так, чтобы убрать с помощью Reactive Streams блокирование и реализовать контроль потока данных между сервисами, то это прекрасный путь развертывания вспомогательного интерфейсного приложения с наименьшими затратами.

Thymeleaf поддерживает также режимы фрагментации и управления данными для поддержки Spring WebFlux, которые подразумевают применение технологии событий, отправленных сервером (Server Sent Events, SSE), и кода JavaScript для передачи данных браузеру. И хотя оба этих режима полностью допустимы, объем необходимого для достижения желаемой цели кода JavaScript может склонить чашу весов в сторону 100 % логики HTML + JavaScript на стороне клиентской части, а не шаблонизации + HTML + JavaScript. Конечно, это решение сильно зависит от конкретных требований и должно приниматься создающим и поддерживающим подобную функциональность разработчиком.

В предыдущем разделе я показал, как перевести функциональность прикладной части на конструкции RS и как Spring Boot обеспечивает работу функциональности клиентской части с помощью Reactor + Thymeleaf, упрощая преобразование блокирующих систем приложений с минимальным временем простоя.

Для текущего сценария использования этого вполне достаточно, чтобы изучить варианты дальнейшего развития функциональности прикладной части, прежде чем вернуться (в одной из следующих глав) к расширению возможностей клиентской части.

RSocket и полностью реактивное взаимодействие между процессами

В этой главе я уже заложил фундамент для межпроцессного взаимодействия отдельных приложений с помощью Reactive Streams. И хотя созданная распределенная система действительно использует реактивные конструкции, она еще не достигла предела своих потенциальных возможностей. Пересечение границы сети посредством высокоуровневых транспортных протоколов на основе HTTP накладывает ограничения, связанные с моделью «запрос — ответ», и один переход на WebSocket не убирает их все. Для гибкого полноценного устранения недостатков межпроцессного взаимодействия был создан RSocket.

Что такое RSocket

RSocket, результат сотрудничества нескольких ведущих IT-компаний и передовых инноваторов, — невероятно быстрый двоичный протокол, который можно использовать поверх транспортных механизмов TCP, WebSocket и Aeron. RSocket поддерживает четыре асинхронные модели взаимодействия:

- «запрос — ответ»;
- «запрос — поток данных»;
- «выстрелил и забыл»;
- канал запроса (двунаправленный поток данных).

RSocket основан на парадигме реактивных потоков данных и Project Reactor, он делает возможными полносвязные системы приложений, предоставляя в то же время механизмы, повышающие гибкость и отказоустойчивость. После установления соединения между двумя приложениями или сервисами исчезают различия между клиентом и сервером и взаимосвязь становится по сути одноранговой. Любая сторона может инициировать любую из четырех приведенных моделей взаимодействия, подходящих для следующих сценариев использования.

- Взаимодействие 1:1, при котором одна из сторон отправляет запрос и получает ответ от другой стороны.
- Взаимодействие 1:*N*, при котором одна из сторон отправляет запрос и получает поток ответов от другой стороны.
- Взаимодействие 1:0, при котором одна из сторон отправляет запрос.
- Полный двунаправленный канал передачи, в котором обе стороны могут отправлять запросы, ответы или потоки данных любого вида без запроса.

Как видите, RSocket исключительно гибок. А как двоичный протокол с упором на быстродействие, он быстр. Помимо этого, RSocket отказоустойчив, обеспечивая восстановление разорванного соединения с автоматическим возобновлением обмена сообщениями с места, на котором он прервался. А поскольку RSocket основан на Reactor, применяющие его разработчики могут рассматривать отдельные приложения как полностью интегрированную систему, ведь сетевые границы теперь никак не ограничивают управление потоком.

Spring Boot с его легендарной автоконфигурацией — вероятно, самый быстрый и удобный для разработчиков Java и Kotlin способ использования RSocket.

Применяем RSocket на практике

В настоящее время приложения PlaneFinder и Aircraft Positions применяют для обмена информацией транспортные протоколы на основе HTTP. Очевидный следующий шаг — перевод обоих приложений на использование RSocket.

Перевод приложения PlaneFinder на RSocket

Прежде всего я добавил зависимость RSocket в файл сборки PlaneFinder:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-rsocket</artifactId>
</dependency>
```

После быстрого повторного импорта Maven можно приступить к рефакторингу кода.

Пока я оставлю существующую конечную точку `/aircraft` без изменений и добавлю конечную точку RSocket в `PlaneController`. Чтобы поместить как конечные точки REST, так и конечные точки RSocket в один класс, я разделил

включенную в аннотацию `@RestController` функциональность на составные части — `@Controller` и `@ResponseBody`.

Замена аннотации уровня класса `@RestController` на `@Controller` означает необходимость снабжения соответствующего метода аннотацией `@ResponseBody` для всех конечных точек REST, из которых необходимо возвращать объекты напрямую в виде JSON, например, для уже существующей конечной точки `/aircraft`, связанной с методом `getCurrentAircraft()`. Преимущество этого кажущегося шага назад в следующем: конечные точки RSocket теперь могут быть определены в том же классе `@Controller`, что и конечные точки REST, сохраняя все входы и выходы PlaneFinder в одном и только одном месте:

```
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;
import reactor.core.publisher.Flux;

import java.io.IOException;
import java.time.Duration;

@Controller
public class PlaneController {
    private final PlaneFinderService pfService;

    public PlaneController(PlaneFinderService pfService) {
        this.pfService = pfService;
    }

    @ResponseBody
    @GetMapping("/aircraft")
    public Flux<Aircraft> getCurrentAircraft() throws IOException {
        return pfService.getAircraft();
    }

    @MessageMapping("acstream")
    public Flux<Aircraft> getCurrentACStream() throws IOException {
        return pfService.getAircraft().concatWith(
                Flux.interval(Duration.ofSeconds(1))
                        .flatMap(l -> pfService.getAircraft()));
    }
}
```

Для создания повторяющегося потока данных о местоположении воздушных судов, отправляемых каждую секунду, я создал метод `getCurrentACStream()` и снабдил его аннотацией `@MessageMapping`. Обратите внимание: поскольку ото-

бражения путей RSocket не строятся от корневого пути, как адреса или конечные точки HTTP, в этом отображении не требуется никакого слеша (/).

Следующий шаг после описания конечной точки и обслуживающего ее метода — указание порта, на котором RSocket будет выполнять прослушивание на предмет запросов соединений. Я сделал это в файле `application.properties`, добавив значение свойства `spring.rsocket.server.port` к уже существующему свойству `server.port`, основанному на протоколе HTTP:

```
server.port=7634
spring.rsocket.server.port=7635
```

Этого одного задания порта сервера RSocket достаточно Spring Boot для конфигурации всего приложения как сервера RSocket, создания всех необходимых компонентов и выполнения всех нужных настроек. Напомним, что хотя изначально одно из двух приложений — участников соединения RSocket должно играть роль сервера, после установления соединения различия между клиентом (приложением, инициирующим соединение) и сервером (приложением, прослушивающим на предмет соединения) исчезают.

Этих небольших изменений достаточно для работы нашего приложения PlaneFinder с RSocket. Просто запустите приложение, и оно будет готово принимать запросы на соединение.

Перевод приложения Aircraft Positions на RSocket

Первый этап добавления RSocket состоит в добавлении зависимости RSocket в файл сборки — в данном случае для приложения Aircraft Positions:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-rsocket</artifactId>
</dependency>
```

Не забудьте, прежде чем продолжать, выполнить повторный импорт, чтобы Maven узнал об изменениях в проекте. А теперь займемся кодом.

Аналогично тому, что мы делали с PlaneFinder, я переделал класс `PositionController` для создания единой точки всех входящих/исходящих сообщений. Замена аннотации уровня класса `@RestController` на `@Controller` позволила включить конечные точки RSocket вместе с конечной точкой на основе HTTP, но в данном случае шаблоноориентированной, вводящей в работу шаблон Thymeleaf `positions.html`.

Чтобы приложение Aircraft Positions играло роль клиента RSocket, я создал компонент `RSocketRequester`, выполнив автосвязывание компонента `RSocketRequester.Builder` с помощью внедрения зависимости через конструктор. Spring Boot автоматически создает компонент `RSocketRequester.Builder` в результате добавления в проект зависимости для RSocket. Внутри конструктора я воспользовался `builder` и его методом `tcp()` для создания TCP-соединения, в данном случае с сервером RSocket приложения PlaneFinder.



Поскольку нужно было внедрить компонент (RSocketRequester.Builder), служащий для создания экземпляра другого объекта (RSocketRequester), мне пришлось создать конструктор. А так как теперь у меня появился конструктор, я удалил аннотацию уровня класса @RequiredArgsConstructor и аннотации Lombok уровня переменных экземпляра @NonNull и просто добавил в написанный конструктор еще и параметр AircraftRepository. В любом случае Spring Boot автоматически связывает компонент, который присваивается переменной экземпляра repository.

Для проверки правильности функционирования соединения RSocket и потока данных я создал конечную точку `/acstream` на основе HTTP, указав, что она должна возвращать поток отправляемых сервером событий (Server Sent Events, SSE) в качестве результата, и снабдив ее аннотацией `@ResponseBody`, указывающей, что ответ должен состоять непосредственно из объектов в формате JSON. С помощью переменной-члена `RSocketRequester`, начальное значение которой задается в конструкторе, я указал маршрут, которому должна соответствовать описанная в PlaneFinder конечная точка RSocket, отправил немного данных (это необязательно, я не передавал в этом конкретном запросе никаких полезных данных) и получил `Flux` объектов `Aircraft`, возвращаемых из PlaneFinder:

```
import org.springframework.http.MediaType;
import org.springframework.messaging.rsocket.RSocketRequester;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;
import org.springframework.web.reactive.function.client.WebClient;
import reactor.core.publisher.Flux;

@Controller
public class PositionController {
    private final AircraftRepository repository;
    private final RSocketRequester requester;
    private WebClient client =
            WebClient.create("http://localhost:7634/aircraft");
```

```
    public PositionController(AircraftRepository repository,
                              RSocketRequester.Builder builder) {
        this.repository = repository;
        this.requester = builder.tcp("localhost", 7635);
    }

    // Конечная точка HTTP, созданный ранее инициатор HTTP-запроса
    @GetMapping("/aircraft")
    public String getCurrentAircraftPositions(Model model) {
        Flux<Aircraft> aircraftFlux = repository.deleteAll()
                .thenMany(client.get()
                        .retrieve()
                        .bodyToFlux(Aircraft.class)
                        .filter(plane -> !plane.getReg().isEmpty())
                        .flatMap(repository::save));
        model.addAttribute("currentPositions", aircraftFlux);
        return "positions";
    }

    // Конечная точка HTTP, клиентская конечная точка RSocket
    @ResponseBody
    @GetMapping(value = "/acstream",
            produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<Aircraft> getCurrentACPositionsStream() {
        return requester.route("acstream")
                .data("Requesting aircraft positions")
                .retrieveFlux(Aircraft.class);
    }
}
```

Для проверки работоспособности соединения RSocket и передачи приложением PlaneFinder данных приложению Aircraft Positions я запустил Aircraft Positions и вернулся к терминалу и HTTPie, добавив флаг `-S` к команде для потоковой обработки данных по мере их поступления, а не ожидания завершения формирования тела ответа. Пример результатов, отредактированных для краткости, приведен далее:

```
mheckler-a01 :: ~ " http -S :8080/acstream
HTTP/1.1 200 OK
Content-Type: text/event-stream;charset=UTF-8
transfer-encoding: chunked

data:{"id":1,"callsign":"RPA3427","squawk":"0526","reg":"N723YX","flightno":
"UA3427","route":"IAD-MCI","type":"E75L","category":"A3","altitude":36000,
"heading":290,"speed":403,"lat":39.183929,"lon":-90.72259,"barometer":0.0,
"vert_rate":64,"selected_altitude":0,"polar_distance":29.06486,
"polar_bearing":297.519943,"is_adsb":true,"is_on_ground":false,
"last_seen_time":"2020-09-20T23:58:51Z",
"pos_update_time":"2020-09-20T23:58:49Z","bds40_seen_time":null}
```

```
data:{"id":2,"callsign":"EDG76","squawk":"3354","reg":"N776RB","flightno":"",
"route":"TEB-VNY","type":"GLF5","category":"A3","altitude":43000,"heading":256,
"speed":419,"lat":38.884918,"lon":-90.363026,"barometer":0.0,"vert_rate":64,
"selected_altitude":0,"polar_distance":9.699159,"polar_bearing":244.237695,
"is_adsb":true,"is_on_ground":false,"last_seen_time":"2020-09-20T23:59:22Z",
"pos_update_time":"2020-09-20T23:59:14Z","bds40_seen_time":null}

data:{"id":3,"callsign":"EJM604","squawk":"3144","reg":"N604SD","flightno":"",
"route":"ENW-HOU","type":"C56X","category":"A2","altitude":38000,"heading":201,
"speed":387,"lat":38.627464,"lon":-90.01416,"barometer":0.0,"vert_rate":-64,
"selected_altitude":0,"polar_distance":20.898095,"polar_bearing":158.9935,
"is_adsb":true,"is_on_ground":false,"last_seen_time":"2020-09-20T23:59:19Z",
"pos_update_time":"2020-09-20T23:59:19Z","bds40_seen_time":null}
```


Эти результаты подтверждают, что данные поступают из PlaneFinder в Aircraft Positions через Reactive Streams по RSocket-соединению с помощью модели *«запрос — поток данных»*. Все системы работают нормально.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Полный код для этой главы находится в ветке chapter8end в репозитории кода.

Резюме

Реактивное программирование для разработчиков — это способ полнее использовать имеющиеся ресурсы. В мире взаимосвязанных систем, который становится все более распределенным, это универсальное решение задач масштабируемости. Оно включает расширение механизмов масштабирования за пределы границ приложения, в каналы обмена информацией. Инициатива Reactive Streams, и в частности Project Reactor, служит мощным, быстродействующим, гибким фундаментом для достижения максимальной масштабируемости всей системы.

В этой главе я познакомил вас с реактивным программированием и продемонстрировал, как Spring лидирует в разработке и совершенствовании многочисленных инструментов и технологий. Я рассказал о блокирующем и неблокирующем обмене информацией и механизмах, предоставляющих эти возможности.

Далее я показал, как обеспечить реактивный доступ к SQL- и NoSQL-базам данных путем рефакторинга приложений PlaneFinder и Aircraft Positions для использования Spring WebFlux/Project Reactor. Проект Reactive Relational Database Connectivity — реактивная замена API Java Persistenc (JPA), работающая с несколькими SQL-базами данных. MongoDB и другие NoSQL-базы данных

предоставляют альтернативные реактивные драйверы, без проблем работающие со Spring Data и Spring Boot.

В главе обсуждались также варианты интеграции реактивных типов данных в клиентской части и был показан предоставляемый Thymeleaf ограниченный путь миграции для приложений, все еще использующих технологии сгенерированных представлений. В следующих главах рассмотрим дополнительные варианты.

Наконец, я продемонстрировал, как поднять межпроцессное взаимодействие на совершенно новый уровень с помощью RSocket. Поддержка RSocket в Spring Boot позволяет быстро добиться высокого быстродействия, масштабируемости, отказоустойчивости и продуктивности разработчиков.

В следующей главе мы займемся тестированием и расскажем, как Spring Boot делает возможными лучшие, более быстрые и простые методы тестирования, как создавать эффективные модульные тесты и как отшлифовать методику тестирования для ускорения цикла сборки и тестирования.

ГЛАВА 9

Тестирование приложений Spring Boot для повышения их готовности к продакшену

В этой главе обсуждаются и демонстрируются основные аспекты тестирования приложений Spring Boot. И хотя тема тестирования очень многогранна, мы сосредоточимся на важнейших элементах тестирования приложений Spring Boot, значительно повышающих готовность любого из них к продакшену. В том числе обсудим такие вопросы, как модульное тестирование, целостное тестирование приложений с помощью `@SpringBootTest`, написание эффективных модульных тестов с помощью JUnit и тестовых срезов Spring Boot для изоляции субъектов тестирования и упрощения тестирования.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Пожалуйста, для начала извлеките из репозитория кода ветку chapter9begin.

Модульное тестирование

Модульное тестирование не случайно предшествует другим видам тестирования приложений: оно позволяет разработчику найти и исправить программные ошибки на самых ранних этапах цикла разработки и развертывания, а значит, с наименьшими затратами.

Если говорить самыми простыми словами, *модульное тестирование* (unit testing) означает проверку определенной единицы кода (модуля), изолированного до максимально возможной разумной степени. Количество возможных исходов

теста растет экспоненциально с ростом размера и сложности. Сокращение объема функциональности, проверяемой в каждом модульном тесте, упрощает его воплощение в жизнь, тем самым повышая вероятность учета всех вероятных и/или возможных исходов.

Только после успешной реализации модульного тестирования в достаточном объеме можно подключать интеграционное тестирование, тестирование UI/UX и т. д. К счастью, у Spring Boot есть возможности, упрощающие модульное тестирование, они включены по умолчанию в каждый создаваемый с помощью Spring Initializr проект, благодаря чему разработчики могут быстро приступить к тестированию, причем сделать именно то, что нужно.

Знакомимся с аннотацией @SpringBootTest

До сих пор я в основном говорил о коде из каталога `src/main/java` в созданных с помощью Spring Initializr проектах, начиная с основного класса приложения. Впрочем, во всех порожденных Spring Initializr приложениях Spring Boot есть соответствующая структура каталогов `src/test/java` с одним заранее созданным, но пока что пустым тестом.

Название ему дается в соответствии с основным классом приложения. Например, если основной класс приложения называется `MyApplication`, то основным тестовым классом будет `MyApplicationTest`. Такое взаимно однозначное соответствие обеспечивает как должную организацию кода, так и единообразие. Внутри класса теста Initializr создает единственный пустой тестовый метод в качестве отправной точки, чтобы разработка начиналась с чистой сборки. Потом можно будет добавить другие тестовые методы или, как делается обычно, дополнительные тестовые классы, соответствующие прочим классам приложения, с одним или более тестовым методом в каждом.

Обычно я рекомендую методику разработки через тестирование (Test Driven Development, TDD), при которой сначала создаются тесты, а код пишется только для того, чтобы они успешно выполнялись. Но поскольку я твердо верю, что для знакомства с тестированием в Boot необходимо понимать ключевые аспекты Spring Boot, то надеюсь, что читатель простит мне вынужденную задержку знакомства с материалами этой главы до той поры, пока мы не обсудим базовые вопросы.

С учетом этого вернемся к приложению Aircraft Positions и напишем для него несколько тестов.

ПРИМЕЧАНИЯ ОТНОСИТЕЛЬНО ТЕСТОВОГО ПОКРЫТИЯ

Существует немало убедительных аргументов в пользу каждого из уровней модульного тестирования, от минимального до 100%-ного тестового покрытия. Я считаю себя прагматиком, понимающим, что слишком мало — это, кхм, слишком мало, но и осознающим ложность принципа «чем больше, тем лучше».

У всего есть своя цена. При слишком малом числе тестов она становится очевидной очень быстро: ошибки и граничные случаи могут просочиться в версию для продакшена, нередко приводя к немалым проблемам и негативно влияя на финансовые последствия. Но и написание тестов для каждого метода доступа и изменения или каждого элемента открытого кода библиотеки или фреймворка также требует немалых затрат труда, зачастую практически впустую. Конечно, методы доступа и изменения могут меняться, да и нижележащий код способен привносить ошибки, но как часто подобное случалось в ваших проектах?

Для этой книги, как и для повседневной работы, я принял на вооружение концепцию достаточного объема тестирования, состоящую в написании тестов лишь для так называемого интересного поведения. Я обычно не пишу тесты для классов предметной области, простых методов доступа и изменения, давно устоявшегося кода Spring или чего-либо еще очень стабильного или максимально защищенного от неправильного использования, за несколькими заметными исключениями, о которых мы поговорим в свое время (см. комментарий ранее относительно интересного поведения). Отмечу также, что в реальных проектах эти оценки кода должны регулярно пересматриваться, ведь программное обеспечение не статично и постоянно развивается.

Лишь вы и ваша компания знаете свой профиль риска и уязвимые места.

Для демонстрации широчайшего спектра возможностей тестирования, предоставляемых Spring Boot самым прозрачным и сжатым образом, я вернусь к JPA-версии приложения Aircraft Positions в качестве основы для обсуждения тестирования в этой главе. Существует еще пару связанных с тестированием вопросов, несколько отклоняющихся от общей темы и дополняющих содержимое главы, но не представленных в этом проекте. Эти родственные вопросы мы рассмотрим в одной из следующих глав.

Важнейшие модульные тесты для приложения Aircraft Positions

Приложение Aircraft Positions в настоящий момент включает лишь один класс, который можно считать интересным поведением. `PositionController` для выдачи конечному пользователю текущих данных о местоположении воздушных

судов непосредственно или через веб-интерфейс предоставляет API, в котором могут производиться следующие действия:

- извлечение текущего местоположения воздушных судов из приложения PlaneFinder;
- сохранение этого местоположения в локальной базе данных;
- извлечение этого местоположения из локальной базы данных;
- возврат текущего местоположения непосредственно или путем добавления в объектную модель документа для веб-страницы.

Если даже забыть на минуту, что эта функциональность взаимодействует с внешним сервисом, она затрагивает каждый слой стека приложения, от пользовательского интерфейса до хранения и извлечения данных. Если вспомнить, что при должном подходе к тестированию требуется изолировать и тестировать маленькие связные элементы функциональности, становится ясно, что необходим итеративный подход к тестированию с пошаговым продвижением от текущего состояния кода без тестов к конечному состоянию в виде оптимизированных организации и тестирования приложения. Это точно отражает типичные, нацеленные на продакшен проекты.



Подобно тому как никогда не бывают завершенными реально используемые приложения, не бывает завершенным и тестирование. По мере развития кода приложения необходимо пересматривать и, возможно, переделывать или удалять или добавлять тесты, чтобы тестирование было эффективным.

Начнем с создания тестового класса, соответствующего классу `PositionController`. Механизм создания тестового класса различается в разных IDE, и конечно, можно создать его и вручную. Поскольку я в основном использую для разработки IntelliJ IDEA, то пользуюсь сочетанием клавиш CMD+N (В Windows Ctrl+N) или нажимаю правую кнопку мыши, а затем выбираю Generate для открытия меню Generate, после чего выбираю пункт Test для создания класса теста. После этого IntelliJ отображает всплывающее окно, приведенное на рис. 9.1.

Во всплывающем окне Create Test я оставил исходную библиотеку тестирования (Testing library) — JUnit 5. С тех пор как стал общедоступным Spring Boot 2.2, по умолчанию для модульных тестов приложений Spring Boot стала использоваться версия 5 библиотеки JUnit. Поддерживаются и многие другие варианты, включая JUnit 3 и 4, Spock, TestNG и пр., но JUnit 5 с ее движком Jupiter — прекрасный вариант с такими возможностями:

- улучшенное тестирование кода Kotlin по сравнению с предыдущими версиями;
- более эффективное одноразовое создание экземпляра, или настройка, или очистка тестового класса для всех тестов с помощью аннотаций `@BeforeAll` и `@AfterAll` уровня методов;
- поддержка тестов как JUnit 4, так и JUnit 5, если, конечно, версия 4 не была явным образом исключена из зависимостей.

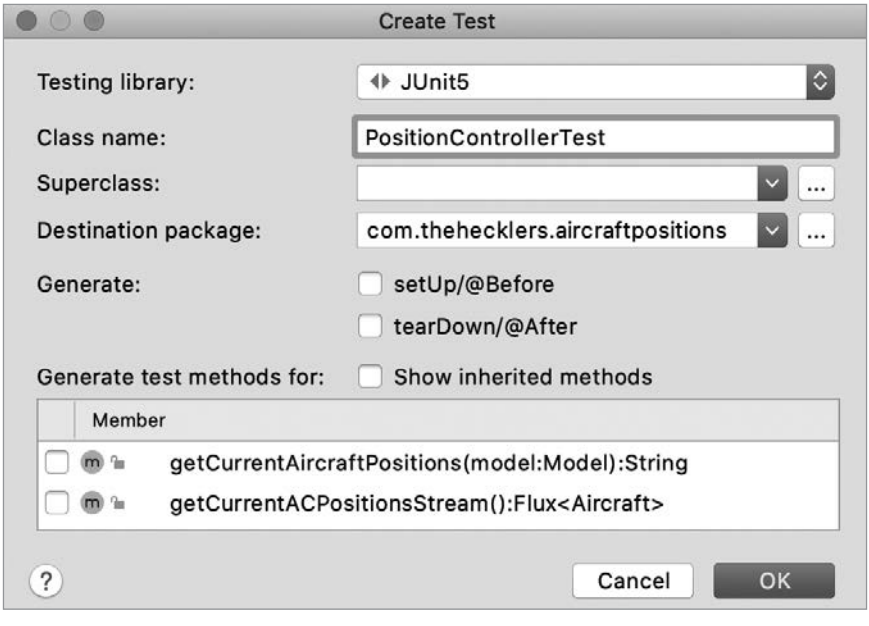


Рис. 9.1. Создание всплывающего окна Create Test, запущенного для класса PositionController

По умолчанию используется движок Jupiter JUnit 5, но предоставляется и старый вариант движка для обратной совместимости с тестами JUnit 4.

Я оставил предложенное название класса `PositionControllerTest`, устанавливаю флажки для генерации методов `setup/@Before` и `tearDown/@After`, а также флажок генерации тестового метода для метода `getCurrentAircraftPositions()` (рис. 9.2).

После нажатия кнопки OK IntelliJ создала класс `PositionControllerTest` с выбранными методами, открыв его в IDE:

```
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
```

```
import org.junit.jupiter.api.Test;

class PositionControllerTest {

    @BeforeEach
    void setUp() {
    }

    @AfterEach
    void tearDown() {
    }

    @Test
    void getCurrentAircraftPositions() {
    }
}
```

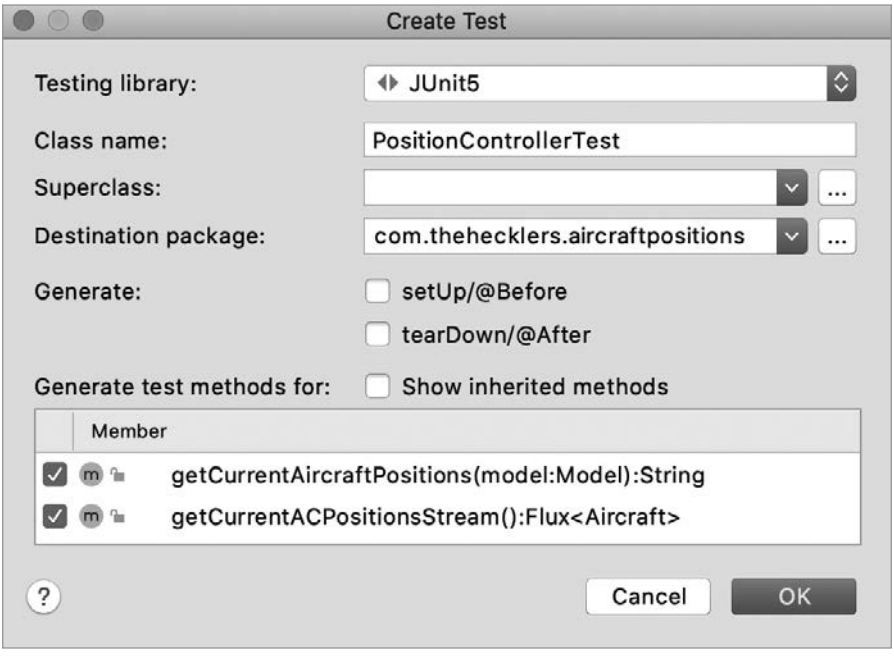


Рис. 9.2. Всплывающее окно Create Test с выбранными опциями

Чтобы после этого быстро приступить к созданию набора тестов, я начал с воспроизведения, насколько это возможно, существующих операций метода `getCurrentAircraftPositions()` класса `PositionController` в том же контексте, в котором он уже успешно работает: `ApplicationContext` Spring Boot.

ПРИМЕЧАНИЯ ОТНОСИТЕЛЬНО APPLICATIONCONTEXT

У каждого приложения Spring Boot есть ApplicationContext, включающий основной контекст — управление взаимодействиями со средой, компонентами приложения, передачей сообщений и т. д., и по умолчанию конкретный тип `ApplicationContext`, необходимый приложению, определяет автоконфигурация Spring Boot.

При тестировании аннотация уровня класса `@SpringBootTest` поддерживает параметр `webEnvironment`, позволяющий выбрать одну из четырех опций:

- `MOCK`;
- `RANDOM_PORT`;
- `DEFINED_PORT`;
- `NONE`.

По умолчанию используется `MOCK`. Ее выбор приводит к загрузке `WebApplicationContext` и задействует макет веб-среды вместо запуска встраиваемого сервера, если веб-среду удается найти по пути к классам приложения. В противном случае загружается обычный `ApplicationContext` без каких-либо веб-возможностей. Для упрощения макетного тестирования веб-API с помощью соответствующих механизмов аннотации `@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.MOCK)` или просто `@SpringBootTest` часто сопровождают `@AutoConfigureMockMVC` или `@AutoConfigureWebTestClient`.

Выбор опции `RANDOM_PORT` приводит к загрузке `WebApplicationContext` и запуску встроенного сервера, предоставляющего настоящую веб-среду, доступную на каком-либо случайно выбранном свободном порте. `DEFINED_PORT` делает то же самое, за одним исключением — прослушивает на порте, заданном в файле `application.properties` или `application.yml/yaml`. Если ни в одном из этих файлов порт не задан, используется порт по умолчанию 8080.

При выборе `NONE` создается `ApplicationContext` вообще без веб-среды, макетной или обычной. Встроенный сервер не запускается.

Я начал с добавления аннотации `@SpringBootTest` на уровне класса. Поскольку наша исходная цель — воспроизвести как можно точнее поведение, наблюдающееся при выполнении приложения, я выбрал опцию запуска встраиваемого сервера, прослушивающего на случайном порте. Для тестирования веб-API собираюсь взять `WebTestClient`, аналогичный `WebClient`, используемому в приложении, но с упором на тестирование:

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureWebTestClient
```

Поскольку у нас пока только один модульный тест и никакой подготовки или демонтажа не требуется, реализуем тестовый метод для `getCurrentAircraftPositions()`:

```
@Test
void getCurrentAircraftPositions(@Autowired WebTestClient client) {
    assert client.get()
            .uri("/aircraft")
            .exchange()
            .expectStatus().isOk()
            .expectBody(Iterable.class)
            .returnResult()
            .getResponseBody()
            .iterator()
            .hasNext();
}
```

Прежде всего следует отметить, что я выполнил автосвязывание компонента `WebTestClient` для использования внутри метода. Для внедрения компонента `WebTestClient` из `ApplicationContext` потребовались минимальные усилия благодаря размещенной мной на уровне класса аннотации `@AutoConfigureWebTestClient`, указывающей Spring Boot создать и автоконфигурировать `WebTestClient`.

Единственный оператор, из которого состоит метод с аннотацией `@Test`, представляет собой утверждение оценки истинности следующего непосредственно за ним выражения. В первой версии теста я воспользовался оператором `assert` языка Java, чтобы проверить равенство конечного результата цепочки операций булеву значению `True`, которое и будет означать, что тест пройден.



В текущей версии теста я пошел как минимум на два небольших компромисса. Во-первых, в нынешнем виде тест не будет пройден, если внешний сервис, поставляющий данные о местоположении воздушных судов (PlaneFinder), недоступен, даже если весь тестируемый код приложения Aircraft Positions работает правильно. Это значит, что тест проверяет не только функциональность, которую должен, а намного больше. Во-вторых, охват теста несколько ограничен, поскольку я проверяю лишь то, что возвращается объект Iterable, включающий как минимум один элемент, а не исследую содержимое самих элементов. Это значит, что тест будет успешно пройден при возврате в Iterable одного элемента любого вида или допустимых элементов с некорректными значениями. Все эти недостатки я исправлю в следующих версиях теста.

В самом выражении применяется внедренный нами компонент `WebTestClient`, отправляющий запрос `GET` к локальной конечной точке `/aircraft`, обслужи-

ваемой методом `getCurrentAircraftPositions()` класса `PositionController`. После обмена запросом и ответом проверяется, равен ли код состояния HTTP 200 (OK), содержит ли тело ответа объект `Iterable`, после чего ответ извлекается. Поскольку ответ состоит из `Iterable`, я воспользовался итератором, чтобы определить, содержится ли в этом объекте `Iterable` хотя бы одно значение. Если да — тест пройден.

Результаты выполнения теста выглядят так, как показано на рис. 9.3, и означают, что тест успешно пройден.

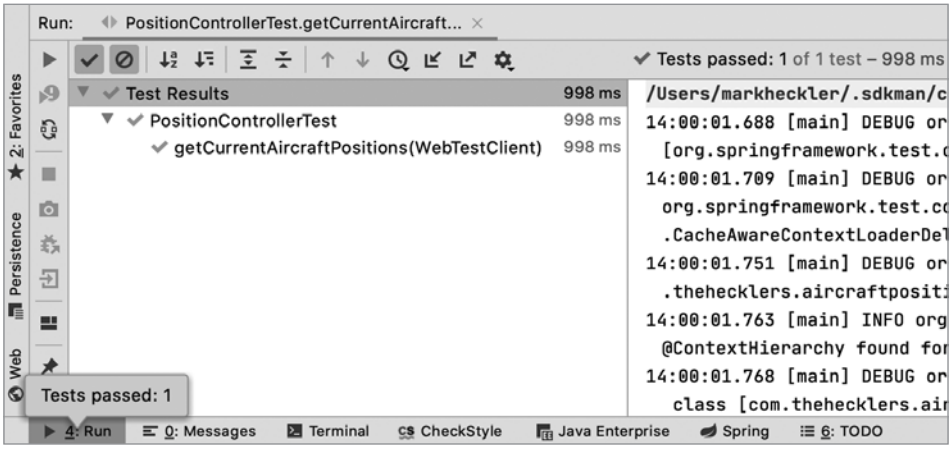


Рис. 9.3. Первый тест пройден

Начало положено, но даже этот отдельный тест можно существенно усовершенствовать. Давайте сделаем это, прежде чем дальше расширять модульное тестирование.

Рефакторинг кода для лучшего тестирования

В абсолютном большинстве случаев загрузка всего `ApplicationContext` со встроенным сервером и всей функциональностью приложения для выполнения горстки тестов — явный перебор. Как упоминалось ранее, модульные тесты должны сосредоточиваться на конкретных элементах функциональности и быть как можно более автономными. Чем меньше область охвата и число внешних зависимостей, тем более целенаправленными их можно сделать. Подобная сфокусированность обладает несколькими достоинствами, в том числе уменьшает число пропущенных сценариев или исходов, тестирование более специфичное

и строгое, тесты более удобочитаемые, а значит, понятные и, что не менее важно, быстрее работают.

Я упоминал ранее, что не имеет смысла писать тесты, не приносящие почти никакой пользы, хотя это и зависит от контекста. Один из факторов, из-за которого разработчики не хотят добавлять полезные тесты, — время, затрачиваемое на их набор. По достижении определенного порогового значения, которое также зависит от контекста, разработчик начинает колебаться, тратить ли время дополнительно к и без того значительному его количеству, необходимому для получения чистой сборки проекта. К счастью, Spring Boot включает средства для повышения качества тестов одновременно с сокращением времени их выполнения.

Если для удовлетворения требований API приложения Aircraft Positions не требовалось никаких вызовов с участием `WebClient` или `WebTestClient`, значит, следующий логичный шаг — удалить параметр `webEnvironment` из аннотации уровня класса `@SpringBootTest`. В результате для тестов класса `PositionControllerTest` будет загружен базовый `ApplicationContext` с использованием макетной веб-среды, что сокращает объем требуемой оперативной памяти и время загрузки. А поскольку `WebClient` — неотъемлемая часть API, а значит, `WebTestClient` — оптимальный вариант ее тестирования, я заменил аннотации уровня класса `@SpringBootTest` и `@AutoConfigureWebTestClient` на `@WebFluxTest` для упрощения `ApplicationContext` с одновременной автоконфигурацией и предоставлением доступа к `WebTestClient`:

```
@WebFluxTest({PositionController.class})
```

Стоит отметить, что аннотация `@WebFluxTest` помимо прочего может принимать параметр `controllers`, указывающий на массив компонентов типов `@Controller`, экземпляры которых будут создаваться тестовым классом, снабженным этой аннотацией. Саму часть `controllers =` можно опустить, как я и сделал, оставив лишь массив классов `@Controller` — в данном случае лишь один, `PositionController`.

Пересматриваем код, чтобы изолировать поведение

Как упоминалось ранее, код класса `PositionController` выполняет несколько операций, включая обращения к базе данных и непосредственное использование `WebClient` для доступа к внешнему сервису. Чтобы лучше изолировать API от базовых действий для разбиения макета на более мелкие единицы, а значит, и ее упрощения и очистки от лишнего, я провел рефакторинг `PositionController`, чтобы удалить код непосредственного определения и применения `WebClient`, а также перенес всю логику метода `getCurrentAircraftPositions()` в класс

PositionRetriever, который далее внедряется в PositionController и используется им:

```
import lombok.AllArgsConstructor;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@AllArgsConstructor
@RestController
public class PositionController {
    private final PositionRetriever retriever;

    @GetMapping("/aircraft")
    public Iterable<Aircraft> getCurrentAircraftPositions() {
        return retriever.retrieveAircraftPositions();
    }
}
```

Первая готовая для макета версия PositionRetriever состоит в основном из кода, ранее находившегося в PositionController. Основная цель этого шага — упростить макетирование метода retrieveAircraftPositions(). Благодаря исключению данной логики из метода getCurrentAircraftPositions() класса PositionController можно макетировать расположенные выше по конвейеру вызовы вместо веб-API, что позволяет протестировать PositionController:

```
import lombok.AllArgsConstructor;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;

@AllArgsConstructor
@Component
public class PositionRetriever {
    private final AircraftRepository repository;
    private final WebClient client =
            WebClient.create("http://localhost:7634");

    Iterable<Aircraft> retrieveAircraftPositions() {
        repository.deleteAll();

        client.get()
                .uri("/aircraft")
                .retrieve()
                .bodyToFlux(Aircraft.class)
                .filter(ac -> !ac.getReg().isEmpty())
                .toStream()
                .forEach(repository::save);

        return repository.findAll();
    }
}
```

Затем можно пересмотреть существующий код тестирования, изолировав функциональность приложения Aircraft Positions от внешних сервисов и сосредоточившись конкретно на веб-API путем макетирования прочих компонентов или функциональности, к которым обращается веб-API, и таким образом упрощая и ускоряя выполнение теста.

Оптимизация теста

Поскольку мы сосредоточились на тестировании веб-API, то чем больше логики, не относящейся к фактическим веб-взаимодействиям, мы сможем имитировать, тем лучше. Метод `PositionController::getCurrentAircraftPositions` теперь обращается к `PositionRetriever` для получения текущего местоположения воздушных судов по запросу, так что `PositionRetriever` — явно первый компонент в очереди на имитацию. Аннотация `@MockBean` фреймворка Mockito включается в проект автоматически с зависимостью `spring-boot-starter-test` — заменяет компонент `PositionRetriever`, который при обычных обстоятельствах создавался бы при запуске приложения с помощью макета и затем внедрялся автоматически:

```
@MockBean
private PositionRetriever retriever;
```



Компоненты-макеты автоматически возвращаются в исходное состояние после выполнения каждого тестового метода.

Далее мы займемся методом, предоставляющим данные о местоположении воздушных судов, — `PositionRetriever::retrieveAircraftPositions`. А поскольку я сейчас внедряю для тестирования макет `PositionRetriever` вместо настоящего, то должен предоставить реализацию метода `retrieveAircraftPositions()`, чтобы он реагировал предсказуемым и пригодным для тестирования образом при вызове объектом `PositionController`.

В качестве примеров данных для тестирования я создал несколько записей о местоположении воздушных судов в классе `PositionControllerTest`, объявив переменные `Aircraft` на уровне класса и присвоив им правдоподобные значения в методе `setUp()`:

```
    private Aircraft ac1, ac2;

    @BeforeEach
    void setUp(ApplicationContext context) {
        // Рейс 001 компании Spring Airlines по пути из STL в SFO
```

```
        // в настоящее время на высоте 9 км
        // над Канзас-сити
        ac1 = new Aircraft(1L, "SAL001", "sqwk", "N12345", "SAL001",
                "STL-SFO", "LJ", "ct",
                30000, 280, 440, 0, 0,
                39.2979849, -94.71921, 0D, 0D, 0D,
                true, false,
                Instant.now(), Instant.now(), Instant.now());

        // Рейс 002 компании Spring Airlines по пути из SFO в STL
        // в настоящее время на высоте 12 км
        // над Денвером
        ac2 = new Aircraft(2L, "SAL002", "sqwk", "N54321", "SAL002",
                "SFO-STL", "LJ", "ct",
                40000, 65, 440, 0, 0,
                39.8560963, -104.6759263, 0D, 0D, 0D,
                true, false,
                Instant.now(), Instant.now(), Instant.now());
    }
```



При использовании в производстве реальных приложений практически всегда извлекается больше одной записи о местоположении воздушных судов, а зачастую значительно больше. С учетом этого используемый при тестировании набор примеров данных должен включать как минимум две записи. В дополнительных тестах из последующих версий набора тестов не помешает рассмотреть граничные случаи: ни одного местоположения, одно или очень большое их количество для аналогичных реальных приложений.

А теперь вернемся к методу `retrieveAircraftPositions()`. Выражение `when...thenReturn` Mockito возвращает указанный ответ при выполнении заданного условия. Теперь, задав примеры данных, я могу указать как условие, так и ответ для возврата при вызове метода `PositionRetriever::retrieveAircraftPositions`:

```
@BeforeEach
void setUp(ApplicationContext context) {
    // Присваивания переменных Aircraft опущены
    // ради краткости

    ...

    Mockito.when(retriever.retrieveAircraftPositions())
        .thenReturn(List.of(ac1, ac2));
}
```

Закончив работу с макетами соответствующих методов, мы можем заняться модульным тестом, размещенным в `PositionControllerTest::getCurrentAircraftPositions`.

Поскольку я указал, что тестовый экземпляр должен загружать компонент `PositionController` с аннотацией уровня класса `@WebFluxTest(controllers = {PositionController.class})`, и создал макет компонента `PositionRetriever`, задав его поведение, то могу теперь переделать часть теста, извлекающую местоположения, с определенной уверенностью в том, что будет возвращено:

```
@Test
void getCurrentAircraftPositions(@Autowired WebTestClient client) {
    final Iterable<Aircraft> acPositions = client.get()
            .uri("/aircraft")
            .exchange()
            .expectStatus().isOk()
            .expectBodyList(Aircraft.class)
            .returnResult()
            .getResponseBody();

    // Необходимо сравнить с ожидаемыми
    // результатами
}
```

Приведенная цепочка операторов должна извлечь список `List<Aircraft>`, состоящий из `ac1` и `ac2`. Чтобы подтвердить правильность результатов, необходимо сравнить фактический результат `acPositions` с ожидаемым. Это можно сделать, например, простым сравнением:

```
assertEquals(List.of(ac1, ac2), acPositions);
```

Этот оператор отрабатывает правильно, и тест будет пройден. Можно было пойти дальше в этом промежуточном шаге, сравнив фактические результаты с полученными посредством имитации вызова к `AircraftRepository`. При добавлении в класс следующих фрагментов кода метод `setUp()` и тестовый метод `getCurrentAircraftPositions()` дают аналогичные (положительные) результаты теста:

```
@MockBean
private AircraftRepository repository;

@BeforeEach
void setUp(ApplicationContext context) {
    // Существующий код setUp опущен для краткости

    ...

    Mockito.when(repository.findAll()).thenReturn(List.of(ac1, ac2));
}

@Test
```

```
void getCurrentAircraftPositions(@Autowired WebTestClient client) {
    // Цепочка операций client.get для краткости опущена

    ...

    assertEquals(repository.findAll(), acPositions);
}
```



Этот вариант также позволяет успешно пройти тест, но несколько противоречит принципу целенаправленного тестирования, поскольку в нем смешиваются понятия тестирования репозитория с тестированием веб-API. А поскольку метод CrudRepository::findAll на самом деле не используется, а лишь имитируется, то никакой реальной пользы этот тест не приносит. Впрочем, подобные тесты порой встречаются, так что я посчитал нужным показать и обсудить его.

Теперь рабочая версия `PlaneControllerTest` должна выглядеть следующим образом:

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.reactive.WebFluxTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.context.ApplicationContext;
import org.springframework.test.web.reactive.server.WebTestClient;

import java.time.Instant;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;

@WebFluxTest(controllers = {PositionController.class})
class PositionControllerTest {
    @MockBean
    private PositionRetriever retriever;

    private Aircraft ac1, ac2;

    @BeforeEach
    void setUp(ApplicationContext context) {
        // Рейс 001 компании Spring Airlines по пути из STL в SFO
        // в настоящее время на высоте 9 км
        // над Канзас-сити
        ac1 = new Aircraft(1L, "SAL001", "sqwk", "N12345", "SAL001",
                "STL-SFO", "LJ", "ct",
                30000, 280, 440, 0, 0,
```

```
                39.2979849, -94.71921, 0D, 0D, 0D,
                true, false,
                Instant.now(), Instant.now(), Instant.now());

        // Рейс 002 компании Spring Airlines по пути из SFO в STL
        // в настоящее время на высоте 12 км над Денвером
        ac2 = new Aircraft(2L, "SAL002", "sqwk", "N54321", "SAL002",
                "SFO-STL", "LJ", "ct",
                40000, 65, 440, 0, 0,
                39.8560963, -104.6759263, 0D, 0D, 0D,
                true, false,
                Instant.now(), Instant.now(), Instant.now());

        Mockito.when(retriever.retrieveAircraftPositions())
            .thenReturn(List.of(ac1, ac2));
    }

    @Test
    void getCurrentAircraftPositions(@Autowired WebTestClient client) {
        final Iterable<Aircraft> acPositions = client.get()
                .uri("/aircraft")
                .exchange()
                .expectStatus().isOk()
                .expectBodyList(Aircraft.class)
                .returnResult()
                .getResponseBody();

        assertEquals(List.of(ac1, ac2), acPositions);
    }
}
```

Если запустить его еще раз, тест будет пройден, а результаты будут подобны приведенным на рис. 9.4.

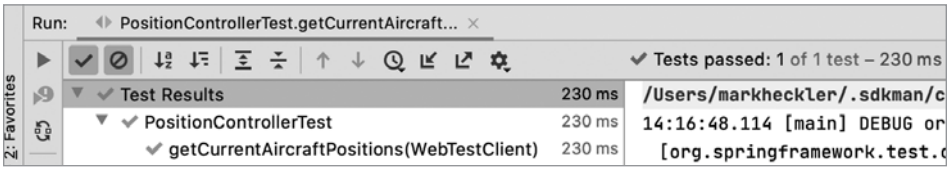


Рис. 9.4. Новый усовершенствованный тест для AircraftRepository::getCurrentAircraftPositions

Поскольку веб-API, который должен соответствовать требованиям приложения либо пользователя, расширяется, то прежде чем создавать код для удовлетворения этих требований, необходимо определиться с модульными тестами, чтобы обеспечить правильные результаты.

Тестовые срезы

Я уже несколько раз упоминал о важности целенаправленного тестирования, и в Spring есть еще один механизм, с помощью которого разработчики могут выполнять его быстро и безболезненно, — тестовые срезы.

В тестировочную зависимость Spring Boot `spring-boot-starter-test` включены несколько аннотаций для автоматической конфигурации этих срезов функциональности. Работают все аннотации тестовых срезов схожим образом: загружают `ApplicationContext` и выбирают компоненты, нужные для текущего среза. В их числе:

- `@JsonTest`;
- `@WebMvcTest`;
- `@WebFluxText` (ранее уже нам встречавшаяся);
- `@DataJpaTest`;
- `@JdbcTest`;
- `@DataJdbcTest`;
- `@JooqTest`;
- `@DataMongoTest`;
- `@DataNeo4jTest`;
- `@DataRedisTest`;
- `@DataLdapTest`;
- `@RestClientTest`;
- `@AutoConfigureRestDocs`;
- `@WebServiceClientTest`.

В предыдущем разделе при использовании аннотации `@WebFluxText` для тестирования и проверки веб-API я упоминал тестирование взаимодействий с хранилищем данных и сознательно исключил его из теста, который был ориентирован на проверку веб-взаимодействий. Теперь мы им займемся, чтобы лучше продемонстрировать, как тестировать работу с данными, а также как тестовые срезы упрощают программирование тестов для конкретной функциональности.

Поскольку в текущей версии приложения Aircraft Positions для хранения и извлечения данных о текущем местоположении применяются JPA и H2, нам идеально подойдет аннотация `@DataJpaTest`. Начнем с создания нового класса для тестирования с помощью IntelliJ IDEA, открыв класс `AircraftRepository`

и воспользовавшись тем же способом создания тестового класса, что и раньше: CMD+N (в Windows Ctrl+N), Test, не меняя JUnit 5 в качестве Testing Library и прочие значения по умолчанию и выбрав опции setUp/@Before и tearDown/@After (рис. 9.5).

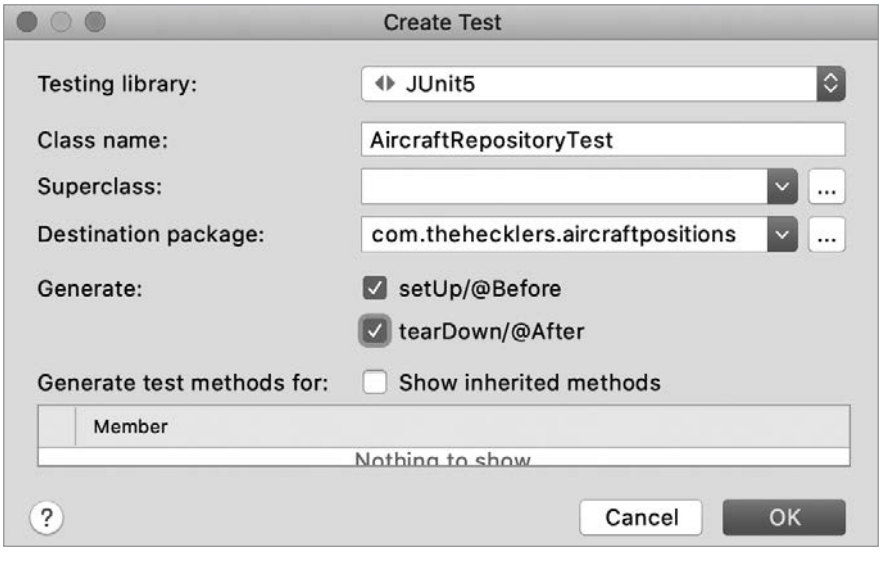


Рис. 9.5. Всплывающее окно Create Test для класса Aircraft Repository



Никакие методы не показаны, поскольку компоненты Spring Data Repository обеспечивают для приложений Spring Boot часто используемые методы посредством автоконфигурации. В качестве примера я добавлю тестовые методы для их проверки. Создаваемые пользовательские методы для репозитория также можно и нужно тестировать.

При нажатии кнопки OK генерируется тестовый класс `AircraftRepositoryTest`:

```
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;

class AircraftRepositoryTest {

    @BeforeEach
    void setUp() {
    }

    @AfterEach
    void tearDown() {
    }
}
```

Первое, что нужно сделать, — снабдить класс `AircraftRepositoryTest` аннотацией тестового среза `@DataJpaTest`:

```
@DataJpaTest
class AircraftRepositoryTest {

    ...

}
```

В результате добавления одной этой аннотации при выполнении тест просканирует классы `@Entity` и настроит репозитории Spring Data JPA — в приложении Aircraft Positions это `Aircraft` и `AircraftRepository`. Если встроенная база данных находится по пути к классам (как H2 у нас), тестовый механизм настроит и ее. Обычные классы, снабженные аннотацией `@Component`, при этом не сканируются.

Для тестирования реальных операций с репозиторием последний нельзя имитировать, а поскольку аннотация `@DataJpaTest` загружает и настраивает компонент `AircraftRepository`, то имитировать его смысла нет. Внедряем компонент репозитория с помощью аннотации `@Autowire` и, точно как в тесте `PositionController` ранее, объявляем переменные `Aircraft`, которые будут в итоге играть роль тестовых данных:

```
@Autowired
private AircraftRepository repository;

private Aircraft ac1, ac2;
```

Для настройки должной среды для тестов внутри класса `AircraftRepositoryTest` я создам два объекта `Aircraft`, присвоив каждый из них объявленной переменной-члену, после чего сохраню их в репозитории в методе `setUp()` с помощью `Repository::saveAll`:

```
@BeforeEach
void setUp() {
    // Рейс 001 компании Spring Airlines по пути из STL в SFO
    // в настоящее время на высоте 9 км над
    // Канзас-сити
    ac1 = new Aircraft(1L, "SAL001", "sqwk", "N12345", "SAL001",
            "STL-SFO", "LJ", "ct",
            30000, 280, 440, 0, 0,
            39.2979849, -94.71921, 0D, 0D, 0D,
            true, false,
            Instant.now(), Instant.now(), Instant.now());

    // Рейс 002 компании Spring Airlines по пути из SFO в STL
    // в настоящее время на высоте 12 км над Денвером
```

```
    ac2 = new Aircraft(2L, "SAL002", "sqwk", "N54321", "SAL002",
            "SFO-STL", "LJ", "ct",
            40000, 65, 440, 0, 0,
            39.8560963, -104.6759263, 0D, 0D, 0D,
            true, false,
            Instant.now(), Instant.now(), Instant.now());

    repository.saveAll(List.of(ac1, ac2));
}
```

Далее я создам тестовый метод для проверки того, что в результате выполнения метода `findAll()` для компонента `AircraftRepository` возвращается именно то, что и должно, — `Iterable<Aircraft>`, содержащий две записи о местоположении воздушных судов, сохраненные в методе `setUp()` теста:

```
@Test
void testFindAll() {
    assertEquals(List.of(ac1, ac2), repository.findAll());
}
```



Интерфейс List расширяет интерфейс Collection, который, в свою очередь, расширяет интерфейс Iterable.

Результаты выполнения теста выглядят так, как показано на рис. 9.6, и демонстрируют, что тест пройден успешно.

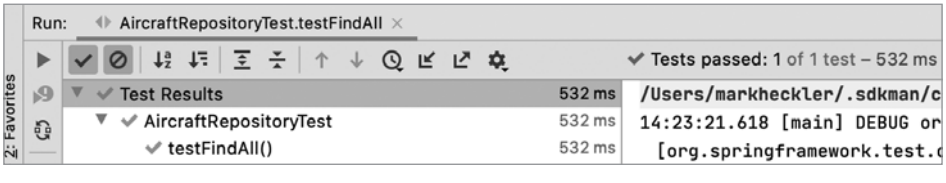


Рис. 9.6. Результаты теста для findAll()

Аналогичным образом создаем тест для метода `AircraftRepository`, который будет искать конкретную запись по полю ID, — `findById()`. Поскольку в методе `Repository::saveAll`, вызывавшемся в `setUp()` тестового класса, были сохранены две записи, я попробую найти их и сравню результаты с ожидаемыми значениями:

```
@Test
void testFindById() {
    assertEquals(Optional.of(ac1), repository.findById(ac1.getId()));
    assertEquals(Optional.of(ac2), repository.findById(ac2.getId()));
}
```

Тест `testFindById()` тоже проходит успешно (рис. 9.7).

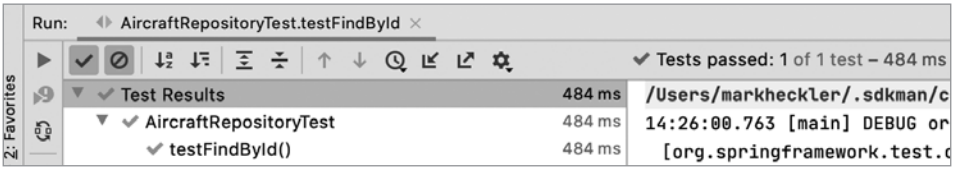


Рис. 9.7. Результаты теста для findById()

ТЕСТИРОВАНИЕ ТЕСТОВ

Если тест пройден успешно, большинство разработчиков считают, что код проверен. Но успешно пройденный тест может означать две вещи:

- код работает;
- тест не работает.

А поэтому я настоятельно рекомендую умышленно нарушать работу, когда это возможно, чтобы убедиться, что тест действительно работает как надо.

Что я подразумеваю под этим?

Проще всего указать неправильные ожидаемые результаты. Если тест при этом внезапно не будет пройден, изучите все подробности. Если он не пройден так, как ожидалось, восстановите правильную функциональность и убедитесь, что он снова работает. Однако если тест будет пройден успешно, даже когда должен провалиться, необходимо исправить его и проверить снова. Когда же он нарушается так, как должен, восстановите правильные ожидаемые результаты и запустите тест еще раз, чтобы убедиться, что он тестирует то, что нужно, так, как нужно.

Заметим, что подобное случается часто, и намного приятнее обнаружить неудачные тесты во время их написания, а не при отладке в тщетных попытках определить, как эта проблема проскользнула через сито тестов и проявилась при продакшене.

Наконец, после выполнения всех тестов не помешает немного убрать за собой. В метод `tearDown()` я вставил один-единственный оператор для удаления всех записей в `AircraftRepository`:

```
@AfterEach
void tearDown() {
    repository.deleteAll();
}
```

Отмечу, что на самом деле в данном случае удалять все записи из репозитория необязательно, поскольку экземпляр базы данных H2 в оперативной памяти инициализируется заново перед каждым тестом. Впрочем, это довольно хорошо отражает типовые операции в методе `tearDown()` тестового класса.

Выполнение всех тестов в `AircraftRepositoryTest` дает результаты, подобные показанным на рис. 9.8 и означающие, что тесты пройдены успешно.

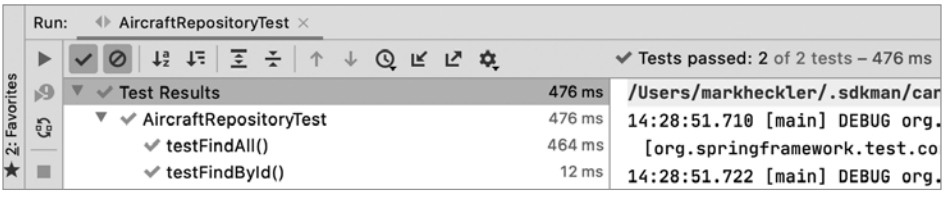


Рис. 9.8. Результаты для всех тестов в AircraftRepositoryTest

ВРЕМЯ ВЫПОЛНЕНИЯ ТЕСТОВ: ЖАЖДА СКОРОСТИ

Как уже упоминалось в этой главе, сокращение сферы действия теста и уменьшение количества компонентов, которые нужно загрузить механизму тестирования в `AircraftContext` для его выполнения, повышают скорость и точность каждого теста. Меньшее количество неизвестных величин обусловливает более разносторонний набор тестов, а более быстрые тесты означают, что этот набор тестов может сделать больше за меньшее время, усерднее работая, чтобы избавить вас от проблем в дальнейшем.

Приближенно оценить экономию времени можно следующим образом: у начальной версии `PositionControllerTest` загрузка `AircraftContext`, выполнение теста и завершение работы занимали 998 мс — почти целую секунду. Небольшой рефакторинг кода и тестов улучшил модульность приложения и сделал соответствующий тест более целенаправленным, одновременно сократив время его выполнения до 230 мс — менее чем четверти секунды. Экономия более чем 3/4 секунды при каждом запуске теста суммарно по нескольким тестам и нескольким сборкам вносит весомый и приятный вклад в ускорение разработки.

Тестирование никогда не завершается, пока приложение развивается. Для текущей же функциональности приложения Aircraft Positions тестов, написанных в этой главе, вполне достаточно для начальной проверки кода и дальнейшего расширения по мере добавления в приложение новой функциональности.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Полный код для этой главы находится в ветке chapter9end в репозитории кода.

Резюме

В этой главе мы обсудили и продемонстрировали основные аспекты тестирования приложений Spring Boot, сделав упор на важнейших элементах тестирования приложений, в наибольшей степени повышающих готовность любого из них к продакшену. В том числе обсудили такие вопросы, как модульное тестирование, целостное тестирование приложений с помощью аннотации `@SpringBootTest`, написание эффективных модульных тестов с помощью JUnit и тестовых срезов Spring Boot для изоляции субъектов тестирования и оптимизации последнего.

Следующая глава посвящена вопросам безопасности, в частности аутентификации и авторизации. Я покажу, как благодаря фреймворку Spring Security реализовать аутентификацию на основе форм для автономных приложений при самых жестких требованиях и как добиться максимальной безопасности и гибкости с помощью OpenID Connect и OAuth2.

ГЛАВА 10

Безопасность приложений Spring Boot

Не понимая основных идей аутентификации и авторизации, невозможно создавать безопасные приложения, поскольку на них основаны проверка личности пользователей и контроль доступа. Spring Security сочетает возможности аутентификации и авторизации с другими механизмами, в частности HTTP-брандмауэром, цепочками фильтров, расширенным применением стандартов и вариантов обмена информацией IETF и Консорциума Всемирной паутины (World Wide Web Consortium, W3C), а также другими инструментами, помогающими защитить приложение от взлома. Принимая на вооружение сложившийся образ мышления, направленный на безопасность, Spring Security оценивает входные параметры и доступные зависимости с помощью обладающей большими возможностями автоконфигурации Spring Boot, обеспечивая максимальную безопасность ее приложений с минимальными усилиями.

В этой главе вы во всех подробностях познакомитесь с основными аспектами безопасности и их применением в приложениях. Я покажу несколько способов включения Spring Security в структуру приложений Spring Boot для повышения уровня безопасности приложения, ликвидации опасных пробелов в сфере его действия и сокращения пространства атаки.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Пожалуйста, для начала извлеките из репозитория кода ветку chapter10begin.

Аутентификация и авторизация

Часто используемые в одном контексте *аутентификация* и *авторизация* — связанные, но не идентичные понятия.

- *Аутентификация* — действие, процесс или метод подтверждения подлинности, истинности чего-либо (личности, произведения искусства или финансовой транзакции).
- *Авторизация*:
  - установление авторства (в юриспруденции);
  - процесс действия по глаголу несовершенного вида «авторизовать»;
  - результат такого действия;
  - идентификация клиента и подтверждение наличия необходимой суммы на его счете (в финансовых расчетах с помощью пластиковых карт).

Второе определение *авторизации* ссылается на термин «*авторизовать*»: *авторизовать* — дать (давать) кому-нибудь разрешение, полномочие на какие-нибудь действия; разрешить (разрешать) распространение своего произведения, изобретения, дать (давать) согласие на это, например авторизовать издание собрания сочинений.

Хотя и довольно интересные, эти определения весьма туманны. Иногда словарные статьи не столь полезны, как хотелось бы. Вот мои собственные определения.

- *Аутентификация* — подтверждение того, что кто-либо — тот, за кого себя выдает.
- *Авторизация* — проверка того, что у кого-либо есть права на доступ к определенному ресурсу или выполнение какой-либо операции.

Аутентификация

Проще говоря, *аутентификация* — подтверждение того, что кто-либо (что-либо) — тот (то в случае устройства, приложения или сервиса), за кого (что) себя выдает.

Можно привести несколько конкретных примеров понятия аутентификации в реальном мире. Если вам случалось предъявлять какое-либо удостоверение личности, например электронный пропуск, водительские права или паспорт, чтобы подтвердить, кто вы такой, значит, вы прошли аутентификацию. Мы все привыкли подтверждать, что мы те, за кого себя выдаем, во множестве различных ситуаций, и по сути аутентификация в реальном мире и аутентификация приложения не слишком различаются.

Аутентификация обычно производится на основе:

- чего-либо, что является неотъемлемой частью нас самих;
- чего-то, что мы знаем;
- чего-то, чем мы обладаем.



Эти три фактора могут использоваться по отдельности или в различных сочетаниях для многофакторной аутентификации (Multi-Factor Authentication, MFA).

Конечно, способы аутентификации в реальном и виртуальном мирах сильно различаются. Вместо сличения фото с вашего удостоверения личности с предъявителем, как часто происходит в реальном мире, аутентификация в приложении обычно означает ввод пароля, использование ключа безопасности или биометрических данных (сканирование радужной оболочки глаза, отпечатков пальцев и т. д.), которые в настоящее время программному обеспечению сверять проще, чем фотографию с человеком. Тем не менее в обоих случаях происходит сравнение хранимых данных с введенными, и при совпадении аутентификация считается успешно пройденной.

Авторизация

После аутентификации у человека появляется возможность получить доступ к ресурсам и/или операциям, разрешенный одному или нескольким субъектам.



В этом контексте в роли субъекта может выступать (и чаще всего выступает) человек, но тот же подход применим и к приложениям, сервисам, устройствам и т. д. в зависимости от контекста.

После подтверждения личности человек получает доступ к приложению на каком-то общем уровне. После чего этот, теперь уже аутентифицированный, пользователь приложения может запросить доступ к чему-либо. Приложение должно определить, обладает ли этот пользователь правами, то есть *авторизован* ли он, на доступ к данному ресурсу. Если да, доступ предоставляется, если нет, пользователя уведомляют, что запрос отклонен из-за отсутствия *полномочий*.

Коротко о Spring Security

Помимо надежных вариантов аутентификации и авторизации, Spring Security предоставляет разработчикам несколько других механизмов, помогающих обезопасить приложения Spring Boot. Благодаря автоконфигурации приложения Spring Boot доводят каждую применимую возможность Spring Security до максимально возможной степени безопасности на основе указанной информации, а иногда и благодаря отсутствию более конкретных предписаний. Конечно, разработчики могут настраивать или делать менее строгими все относящиеся к безопасности возможности в соответствии с требованиями, установленными в своей организации.

Возможности Spring Security слишком многочисленны, чтобы подробно описать их все в этой главе, но три из них я считаю совершенно необходимыми для понимания модели и базового устройства Spring Security. Это HTTP-брандмауэр, цепочки фильтров безопасности, а также широкое использование Spring Security стандартов и вариантов запросов — ответов IETF и W3C.

HTTP-брандмауэр

Хотя конкретные цифры указать нелегко, многие нарушения безопасности начинаются с запроса, в котором неправильно сформирован URI, и неожиданной реакции системы на него. Фактически это первый эшелон защиты приложения, а поэтому — проблема, которую следует решить прежде всех прочих попыток обеспечения безопасности приложения.

Начиная с версии 5.0 Spring Security включает в себя HTTP-брандмауэр, тщательно анализирующий все входящие запросы на предмет сомнительного форматирования. При любых проблемах с запросом, например некорректных значениях заголовков или неверном форматировании, запрос отклоняется. По умолчанию, если разработчик не переопределит такое поведение, используется реализация, красноречиво названная `StrictHttpFirewall`, сразу закрывающая первую и, вероятно, самую очевидную брешь в профиле безопасности приложения.

Цепочки фильтров безопасности

В качестве более специфичного фильтра следующего уровня для входящих запросов Spring Security использует цепочки фильтров для обработки сформированных должным образом запросов, прошедших через HTTP-брандмауэр.

Если сказать проще, то для большинства приложений разработчик просто задает цепочку условий фильтров, через которые проходит входящий запрос, пока не окажется, что он удовлетворяет одному из них. Когда входящий запрос удовлетворяет фильтру, вычисляются соответствующие условия, чтобы определить, можно ли его выполнить. Например, когда поступает запрос к определенной конечной точке API и оказывается, что он удовлетворяет условию фильтра в цепочке фильтров, может быть проверено, обладает ли выполнивший запрос пользователь соответствующими ролью или полномочиями для доступа к запрашиваемому ресурсу. При положительном результате такой проверки запрос выполняется, в противном случае он отклоняется, обычно с кодом состояния 403 Запрещено.

Если запрос проходит все заданные фильтры в цепочке и не удовлетворяет ни одному из них, запрос отбраковывается.

Заголовки запросов и ответов

IETF и W3C создали несколько спецификаций и стандартов обмена информацией на основе протокола HTTP, часть из которых связаны с безопасным обменом информацией. В них описаны несколько заголовков для взаимодействия между агентами пользователей — утилитами командной строки, веб-браузерами и т. п. — и сервером или облачными приложениями/сервисами. С помощью этих заголовков запрашивается конкретное поведение или сообщается о нем, в них описываются допустимые значения и поведенческие реакции, они широко применяются в Spring Security для повышения уровня безопасности приложений Spring Boot.

С учетом того, что различные агенты пользователей могут поддерживать некоторые из этих стандартов и спецификаций или их все, полностью или частично, Spring Security берет на вооружение подход максимального охвата, проверяет все известные опции заголовков, применяя их повсеместно, ищет их в запросах и передает в ответах в зависимости от обстоятельств.

Реализация аутентификации и авторизации на основе форм с помощью Spring Security

Бесчисленные приложения, применяющие метод аутентификации «нечто, что мы знаем», используются каждый день. Внутренние ли это приложения организации, веб-приложения, доступные потребителям через интернет, или

нативные приложения мобильного устройства, ввод идентификатора пользователя и пароля — привычное дело как для разработчиков, так и для обычных пользователей. И в большинстве случаев безопасности, обеспечиваемой этим методом, вполне достаточно для поставленной задачи.

Spring Security обеспечивает приложения Spring Boot превосходной готовой поддержкой парольной аутентификации посредством автоконфигурации и понятных абстракций. В данном разделе показаны различные узловые точки этого процесса, для чего мы с помощью Spring Security выполним рефакторинг приложения Aircraft Positions, встроив в него аутентификацию на основе форм.

Добавление зависимостей Spring Security

При создании нового проекта Spring Boot можно легко добавить посредством Spring Initializr еще одну зависимость для *Spring Security* — и безопасность базового уровня нашему незрелому приложению будет обеспечена, причем дополнительная настройка не потребуется (рис. 10.1).



Рис. 10.1. Зависимость Spring Security в Spring Initializr

Модифицировать наше уже существующее приложение лишь немногим сложнее. Я добавил в файл сборки Maven `pom.xml` те же две вспомогательные зависимости, которые добавляет Initializr, — одну для самого Spring Security, а вторую для тестирования:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>
</dependency>
```

ПРИМЕЧАНИЯ ОТНОСИТЕЛЬНО ПОКРЫТИЯ ТЕСТАМИ

Чтобы в этой главе сосредоточиться сугубо на ключевых концепциях Spring Security, я с некоторой неохотой проигнорирую тесты, которые полагалось бы создать при добавлении дополнительных зависимостей. Это решение связано исключительно с желанием упростить содержание главы, а не с особенностями процесса разработки.

Для сборки проектов в этой и последующих главах может понадобиться добавить флаг `-DskipTests` при работе из командной строки или не забыть выбрать конфигурацию для приложения, а не тестов, из раскрывающегося меню при сборке из IDE.

Убедившись, что Spring Security добавлен в путь к классам, и не меняя код или конфигурацию приложения, я перезапустил Aircraft Positions для быстрой проверки функциональности — это прекрасная возможность посмотреть, какие возможности Spring Security сразу же предоставляет разработчику.

Запустив оба приложения, PlaneFinder и Aircraft Positions, я вернулся в окно терминала и снова обратился к конечной точке `/aircraft` приложения Aircraft Positions, как показано далее:

```
mheckler-a01 :: ~ " http :8080/aircraft
HTTP/1.1 401
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Expires: 0
Pragma: no-cache
Set-Cookie: JSESSIONID=347DD039FE008DE50F457B890F2149C0; Path=/; HttpOnly
WWW-Authenticate: Basic realm="Realm"
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block

{
    "error": "Unauthorized",
    "message": "",
    "path": "/aircraft",
    "status": 401,
    "timestamp": "2020-10-10T17:26:31.599+00:00"
}
```



Некоторые заголовки ответа были убраны для большей ясности.

Как видите, у меня больше нет доступа к конечной точке `/aircraft`, на мой запрос возвращается код состояния `401 Unauthorized (Не авторизован)`. Поскольку конечная точка `/aircraft` пока что единственное средство доступа к информации из приложения Aircraft Positions, фактически это означает, что наше приложение полностью защищено от нежелательного доступа. Это прекрасная новость, но важно понимать, как это произошло и как возобновить доступ для законных пользователей.

Как я уже упоминал, в Spring Security принята политика, при которой разработчик получает «безопасность по умолчанию» при любых настройках конфигурации, даже при ее отсутствии. Когда Spring Boot обнаруживает Spring Security на пути к классам, безопасность настраивается в соответствии с разумными значениями по умолчанию. Даже если разработчик не задал никаких пользователей или паролей, уже само включение Spring Security в проект указывает на то, что цель — создание защищенного приложения.

Как вы можете догадаться, это не так уж много. Но автоконфигурация Spring Boot + Security создает много важнейших компонентов для реализации основных возможностей безопасности, связанных с аутентификацией на основе форм и авторизацией пользователей по их идентификаторам и паролям. Отсюда логически вытекают следующие вопросы: «Какие пользователи? Какие пароли?»

Возвращаясь к журналам запуска приложения Aircraft Positions, найти ответ на эти вопросы можно в следующей строке:

```
Using generated security password: 1ad8a0fc-1a0c-429e-8ed7-ba0e3c3649ef
```

Если в приложении не заданы никакие пользователи или пароли и не обеспечены возможности для доступа с помощью других средств, в приложениях Spring Boot с поддержкой безопасности по умолчанию применяется одна пользовательская учетная запись `user` с уникальным паролем, генерируемым заново при каждом запуске приложения. Вернувшись в окно терминала, я попробовал получить доступ к приложению еще раз, теперь с указанными учетными данными:

```
mheckler-a01 :: ~ " http :8080/aircraft
    --auth user:1ad8a0fc-1a0c-429e-8ed7-ba0e3c3649ef
HTTP/1.1 200
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Expires: 0
Pragma: no-cache
Set-Cookie: JSESSIONID=94B52FD39656A17A015BC64CF6BF7475; Path=/; HttpOnly
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
```

```
[
    {
        "altitude": 40000,
        "barometer": 1013.6,
        "bds40_seen_time": "2020-10-10T17:48:02Z",
        "callsign": "SWA2057",
        "category": "A3",
        "flightno": "WN2057",
        "heading": 243,
        "id": 1,
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-10-10T17:48:06Z",
        "lat": 38.600372,
        "lon": -90.42375,
        "polar_bearing": 207.896382,
        "polar_distance": 24.140226,
        "pos_update_time": "2020-10-10T17:48:06Z",
        "reg": "N557WN",
        "route": "IND-DAL-MCO",
        "selected_altitude": 40000,
        "speed": 395,
        "squawk": "2161",
        "type": "B737",
        "vert_rate": -64
    },
    {
        "altitude": 3500,
        "barometer": 0.0,
        "bds40_seen_time": null,
        "callsign": "N6884J",
        "category": "A1",
        "flightno": "",
        "heading": 353,
        "id": 2,
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-10-10T17:47:45Z",
        "lat": 39.062851,
        "lon": -90.084965,
        "polar_bearing": 32.218696,
        "polar_distance": 7.816637,
        "pos_update_time": "2020-10-10T17:47:45Z",
        "reg": "N6884J",
        "route": "",
        "selected_altitude": 0,
        "speed": 111,
        "squawk": "1200",
        "type": "P28A",
        "vert_rate": -64
    },
```

```
    {
        "altitude": 39000,
        "barometer": 0.0,
        "bds40_seen_time": null,
        "callsign": "ATN3425",
        "category": "A5",
        "flightno": "",
        "heading": 53,
        "id": 3,
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-10-10T17:48:06Z",
        "lat": 39.424159,
        "lon": -90.419739,
        "polar_bearing": 337.033437,
        "polar_distance": 30.505314,
        "pos_update_time": "2020-10-10T17:48:06Z",
        "reg": "N419AZ",
        "route": "AFW-ABE",
        "selected_altitude": 0,
        "speed": 524,
        "squawk": "2224",
        "type": "B763",
        "vert_rate": 0
    },
    {
        "altitude": 45000,
        "barometer": 1012.8,
        "bds40_seen_time": "2020-10-10T17:48:06Z",
        "callsign": null,
        "category": "A2",
        "flightno": "",
        "heading": 91,
        "id": 4,
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-10-10T17:48:06Z",
        "lat": 39.433982,
        "lon": -90.50061,
        "polar_bearing": 331.287125,
        "polar_distance": 32.622134,
        "pos_update_time": "2020-10-10T17:48:05Z",
        "reg": "N30GD",
        "route": "",
        "selected_altitude": 44992,
        "speed": 521,
        "squawk": null,
        "type": "GLF4",
        "vert_rate": 64
    }
]
```

ВАЖНАЯ ИНФОРМАЦИЯ О ЗАГОЛОВКАХ ОТВЕТОВ

Как я уже мельком упоминал, несколько опций заголовков из стандартов IETF и W3C стали официальными и/или рекомендуемыми для применения в браузерах и прочих агентах пользователей для повышения безопасности приложений. Spring Security принял их на вооружение и досконально реализовал в попытке всеми доступными средствами обеспечить максимальную безопасность.

В числе заголовков ответов Spring Security по умолчанию, соответствующих этим стандартам и рекомендациям, следующие.

- Заголовок `Cache-Control` по умолчанию принимает значение `no-cache` с директивой `no-store` и директивами must-revalidate и `max-age` (равна 0). Кроме того, заголовок `Pragma` возвращается с директивой `no-cache`, а для заголовка `Expires` задается значение `0`. Все эти механизмы предназначены для устранения возможных пробелов в охвате функций браузера/пользовательского агента, чтобы как можно лучше контролировать кэширование, то есть отключать его всегда, когда пользователь выходит с сайта, чтобы злоумышленник не смог просто нажать кнопку Назад браузера и вернуться на защищенный сайт, авторизовавшись там с учетными данными жертвы.
- Заголовок `X-Content-Type-Options` установлен в `nosniff`, чтобы отключить анализ трафика. Браузеры могут (и часто так и делают) выяснять тип запрошенного контента, чтобы отобразить его соответствующим образом. Например, если запрашивается файл .jpg, браузер может отображать его в виде графического изображения. Это кажется довольно хорошей функцией, но если в анализируемый контент встроен вредоносный код, он тоже незаметно будет выполнен в обход строгих в остальном мер безопасности. Spring Security с настройкой `nosniff` по умолчанию закрывает этот вектор атаки.
- Заголовок `X-Frame-Options` установлен в значение DENY, чтобы запретить браузерам отображать контент во встраиваемых фреймах. Соответствующая атака, называемая «перехват щелчка мышью» (clickjacking), может быть реализована, когда поверх отображаемого элемента управления выводится невидимый фрейм, в результате чего пользователь запускает выполнение нежелательного действия вместо того, которое хотел, то есть происходит «перехват» выполняемого пользователем щелчка мышью. Spring Security по умолчанию отключает поддержку фреймов, закрывая таким образом дорогу подобным атакам.
- Заголовок `X-XSS-Protection` установлен в значение `1`, чтобы активировать защиту браузера от атак межсайтового выполнения сценариев (Cross Site Scripting, XSS). После включения этой возможности браузер может реагировать на возможные атаки множеством способов. Spring Security по умолчанию применяет наиболее безопасную настройку `mode=block`, чтобы гарантировать, что исполненная благих намерений попытка браузера модифицировать и безопасно обработать контент не сделает пользователя уязвимым. Блокирование контента закрывает эту потенциальную уязвимость.

Обратите внимание, что при использовании сетей поставки контента может понадобиться откорректировать настройки XSS для правильной обработки. Эту настройку, как и другие, разработчик может гибко настраивать. Если же никаких указаний вы, как разработчик, не дадите, Spring Security будет стремиться использовать политику «безопасность по умолчанию» в максимально возможной степени при доступной информации.



Как и ранее, часть заголовков ответа была убрана для большей ясности.

При правильном идентификаторе пользователя и сгенерированном пароле я получил ответ с кодом состояния `200 OK`, так что у меня снова появился доступ к конечной точке `/aircraft`, а значит, и к приложению Aircraft Positions.

Возвращаясь к приложению Aircraft Positions: существует несколько проблем с текущим состоянием его безопасности. Прежде всего, то, что есть лишь одна учетная запись пользователя, которую приходится применять всем, кому нужен доступ к приложению. Это прямо противоречит принципу безопасности, говорящему о необходимости учета, и даже принципам аутентификации, ведь никто из пользователей не доказывает однозначно, что он — тот, за кого себя выдает. Возвращаясь к вопросу учета: как в подобном случае определить, кто выполнил несанкционированный доступ или участвовал в нем? Не говоря уже о том, что отключение единственной учетной записи пользователя в случае несанкционированного доступа сделает невозможным доступ для всех пользователей. Но пока что у нас нет способа избежать этого.

Вторая проблема нынешней архитектуры безопасности — то, как мы обращаемся с единственным паролем. При каждом запуске приложения генерируется новый пароль, которым затем необходимо поделиться со всеми пользователями. И хотя масштабирование приложения мы пока что не обсуждали, каждый запускаемый экземпляр приложения Aircraft Positions будет генерировать свой уникальный пароль, так что пользователю, пытающемуся войти в данный конкретный экземпляр приложения, необходимо будет ввести именно этот конкретный пароль. Явно эту архитектуру можно и нужно усовершенствовать.

Добавляем аутентификацию

В центре возможностей аутентификации Spring Security лежит идея `UserDetailsService`. Так называется интерфейс, включающий один метод `loadUserByUsername(String username)`. Этот метод, будучи реализованным, возвращает объект, реализующий интерфейс `UserDetails`, который содержит ключевую информацию о пользователе: его имя, пароль, предоставленные ему полномочия и состояние учетной записи. Подобная гибкость делает возможными множество реализаций при помощи различных технологий. Для приложения не важны нюансы реализации, лишь бы `UserDetailsService` возвращал `UserDetails`.

Для создания компонента `UserDetails` я создам класс конфигурации, в котором и опишу метод создания компонента.

СОЗДАНИЕ КОМПОНЕНТА

Как упоминалось ранее, методы создания компонентов можно помещать в любой снабженный аннотацией `@Configuration` класс, в том числе имеющий метааннотации, которые включают аннотацию `@Configuration`, например основной класс приложения. Однако лучше создавать класс конфигурации для связанных групп компонентов. Если и приложение, и число компонентов невелико, достаточно одного класса.

За исключением самых маленьких и «одноразовых» приложений, я обычно не помещаю методы создания компонентов в основной класс приложения, снабженный аннотацией `@SpringBootApplication`. Размещение их в отдельном классе(-ах) упрощает тестирование за счет уменьшения числа компонентов, которые необходимо создать или имитировать. А в тех редких случаях, когда разработчику необходимо отключить группу компонентов, например класс `@Component` или `@Configuration`, загружающий данные или выполняющий аналогичные операции, достаточно удалить или закомментировать одну аннотацию уровня класса, чтобы отключить соответствующую функциональность, сохранив возможность легко ее возвратить. И хотя не следует применять стратегию «закомментировать и оставить» везде и всюду, в строго определенных случаях она вполне оправдывает себя.

Этот отдельный класс пригодится нам и в следующей версии, упрощая и ускоряя ее создание благодаря естественному разделению ответственности.

Прежде всего я создал класс `SecurityConfig` и снабдил его аннотацией `@Configuration`, чтобы Spring Boot мог найти и выполнить размещенные в нем методы создания компонентов. Для аутентификации необходим компонент, реализующий интерфейс `UserDetailsService`, так что я создал метод `authentication()`, который создает и возвращает этот компонент. Вот первый, умышленно неполный вариант его кода:

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

@Configuration
public class SecurityConfig {
    @Bean
    UserDetailsService authentication() {
```

```
        UserDetails peter = User.builder()
                .username("peter")
                .password("ppassword")
                .roles("USER")
                .build();

        UserDetails jodie = User.builder()
                .username("jodie")
                .password("jpassword")
                .roles("USER", "ADMIN")
                .build();

        System.out.println("   >>> Peter's password: " + peter.getPassword());
        System.out.println("   >>> Jodie's password: " + jodie.getPassword());

        return new InMemoryUserDetailsManager(peter, jodie);
    }
}
```

В методе `UserDetailsService authentication()` я создал два объекта приложения, реализующих требования интерфейса `UserDetails` с помощью метода `builder()` класса `User`, которые задают имя пользователя, пароль и роли/полномочия этого пользователя. Далее я выполняю `build()` для этих объектов `User` и присваиваю каждый из них локальной переменной.

Затем я отображаю пароли, *исключительно в целях демонстрации*, чтобы пояснить еще одно понятие, которое будет рассматриваться далее в этой главе. Но, повторюсь, *исключительно в целях демонстрации*.



Журналирование паролей — самая худшая разновидность антипаттерна. Никогда не заносите пароли в журнал в приложении, работающем в продакшене.

Наконец, я создаю экземпляр `InMemoryUserDetailsManager` на основе двух созданных ранее объектов `User` и возвращаю его в виде компонента Spring. `InMemoryUserDetailsManager` реализует интерфейсы `UserDetailsManager` и `UserDetailsPasswordService`, делая возможными такие задачи управления пользователями, как определение того, существует ли конкретный пользователь, создание, обновление и удаление пользователя, а также изменение или обновление пароля пользователя. Я применил `InMemoryUserDetailsManager` ради ясности демонстрации идеи (благодаря отсутствию внешних зависимостей), но роль компонента аутентификации может играть любой реализующий интерфейс `UserDetailsService` компонент.

Перезапустив приложение Aircraft Positions, я попытался аутентифицироваться и извлечь список текущих мест расположения воздушных судов. Получил следующий результат (часть заголовков удалена для краткости):

```
mheckler-a01 :: ~ " http :8080/aircraft --auth jodie:jpassword
HTTP/1.1 401
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Content-Length: 0
Expires: 0
Pragma: no-cache
WWW-Authenticate: Basic realm="Realm"
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
```

Придется найти причину проблемы. Возвращаясь в IDE, находим в трассе вызовов полезный фрагмент информации:

```
java.lang.IllegalArgumentException: There is no PasswordEncoder
    mapped for the id "null"
        at org.springframework.security.crypto.password
        .DelegatingPasswordEncoder$UnmappedIdPasswordEncoder
            .matches(DelegatingPasswordEncoder.java:250)
                ~[spring-security-core-5.3.4.RELEASE.jar:5.3.4.RELEASE]
```

По этой информации можно догадаться о причине проблемы. Первый же взгляд на занесенные в журнал пароли (напоминаю: *исключительно в целях демонстрации*) подтверждает догадку:

```
>>> Peter's password: ppassword
>>> Jodie's password: jpassword
```

Эти пароли, как можно заметить, передаются открытым текстом, без какого-либо кодирования. Следующий шаг по направлению к работоспособной защищенной аутентификации — добавление внутрь класса `SecurityConfig` средства кодирования паролей, как показано в следующем коде:

```
private final PasswordEncoder pwEncoder =
        PasswordEncoderFactories.createDelegatingPasswordEncoder();
```

Одна из проблем создания и сопровождения безопасных приложений состоит в том, что защита должна постоянно совершенствоваться. Поэтому в Spring Security не просто выделен один подключаемый кодировщик, а применяется фабрика с несколькими возможными кодировщиками, один из которых и получает задания.

Конечно, если конкретный кодировщик не указан, как в предыдущем примере, должен использоваться кодировщик по умолчанию. В настоящее время кодировщиком по умолчанию служит *BCrypt*, но гибкая архитектура Spring Security позволяет легко заменить один кодировщик другим при изменении стандарта и/или требований. Подобный изящный подход дает возможность легко переносить учетные данные с одного кодировщика на другой при входе пользователя в приложение, а значит, сократить число задач, которые не представляют непосредственной ценности для компании, но тем не менее критически важны для правильной и быстрой работы приложения.

Теперь, когда у нас есть кодировщик, можно шифровать с его помощью пароли пользователей. Для этого достаточно подключить вызов метода `encode()` кодировщика паролей, передав в него простой текстовый пароль и получив в ответ зашифрованный результат.



Строго говоря, шифрование значения означает также его кодирование, но не все кодировщики производят шифрование. Например, хеширование означает кодирование значения, но не обязательно шифрование. Все поддерживаемые Spring Security алгоритмы кодирования производят шифрование, но некоторые из поддерживаемых алгоритмов, оставленных для поддержки унаследованных приложений, намного менее безопасны, чем другие. Всегда используйте рекомендованный Spring Security кодировщик или кодировщик по умолчанию, который можно получить с помощью PasswordEncoderFactories.createDelegatingPasswordEncoder().

Переработанная версия класса `SecurityConfig` с поддержкой аутентификации:

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

@Configuration
public class SecurityConfig {
    private final PasswordEncoder pwEncoder =
            PasswordEncoderFactories.createDelegatingPasswordEncoder();

    @Bean
    UserDetailsService authentication() {
        UserDetails peter = User.builder()
                .username("peter")
```

```
                .password(pwEncoder.encode("ppassword"))
                .roles("USER")
                .build();

        UserDetails jodie = User.builder()
                .username("jodie")
                .password(pwEncoder.encode("jpassword"))
                .roles("USER", "ADMIN")
                .build();

        System.out.println("   >>> Peter's password: " + peter.getPassword());
        System.out.println("   >>> Jodie's password: " + jodie.getPassword());

        return new InMemoryUserDetailsManager(peter, jodie);
    }
}
```

Перезапустив приложение Aircraft Positions, я еще раз пробую пройти аутентификацию и извлечь список текущих местоположений воздушных судов. Получаю следующие результаты (часть заголовков и результатов удалены для краткости):

```
mheckler-a01 :: ~ " http :8080/aircraft --auth jodie:jpassword
HTTP/1.1 200
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Expires: 0
Pragma: no-cache
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block

[
    {
        "altitude": 24250,
        "barometer": 0.0,
        "bds40_seen_time": null,
        "callsign": null,
        "category": "A2",
        "flightno": "",
        "heading": 118,
        "id": 1,
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-10-12T16:13:26Z",
        "lat": 38.325119,
        "lon": -90.154159,
        "polar_bearing": 178.56009,
        "polar_distance": 37.661127,
        "pos_update_time": "2020-10-12T16:13:24Z",
```

```
        "reg": "N168ZZ",
        "route": "FMY-SUS",
        "selected_altitude": 0,
        "speed": 404,
        "squawk": null,
        "type": "LJ60",
        "vert_rate": 2880
    }
]
```

Полученные результаты подтверждают успешность аутентификации (ради экономии места опущен сценарий с преднамеренной неудачей аутентификации при указании неправильного пароля), так что зарегистрированные пользователи снова могут обращаться к предоставляемому API.

Возвращаясь к занесенным в журнал теперь уже закодированным паролям, видим в выводе IDE значения, подобные следующим:

```
>>> Peter's password:
    {bcrypt}$2a$10$rLKBzRBvtTtNcV9o8JHzFeaIskJIPXnYgVtCPs5H0GINZtk1WzsBu
>>> Jodie's password: {
    bcrypt}$2a$10$VR33/dlbSsEPPq6nlpnE/.ZQt0M4.bjvO5UYmw0ZW1aptO4G8dEkW
```

Эти значения подтверждают, что оба указанных в коде пароля из нашего примера были успешно закодированы с помощью назначенного кодировщика паролей BCrypt.

ПРИМЕЧАНИЕ ОТНОСИТЕЛЬНО ФОРМАТА ЗАКОДИРОВАННЫХ ПАРОЛЕЙ

Нужный кодировщик паролей выбирается автоматически на основе формата закодированного пароля, чтобы закодировать введенный аутентифицируемым пользователем пароль и сравнить его с хранимым. Spring Security для удобства предваряет закодированное значение ключом, указывающим, какой алгоритм применялся, что иногда приводит разработчиков в замешательство. Не будет ли выдана жизненно важная информация, если зашифрованный пароль попадет к злоумышленнику? Не упрощает ли эта информация расшифровку?

Короткий ответ: нет. Источник стойкости шифрования — в самом шифре, а не в кажущейся неясности.

Как же убедиться в стойкости?

У большинства методов шифрования есть отличительные признаки, говорящие о том, с помощью чего было зашифровано значение. Обратите внимание на два вышеупомянутых пароля. Оба закодированных значения начинаются с символьной строки `$2a$10$`, как, собственно, и все зашифрованные с помощью BCrypt значения. И хотя есть алгоритмы шифрования, на которые никак не указывают полученные с их помощью закодированные значения, они являются скорее исключением, чем правилом.

Авторизация

Приложение Aircraft Positions теперь успешно аутентифицирует пользователей, давая только прошедшим проверку доступ к API. У нашей схемы безопасности, впрочем, есть одна довольно серьезная проблема: возможность доступа к любой части API означает доступ ко всему API вне зависимости от ролей либо полномочий пользователя, точнее, вне зависимости от *отсутствия* у него ролей.

Для демонстрации простейшего примера этого изъяна в безопасности я добавлю в API приложения Aircraft Positions еще одну конечную точку посредством копирования, переименования и связывания с другим URI уже существующего метода `getCurrentAircraftPositions()` в классе `PositionController`. В завершенном виде `PositionController` выглядит следующим образом:

```
import lombok.AllArgsConstructor;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@AllArgsConstructor
@RestController
public class PositionController {
    private final PositionRetriever retriever;

    @GetMapping("/aircraft")
    public Iterable<Aircraft> getCurrentAircraftPositions() {
        return retriever.retrieveAircraftPositions();
    }

    @GetMapping("/aircraftadmin")
    public Iterable<Aircraft> getCurrentAircraftPositionsAdminPrivs() {
        return retriever.retrieveAircraftPositions();
    }
}
```

Мы хотим добиться того, чтобы доступ ко второму методу, `getCurrentAircraftPositionsAdminPrivs()`, был только у пользователей с ролью `ADMIN`. И хотя в этой версии примера возвращаемые значения идентичны возвращаемым `getCurrentAircraftPositions()`, вероятно, при расширении приложения это изменится, но общая концепция все равно останется применимой.

Перезапустив приложение Aircraft Positions и опять перейдя в командную строку, я вошел в приложение сначала как пользователь Jodie, чтобы проверить, доступна ли новая конечная точка (обращение к первой конечной точке прошло успешно, но опущено ради экономии места, часть заголовков и результатов также исключены для краткости):

```
mheckler-a01 :: ~ " http :8080/aircraftadmin --auth jodie:jpassword
HTTP/1.1 200
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Expires: 0
Pragma: no-cache
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block

[
    {
        "altitude": 24250,
        "barometer": 0.0,
        "bds40_seen_time": null,
        "callsign": null,
        "category": "A2",
        "flightno": "",
        "heading": 118,
        "id": 1,
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-10-12T16:13:26Z",
        "lat": 38.325119,
        "lon": -90.154159,
        "polar_bearing": 178.56009,
        "polar_distance": 37.661127,
        "pos_update_time": "2020-10-12T16:13:24Z",
        "reg": "N168ZZ",
        "route": "FMY-SUS",
        "selected_altitude": 0,
        "speed": 404,
        "squawk": null,
        "type": "LJ60",
        "vert_rate": 2880
    },
    {
        "altitude": 38000,
        "barometer": 1013.6,
        "bds40_seen_time": "2020-10-12T20:24:48Z",
        "callsign": "SWA1828",
        "category": "A3",
        "flightno": "WN1828",
        "heading": 274,
        "id": 2,
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-10-12T20:24:48Z",
        "lat": 39.348862,
        "lon": -90.751668,
        "polar_bearing": 310.510201,
        "polar_distance": 35.870036,
```

```
        "pos_update_time": "2020-10-12T20:24:48Z",
        "reg": "N8567Z",
        "route": "TPA-BWI-OAK",
        "selected_altitude": 38016,
        "speed": 397,
        "squawk": "7050",
        "type": "B738",
        "vert_rate": -128
    }
]
```

Далее я вошел в приложение как Peter. У него не должно быть доступа к методу `getCurrentAircraftPositionsAdminPrivs()`, связанному с конечной точкой `/aircraftadmin`. Но на практике все не так — в настоящее время у Peter, аутентифицированного пользователя, есть полный доступ к приложению:

```
mheckler-a01 :: ~ " http :8080/aircraftadmin --auth peter:ppassword
HTTP/1.1 200
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Expires: 0
Pragma: no-cache
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block

[
    {
        "altitude": 24250,
        "barometer": 0.0,
        "bds40_seen_time": null,
        "callsign": null,
        "category": "A2",
        "flightno": "",
        "heading": 118,
        "id": 1,
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-10-12T16:13:26Z",
        "lat": 38.325119,
        "lon": -90.154159,
        "polar_bearing": 178.56009,
        "polar_distance": 37.661127,
        "pos_update_time": "2020-10-12T16:13:24Z",
        "reg": "N168ZZ",
        "route": "FMY-SUS",
        "selected_altitude": 0,
        "speed": 404,
        "squawk": null,
        "type": "LJ60",
```

```
        "vert_rate": 2880
    },
    {
        "altitude": 38000,
        "barometer": 1013.6,
        "bds40_seen_time": "2020-10-12T20:24:48Z",
        "callsign": "SWA1828",
        "category": "A3",
        "flightno": "WN1828",
        "heading": 274,
        "id": 2,
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-10-12T20:24:48Z",
        "lat": 39.348862,
        "lon": -90.751668,
        "polar_bearing": 310.510201,
        "polar_distance": 35.870036,
        "pos_update_time": "2020-10-12T20:24:48Z",
        "reg": "N8567Z",
        "route": "TPA-BWI-OAK",
        "selected_altitude": 38016,
        "speed": 397,
        "squawk": "7050",
        "type": "B738",
        "vert_rate": -128
    }
]
```


Чтобы приложение Aircraft Positions могло не только аутентифицировать пользователей, но и проверять их полномочия на доступ к конкретным ресурсам, я провел рефакторинг класса `SecurityConfig`.

Первый шаг — заменить аннотацию уровня класса `@Configuration` на `@EnableWebSecurity`. `@EnableWebSecurity` — метааннотация, включающая удаленную аннотацию `@Configuration` (это позволяет объявлять в аннотированном классе методы создания компонентов), а также аннотацию `@EnableGlobalAuthentication` для расширенной автоконфигурации безопасности приложения со стороны Spring Boot. Теперь приложение Aircraft Positions готово к следующему шагу — описанию самого механизма авторизации.

Я переделаю класс `SecurityConfig` так, чтобы он расширял `WebSecurityConfigurerAdapter` — абстрактный класс с многочисленными переменными экземпляра и методами, удобными для расширения базовой конфигурации веб-безопасности приложения. В частности, класс `WebSecurityConfigurerAdapter` включает метод `configure(HttpSecurity http)`, обеспечивающий базовую реализацию авторизации пользователей:

```
protected void configure(HttpSecurity http) throws Exception {
    // Операторы вывода в журнал опущены

    http
        .authorizeRequests()
            .anyRequest().authenticated()
            .and()
        .formLogin().and()
        .httpBasic();
}
```

В предшествующей реализации имеются следующие директивы.

- Авторизовать все запросы от аутентифицированных пользователей.
- Предоставить созданные разработчиком простые переопределяемые формы входа в приложение и выхода из него.
- Обеспечить возможность базовой HTTP-аутентификации для небраузерных агентов пользователя, например утилит командной строки.

Это вполне достаточная политика безопасности на случай, если разработчик никак не конкретизировал авторизацию. Следующий шаг — переопределение такого поведения за счет этой самой конкретики.

Для открытия меню Generate в IntelliJ для Mac можно применить сочетание горячих клавиш Ctrl+O (или нажать правую кнопку мыши, а затем выбрать Generate), а далее выбрать пункт меню Override methods для отображения методов, которые можно переопределить или реализовать. Если выбрать метод с сигнатурой `configure(http:HttpSecurity):void`, будет сгенерирован следующий метод:

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    super.configure(http);
}
```

Теперь заменяем вызов метода суперкласса следующим кодом:

```
// Авторизация пользователя
@Override
protected void configure(HttpSecurity http) throws Exception {
    http.authorizeRequests()
            .mvcMatchers("/aircraftadmin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
            .and()
            .formLogin()
            .and()
            .httpBasic();
}
```

Эта реализация метода `configure(HttpSecurity http)` запускает следующие действия.

- С помощью средства сопоставления шаблонов строковых значений выполняется поиск совпадения пути запроса с `/aircraftadmin` и всеми дочерними для него путями.
- При успешном нахождении соответствия пользователь авторизуется на выполнение запроса, если у него есть роль либо полномочия `ADMIN`.
- Выполняются все прочие запросы от любых аутентифицированных пользователей.
- Предоставляются созданные разработчиком простые переопределяемые формы входа в приложение и выхода из него.
- Обеспечивается возможность базовой HTTP-аутентификации для небраузерных агентов пользователя — утилит командной строки и т. п.

Подобный минималистичный механизм авторизации включает в цепочку фильтров безопасности два фильтра: один для проверки на совпадение пути и привилегии администратора, а второй для всех прочих путей и аутентифицированного пользователя. Многоуровневый подход позволяет описывать сложные сценарии при помощи довольно простой и понятной логики.

Окончательная версия (для безопасности на основе форм) класса `SecurityConfig` выглядит следующим образом:

```
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration
    .EnableWebSecurity;
import org.springframework.security.config.annotation.web.configuration
    .WebSecurityConfigurerAdapter;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    private final PasswordEncoder pwEncoder =
            PasswordEncoderFactories.createDelegatingPasswordEncoder();

    @Bean
    UserDetailsService authentication() {
        UserDetails peter = User.builder()
                .username("peter")
```

```
                .password(pwEncoder.encode("ppassword"))
                .roles("USER")
                .build();

        UserDetails jodie = User.builder()
                .username("jodie")
                .password(pwEncoder.encode("jpassword"))
                .roles("USER", "ADMIN")
                .build();

        System.out.println("   >>> Peter's password: " + peter.getPassword());
        System.out.println("   >>> Jodie's password: " + jodie.getPassword());

        return new InMemoryUserDetailsManager(peter, jodie);
    }

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.authorizeRequests()
                .mvcMatchers("/aircraftadmin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
                .and()
                .formLogin()
                .and()
                .httpBasic();
    }
}
```

ДОПОЛНИТЕЛЬНЫЕ ПРИМЕЧАНИЯ ОТНОСИТЕЛЬНО БЕЗОПАСНОСТИ

Очень важно размещать более конкретные критерии перед более общими.

Каждый из запросов проходит цепочку фильтров безопасности, пока не натыкается на соответствующий ему фильтр, после чего обрабатывается в соответствии с заданными условиями. Если, например, разместить условие `.anyRequest().authenticated()` перед `.mvcMatchers("/aircraftadmin/**").hasRole("ADMIN")`, все запросы будут ему соответствовать, а значит, все аутентифицированные пользователи получат доступ ко всем предоставляемым ресурсам, включая все ресурсы в `/aircraftadmin`.

Размещение же `.mvcMatchers("/aircraftadmin/**").hasRole("ADMIN")` и всех более конкретных критериев перед критерием для `anyRequest()` возвращает `anyRequest()` в состояние перехвата всего, что только можно. Это очень удобный вариант, когда необходимо предоставить всем аутентифицированным пользователям доступ к определенным областям приложения, например общей начальной странице, меню и т. д.

Обратите внимание также на небольшие различия в аннотациях, названиях классов или компонентов, возвращаемых типах данных и подходах к аутентификации и авторизации в основанных на Spring MVC (нереактивных) и на Spring WebFlux (реактивных) приложениях Spring Boot, хотя очень многое в них совпадает. В какой-то мере мы обсудим это в следующем разделе.

Теперь убедимся, что все работает как должно. Перезапускаем приложение Aircraft Positions и обращаемся к конечной точке `/aircraftadmin` от имени пользователя Jodie из командной строки (обращение к первой конечной точке прошло успешно, но опущено ради экономии места, часть заголовков и результатов также исключена для краткости):

```
mheckler-a01 :: ~ " http :8080/aircraftadmin --auth jodie:jpassword
HTTP/1.1 200
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Expires: 0
Pragma: no-cache
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block

[
    {
        "altitude": 36000,
        "barometer": 1012.8,
        "bds40_seen_time": "2020-10-13T19:16:10Z",
        "callsign": "UPS2806",
        "category": "A5",
        "flightno": "5X2806",
        "heading": 289,
        "id": 1,
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-10-13T19:16:14Z",
        "lat": 38.791122,
        "lon": -90.21286,
        "polar_bearing": 189.515723,
        "polar_distance": 9.855602,
        "pos_update_time": "2020-10-13T19:16:12Z",
        "reg": "N331UP",
        "route": "SDF-DEN",
        "selected_altitude": 36000,
        "speed": 374,
        "squawk": "6652",
        "type": "B763",
        "vert_rate": 0
    },
    {
        "altitude": 25100,
        "barometer": 1012.8,
        "bds40_seen_time": "2020-10-13T19:16:13Z",
        "callsign": "ASH5937",
        "category": "A3",
        "flightno": "AA5937",
        "heading": 44,
        "id": 2,
```

```
        "is_adsb": true,
        "is_on_ground": false,
        "last_seen_time": "2020-10-13T19:16:13Z",
        "lat": 39.564148,
        "lon": -90.102459,
        "polar_bearing": 5.201331,
        "polar_distance": 36.841422,
        "pos_update_time": "2020-10-13T19:16:13Z",
        "reg": "N905J",
        "route": "DFW-BMI-DFW",
        "selected_altitude": 11008,
        "speed": 476,
        "squawk": "6270",
        "type": "CRJ9",
        "vert_rate": -2624
    }
]
```

Как и следовало ожидать, у Jodie есть доступ к конечной точке `/aircraftadmin` благодаря роли `ADMIN`. Далее попробуем войти в приложение от имени Peter. Отметим, что первое обращение к конечной точке прошло успешно, но опущено ради экономии места, часть заголовков и результатов также исключена для краткости:

```
mheckler-a01 :: ~ " http :8080/aircraftadmin --auth peter:ppassword
HTTP/1.1 403
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Expires: 0
Pragma: no-cache
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block

{
    "error": "Forbidden",
    "message": "",
    "path": "/aircraftadmin",
    "status": 403,
    "timestamp": "2020-10-13T19:18:10.961+00:00"
}
```

Именно так и должно быть, поскольку у Peter есть только роль `USER`, а не `ADMIN`. Система работает.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Полный код примера на основе форм находится в ветке chapter10forms в репозитории кода.

Реализация OpenID Connect и OAuth2 для аутентификации и авторизации

Хотя аутентификация на основе форм и внутренняя авторизация удобны для большого числа приложений, существует немало сценариев использования, в которых методы аутентификации типа «нечто, что мы знаем» не вполне подходят или даже недостаточны для желаемого или необходимого уровня безопасности. Вот некоторые, но далеко не все примеры.

- Бесплатные сервисы, требующие аутентификации, которым не нужно никакой информации о пользователе (или им нежелательно иметь такую информацию по юридическим и иным причинам).
- Ситуации, в которых однофакторная аутентификация не дает достаточной степени защиты, так что необходима поддержка многофакторной аутентификации (MFA).
- Создание и сопровождение инфраструктуры защищенного ПО для управления паролями, ролями либо полномочиями и прочими необходимыми механизмами.
- Вопросы возможной ответственности в случае взлома.

Для всех этих вопросов и целей не существует простого решения, но несколько компаний создали и поддерживают ресурсы для обеспечения отказоустойчивой защищенной инфраструктуры аутентификации и авторизации, предлагая их для всеобщего применения за очень низкую плату, а то и вовсе бесплатно. В их числе Okta, ведущий поставщик безопасности, и прочие компании, чей бизнес требует надежной аутентификации пользователей и проверки их полномочий, в том числе Facebook, GitHub и Google. Spring Security поддерживает все эти и многие другие варианты посредством OpenID Connect и OAuth2.

OAuth2 был создан как стороннее средство авторизации пользователей в конкретных ресурсах, например облачных сервисах, виртуальных хранилищах и приложениях. OpenID Connect на основе OAuth2 открывает возможности для единообразной, стандартизированной аутентификации с помощью одного или нескольких факторов:

- нечто, что мы знаем, например пароль;
- нечто, чем мы обладаем, например аппаратный ключ;
- нечто, что является частью нас самих, например биометрический идентификатор.

Spring Boot и Spring Security включают готовую поддержку автоконфигурации для реализаций OpenID Connect и OAuth2, предлагаемых Facebook, GitHub, Google и Okta, с дополнительными легко настраиваемыми (благодаря опубликованным стандартам OpenID Connect и OAuth2, а также расширяемой архитектуре Spring Security) поставщиками. Для следующих примеров задействуются библиотеки Okta и механизмы аутентификации + авторизации, но различия между поставщиками в основном непринципиальны. Можете без колебаний использовать того поставщика безопасности, который вам лучше подходит.

РАЗЛИЧНЫЕ РОЛИ ПРИЛОЖЕНИЙ/СЕРВИСОВ ДЛЯ OPENID CONNECT И OAUTH2

Хотя этот раздел посвящен непосредственно ролям, выполняемым различными сервисами с помощью OpenID Connect и OAuth2 для аутентификации и авторизации соответственно, на самом деле он частично или полностью применим к любым сторонним механизмам аутентификации и авторизации.

Приложения/сервисы играют три основные роли:

- клиент;
- сервер авторизации;
- сервер ресурсов.

Обычно один или несколько сервисов играют роль клиентов, приложений/сервисов, с которыми взаимодействует конечный пользователь, работающих с одним или несколькими поставщиками безопасности для аутентификации и получения авторизации (ролей или полномочий), предоставляемых пользователям для различных ресурсов.

За аутентификацию пользователя и возврат клиенту(-ам) информации о полномочиях пользователей отвечает один или несколько серверов авторизации. Они выпускают выдаваемые на определенное время авторизации и при необходимости продлевают их.

Серверы ресурсов обеспечивают доступ к защищенным ресурсам в зависимости от предъявляемых клиентами полномочий.

Spring Security позволяет разработчикам создавать все три типа приложений/сервисов, но в этой книге я сосредоточусь на создании клиента и сервера ресурсов. В настоящее время Spring Authorization Server (https://oreil.ly/spraut) находится на экспериментальной стадии, но быстро развивается и очень удобен для нескольких сценариев использования. Впрочем, для множества организаций и многих из перечисленных ранее целей разумнее будет задействовать сторонние сервисы авторизации. Как всегда, следует выбирать исходя из конкретных требований.

В этом примере я переделаю приложение Aircraft Positions, которое теперь будет играть роль клиентского приложения OpenID Connect и OAuth2, проверяющего,

тот ли пользователь, за кого он себя выдает, с помощью функциональности Okta и получающего полномочия пользователя для доступа к ресурсам, предоставляемым сервером ресурсов. А затем переделаю приложение PlaneFinder, чтобы оно в качестве сервера ресурсов OAuth2 предоставляло ресурсы в зависимости от учетных данных, передаваемых с запросами клиентским приложением Aircraft Positions.

Клиентское приложение Aircraft Positions

Обычно я начинаю с самого дальнего по стеку приложения, но в данном случае мне кажется, что лучше поступить наоборот из-за потоков, соответствующих получению (или неполучению) пользователем доступа к ресурсу.

Пользователь обращается к клиентскому приложению, применяющему какой-либо механизм аутентификации. После аутентификации запросы пользователей на ресурсы перенаправляются так называемым серверам ресурсов, отвечающим за хранение и обработку последних. Это хорошо знакомый большинству из нас логический поток. Включение систем безопасности в том же порядке — сначала клиент, затем сервер ресурсов — прекрасно согласуется с нашим собственным ожидаемым потоком авторизации.

Добавление зависимостей OpenID Connect и OAuth2 в приложение Aircraft Positions

Как и при обеспечении безопасности на основе форм, при создании нового проекта клиента Spring Boot можно легко добавить дополнительные зависимости с помощью Spring Initializr, чтобы попробовать применить OpenID Connect и OAuth2 в разрабатываемом с нуля клиентском приложении (рис. 10.2).

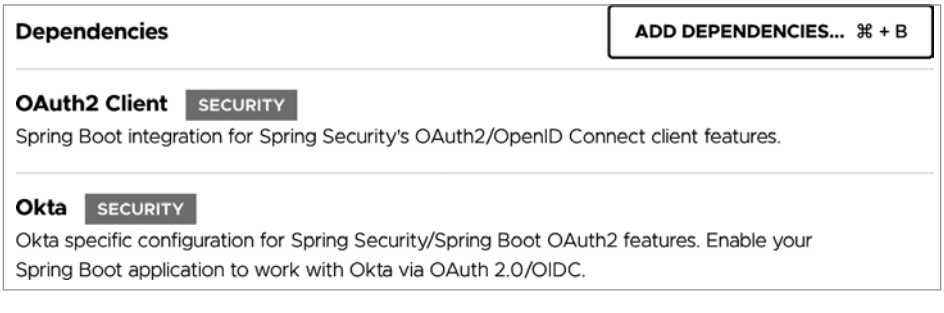


Рис. 10.2. Зависимости в Spring Initializr для клиентского приложения OpenID Connect и OAuth2, использующего Okta

Для модификации уже существующего приложения требуется лишь немногим больше усилий. Поскольку мы заменяем текущий вариант обеспечения безопасности на основе форм, то сначала удаляем существующую зависимость для Spring Security, добавленную в предыдущем разделе. А затем добавляем в файл сборки Maven `pom.xml` приложения Aircraft Positions те же самые две зависимости, которые Spring Initializr добавляет для OpenID Connect и OAuth2, одну — для OAuth2 Client (включающего функциональность аутентификации OpenID Connect и прочие необходимые компоненты) и одну — для Okta, поскольку мы хотим использовать их инфраструктуру для аутентификации и управления полномочиями:

```
<!--  Закомментируйте или удалите это  -->
<!--<dependency>-->
<!--  <groupId>org.springframework.boot</groupId>-->
<!--  <artifactId>spring-boot-starter-security</artifactId>-->
<!--</dependency>-->

<!--  И добавьте это              -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
<dependency>
    <groupId>com.okta.spring</groupId>
    <artifactId>okta-spring-boot-starter</artifactId>
    <version>1.4.0</version>
</dependency>
```



Включаемая в настоящее время версия библиотеки Spring Boot Starter для Okta, 1.4.0, протестирована и проверена на совместимость с текущей версией Spring Boot. При добавлении зависимостей в файл сборки вручную разработчику рекомендуется зайти на сайт Spring Initializr (https://start.spring.io), выбрать актуальную в данный момент версию Boot, добавить Okta или другую зависимость с конкретной версией и выполнить Explore, чтобы проверить рекомендуемый номер версии.

После повторной сборки можно переделать код, чтобы приложение Aircraft Positions могло осуществлять аутентификацию с помощью Okta и получать информацию о полномочиях пользователей.

Рефакторинг приложения Aircraft Positions для аутентификации и авторизации

Для настройки нынешнего приложения Aircraft Positions в качестве клиентского приложения OAuth2 необходимо сделать три вещи:

- удалить конфигурацию безопасности на основе форм;
- добавить конфигурацию OAuth2 в `WebClient`, используемый для доступа к конечным точкам PlaneFinder;
- указать учетные данные зарегистрированного клиента OpenID Connect + + OAuth2 и URI поставщика безопасности — в данном случае Okta.

Первые две задачи я решил совместно, начав с удаления целиком тела класса `SecurityConfig`. При желательности либо необходимости сохранить контроль за доступом к ресурсам, предоставляемым локально приложением Aircraft Positions, можно, конечно, оставить `SecurityConfig` в нынешнем или слегка измененном виде. Впрочем, для этого примера приложение PlaneFinder играет роль сервера ресурсов и как таковое отвечает за контроль или запрет доступа к запрашиваемым важным ресурсам. Приложение Aircraft Positions играет просто роль клиента пользователя, взаимодействующего с инфраструктурой безопасности для аутентификации пользователей, а затем передающего запросы на ресурсы серверу(-ам) ресурсов.

Я заменил аннотацию `@EnableWebSecurity` на `@Configuration`, поскольку автоконфигурация для локальной аутентификации больше не нужна. Также я убрал `extends WebSecurityConfigurerAdapter`, поскольку данная конкретная версия приложения Aircraft Positions не ограничивает запросы к конечным точкам, а передает полномочия пользователей вместе с запросами приложению PlaneFinder для сравнения с полномочиями, необходимыми для доступа к ресурсам, и соответствующих действий.

Далее я создал компонент `WebClient` в классе `SecurityConfig` для использования во всем приложении Aircraft Positions. Это требование не доставляет проблем, поскольку можно просто включить конфигурацию OAuth2 в создание присваиваемого переменной-члену экземпляра `WebClient` внутри `PositionRetriever`, и в пользу такого варианта есть серьезные аргументы. В то же время `PositionRetriever` необходим доступ к `WebClient`, но настройка `WebClient` для осуществления конфигурации OpenID Connect и OAuth2 выходит далеко за пределы основной задачи `PositionRetriever` — извлечения данных о местоположении воздушных судов.

Создание и настройка `WebClient` для аутентификации и авторизации прекрасно соответствуют сфере обязанностей класса с названием `SecurityConfig`:

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.client.registration
    .ClientRegistrationRepository;
import org.springframework.security.oauth2.client.web
```

```
    .OAuth2AuthorizedClientRepository;
import org.springframework.security.oauth2.client.web.reactive.function.client
    .ServletOAuth2AuthorizedClientExchangeFilterFunction;
import org.springframework.web.reactive.function.client.WebClient;

@Configuration
public class SecurityConfig {
    @Bean
    WebClient client(ClientRegistrationRepository regRepo,
                     OAuth2AuthorizedClientRepository cliRepo) {
        ServletOAuth2AuthorizedClientExchangeFilterFunction filter =
                new ServletOAuth2AuthorizedClientExchangeFilterFunction
                    (regRepo, cliRepo);

        filter.setDefaultOAuth2AuthorizedClient(true);

        return WebClient.builder()
                .baseUrl("http://localhost:7634/")
                .apply(filter.oauth2Configuration())
                .build();
    }
}
```

В метод создания компонента `client()` автоматически связываются два компонента:

- `ClientRegistrationRepository` — список клиентов OAuth2, указанных как используемые приложением, обычно в файле свойств, например `application.yml`;
- `OAuth2AuthorizedClientRepository` — список клиентов OAuth2, соответствующих аутентифицированному пользователю, отвечающих за `OAuth2AccessToken` этого пользователя.

Для создания и настройки внутри этого метода компонента `WebClient` я произвел следующие действия.

1. Инициализировал функцию `filter` двумя внедряемыми объектами репозиториев.
2. Подтвердил, что необходимо использовать авторизованный клиент, принятый по умолчанию. Обычно так и происходит — в конце концов аутентифицированный пользователь обычно является владельцем ресурса, который желает получить к нему доступ, но иногда в случаях делегирования доступа может потребоваться другой авторизованный клиент. Я задал URI и применил к `WebClient.builder` фильтр, настроенный для OAuth2, после чего выполнил операцию `build()` объекта `WebClient`, вернув компонент Spring и добавив его в `ApplicationContext`. Теперь мы можем использовать `WebClient` с поддержкой OAuth2 везде в приложении Aircraft Positions.

Поскольку мы теперь создаем компонент `WebClient` посредством метода создания компонента, можно удалить оператор создания и прямого присвоения объекта переменной экземпляра внутри класса `PositionRetriever`, заменив его простым объявлением переменной-члена. Поскольку класс снабжен аннотацией Lombok `@AllArgsConstructor`, Lombok автоматически добавляет параметр `WebClient` в конструктор с полным набором параметров, генерируемый для этого класса. Благодаря доступности компонента `WebClient` в `ApplicationContext` Spring Boot автоматически связывает его в `PositionRetriever`, где он автоматически присваивается переменной-члену `WebClient`. Переделанный нами только что класс `PositionRetriever` теперь выглядит следующим образом:

```
import lombok.AllArgsConstructor;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;

@AllArgsConstructor
@Component
public class PositionRetriever {
    private final AircraftRepository repository;
    private final WebClient client;

    Iterable<Aircraft> retrieveAircraftPositions() {
        repository.deleteAll();

        client.get()
                .uri("/aircraft")
                .retrieve()
                .bodyToFlux(Aircraft.class)
                .filter(ac -> !ac.getReg().isEmpty())
                .toStream()
                .forEach(repository::save);
        return repository.findAll();
    }
}
```

Ранее в этом разделе я упоминал `ClientRegistrationRepository` — список клиентов OAuth2, которые должны использоваться приложением. Существует немало способов наполнения этого репозитория, но обычно записи задаются в виде свойств приложения. В этом примере я добавляю следующую информацию в файл `application.yml` приложения Aircraft Positions (здесь указаны фиктивные значения):

```
spring:
  security:
    oauth2:
```

```
client:
  registration:
    okta:
      client-id: <идентификатор_назначенного_клиента>
      client-secret: <секрет_назначенного_клиента>
  provider:
    okta:
      issuer-uri: https://<назначенный_поддомен>
                        .oktapreview.com/oauth2/default
```

ПОЛУЧЕНИЕ ИНФОРМАЦИИ О КЛИЕНТЕ И ЗАПРАШИВАЮЩЕЙ СТОРОНЕ ОТ ПОСТАВЩИКА OPENID CONNECT + OAUTH2

Поскольку в центре внимания этого раздела — разработка защищенных приложений Spring Boot должным образом — посредством взаимодействия с инфраструктурой безопасности, предоставляемой доверенными третьими сторонами, подробное пошаговое описание создания учетных записей в этих многочисленных поставщиках безопасности, регистрация приложений и описание полномочий пользователей для различных ресурсов выходят за рамки декларируемых целей этой главы. К счастью, процедуры выполнения этих внешних по отношению к системе действий описаны в примере репозитория OAuth2 для Spring Security. Найти описание настройки Okta в качестве поставщика аутентификации можно по следующей ссылке: https://oreil.ly/sbokta. В том же документе можно найти описание настройки и для других поддерживаемых поставщиков.

При этих настройках у компонента `ClientRegistrationRepository` приложения Aircraft Positions будет одна запись для Okta, используемая автоматически при попытке входа пользователя в приложение.



Если описано несколько поставщиков, при первом запросе будет выводиться веб-страница с просьбой к пользователю выбрать поставщика.

Я внес одно маленькое изменение в приложение Aircraft Positions (и небольшое вытекающее из него изменение в `PositionRetriever`) для более наглядной демонстрации успешной и неудачной авторизации пользователей. Продублировал единственную описанную в настоящий момент в классе `PositionController` конечную точку, переименовал ее и связал с URI, подразумевающим доступ «только для администратора»:

```
import lombok.AllArgsConstructor;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
@AllArgsConstructor
@RestController
public class PositionController {
    private final PositionRetriever retriever;

    @GetMapping("/aircraft")
    public Iterable<Aircraft> getCurrentAircraftPositions() {
        return retriever.retrieveAircraftPositions("aircraft");
    }

    @GetMapping("/aircraftadmin")
    public Iterable<Aircraft> getCurrentAircraftPositionsAdminPrivs() {
        return retriever.retrieveAircraftPositions("aircraftadmin");
    }
}
```

Чтобы можно было обращаться к обеим конечным точкам приложения PlaneFinder с помощью одного метода класса `PositionRetriever`, я сделал так, что его метод `retrieveAircraftPositions()` принимает теперь динамический параметр пути `String endpoint`, используя его при формировании клиентского запроса. Модифицированный класс `PositionRetriever` выглядит следующим образом:

```
import lombok.AllArgsConstructor;
import org.springframework.stereotype.Component;
import org.springframework.web.reactive.function.client.WebClient;

@AllArgsConstructor
@Component
public class PositionRetriever {
    private final AircraftRepository repository;
    private final WebClient client;

    Iterable<Aircraft> retrieveAircraftPositions(String endpoint) {
        repository.deleteAll();

        client.get()
                .uri((null != endpoint) ? endpoint : "")
                .retrieve()
                .bodyToFlux(Aircraft.class)
                .filter(ac -> !ac.getReg().isEmpty())
                .toStream()
                .forEach(repository::save);

        return repository.findAll();
    }
}
```

Приложение Aircraft Positions теперь представляет собой полностью настроенное клиентское приложение OpenID Connect и OAuth2. Далее я переделаю

приложение PlaneFinder, чтобы оно служило сервером ресурсов OAuth2, предоставляющим по запросу ресурсы авторизованным пользователям.

Сервер ресурсов PlaneFinder

Как и при любом рефакторинге, связанном с изменением зависимостей, мы начнем с файла сборки.

Добавление зависимостей OpenID Connect и OAuth2 в приложение PlaneFinder

Как упоминалось ранее, можно легко добавить зависимость или две с помощью Spring Initializr при создании нового проекта сервера ресурсов OAuth2 Spring Boot в разрабатываемом с нуля клиентском приложении (рис. 10.3).

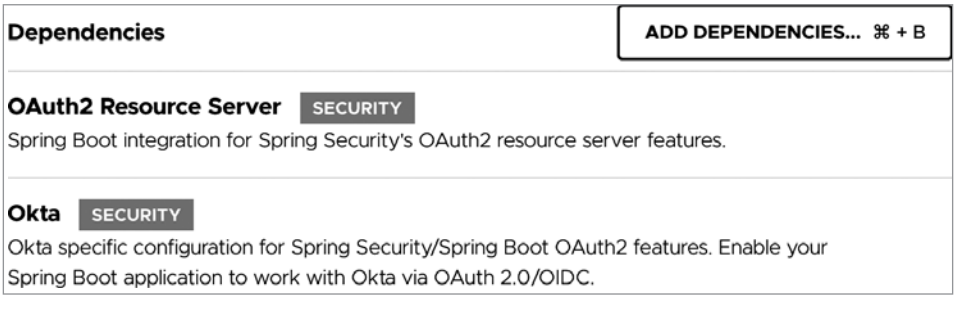


Рис. 10.3. Зависимости в Spring Initializr для сервера ресурсов OAuth2, использующего Okta

Модификация существующего приложения PlaneFinder не представляет сложностей. Добавляем в файл сборки `pom.xml` Maven приложения PlaneFinder те же самые две зависимости, которые Initializr добавляет для сервера ресурсов OAuth2 и Okta, поскольку собираемся использовать их инфраструктуру для проверки полномочий:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
<dependency>
    <groupId>com.okta.spring</groupId>
    <artifactId>okta-spring-boot-starter</artifactId>
    <version>1.4.0</version>
</dependency>
```

ПРИМЕЧАНИЯ ОТНОСИТЕЛЬНО ЭТИХ ЗАВИСИМОСТЕЙ

Поскольку мы выбрали Okta для механизмов OpenID Connect и OAuth2 (инфраструктуры и библиотек безопасности) в этих примерах, то могли получить те же результаты, добавив в проект одну только зависимость стартового пакета Okta Spring Boot. Зависимость для Okta включает все прочие необходимые библиотеки как для клиентских приложений, так и для серверов ресурсов OAuth2:

- Spring Security Config;
- Spring Security Core;
- Spring Security OAuth2 Client;
- Spring Security Core;
- Spring Security JOSE;
- Spring Security Resource Server;
- Spring Security Web.

Добавление в Initializr зависимости для сервера ресурсов OAuth2 не перегружает новое приложение, поскольку она включает те же самые зависимости, за исключением зависимости OAuth2 Client, а зависимости Okta охватывают их все. В основном я поступил так для наглядности, а также чтобы не гоняться за зависимостями потом, если решу поменять поставщиков аутентификации и авторизации. Тем не менее я рекомендую всегда тщательно изучать дерево зависимостей своего приложения и удалять ненужные.

После повторной сборки можно переделать код, чтобы приложение PlaneFinder проверяло полномочия пользователей, передаваемые с входящими запросами, для проверки прав доступа пользователей и предоставления им доступа к ресурсам приложения PlaneFinder или отказа в нем.

Рефакторинг приложения PlaneFinder для авторизации на доступ к ресурсам

Значительную часть того, что нужно для аутентификации и авторизации OpenID Connect и OAuth2 с помощью Okta в нашей распределенной системе, мы уже сделали. Рефакторинг приложения PlaneFinder для выполнения им обязанностей сервера ресурсов OAuth2 должным образом требует минимальных усилий:

- добавить поддержку JWT (JSON Web Token);
- сравнить передаваемые с JWT полномочия с теми, что требуются для доступа к запрашиваемым ресурсам.

Для решения обеих задач достаточно создать один компонент `SecurityWebFilterChain`, с помощью которого Spring Security будет извлекать содержимое входящих запросов JWT, проверять его и сопоставлять с требуемыми полномочиями.

Снова создаем класс `SecurityConfig` и снабжаем его аннотацией `@Configuration`, чтобы выделить особое место для методов создания компонентов. А затем создаем метод `securityWebFilterChain()` следующего вида:

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.web.server.ServerHttpSecurity;
import org.springframework.security.web.server.SecurityWebFilterChain;

@Configuration
public class SecurityConfig {
    @Bean
    public SecurityWebFilterChain securityWebFilterChain(ServerHttpSecurity http) {
        http
           .authorizeExchange()
           .pathMatchers("/aircraft/**").hasAuthority("SCOPE_closedid")
           .pathMatchers("/aircraftadmin/**").hasAuthority("SCOPE_openid")
           .and().oauth2ResourceServer().jwt();

        return http.build();
    }
}
```

Для создания этой цепочки фильтров производим автосвязывание существующего компонента `ServerHttpSecurity`, предоставляемого автоконфигурацией безопасности Spring Boot. Этот компонент используется вместе с приложениями при поддержке WebFlux, то есть при наличии по пути к классам зависимости `spring-boot-starter-webflux`.



Приложения, у которых нет WebFlux по пути к классам, используют вместо этого компонент HttpSecurity и его соответствующие методы, как происходило в примере аутентификации на основе форм ранее в этой главе.

Далее настраиваем критерии безопасности компонента `ServerHttpSecurity`, указывая, как должны обрабатываться запросы. Для этого я задал два пути к ресурсам для сопоставления с запросами и соответствующими полномочиями пользователей, а также включил поддержку сервера ресурсов OAuth2 с JWT в качестве носителей информации о пользователе.



JWT иногда называют токенами-носителями (bearer token), поскольку они несут авторизацию пользователя на доступ к ресурсам.

Наконец, формируем объект `SecurityWebFilterChain` из компонента `ServerHttpSecurity` и возвращаем его, чтобы он был доступен в качестве компонента во всем приложении PlaneFinder.

При поступлении запроса указанный в нем путь сравнивается с путями из цепочки фильтров, пока не будет найдено совпадение. После чего приложение проверяет действительность токена при помощи поставщика OAuth2 — в данном случае Okta, а затем сравнивает содержащиеся в нем полномочия с полномочиями, требующимися для доступа к указанным ресурсам. При обнаружении соответствия доступ разрешается, в противном случае приложение возвращает код состояния `403 Forbidden (Запрещено)`.

Возможно, вы обратили внимание на то, что в приложении PlaneFinder пока нет пути к ресурсу во втором вызове `pathMatchers`. Я добавил этот путь в класс `PlaneController` исключительно для того, чтобы привести пример как успешной, так и провальной проверки полномочий.

Поставщики OAuth2 могут содержать несколько полномочий по умолчанию, в том числе `openid`, `email`, `profile` и др. В этом примере цепочки фильтров проверяются несуществующие (при наших настройках поставщика и полномочий OAuth2) полномочия `closedid`, соответственно, все запросы ресурсов с путем, начинающимся с `/aircraft`, завершатся ошибкой. А все входящие запросы ресурсов, начинающихся с пути `/aircraftadmin`, содержащие действительный токен, будут выполнены успешно.



Spring Security добавляет "SCOPE_" в начало предоставляемых поставщиком OAuth2 полномочий, взаимно однозначно сопоставляя внутреннее представление Spring Security областей видимости с полномочиями OAuth2. Использующим Spring Security с OAuth2 разработчикам следует знать об этом, но никаких практических последствий это отличие не несет.

В завершение рефакторинга кода я добавлю в класс `PlaneController` приложения PlaneFinder соответствие для конечной точки `/aircraftadmin`, указанное в предыдущем `pathMatchers`. Для этого просто скопирую функциональность

уже существующей конечной точки `/aircraft`, чтобы показать две конечные точки с различными критериями доступа:

```
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;
import reactor.core.publisher.Flux;

import java.io.IOException;
import java.time.Duration;

@Controller
public class PlaneController {
    private final PlaneFinderService pfService;

    public PlaneController(PlaneFinderService pfService) {
        this.pfService = pfService;
    }

    @ResponseBody
    @GetMapping("/aircraft")
    public Flux<Aircraft> getCurrentAircraft() throws IOException {
        return pfService.getAircraft();
    }

    @ResponseBody
    @GetMapping("/aircraftadmin")
    public Flux<Aircraft> getCurrentAircraftByAdmin() throws IOException {
        return pfService.getAircraft();
    }

    @MessageMapping("acstream")
    public Flux<Aircraft> getCurrentACStream() throws IOException {
        return pfService.getAircraft().concatWith(
                Flux.interval(Duration.ofSeconds(1))
                        .flatMap(l -> pfService.getAircraft()));
    }
}
```

Наконец, необходимо указать приложению путь для обращения к поставщику OAuth2, чтобы проверить входящие JWT. Это можно делать по-разному, поскольку спецификация конечных точек поставщиков OAuth2 допускает вольную трактовку, но Okta успешно реализует URI запрашивающей стороны, играющее роль основного URI конфигурации, из которого затем можно получить прочие необходимые URI. Так что разработчикам приложения нужно добавить всего одно свойство.

Я преобразовал файл `application.properties` из формата пары «ключ — значение» в `application.yml` со структурированным деревом свойств, немного сократив таким образом число повторов. Делать это не обязательно, но полезно при большом количестве дублирований ключей свойств:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://<your_assigned_subdomain_here>.oktapreview.com/
              oauth2/default

  rsocket:
    server:
      port: 7635

server:
  port: 7634
```

Теперь, когда все готово, перезапускаем сервер ресурсов OAuth2 PlaneFinder и клиентское приложение OpenID Connect + OAuth2 Aircraft Positions, чтобы проверить результаты. Переходим в браузере по адресу конечной точки `/aircraftadmin` API приложения Aircraft Positions (http://localhost:8080/aircraftadmin), при этом происходит перенаправление в Okta для аутентификации, как показано на рис. 10.4.

После ввода мной корректных учетных данных пользователя Okta перенаправляет аутентифицированного пользователя (меня) в клиентское приложение Aircraft Positions. Запрошенная мной конечная точка, в свою очередь, запрашивает данные о местоположении воздушных судов у приложения PlaneFinder, передавая ему предоставленный Okta JWT. Когда приложение PlaneFinder находит соответствие запрошенному пути среди путей ресурсов и проверяет действительность JWT и содержащиеся в нем полномочия, оно возвращает текущие местоположения воздушных судов клиентскому приложению Aircraft Positions, которое выдает их мне (рис. 10.5).

А что будет, если запросить ресурс, на доступ к которому я не авторизован? Для демонстрации примера неудачной авторизации попробую обратиться к конечной точке `/aircraft` приложения Aircraft Positions по адресу http://localhost:8080/aircraft. Результаты приведены на рис. 10.6. Обратите внимание: поскольку я уже аутентифицирован, для доступа в дальнейшем к приложению Aircraft Positions мне не нужно повторно проходить аутентификацию.

Обратите внимание: в ответе не приводится подробная информация о том, почему не удалось извлечь данные. Рекомендуемая практика безопасности — не

сообщать лишней информации, которая может помочь возможным злоумышленникам в конце концов взломать систему. Впрочем, если заглянуть в журналы приложения Aircraft Positions, мы увидим следующую дополнительную информацию:

```
Forbidden: 403 Forbidden from GET http://localhost:7634/aircraft with root cause
```

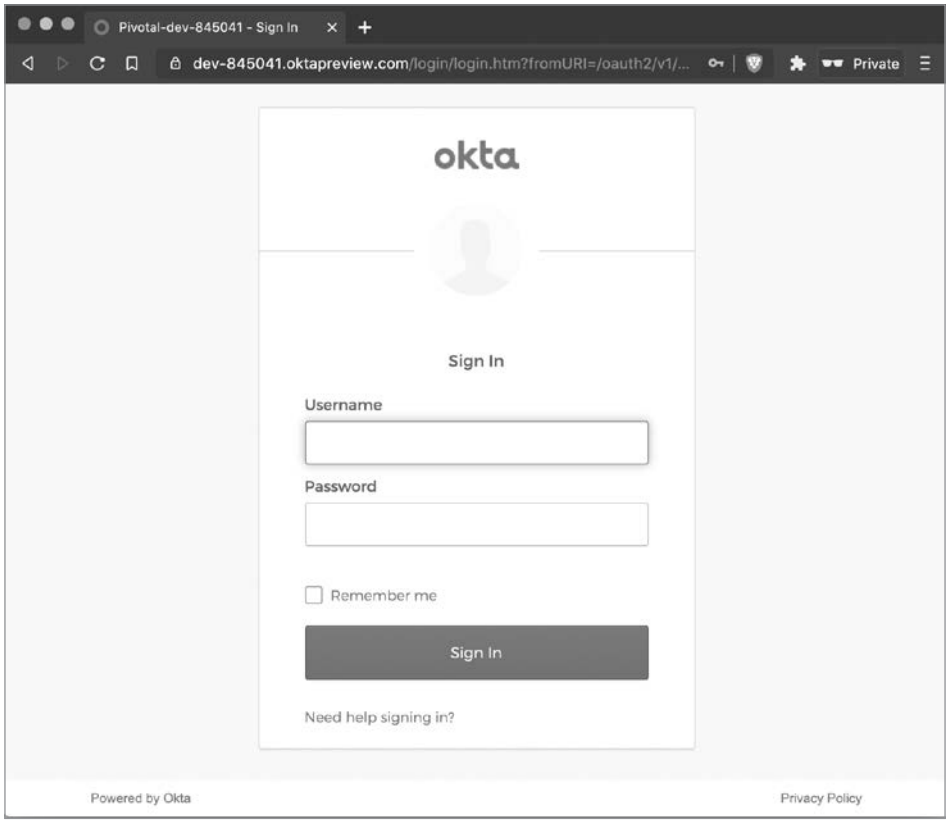


Рис. 10.4. Приглашение на вход в систему, выдаваемое поставщиком OpenID Connect (Okta)

Именно этот ответ ожидался, ведь фильтр приложения PlaneFinder, соответствующий запросам на ресурсы по пути `/aircraft` или дочерним путям, требует не описанных нами полномочий `closedid`, которые, естественно, не были предоставлены.

Эти примеры отражают квинтэссенцию ключевых аспектов аутентификации OpenID Connect и авторизации OAuth2 с помощью сторонних поставщиков

безопасности. На этих базовых принципах и шагах основаны все прочие модификации и расширения этого типа аутентификации и авторизации для приложений Spring Boot.



Рис. 10.5. Успешный возврат данных о местоположении воздушных судов

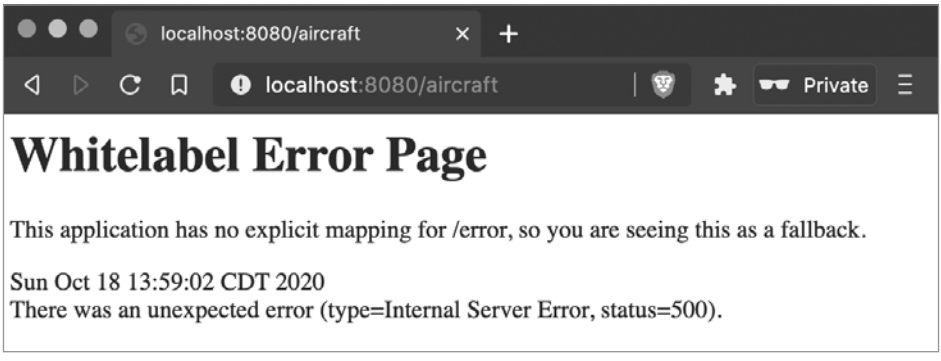


Рис. 10.6. Результаты неудачной авторизации

СЛЕДУЕМ ЗА ПОТОКОМ

В примерах из этого раздела используется поток кода авторизации (Authorization Code Flow), результатом которого является предоставление кода авторизации (Authorization Code Grant). Поток кода авторизации представляет собой процесс, обычно лежащий в основе безопасных веб-приложений и играющий центральную роль в рекомендуемом для нативных приложений потоке кода авторизации с PKCE (Proof Key for Code Exchange).

Существуют и другие потоки авторизации, в частности поток паролей владельцев ресурсов (Resource Owner Password Flow), неявный поток (Implicit Flow) и поток учетных данных клиентов (Client Credentials Flow), обсуждение которых, а также их ограничений и сценариев использования выходит за рамки данной главы.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Полный код для этой главы находится в ветке chapter10end в репозитории кода.

Резюме

Не понимая основных концепций аутентификации и авторизации, невозможно создавать безопасные приложения, поскольку на них основана проверка личности пользователей и контроль доступа. Spring Security сочетает возможности аутентификации и авторизации с другими механизмами, в частности HTTP-брандмауэром, цепочками фильтров, широким использованием стандартов и вариантов обмена информацией IETF и W3C, а также многим другим, что

помогает защитить приложение от взлома. Spring Security оценивает входные параметры и доступные зависимости с помощью автоконфигурации Spring Boot и обеспечивает максимальную безопасность приложений Spring Boot с минимальными усилиями.

В этой главе вы познакомились с основными аспектами безопасности и их применением в приложениях. Я продемонстрировал несколько способов включения Spring Security в структуру приложений Spring Boot для укрепления позиций приложения в смысле безопасности, ликвидации опасных пробелов в ее охвате и сокращения поверхности атаки.

В следующей главе мы обсудим способы развертывания приложений Spring Boot в различных целевых средах, а также сравним их достоинства. Кроме того, я продемонстрирую, как создавать артефакты развертывания, рассмотрю варианты их оптимального выполнения и покажу, как проверить их компоненты и происхождение.

ГЛАВА 11

Развертывание приложений Spring Boot

В разработке ПО развертывание — это переход к продакшену приложения.

Все обещанные конечным пользователям возможности приложения остаются пустым звуком, пока пользователи не смогут это приложение применить. Фигурально, а часто и вполне реально развертывание окупает приложение.

Благодаря Spring Initializr многие разработчики знают, что приложения Spring Boot можно создавать в виде файлов WAR или JAR. Большинство этих разработчиков также знают, что есть немало причин (часть из них упоминалась ранее в этой книге) отказаться от варианта WAR и создавать исполняемые файлы JAR и совсем немного причин поступать наоборот. Но в чем многие разработчики не отдают себе отчета, так это в том, что существует множество вариантов создания исполняемого JAR-файла Spring Boot для удовлетворения разнообразных требований и сценариев использования.

В этой главе мы изучим способы развертывания приложений Spring Boot, а также параметры, подходящие для различных целевых сред, и обсудим их относительные достоинства. А затем я продемонстрирую, как создавать артефакты развертывания, рассмотрю варианты их оптимального выполнения и покажу, как проверить их компоненты и происхождение. Вам наверняка доступно гораздо больше инструментов развертывания приложений Spring Boot, чем вы думаете.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Пожалуйста, для начала извлеките из репозитория кода ветку chapter11begin.

Возвращаемся к исполняемым JAR-файлам Spring Boot

Как обсуждалось в главе 1, исполняемые JAR-файлы Spring Boot обеспечивают максимум пользы и универсальности в одном самодостаточном, тестируемом и развертываемом компоненте. Они быстро создаются, сами динамически приспосабливаются к изменениям среды и исключительно просты в распространении и поддержке.

У всех поставщиков облачных сервисов есть варианты хостинга приложений, подходящие для создания прототипов посредством развертывания в реальной среде. Большинство этих платформ приложений ориентированы на более или менее самодостаточные развертываемые приложения и предлагают лишь самые необходимые элементы среды. JAR-файлы Spring Boot прекрасно подходят для подобной «чистой» среды, требуя для беспроблемного запуска лишь наличия JDK. Некоторые платформы даже явно упоминают Spring Boot, так хорошо он подходит для хостинга приложений. Приложения Spring Boot благодаря наличию механизмов для внешних взаимодействий по HTTP, обмена сообщениями и прочего позволяют отказаться от инсталляции, настройки и сопровождения сервера приложений и тому подобных внешних средств. Это резко сокращает нагрузку на разработчика и дополнительные расходы ресурсов платформы приложения.

Благодаря полному контролю приложения Spring Boot над библиотеками, от которых оно зависит, можно не бояться, что какие-либо из них изменятся. За прошедшие годы произошло бесчисленное множество сбоев не-Boot-приложений из-за запланированных обновлений сервера приложений, механизма сервлетов, базы данных, библиотек обмена сообщениями и прочих критически важных компонентов. В этих приложениях, полагающихся на контролируемые платформой приложения внешние компоненты, разработчикам приходится постоянно сохранять бдительность, отслеживая незапланированные простои, когда почва буквально уходит из-под ног лишь из-за выпуска младшей версии одной-единственной библиотеки-зависимости. Просто замечательно.

В случае же приложений Spring Boot обновление любых зависимостей — как библиотек ядра Spring, так и зависимостей второго (третьего, четвертого и т. д.) уровня — проходит намного проще и безболезненнее. Разработчик приложения обновляет и тестирует приложение, а развертывает обновление (обычно посредством *сине-зеленого развертывания* (https://en.wikipedia.org/wiki/Blue-green_deployment)), только убедившись, что все работает как должно. А поскольку зависимости уже не являются внешними по отношению к приложению, а упакованы вместе с ним, разработчик полностью контролирует версии зависимостей и расписание обновлений.

JAR-файлы Spring Boot благодаря плагинам Spring Boot Maven и Gradle обладают еще одним полезным свойством — возможностью создания «полностью исполняемых» (fully executable) JAR-файлов. Кавычки не случайны, в официальной документации написание этого термина такое же, поскольку для функционирования приложения все равно нужен JDK. Так что же понимается под полностью исполняемым приложением Spring Boot и как его создать?

Начнем со второй части вопроса.

Сборка «полностью исполняемого» JAR-файла Spring Boot

Для этого примера я воспользуюсь приложением PlaneFinder. Ради удобства сравнения произведем сборку проекта в неизменном виде из командной строки с помощью команды `mvn clean package`. В результате в каталоге `target` проекта будет создан следующий JAR-файл (результаты обрезаны, чтобы помещались на странице):

```
" ls -lb target/*.jar
-rw-r--r--  1 markheckler  staff  27085204 target/planefinder-0.0.1-SNAPSHOT.jar
```

Такой JAR-файл Spring Boot называется *исполняемым* (executable), поскольку он включает все приложение и не требует внешних зависимостей — для его исполнения достаточно JVM, которая устанавливается вместе с JDK. Запуск приложения в его нынешнем состоянии выглядит примерно так (результаты обрезаны и отредактированы, чтобы помещались на странице):

```
" java -jar target/planefinder-0.0.1-SNAPSHOT.jar

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v2.4.0)

: Starting PlanefinderApplication v0.0.1-SNAPSHOT
: No active profile set, falling back to default profiles: default
: Bootstrapping Spring Data R2DBC repositories in DEFAULT mode.
: Finished Spring Data repository scanning in 132 ms. Found 1 R2DBC
  repository interfaces.
: Netty started on port(s): 7634
: Netty RSocket started on port(s): 7635
: Started PlanefinderApplication in 2.75 seconds (JVM running for 3.106)
```

Все работает как должно, конечно, это наша отправная точка для последующих действий. Теперь обратимся к файлу `pom.xml` приложения PlaneFinder и добавим указанный фрагмент XML к уже существующему разделу для `spring-boot-maven-plug-in` (рис. 11.1).

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <version>2.4.0</version>
            <configuration>
                <executable>true</executable>
            </configuration>
        </plugin>
    </plugins>
</build>
```

Рис. 11.1. Раздел plugins файла pom.xml приложения PlaneFinder

Возвращаясь в терминал, опять производим сборку проекта из командной строки с помощью команды `mvn clean package`. На этот раз созданный в каталоге `target` проекта JAR-файл существенно отличается от прежнего, как видно из выводимой информации (результаты обрезаны, чтобы помещались на странице):

```
" ls -lb target/*.jar

-rwxr--r--  1 markheckler  staff  27094314 target/planefinder-0.0.1-SNAPSHOT.jar
```

Лишь чуть-чуть больше стандартного исполняемого JAR-файла Boot, а именно на 9110 байт, то есть чуть меньше, чем на 9 Кбайт. Чем же он лучше?

JAR-файлы Java читаются от конца к началу — да, все правильно, — пока не будет найден маркер конца файла. При создании так называемого полностью исполняемого JAR-файла плагин Maven Spring Boot ловко добавляет в начало обычного исполняемого JAR-файла Spring Boot скрипт, позволяющий выполнять его точно так же, как любой другой исполняемый бинарный файл (при наличии JDK, конечно) в системе на основе Unix или Linux, включая регистрацию в `init.d` или `systemd`. Если взглянуть на JAR-файл приложения PlaneFinder в редакторе, увидим следующее (для краткости приводим только часть заголовка скрипта — он довольно большой):

```
#!/bin/bash
#
#    .   ____          _            __ _ _
#   /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
#  ( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
#   \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
#    '  |____| .__|_| |_|_| |_\__, | / / / /
#   =========|_|==============|___/=/_/_/_/
#   :: Spring Boot Startup Script ::
#
### BEGIN INIT INFO
# Provides:          planefinder
# Required-Start:    $remote_fs $syslog $network
# Required-Stop:     $remote_fs $syslog $network
# Default-Start:     2 3 4 5
# Default-Stop:      0 1 6
# Short-Description: planefinder
# Description:       Data feed for SBUR
# chkconfig:         2345 99 01
### END INIT INFO

...

# Action functions[1]
start() {
  if [[ -f "$pid_file" ]]; then
    pid=$(cat "$pid_file")
    isRunning "$pid" && { echoYellow "Already running [$pid]"; return 0; }
  fi
  do_start "$@"
}

do_start() {
  working_dir=$(dirname "$jarfile")
  pushd "$working_dir" > /dev/null
  if [[ ! -e "$PID_FOLDER" ]]; then
    mkdir -p "$PID_FOLDER" &> /dev/null
    if [[ -n "$run_user" ]]; then
      chown "$run_user" "$PID_FOLDER"
    fi
  fi
  if [[ ! -e "$log_file" ]]; then
    touch "$log_file" &> /dev/null
    if [[ -n "$run_user" ]]; then
      chown "$run_user" "$log_file"
    fi
  fi
  if [[ -n "$run_user" ]]; then
```

[1] Функции действий. — *Примеч. пер.*

```
    checkPermissions || return $?
    if [ $USE_START_STOP_DAEMON = true ] && type start-stop-daemon >
        /dev/null 2>&1; then
      start-stop-daemon --start --quiet \
        --chuid "$run_user" \
        --name "$identity" \
        --make-pidfile --pidfile "$pid_file" \
        --background --no-close \
        --startas "$javaexe" \
        --chdir "$working_dir" \
       –"${arguments[@]}" \
        >> "$log_file" 2>&1
      await_file "$pid_file"
    else
      su -s /bin/sh -c "$javaexe $(printf "\"%s\" " "${arguments[@]}") >>
        \"$log_file\" 2>&1 & echo \$!" "$run_user" > "$pid_file"
    fi
    pid=$(cat "$pid_file")
  else
    checkPermissions || return $?
    "$javaexe" "${arguments[@]}" >> "$log_file" 2>&1 &
    pid=$!
    disown $pid
    echo "$pid" > "$pid_file"
  fi
  [[ -z $pid ]] && { echoRed "Failed to start"; return 1; }
  echoGreen "Started [$pid]"
}

stop() {
  working_dir=$(dirname "$jarfile")
  pushd "$working_dir" > /dev/null
  [[ -f $pid_file ]] ||
    { echoYellow "Not running (pidfile not found)"; return 0; }
  pid=$(cat "$pid_file")
  isRunning "$pid" || { echoYellow "Not running (process ${pid}).
    Removing stale pid file."; rm -f "$pid_file"; return 0; }
  do_stop "$pid" "$pid_file"
}

do_stop() {
  kill "$1" &> /dev/null || { echoRed "Unable to kill process $1"; return 1; }
  for ((i = 1; i <= STOP_WAIT_TIME; i++)); do
    isRunning "$1" || { echoGreen "Stopped [$1]"; rm -f "$2"; return 0; }
    [[ $i -eq STOP_WAIT_TIME/2 ]] && kill "$1" &> /dev/null
    sleep 1
  done
  echoRed "Unable to kill process $1";
  return 1;
}
```

```
force_stop() {
  [[ -f $pid_file ]] ||
    { echoYellow "Not running (pidfile not found)"; return 0; }
  pid=$(cat "$pid_file")
  isRunning "$pid" ||
    { echoYellow "Not running (process ${pid}). Removing stale pid file.";
    rm -f "$pid_file"; return 0; }
  do_force_stop "$pid" "$pid_file"
}

do_force_stop() {
  kill -9 "$1" &> /dev/null ||
      { echoRed "Unable to kill process $1"; return 1; }
  for ((i = 1; i <= STOP_WAIT_TIME; i++)); do
    isRunning "$1" || { echoGreen "Stopped [$1]"; rm -f "$2"; return 0; }
    [[ $i -eq STOP_WAIT_TIME/2 ]] && kill -9 "$1" &> /dev/null
    sleep 1
  done
  echoRed "Unable to kill process $1";
  return 1;
}
restart() {
  stop && start
}

force_reload() {
  working_dir=$(dirname "$jarfile")
  pushd "$working_dir" > /dev/null
  [[ -f $pid_file ]] || { echoRed "Not running (pidfile not found)";
      return 7; }
  pid=$(cat "$pid_file")
  rm -f "$pid_file"
  isRunning "$pid" || { echoRed "Not running (process ${pid} not found)";
      return 7; }
  do_stop "$pid" "$pid_file"
  do_start
}

status() {
  working_dir=$(dirname "$jarfile")
  pushd "$working_dir" > /dev/null
  [[ -f "$pid_file" ]] || { echoRed "Not running"; return 3; }
  pid=$(cat "$pid_file")
  isRunning "$pid" || { echoRed "Not running (process ${pid} not found)";
      return 1; }
  echoGreen "Running [$pid]"
  return 0
}

run() {
```

```
  pushd "$(dirname "$jarfile")" > /dev/null
  "$javaexe" "${arguments[@]}"
  result=$?
  popd > /dev/null
  return "$result"
}

# Call the appropriate action function[1]
case "$action" in
start)
  start "$@"; exit $?;;
stop)
  stop "$@"; exit $?;;
force-stop)
  force_stop "$@"; exit $?;;
restart)
  restart "$@"; exit $?;;
force-reload)
  force_reload "$@"; exit $?;;
status)
  status "$@"; exit $?;;
run)
  run "$@"; exit $?;;
*)
  echo "Usage: $0 {start|stop|force-stop|restart|force-reload|status|run}";
    exit 1;
esac
exit 0
<binary portion omitted>
```

Плагин Maven (или Gradle, если выбрать его в качестве системы сборки) Spring Boot также задает для выходного JAR-файла права доступа на чтение, запись и выполнение (`rwx`), благодаря чему его можно выполнять так, как было указано ранее, а скрипт из заголовка может найти JDK, подготовить приложение к выполнению и запустить его вот так (результаты обрезаны и отредактированы, чтобы помещались на странице):

```
" target/planefinder-0.0.1-SNAPSHOT.jar

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v2.4.0)
```

[1] Вызов соответствующей функции действия. — *Примеч. пер.*

```
: Starting PlanefinderApplication v0.0.1-SNAPSHOT
: No active profile set, falling back to default profiles: default
: Bootstrapping Spring Data R2DBC repositories in DEFAULT mode.
: Finished Spring Data repository scanning in 185 ms.
  Found 1 R2DBC repository interfaces.
: Netty started on port(s): 7634
: Netty RSocket started on port(s): 7635
: Started PlanefinderApplication in 2.938 seconds (JVM running for 3.335)
```

Я показал, как все происходит, пришло время разобраться, что эта возможность нам дает.

Что это нам дает

Возможность создания «полностью исполняемых» JAR-файлов не панацея от всех проблем, но уникальная возможность более глубокой интеграции с базовыми Unix- и Linux-системами при необходимости. Запуск приложения Spring Boot в виде сервиса становится тривиальной задачей благодаря встраиваемому сценарию запуска и правам на выполнение.

Если в нынешней среде выполнения вашего приложения эта возможность не нужна или не может быть применена, можете создавать обычные исполняемые JAR-файлы для выполнения с помощью команды `java -jar`. Это просто дополнительный инструмент в вашем наборе инструментов, не влекущий никаких дополнительных накладных расходов, применение которого, если понадобится, не требует практически никаких усилий.

Разобранные JAR-файлы

Новаторский подход Spring Boot с вложением зависимых JAR-файлов в неизменном виде в исполняемый JAR-файл Boot идеально подходит для таких дальнейших операций, как их извлечение. Обратный процесс добавления их в исполняемый JAR-файл Spring Boot позволяет получить компоненты-артефакты в исходном, неизменном виде. Звучит просто, потому что это *действительно* просто.

Существует немало потенциальных причин для разделения JAR-файлов на составные части.

- Распакованные приложения Boot выполняются чуть быстрее. Это редко оправдывает разбиение само по себе, но как дополнительный бонус — неплохо.

- Распакованные зависимости представляют собой легко заменяемые отдельные модули. Это позволяет обновлять приложения быстрее и/или при более низкой пропускной способности сети, поскольку можно развертывать только изменившиеся файлы.
- Во многих облачных платформах, например Heroku и всех сборках или версиях/вариантах Cloud Foundry, распаковка является частью процесса развертывания приложения. Зеркалирование локальной и удаленной среды в максимально возможной степени повышает согласованность и при необходимости помогает выяснять причины проблем.

Как стандартные исполняемые JAR-файлы Spring Boot, так и «полностью исполняемые» JAR-файлы можно распаковать следующим образом с помощью команды `jar xvf <spring_boot_jar>` (большая часть элементов списка файлов удалена для краткости):

```
" mkdir expanded
" cd expanded
" jar -xvf ../target/planefinder-0.0.1-SNAPSHOT.jar
  created: META-INF/
 inflated: META-INF/MANIFEST.MF
  created: org/
  created: org/springframework/
  created: org/springframework/boot/
  created: org/springframework/boot/loader/
  created: org/springframework/boot/loader/archive/
  created: org/springframework/boot/loader/data/
  created: org/springframework/boot/loader/jar/
  created: org/springframework/boot/loader/jarmode/
  created: org/springframework/boot/loader/util/
  created: BOOT-INF/
  created: BOOT-INF/classes/
  created: BOOT-INF/classes/com/
  created: BOOT-INF/classes/com/thehecklers/
  created: BOOT-INF/classes/com/thehecklers/planefinder/
  created: META-INF/maven/
  created: META-INF/maven/com.thehecklers/
  created: META-INF/maven/com.thehecklers/planefinder/
 inflated: BOOT-INF/classes/schema.sql
 inflated: BOOT-INF/classes/application.properties
 inflated: META-INF/maven/com.thehecklers/planefinder/pom.xml
 inflated: META-INF/maven/com.thehecklers/planefinder/pom.properties
  created: BOOT-INF/lib/
 inflated: BOOT-INF/classpath.idx
 inflated: BOOT-INF/layers.idx
"
```

После распаковки файлов полезно представить структуру более наглядным образом с помощью команды `tree` *nix-систем:

```
" tree
.
├── BOOT-INF
│   ├── classes
│   │   ├── application.properties
│   │   ├── com
│   │   │   └── thehecklers
│   │   │       └── planefinder
│   │   │           ├── Aircraft.class
│   │   │           ├── DbConxInit.class
│   │   │           ├── PlaneController.class
│   │   │           ├── PlaneFinderService.class
│   │   │           ├── PlaneRepository.class
│   │   │           ├── PlanefinderApplication.class
│   │   │           └── User.class
│   │   └── schema.sql
│   ├── classpath.idx
│   ├── layers.idx
│   └── lib
│       ├── h2-1.4.200.jar
│       ├── jackson-annotations-2.11.3.jar
│       ├── jackson-core-2.11.3.jar
│       ├── jackson-databind-2.11.3.jar
│       ├── jackson-dataformat-cbor-2.11.3.jar
│       ├── jackson-datatype-jdk8-2.11.3.jar
│       ├── jackson-datatype-jsr310-2.11.3.jar
│       ├── jackson-module-parameter-names-2.11.3.jar
│       ├── jakarta.annotation-api-1.3.5.jar
│       ├── jul-to-slf4j-1.7.30.jar
│       ├── log4j-api-2.13.3.jar
│       ├── log4j-to-slf4j-2.13.3.jar
│       ├── logback-classic-1.2.3.jar
│       ├── logback-core-1.2.3.jar
│       ├── lombok-1.18.16.jar
│       ├── netty-buffer-4.1.54.Final.jar
│       ├── netty-codec-4.1.54.Final.jar
│       ├── netty-codec-dns-4.1.54.Final.jar
│       ├── netty-codec-http-4.1.54.Final.jar
│       ├── netty-codec-http2-4.1.54.Final.jar
│       ├── netty-codec-socks-4.1.54.Final.jar
│       ├── netty-common-4.1.54.Final.jar
│       ├── netty-handler-4.1.54.Final.jar
│       ├── netty-handler-proxy-4.1.54.Final.jar
│       ├── netty-resolver-4.1.54.Final.jar
│       ├── netty-resolver-dns-4.1.54.Final.jar
```

```
│       ├── netty-transport-4.1.54.Final.jar
│       ├── netty-transport-native-epoll-4.1.54.Final-linux-x86_64.jar
│       ├── netty-transport-native-unix-common-4.1.54.Final.jar
│       ├── r2dbc-h2-0.8.4.RELEASE.jar
│       ├── r2dbc-pool-0.8.5.RELEASE.jar
│       ├── r2dbc-spi-0.8.3.RELEASE.jar
│       ├── reactive-streams-1.0.3.jar
│       ├── reactor-core-3.4.0.jar
│       ├── reactor-netty-core-1.0.1.jar
│       ├── reactor-netty-http-1.0.1.jar
│       ├── reactor-pool-0.2.0.jar
│       ├── rsocket-core-1.1.0.jar
│       ├── rsocket-transport-netty-1.1.0.jar
│       ├── slf4j-api-1.7.30.jar
│       ├── snakeyaml-1.27.jar
│       ├── spring-aop-5.3.1.jar
│       ├── spring-beans-5.3.1.jar
│       ├── spring-boot-2.4.0.jar
│       ├── spring-boot-autoconfigure-2.4.0.jar
│       ├── spring-boot-jarmode-layertools-2.4.0.jar
│       ├── spring-context-5.3.1.jar
│       ├── spring-core-5.3.1.jar
│       ├── spring-data-commons-2.4.1.jar
│       ├── spring-data-r2dbc-1.2.1.jar
│       ├── spring-data-relational-2.1.1.jar
│       ├── spring-expression-5.3.1.jar
│       ├── spring-jcl-5.3.1.jar
│       ├── spring-messaging-5.3.1.jar
│       ├── spring-r2dbc-5.3.1.jar
│       ├── spring-tx-5.3.1.jar
│       ├── spring-web-5.3.1.jar
│       └── spring-webflux-5.3.1.jar
├── META-INF
│   ├── MANIFEST.MF
│   └── maven
│       └── com.thehecklers
│           └── planefinder
│               ├── pom.properties
│               └── pom.xml
└── org
    └── springframework
        └── boot
            └── loader
                ├── ClassPathIndexFile.class
                ├── ExecutableArchiveLauncher.class
                ├── JarLauncher.class
                ├── LaunchedURLClassLoader$DefinePackageCallType.class
                ├── LaunchedURLClassLoader
                │   $UseFastConnectionExceptionsEnumeration.class
                ├── LaunchedURLClassLoader.class
```

```
├── Launcher.class
├── MainMethodRunner.class
├── PropertiesLauncher$1.class
├── PropertiesLauncher$ArchiveEntryFilter.class
├── PropertiesLauncher$ClassPathArchives.class
├── PropertiesLauncher$PrefixMatchingArchiveFilter.class
├── PropertiesLauncher.class
├── WarLauncher.class
├── archive
│   ├── Archive$Entry.class
│   ├── Archive$EntryFilter.class
│   ├── Archive.class
│   ├── ExplodedArchive$AbstractIterator.class
│   ├── ExplodedArchive$ArchiveIterator.class
│   ├── ExplodedArchive$EntryIterator.class
│   ├── ExplodedArchive$FileEntry.class
│   ├── ExplodedArchive$SimpleJarFileArchive.class
│   ├── ExplodedArchive.class
│   ├── JarFileArchive$AbstractIterator.class
│   ├── JarFileArchive$EntryIterator.class
│   ├── JarFileArchive$JarFileEntry.class
│   ├── JarFileArchive$NestedArchiveIterator.class
│   └── JarFileArchive.class
├── data
│   ├── RandomAccessData.class
│   ├── RandomAccessDataFile$1.class
│   ├── RandomAccessDataFile$DataInputStream.class
│   ├── RandomAccessDataFile$FileAccess.class
│   └── RandomAccessDataFile.class
├── jar
│   ├── AbstractJarFile$JarFileType.class
│   ├── AbstractJarFile.class
│   ├── AsciiBytes.class
│   ├── Bytes.class
│   ├── CentralDirectoryEndRecord$1.class
│   ├── CentralDirectoryEndRecord$Zip64End.class
│   ├── CentralDirectoryEndRecord$Zip64Locator.class
│   ├── CentralDirectoryEndRecord.class
│   ├── CentralDirectoryFileHeader.class
│   ├── CentralDirectoryParser.class
│   ├── CentralDirectoryVisitor.class
│   ├── FileHeader.class
│   ├── Handler.class
│   ├── JarEntry.class
│   ├── JarEntryCertification.class
│   ├── JarEntryFilter.class
│   ├── JarFile$1.class
│   ├── JarFile$JarEntryEnumeration.class
│   ├── JarFile.class
│   ├── JarFileEntries$1.class
```

```
│   ├── JarFileEntries$EntryIterator.class
│   ├── JarFileEntries.class
│   ├── JarFileWrapper.class
│   ├── JarURLConnection$1.class
│   ├── JarURLConnection$JarEntryName.class
│   ├── JarURLConnection.class
│   ├── StringSequence.class
│   └── ZipInflaterInputStream.class
├── jarmode
│   ├── JarMode.class
│   ├── JarModeLauncher.class
│   └── TestJarMode.class
└── util
    └── SystemPropertyUtils.class

19 directories, 137 files
"
```

Команда `tree` позволяет просмотреть, из чего состоит приложение, в удобном иерархическом виде. А также выявляет многочисленные зависимости, которые в совокупности обеспечивают выбранные для приложения элементы функциональности. Список файлов в каталоге `BOOT-INF/lib` подтверждает, что библиотеки компонентов остаются неизменными в процессе сборки JAR-файла Spring Boot и последующего извлечения его содержимого, вплоть до меток даты/времени исходных компонентов JAR-файлов, как видно из следующего кода (большинство записей удалено ради экономии места):

```
" ls -l BOOT-INF/lib
total 52880
-rw-r--r--  1 markheckler  staff  2303679 Oct 14  2019 h2-1.4.200.jar
-rw-r--r--  1 markheckler  staff    68215 Oct  1 22:20 jackson-annotations-
 2.11.3.jar
-rw-r--r--  1 markheckler  staff   351495 Oct  1 22:25 jackson-core-
 2.11.3.jar
-rw-r--r--  1 markheckler  staff  1421699 Oct  1 22:38 jackson-databind-
 2.11.3.jar
-rw-r--r--  1 markheckler  staff    58679 Oct  2 00:17 jackson-dataformat-cbor-
 2.11.3.jar
-rw-r--r--  1 markheckler  staff    34335 Oct  2 00:25 jackson-datatype-jdk8-
 2.11.3.jar
-rw-r--r--  1 markheckler  staff   111008 Oct  2 00:25 jackson-datatype-jsr310-
 2.11.3.jar
-rw-r--r--  1 markheckler  staff     9267 Oct  2 00:25 jackson-module-parameter-
 names-2.11.3.jar
 ...
-rw-r--r--  1 markheckler  staff   374303 Nov 10 09:01 spring-aop-5.3.1.jar
-rw-r--r--  1 markheckler  staff   695851 Nov 10 09:01 spring-beans-5.3.1.jar
```

```
-rw-r--r--  1 markheckler  staff  1299025 Nov 12 13:56 spring-boot-2.4.0.jar
-rw-r--r--  1 markheckler  staff  1537971 Nov 12 13:55 spring-boot-
 autoconfigure-2.4.0.jar
-rw-r--r--  1 markheckler  staff    32912 Feb  1  1980 spring-boot-jarmode-
 layertools-2.4.0.jar
-rw-r--r--  1 markheckler  staff  1241939 Nov 10 09:01 spring-context-5.3.1.jar
-rw-r--r--  1 markheckler  staff  1464734 Feb  1  1980 spring-core-5.3.1.jar
-rw-r--r--  1 markheckler  staff  1238966 Nov 11 12:03 spring-data-commons-
 2.4.1.jar
-rw-r--r--  1 markheckler  staff   433079 Nov 11 12:08 spring-data-r2dbc-
 1.2.1.jar
-rw-r--r--  1 markheckler  staff   339745 Nov 11 12:05 spring-data-relational-
 2.1.1.jar
-rw-r--r--  1 markheckler  staff   282565 Nov 10 09:01 spring-expression-
 5.3.1.jar
-rw-r--r--  1 markheckler  staff    23943 Nov 10 09:01 spring-jcl-5.3.1.jar
-rw-r--r--  1 markheckler  staff   552895 Nov 10 09:01 spring-messaging-
 5.3.1.jar
-rw-r--r--  1 markheckler  staff   133156 Nov 10 09:01 spring-r2dbc-5.3.1.jar
-rw-r--r--  1 markheckler  staff   327956 Nov 10 09:01 spring-tx-5.3.1.jar
-rw-r--r--  1 markheckler  staff  1546053 Nov 10 09:01 spring-web-5.3.1.jar
-rw-r--r--  1 markheckler  staff   901591 Nov 10 09:01 spring-webflux-5.3.1.jar
"
```

После извлечения всех файлов из JAR-файла Spring Boot запустить приложение можно несколькими способами. Рекомендуемый подход — воспользоваться `JarLauncher`, сохраняющим одинаковый порядок загрузки классов от запуска к запуску, как показано далее (результаты обрезаны и отредактированы, чтобы помещались на странице):

```
" java org.springframework.boot.loader.JarLauncher

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v2.4.0)

: Starting PlanefinderApplication v0.0.1-SNAPSHOT
: No active profile set, falling back to default profiles: default
: Bootstrapping Spring Data R2DBC repositories in DEFAULT mode.
: Finished Spring Data repository scanning in 95 ms. Found 1 R2DBC
  repository interfaces.
: Netty started on port(s): 7634
: Netty RSocket started on port(s): 7635
: Started PlanefinderApplication in 1.935 seconds (JVM running for 2.213)
```

В данном случае приложение PlaneFinder в разобранном виде запускается на секунду с лишним быстрее, чем «полностью исполняемый» JAR-файл Spring Boot. Этот положительный момент, возможно, перевесит преимущества единого, полностью самодостаточного развертываемого модуля, а может, и нет, и скорее нет, чем да. Но в сочетании с возможностью отправлять обновления приложения, состоящие всего из нескольких изменившихся файлов и в определенных случаях лучше соответствующие локальной и удаленной средам, возможность выполнения разобранных приложений Spring Boot может оказаться весьма полезной.

Развертывание приложений Spring Boot в контейнерах

Как уже упоминалось, некоторые облачные платформы — как локальные/частные, так и общедоступные облачные сервисы — поддерживают создание для развертываемых приложений образа контейнера вместо разработчика на основе хорошо оптимизированных параметров как по умолчанию, так и указанных разработчиком приложения.

Далее на основе этих образов, настроек репликации и способа использования приложения создаются и уничтожаются контейнеры с работающими приложениями. Такие платформы, как Heroku и разнообразные версии Cloud Foundry, позволяют разработчику отправлять исполняемый JAR-файл Spring Boot, указывая все необходимые параметры конфигурации или просто принимая параметры по умолчанию, в то время как все остальное берет на себя платформа. Другие платформы, например Tanzu Application Service для Kubernetes от VMware, тоже способны на это, и список возможностей, обеспечивающих как широту охвата, так и гибкость выполнения, постоянно растет.

Существует множество платформ и целевых сред развертывания, не поддерживающих предоставление разработчику подобных возможностей. Используете ли вы либо ваша компания подобные варианты или другие требования ведут вас в ином направлении, Spring Boot обо всем позаботится.

Можно создавать вручную собственные образы контейнеров для своих приложений Spring Boot, но это не оптимальное решение. Никакой пользы приложению это не несет и обычно считается в лучшем случае неизбежным злом, необходимым для перехода от разработки к продакшену. Не более того.

Spring Boot, используя многие из инструментов, применяемых в вышеупомянутых платформах для продуманной контейнеризации приложений,

включает в свои плагины Maven и Gradle возможность просто и удобно собирать полностью совместимые с Open Container Initiative (OCI) образы, работающие с Docker, Kubernetes и всеми прочими движками/механизмами для использования контейнеров. Встроенные плагины Spring Boot, основанные на ведущих проектах для работы с пакетами компоновки Cloud Native Buildpacks (https://buildpacks.io) и Paketo (https://paketo.io), позволяют с помощью установленного и запущенного на локальной машине демона Docker создать образ OCI и в дальнейшем поместить его в локальный или удаленный репозиторий образов.

Создание образа для приложения с помощью плагина Spring Boot наилучшим образом поддерживается его хорошо продуманной средой выполнения (настройками по умолчанию). Для оптимизации этого процесса путем послойного размещения его содержимого с разделением кода/библиотек в зависимости от предполагаемой частоты изменений каждого блока кода используется своеобразная «автоконфигурация». Верный своей философии относительно автоконфигурации и мнений, Spring Boot позволяет переопределять и направлять этот процесс иерархического размещения при необходимости настройки конфигурации под свои нужды. Такая необходимость возникает редко, но если требования относятся к одному из подобных исключительных случаев, реализовать ее несложно.

При настройках по умолчанию во всех версиях Spring Boot, начиная с 2.4.0 Milestone 2, создаются следующие слои:

- `dependencies` — включает регулярно выпускаемые зависимости, то есть общедоступные версии;
- `spring-boot-loader` — включает все файлы из `org/springframework/boot/loader`;
- `snapshot-dependencies` — включает перспективные выпуски, пока еще не относимые к общедоступным;
- `Application` — включает классы приложения и соответствующие ресурсы (шаблоны, файлы свойств, скрипты и т. д.).

Нестабильность кода, то есть его склонность к изменениям и их частота, обычно возрастает по мере продвижения по списку слоев сверху вниз. Использование отдельных слоев для кода одинаковой степени нестабильности существенно повышает и ускоряет последующее создание образа. А это *резко* сокращает время и ресурсы, затрачиваемые на повторные сборки развертываемого артефакта за весь срок жизни приложения.

Создание образа контейнера из IDE

Очень легко создать многослойный образ контейнера на основе приложения Spring Boot из IDE. Для этого примера я воспользуюсь IntelliJ, но подобные возможности есть практически во всех основных IDE.



Для создания образов должна быть запущена локальная версия Docker — в моем случае Docker Desktop для Mac.

Для создания образа я открыл панель Maven, развернув вкладку Maven в правом поле IntelliJ, затем развернул Plugins, выбрал и развернул плагин `spring-boot`, после чего дважды щелкнул на опции spring-boot:build-image для выполнения цели сборки (рис. 11.2).

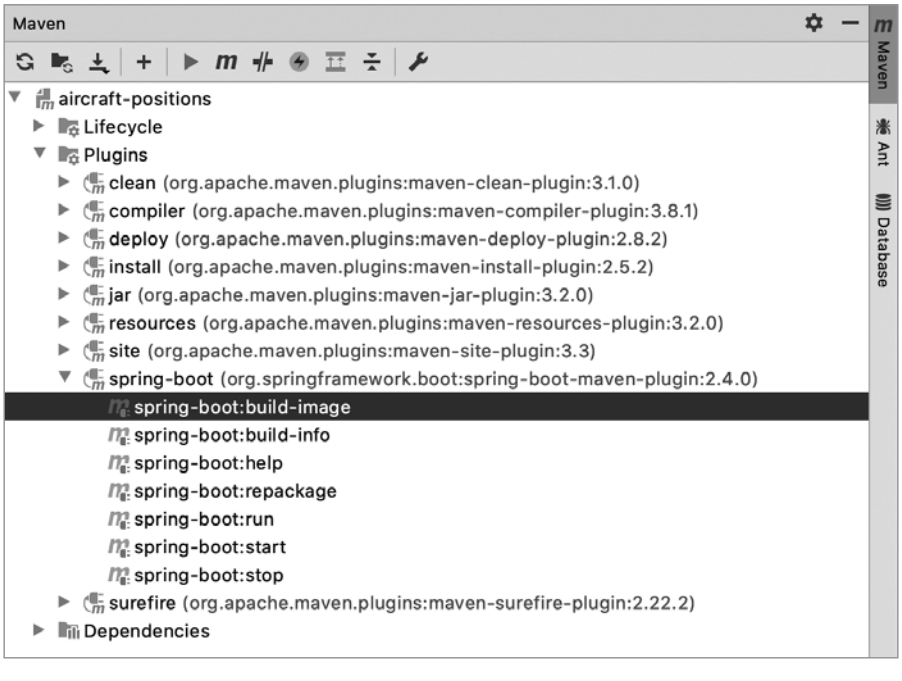


Рис. 11.2. Сборка образа контейнера приложения Spring Boot из панели Maven IntelliJ

При создании образа генерируется довольно длинный журнал действий. Особенно интересные записи приведены далее:

```
[INFO]     [creator]     Paketo Executable JAR Buildpack 3.1.3
[INFO]     [creator]       https://github.com/paketo-buildpacks/executable-jar
[INFO]     [creator]         Writing env.launch/CLASSPATH.delim
[INFO]     [creator]         Writing env.launch/CLASSPATH.prepend
[INFO]     [creator]       Process types:
[INFO]     [creator]         executable-jar: java org.springframework.boot.
    loader.JarLauncher
[INFO]     [creator]         task:           java org.springframework.boot.
    loader.JarLauncher
[INFO]     [creator]         web:            java org.springframework.boot.
    loader.JarLauncher
[INFO]     [creator]
[INFO]     [creator]     Paketo Spring Boot Buildpack 3.5.0
[INFO]     [creator]       https://github.com/paketo-buildpacks/spring-boot
[INFO]     [creator]       Creating slices from layers index
[INFO]     [creator]         dependencies
[INFO]     [creator]         spring-boot-loader
[INFO]     [creator]         snapshot-dependencies
[INFO]     [creator]         application
[INFO]     [creator]       Launch Helper: Contributing to layer
[INFO]     [creator]         Creating /layers/paketo-buildpacks_spring-boot/
    helper/exec.d/spring-cloud-bindings
[INFO]     [creator]         Writing profile.d/helper
[INFO]     [creator]       Web Application Type: Contributing to layer
[INFO]     [creator]         Reactive web application detected
[INFO]     [creator]         Writing env.launch/BPL_JVM_THREAD_COUNT.default
[INFO]     [creator]       Spring Cloud Bindings 1.7.0: Contributing to layer
[INFO]     [creator]         Downloading from
    https://repo.spring.io/release/org/springframework/cloud/
    spring-cloud-bindings/1.7.0/spring-cloud-bindings-1.7.0.jar
[INFO]     [creator]         Verifying checksum
[INFO]     [creator]         Copying to
    /layers/paketo-buildpacks_spring-boot/spring-cloud-bindings
[INFO]     [creator]       4 application slices
```

Как уже упоминалось, слои образа, именуемые *срезами* (slices) в предыдущем листинге, и их содержимое можно при необходимости (в редких случаях) модифицировать.

После создания образа журнал завершает примерно следующая информация:

```
[INFO] Successfully built image 'docker.io/library/aircraft-positions:
       0.0.1-SNAPSHOT'
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  25.851 s
[INFO] Finished at: 2020-11-28T20:09:48-06:00
[INFO] ------------------------------------------------------------------------
```

Создание образа контейнера из командной строки

Конечно, можно — и довольно просто — создать тот же образ контейнера из командной строки. Перед этим я хотел бы немного поменять наименования в итоговом образе.

Для удобства я предпочитаю создавать образы, соответствующие соглашениям о наименованиях и вообще моей учетной записи Docker Hub (https://hub.docker.com). Вероятно, у выбранного вами репозитория образов также будут свои соглашения. Плагины сборки Spring Boot позволяют указывать в разделе `<configuration>` параметры, упрощающие отправку образа в репозиторий или каталог. Я добавил одну строку с соответствующими тегами в раздел `<plugins>` файла `pom.xml` приложения Aircraft Positions, чтобы удовлетворить свои требования либо предпочтения:

```
<build>
  <plug-ins>
    <plug-in>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plug-in</artifactId>
      <configuration>
        <image>
          <name>hecklerm/${project.artifactId}</name>
        </image>
      </configuration>
    </plug-in>
  </plug-ins>
</build>
```

Далее я выполнил следующую команду из каталога проекта в окне терминала для пересоздания образа контейнера приложения и вскоре после этого получил приведенные результаты:

```
" mvn spring-boot:build-image

... (Intermediate logged results omitted for brevity)

[INFO] Successfully built image 'docker.io/hecklerm/aircraft-positions:latest'
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  13.257 s
[INFO] Finished at: 2020-11-28T20:23:40-06:00
[INFO] ------------------------------------------------------------------------
```

Обратите внимание, что получившийся образ больше не называется `docker.io/library/aircraft-positions:0.0.1-SNAPSHOT`, как было, когда я собирал его при

настройках по умолчанию в IDE. Новые координаты образа соответствуют указанным мной в файле `pom.xml`: `docker.io/hecklerm/aircraft-positions:latest`.

Проверяем наличие образа

Чтобы проверить, что созданные в предыдущих двух разделах образы были загружены в локальный репозиторий, я выполнил следующую команду из окна терминала, отфильтровав результаты по названию (и обрезав по ширине страницы), чтобы получить в итоге следующее:

```
" docker images | grep -in aircraft-positions
aircraft-positions           0.0.1-SNAPSHOT   a7ed39a3d52e    277MB
hecklerm/aircraft-positions  latest           924893a0f1a9    277MB
```

Отправить последний из показанных в предыдущем выводе образов (поскольку теперь он соответствует ожидаемым и желательным учетной записи и соглашениям о наименованиях) в Docker Hub можно следующим образом вот с такими результатами:

```
" docker push hecklerm/aircraft-positions
The push refers to repository [docker.io/hecklerm/aircraft-positions]
1dc94a70dbaa: Pushed
4672559507f8: Pushed
e3e9839150af: Pushed
5f70bf18a086: Layer already exists
a3abfb734aa5: Pushed
3c14fe2f1177: Pushed
4cc7b4eb8637: Pushed
fcc507beb4cc: Pushed
c2e9ddddd4ef: Pushed
108b6855c4a6: Pushed
ab39aa8fd003: Layer already exists
0b18b1f120f4: Layer already exists
cf6b3a71f979: Pushed
ec0381c8f321: Layer already exists
7b0fc1578394: Pushed
eb0f7cd0acf8: Pushed
1e5c1d306847: Mounted from paketobuildpacks/run
23c4345364c2: Mounted from paketobuildpacks/run
a1efa53a237c: Mounted from paketobuildpacks/run
fe6d8881187d: Mounted from paketobuildpacks/run
23135df75b44: Mounted from paketobuildpacks/run
b43408d5f11b: Mounted from paketobuildpacks/run
latest: digest:
  sha256:a7e5d536a7426d6244401787b153ebf43277fbadc9f43a789f6c4f0aff6d5011
    size: 5122
"
```

Посетив Docker Hub, убеждаемся в успешном публичном развертывании нашего образа (рис. 11.3).



Рис. 11.3. Образ контейнера приложения Spring Boot в Docker Hub

Развертывание в Docker Hub или любом другом репозитории образов контейнеров, доступном извне локальной машины, — последний этап перед более масштабным (и, надеемся, в продакшене) развертыванием контейнеризованного приложения Spring Boot.

Запуск контейнеризованного приложения

Для запуска приложения я использую команду `docker run`. Вероятно, в вашей компании есть какой-либо конвейер развертывания для перемещения приложений из образов контейнеров, извлекаемых из репозиториев, в запущенные контейнеризованные приложения. И, вероятно, он состоит из тех же этапов, только более автоматизированных и с меньшим количеством набираемых команд.

Поскольку у меня уже есть локальная копия образа, извлекать его из удаленного репозитория не нужно. В противном случае для извлечения удаленного образа и/или слоев для его локального воссоздания перед запуском контейнера на его основе процессу-демону понадобился бы удаленный доступ к репозиторию образов.

Для запуска контейнеризованного приложения Aircraft Positions я выполнил следующую команду с вот такими результатами (результаты обрезаны и отредактированы, чтобы помещались на странице):

```
" docker run --name myaircraftpositions -p8080:8080
  hecklerm/aircraft-positions:latest
Setting Active Processor Count to 6
WARNING: Container memory limit unset. Configuring JVM for 1G container.
Calculated JVM Memory Configuration: -XX:MaxDirectMemorySize=10M -Xmx636688K
  -XX:MaxMetaspaceSize=104687K -XX:ReservedCodeCacheSize=240M -Xss1M
  (Total Memory: 1G, Thread Count: 50, Loaded Class Count: 16069, Headroom: 0%)
Adding 138 container CA certificates to JVM truststore
```

```
Spring Cloud Bindings Enabled
Picked up JAVA_TOOL_OPTIONS:
-Djava.security.properties=/layers/paketo-buildpacks_bellsoft-liberica/
    java-security-properties/java-security.properties
  -agentpath:/layers/paketo-buildpacks_bellsoft-liberica/jvmkill/
    jvmkill-1.16.0-RELEASE.so=printHeapHistogram=1
  -XX:ActiveProcessorCount=6
  -XX:MaxDirectMemorySize=10M
  -Xmx636688K
  -XX:MaxMetaspaceSize=104687K
  -XX:ReservedCodeCacheSize=240M
  -Xss1M
  -Dorg.springframework.cloud.bindings.boot.enable=true

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v2.4.0)

: Starting AircraftPositionsApplication v0.0.1-SNAPSHOT
: Netty started on port(s): 8080
: Started AircraftPositionsApplication in 10.7 seconds (JVM running for 11.202)
```

Теперь самое время заглянуть внутрь образа, созданного плагином Spring Boot.

Утилиты для исследования образов контейнеров приложений Spring Boot

Существует множество утилит для работы с образами контейнеров, и немалая часть их функциональности выходит далеко за рамки темы данной книги. Я хотел бы вкратце упомянуть две из них, полезные при определенных обстоятельствах: `pack` и `dive`.

Pack

Чтобы просмотреть материалы для создания образа контейнера приложения Spring Boot с помощью Cloud Native (Paketo) Buildpacks и самих пакетов компоновки, можно воспользоваться утилитой `pack`. `pack` — специализированным CLI для сборки приложений с помощью Cloud Native Buildpacks, доступ к которому можно получить несколькими способами. На своем Mac я задействовал систему

управления пакетами `homebrew` для извлечения и установки его с помощью простой команды `brew install pack`.

Выполнение `pack` для созданного ранее образа приводит к следующим результатам:

```
" pack inspect-image hecklerm/aircraft-positions
Inspecting image: hecklerm/aircraft-positions

REMOTE:

Stack: io.buildpacks.stacks.bionic

Base Image:
  Reference: f5caea10feb38ae882a9447b521fd1ea1ee93384438395c7ace2d8cfaf808e3d
  Top Layer: sha256:1e5c1d306847275caa0d1d367382dfdcfd4d62b634b237f1d7a2e
             746372922cd

Run Images:
  index.docker.io/paketobuildpacks/run:base-cnb
  gcr.io/paketo-buildpacks/run:base-cnb

Buildpacks:
  ID                                     VERSION
  paketo-buildpacks/ca-certificates      1.0.1
  paketo-buildpacks/bellsoft-liberica    5.2.1
  paketo-buildpacks/executable-jar       3.1.3
  paketo-buildpacks/dist-zip             2.2.2
  paketo-buildpacks/spring-boot          3.5.0

Processes:
  TYPE           SHELL  COMMAND  ARGS
  web (default)  bash   java     org.springframework.boot.loader.JarLauncher
  executable-jar bash   java     org.springframework.boot.loader.JarLauncher
  task           bash   java     org.springframework.boot.loader.JarLauncher

LOCAL:

Stack: io.buildpacks.stacks.bionic

Base Image:
  Reference: f5caea10feb38ae882a9447b521fd1ea1ee93384438395c7ace2d8cfaf808e3d
  Top Layer: sha256:1e5c1d306847275caa0d1d367382dfdcfd4d62b634b237f1d7a2e
             746372922cd

Run Images:
  index.docker.io/paketobuildpacks/run:base-cnb
  gcr.io/paketo-buildpacks/run:base-cnb
```

```
Buildpacks:
  ID                                          VERSION
  paketo-buildpacks/ca-certificates           1.0.1
  paketo-buildpacks/bellsoft-liberica         5.2.1
  paketo-buildpacks/executable-jar            3.1.3
  paketo-buildpacks/dist-zip                  2.2.2
  paketo-buildpacks/spring-boot               3.5.0

Processes:
  TYPE            SHELL  COMMAND  ARGS
  web (default)   bash   java     org.springframework.boot.loader.JarLauncher
  executable-jar  bash   java     org.springframework.boot.loader.JarLauncher
  task            bash   java     org.springframework.boot.loader.JarLauncher
```

Команда `inspect-image` утилиты `pack` позволяет получить ключевую информацию об образе, в частности следующую:

- какой базовый образ Docker или версия Linux (bionic) использовались в качестве основы данного образа;
- какие пакеты компоновки применялись для наполнения данного образа (перечислены пять пакетов компоновки Paketo);
- какие процессы будут запущены и посредством чего (выполнение команд Java в командной оболочке).

Обратите внимание, что для указанного образа опрашивается как локальный, так и связанный с ним удаленный репозиторий и выводится информация об обоих. Это особенно полезно при диагностике проблем, вызванных устаревшими образами контейнеров в том или ином месте.

Dive

Утилита `dive` создана Алексом Гудманом (Alex Goodman) в качестве способа заглянуть в образ контейнера и *детально* просмотреть все слои OCI образа и древовидную структуру его файловой системы.

`dive` способна заглянуть намного ниже слоев уровня приложения многослойной конструкции Spring Boot — на уровень операционной системы. Она представляется мне менее удобной, чем `pack`, поскольку ориентирована на операционную систему, а не на приложение, но прекрасно подходит для проверки наличия/отсутствия конкретных файлов, прав доступа к файлам и прочих существенных низкоуровневых проблем. Используется эта утилита редко, но бывает незаменима, если необходима настолько подробная информация и контроль над выводом.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Полный код для этой главы находится в ветке chapter11end в репозитории кода.

Резюме

Все обещанные конечным пользователям возможности приложения остаются пустым звуком, пока пользователи не смогут с этим приложением работать. Фигурально, а часто и вполне реально развертывание окупает приложение.

Многие разработчики знают, что приложения Spring Boot можно создавать в виде файлов WAR или JAR. Большинство этих разработчиков также знают, что есть немало причин отказаться от варианта WAR и создавать исполняемые файлы JAR и совсем немного причин поступать наоборот. Но в чем многие разработчики не отдают себе отчета, так это в том, что существует множество вариантов создания исполняемого JAR-файла Spring Boot для удовлетворения разнообразных требований и сценариев использования.

В этой главе я исследовал способы развертывания приложений Spring Boot, а также параметры, подходящие для различных целевых сред, и обсудил их относительные достоинства. А затем продемонстрировал, как создавать артефакты развертывания, рассмотрел варианты их оптимального выполнения и показал, как проверить их компоненты и происхождение. В числе целей сборки — стандартные исполняемые JAR-файлы Spring Boot, «полностью исполняемые» JAR-файлы Spring Boot, разобранные/развернутые JAR-файлы, а также собранные с помощью Cloud Native (Paketo) Buildpacks образы контейнеров, запускаемые в Docker, Kubernetes и на всех прочих основных движках/механизмах для работы с контейнерами. Spring Boot предоставляет множество вариантов беспроблемного развертывания, добавляя к сверхспособностям разработки еще и сверхспособности развертывания.

В следующей, последней главе я завершу эту книгу и все наше путешествие, углубившись в две чуть более продвинутые темы. Если вы хотите узнать больше о тестировании и отладке реактивных приложений, не пропускайте ее.

ГЛАВА 12

Углубляемся в реактивное программирование

Как уже говорилось, реактивное программирование позволяет разработчикам лучше использовать ресурсы распределенных систем и даже распространять мощные механизмы масштабирования за границы приложений и на каналы связи. Разработчики, имеющие опыт применения только господствующих приемов разработки Java — их часто называют *императивным* языком Java из-за явной последовательной логики, в отличие от более декларативного подхода, применяемого обычно в реактивном программировании, хотя этот ярлык, как и большинство ярлыков, не идеален, — могут обнаружить, что подобные реактивные возможности влекут за собой нежелательные дополнительные затраты. Помимо ожидаемой кривой обучения, которую Spring делает более покатой благодаря параллельным, дополняющим друг друга реализациям WebMVC и WebFlux, есть определенные ограничения в смысле инструментария, его зрелости и установившихся способов выполнения основных операций, например тестирования, поиска проблем и отладки.

И хотя реактивная разработка на Java делает лишь первые шаги по сравнению со своей императивной кузиной, их родство привело к намного более быстрой разработке и наработке полезных утилит и процессов. Как уже упоминалось, Spring в разработке основывается на накопленных императивных знаниях, сжимая десятилетия эволюции в готовые к продакшену компоненты, доступные *прямо сейчас*.

В этой главе я познакомлю вас с последними достижениями в сфере тестирования и диагностики/отладки проблем, с которыми вы можете столкнуться при развертывании реактивных приложений Spring Boot, а также покажу, как применить на деле WebFlux или Reactor еще до перехода к продакшену и как они могут помочь вам в этом.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Пожалуйста, для начала извлеките из репозитория кода ветку chapter12begin.

Когда следует использовать реактивное программирование

Реактивное программирование, в частности приложения, работающие с реактивными потоками данных, открывает возможности масштабирования всей системы, несравнимые с тем, чего можно достичь при помощи других доступных сегодня средств. Однако не всем приложениям нужна экстремальная сквозная масштабируемость либо они на протяжении длительного времени неплохо работают (или ожидается, что будут работать) при более или менее предсказуемых нагрузках. Императивные приложения уже давно удовлетворяют производственные потребности компаний по всему миру и никуда не исчезнут просто потому, что появился новый вариант.

И хотя реактивное программирование, безусловно, предлагает множество восхитительных возможностей, команда Spring четко заявляет, что реактивный код не заменит весь императивный код как минимум в ближайшем будущем. Как указывается в справочной документации для Spring WebFlux (https://oreil.ly/SFRefDoc): «Если у вас большая команда разработчиков, учтите, что кривая обучения при переходе на неблокирующее, функциональное и декларативное программирование довольно крута. На практике, чтобы не переходить сразу полностью, удобно воспользоваться реактивным WebClient. Помимо этого, начинайте с небольших шагов и оценивайте положительный эффект от каждого из них. Мы полагаем, что для широкого спектра приложений такой переход не нужен. Если вы не уверены в том, какой положительный эффект получите, начните с изучения принципов работы неблокирующего ввода/вывода (например, параллелизма в однопоточном Node.js) и его результатов».

Если вкратце, принятие на вооружение реактивного программирования и Spring WebFlux — лишь один из вариантов. Да, он замечательный, возможно, наилучший для удовлетворения определенных требований, но все же один из нескольких, и выбирать его следует лишь после тщательного учета всех требований и запросов конкретной системы. Реактивный или нет, Spring Boot открывает непревзойденные возможности разработки критически важного для бизнеса ПО, берущего на себя основную нагрузку в продакшене.

Тестирование реактивных приложений

Чтобы не отвлекаться от ключевых аспектов тестирования реактивных приложений Spring Boot, я предпринял определенные шаги для уменьшения объема рассматриваемого кода. Подобно увеличению объекта при фотографировании, остальной код проекта никуда не пропадает, просто он не столь важен для излагаемого в этом разделе материала.

ДОПОЛНИТЕЛЬНЫЕ ПРИМЕЧАНИЯ ОТНОСИТЕЛЬНО ТЕСТИРОВАНИЯ

Я уже обсуждал тестирование и частично свои взгляды на него в главе 9. Чтобы углубиться в обсуждаемые в текущей главе аспекты тестирования и пояснить предпринимаемые шаги, мне придется поделиться некоторыми своими соображениями. Поскольку эта книга посвящена в основном Spring Boot, а прочие связанные с ним вопросы — второстепенные, я попытался и буду продолжать пытаться найти достаточное количество дополнительной информации, необходимой для понимания контекста, но без излишних уточнений. Как вы догадываетесь, найти такую точку баланса невозможно, ведь для каждого читателя она своя, но я попытаюсь приблизиться к ней настолько, насколько возможно.

Тестирование определяет структуру кода. При настоящей разработке через тестирование (TDD) структурные ориентиры задаются с самого начала разработки приложения. Реализация тестов после завершения написания кода — так я делал в нескольких главах этой книги, чтобы сосредоточиться на концепциях Spring Boot, а не на соответствующей тестовой оснастке, — может потребовать больших затрат труда на рефакторинг, чтобы лучше изолировать и разделить поведение для тестирования конкретных компонентов и результатов. Несмотря на кажущуюся деструктивность такого подхода, обычно он дает код с более четкими границами, а значит, лучше поддающийся тестированию и более отказоустойчивый.

Код этой главы не исключение. Для изоляции и должного тестирования желаемого поведения придется произвести определенный рефакторинг уже существующего, работающего кода. Это не займет много времени, а результат в итоге, вероятно, будет лучшим.

В этом разделе я сосредоточусь на внешнем тестировании API, открывающих для доступа реактивные потоки данных, относящиеся к типам `Publisher`, которые могут быть как `Flux`, так и `Mono`, вместо обычных блокирующих типов `Iterable` и `Object`. Начнем с предоставляющего внешние API класса `PositionController` в приложении Aircraft Positions.



Если вы еще не извлекли из репозитория код для главы 12, как предлагалось в начале, самое время сделать это.

Но сначала — рефакторинг

Хотя код в классе `PositionController` вполне работоспособен, с точки зрения тестирования он весьма запутан. Первым делом необходимо четко разделить обязанности, и я начну с переноса кода создания объекта `RSocketRequester` в класс с аннотацией `@Configuration`, который будет создавать его в виде компонента Spring, доступного в любом месте приложения:

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.rsocket.RSocketRequester;

@Configuration
public class RSocketRequesterConfig {
    @Bean
    RSocketRequester requester(RSocketRequester.Builder builder) {
        return builder.tcp("localhost", 7635);
    }
}
```

Конструктор класса `PositionController` при этом упрощается, а объект `RSocketRequester` создается там, где и должен, — далеко за пределами класса-контроллера. А для использования компонента `RSocketRequester` в классе `PositionController` я просто выполняю его автосвязывание с помощью предоставляемой Spring Boot возможности внедрения зависимости через конструктор:

```
public PositionController(AircraftRepository repository,
                          RSocketRequester requester) {
    this.repository = repository;
    this.requester = requester;
}
```



Для проверки соединения RSocket нам понадобится интеграционное тестирование. И хотя данный раздел посвящен обсуждению модульного, а не интеграционного тестирования, для изоляции и должного модульного тестирования класса PositionController важно все же разделить формирование RSocketRequester и PositionController.

Еще один источник логики, выходящей далеко за рамки оставшейся функциональности контроллера, связан с получением, а затем сохранением и извлечением данных о местоположении воздушных судов с помощью компонента `AircraftRepository`. Обычно, когда в конкретный класс попадает не относящаяся к нему сложная логика, лучше вынести ее за пределы этого класса, как я и по-

ступил с компонентом `RSocketRequester`. Чтобы вынести этот довольно сложный и независимый код за пределы класса `PositionController`, я создал класс `PositionService`, описав его в виде компонента `@Service`, доступного в любом месте приложения. Аннотация `@Service` представляет собой более наглядное описание часто применяемой аннотации `@Component`:

```
import org.springframework.stereotype.Service;
import org.springframework.web.reactive.function.client.WebClient;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

@Service
public class PositionService {
    private final AircraftRepository repo;
    private WebClient client = WebClient.create(
        "http://localhost:7634/aircraft");

    public PositionService(AircraftRepository repo) {
        this.repo = repo;
    }

    public Flux<Aircraft> getAllAircraft() {
        return repo.deleteAll()
                .thenMany(client.get()
                        .retrieve()
                        .bodyToFlux(Aircraft.class)
                        .filter(plane -> !plane.getReg().isEmpty()))
                .flatMap(repo::save)
                .thenMany(repo.findAll());
    }

    public Mono<Aircraft> getAircraftById(Long id) {
        return repo.findById(id);
    }

    public Flux<Aircraft> getAircraftByReg(String reg) {
        return repo.findAircraftByReg(reg);
    }
}
```



В настоящее время в AircraftRepository не определен метод findAircraftByReg(). Я исправлю это перед написанием тестов.

Хотя тут осталось немало дополнительной работы, особенно связанной с переменной экземпляра `WebClient`, пока что достаточно перенести сложную логику,

видимую в `PositionService::getAllAircraft`, со старого места в `PositionController::getCurrentAircraftPositions` и внедрить компонент `PositionService` в контроллер для использования, в результате чего получится намного более аккуратный и узконаправленный класс контроллера:

```
import org.springframework.http.MediaType;
import org.springframework.messaging.rsocket.RSocketRequester;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;
import reactor.core.publisher.Flux;

@Controller
public class PositionController {
    private final PositionService service;
    private final RSocketRequester requester;

    public PositionController(PositionService service,
            RSocketRequester requester) {
        this.service = service;
        this.requester = requester;
    }

    @GetMapping("/aircraft")
    public String getCurrentAircraftPositions(Model model) {
        model.addAttribute("currentPositions", service.getAllAircraft());

        return "positions";
    }

    @ResponseBody
    @GetMapping(value = "/acstream", produces =
        MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<Aircraft> getCurrentACPositionsStream() {
        return requester.route("acstream")
                .data("Requesting aircraft positions")
                .retrieveFlux(Aircraft.class);
    }
}
```

Ревизия существующих конечных точек класса `PositionController` показывает, что они основаны на шаблоне Thymeleaf (`public String getCurrentAircraftPositions(Model model)`) или требуют внешнего соединения `RSocket` (`public Flux<Aircraft> getCurrentACPositionsStream()`). Для изоляции и тестирования предоставляемого приложением Aircraft Positions внешнего API необходимо расширить список описанных в настоящий момент конечных точек. Я добавлю еще две конечные точки, соответствующие путям `/acpos` и `/acpos/`

search, чтобы создать простой, но гибкий API, использующий созданные мной в PositionService методы.

Сначала я создал метод для получения и возврата в виде JSON всех местоположений воздушных судов в зоне досягаемости нашего устройства, опрашиваемого сервисом PlaneFinder. Метод getCurrentACPositions() вызывает PositionService::getAllAircraft подобно его аналогу getCurrentAircraftPositions(Model model), но возвращает значение в виде объектов JSON, вместо того чтобы добавлять их в DOM и перенаправлять в шаблонизатор для отображения HTML-страницы.

Далее я создал метод для поиска текущих местоположений воздушных судов по уникальному идентификатору записи и регистрационному номеру воздушного судна. Идентификатор записи (фактически документа, ведь в данной версии приложения Aircraft Positions используется MongoDB) представляет собой уникальный (среди хранимых местоположений воздушных судов, полученных в последний раз от PlaneFinder) идентификатор базы данных, удобный для получения записи о конкретном местоположении. Но для воздушного судна большую пользу приносит возможность поиска по уникальному регистрационному номеру.

Что любопытно, приложение PlaneFinder может выдавать в ответ на запрос небольшое количество местоположений, относящихся к одному воздушному судну, вследствие практически неизменных сообщений о местоположении, отправляемых летящим самолетом. Для нас это означает, что поиск по уникальному регистрационному номеру воздушного судна в списке текущих местоположений может вернуть более одного местоположения для конкретного рейса.

Существует много вариантов создания гибкого поискового механизма, позволяющего задавать различные критерии поиска и получать в ответ различное количество возможных результатов, но я решил объединить все варианты в одном методе:

```
@ResponseBody
@GetMapping("/acpos/search")
public Publisher<Aircraft>
        searchForACPosition(@RequestParam Map<String, String> searchParams) {

    if (!searchParams.isEmpty()) {
        Map.Entry<String, String> setToSearch =
                searchParams.entrySet().iterator().next();
        if (setToSearch.getKey().equalsIgnoreCase("id")) {
            return service.getAircraftById(Long.valueOf(setToSearch.getValue()));
        } else {
            return service.getAircraftByReg(setToSearch.getValue());
```

```
        }
    } else {
        return Mono.empty();
    }
}
```

ПРИМЕЧАНИЯ ОТНОСИТЕЛЬНО АРХИТЕКТУРЫ МЕТОДА SEARCHFORACPOSITION И ПРИНЯТЫХ ПРИ РЕАЛИЗАЦИИ РЕШЕНИЙ

Прежде всего необходима аннотация `@ResponseBody`, поскольку я решил объединить конечные точки REST с конечными точками, управляемыми шаблонами, в одном классе контроллера. Как уже упоминалось, метааннотация `@RestController` включает функциональность как `@Controller`, так и `@ResponseBody`, указывая на то, что значения `Object` возвращаются непосредственно, а не с помощью объектной модели предметной области (DOM). А поскольку класс `PositionController` снабжен только аннотацией `@Controller`, необходимо добавлять `@ResponseBody` ко всем методам, которые должны возвращать напрямую значения `Object`.

Далее аннотация `@RequestParam` позволяет пользователю указывать ноль или более параметров запроса, присоединяя знак вопроса (`?`) к соответствующей конечной точке URI и задавая параметры в формате «ключ — значение» с разделением их запятыми. В этом примере я осознанно проверяю только первый параметр (при его наличии) для ключа `id`. Если запрос включает параметр `id`, документ запрашивается о местоположении воздушного судна по его идентификатору базы данных. Если же параметр — не `id`, по умолчанию мы ищем регистрационные номера воздушных судов в списке текущих местоположений.

Здесь делается несколько неявных допущений, которые я вряд ли использовал бы в реальной системе: поиск по умолчанию регистрационных номеров, умышленное отбрасывание всех поисковых параметров, кроме первого, и т. д. Оставляю это в качестве упражнения на будущее для себя самого и для читателей.

Относительно сигнатуры нашего метода стоит отметить один нюанс: я возвращаю объект `Publisher`, а не конкретно `Flux` или `Mono`. Этого требует принятое мной решение о совмещении параметров поиска в одном методе и то, что хотя при поиске документа о местоположении в базе данных по идентификатору базы данных возвращается не более одного найденного соответствия, при поиске по регистрационному номеру воздушного судна могут возвращаться несколько близко сгруппированных отчетов о местоположении. Указание `Publisher` в качестве возвращаемого типа для метода позволяет возвращать как `Mono`, так и `Flux`, ведь оба этих типа данных реализуют `Publisher`.

Наконец, если пользователь не указал никаких параметров поиска, я возвращаю пустой `Mono` с помощью вызова `Mono.empty()`. Ваша ситуация может потребовать того же, либо вы можете решить (или будете вынуждены) вернуть другой результат, например все местоположения воздушных судов. При любом проектном решении исход должен определяться принципом наименьшего удивления.

Окончательная (пока что) версия класса `PositionController` должна выглядеть примерно так:

```
import org.reactivestreams.Publisher;
import org.springframework.http.MediaType;
import org.springframework.messaging.rsocket.RSocketRequester;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseBody;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.util.Map;

@Controller
public class PositionController {
    private final PositionService service;
    private final RSocketRequester requester;

    public PositionController(PositionService service,
            RSocketRequester requester) {
        this.service = service;
        this.requester = requester;
    }

    @GetMapping("/aircraft")
    public String getCurrentAircraftPositions(Model model) {
        model.addAttribute("currentPositions", service.getAllAircraft());

        return "positions";
    }

    @ResponseBody
    @GetMapping("/acpos")
    public Flux<Aircraft> getCurrentACPositions() {
        return service.getAllAircraft();
    }

    @ResponseBody
    @GetMapping("/acpos/search")
    public Publisher<Aircraft> searchForACPosition(@RequestParam Map<String,
            String> searchParams) {

        if (!searchParams.isEmpty()) {
            Map.Entry<String, String> setToSearch =
                searchParams.entrySet().iterator().next();

            if (setToSearch.getKey().equalsIgnoreCase("id")) {
```

```
                return service.getAircraftById(Long.valueOf
                    (setToSearch.getValue()));
            } else {
                return service.getAircraftByReg(setToSearch.getValue());
            }
        } else {
            return Mono.empty();
        }
    }

    @ResponseBody
    @GetMapping(value = "/acstream", produces =
            MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<Aircraft> getCurrentACPositionsStream() {
        return requester.route("acstream")
                .data("Requesting aircraft positions")
                .retrieveFlux(Aircraft.class);
    }
}
```

Далее возвращаемся к классу `PositionService`. Как упоминалось ранее, его метод `public Flux<Aircraft> getAircraftByReg(String reg)` ссылается на пока что не определенный метод из `AircraftRepository`. Для решения этой проблемы добавим в описание интерфейса `AircraftRepository` метод `Flux<Aircraft> findAircraftByReg(String reg)`:

```
import org.springframework.data.repository.reactive.ReactiveCrudRepository;
import reactor.core.publisher.Flux;

public interface AircraftRepository extends
        ReactiveCrudRepository<Aircraft, Long> {
    Flux<Aircraft> findAircraftByReg(String reg);
}
```

Этот интересный фрагмент кода — сигнатура этого одного метода — демонстрирует замечательную концепцию Spring Data — вывод запроса на основе набора широко применяемых соглашений: операторов наподобие `find`, `search` и `get`, указания типа хранимых/извлекаемых/контролируемых объектов (в данном случае `Aircraft`) и имен переменных-членов, например `reg`. Благодаря объявлению сигнатуры метода с параметрами и типами с помощью вышеупомянутых соглашений о наименованиях методов Spring Data может создать для вас реализацию метода.

При необходимости конкретизации или предоставления подсказок можно также снабдить сигнатуру метода аннотацией `@Query`, указав желаемые или необходимые сведения. В данном случае это не нужно, поскольку информации о том, что мы хотим искать местоположение воздушных судов по регистрационному номе-

ру и возвращаем ноль или более значений в реактивных потоках данных `Flux`, вполне достаточно для Spring Data, чтобы создать соответствующую реализацию.

Возвращаясь к `PositionService`: IDE-среда сообщает, что `repo.findAircraftByReg(reg)` — допустимый вызов метода.



Еще одно решение относительно архитектуры этого примера связано с тем, что оба метода getAircraftByXxx запрашивают документы о текущем местоположении. То есть предполагается, что в базе данных существуют какие-либо документы о местоположении или что пользователь не заинтересован в новом поиске, если в базе данных не содержится никаких данных о местоположении. Ваши требования могут привести к выбору другого варианта, например проверке наличия данных о местоположении до поиска, если не выполняется новый поиск с помощью вызова getAllAircraft.

А теперь — тестирование

В предыдущем примере тестирования для проверки ожидаемых результатов применялись стандартные типы `Object`. Я использовал `WebClient` и `WebTestClient`, но лишь в качестве предпочтительного инструмента взаимодействия со всеми HTTP-конечными точками вне зависимости от того, возвращают они типы реактивных потоков данных `Publisher` или нет. Теперь самое время протестировать семантику этих реактивных потоков данных.

Воспользовавшись уже существующим классом `PositionControllerTest` в качестве отправной точки, я переделал его для работы с новыми реактивными конечными точками, открытыми для доступа соответствующим ему классом `PositionController`. Вот все подробности на уровне класса:

```
@WebFluxTest(controllers = {PositionController.class})
class PositionControllerTest {
    @Autowired
    private WebTestClient client;

    @MockBean
    private PositionService service;
    @MockBean
    private RSocketRequester requester;

    private Aircraft ac1, ac2, ac3;

    ...

}
```

Во-первых, я указал аннотацию уровня класса `@WebFluxTest(controllers = {PositionController.class})`. Я по-прежнему использую реактивный `WebTestClient` и хотел бы ограничить этот тестовый класс рамками возможностей `WebFlux`, так что загружать полный контекст приложения Spring Boot не требуется и только означало бы пустую трату времени и ресурсов.

Во-вторых, я автосвязываю компонент `WebTestClient`. В одной из предыдущих глав, посвященной тестированию, я напрямую внедрял компонент `WebTestClient` в один тестовый метод, но поскольку сейчас он понадобится в нескольких методах, имеет смысл создать переменную-член класса, чтобы ссылаться на него.

В-третьих, я создал фиктивные компоненты с помощью аннотации `@MockBean` Moskito. Я создал макет компонента `RSocketRequester` просто потому, что классу `PositionController`, который мы хотим и должны загружать в аннотации уровня класса, необходим компонент `RSocketRequester`, имитируемый или настоящий. А компонент `PositionService` я макетирую, чтобы имитировать и использовать его поведение в тестах данного класса. Имитация `PositionService` позволяет убедиться в его правильном поведении, протестировать потребителя результатов его работы (`PositionController`) и сравнивать фактические результаты с известными ожидаемыми результатами.

Наконец, я создаю три экземпляра `Aircraft` для применения в тестах.

Перед выполнением метода `@Test` JUnit запускается метод, снабженный аннотацией `@BeforeEach`, для задания настроек сценария и ожидаемых результатов. Вот метод `setUp()`, с помощью которого я готовлю среду тестирования перед каждым тестовым методом:

```
@BeforeEach
void setUp(ApplicationContext context) {
    // Рейс 001 компании Spring Airlines по пути из STL в SFO
    // в настоящее время на высоте 9 км над Канзас-сити
    ac1 = new Aircraft(1L, "SAL001", "sqwk", "N12345", "SAL001",
            "STL-SFO", "LJ", "ct",
            30000, 280, 440, 0, 0,
            39.2979849, -94.71921, 0D, 0D, 0D,
            true, false,
            Instant.now(), Instant.now(), Instant.now());

    // Рейс 002 компании Spring Airlines по пути из SFO в STL
    // в настоящее время на высоте 12 км над Денвером
    ac2 = new Aircraft(2L, "SAL002", "sqwk", "N54321", "SAL002",
            "SFO-STL", "LJ", "ct",
            40000, 65, 440, 0, 0,
            39.8560963, -104.6759263, 0D, 0D, 0D,
            true, false,
```

```
            Instant.now(), Instant.now(), Instant.now());

    // Рейс 002 компании Spring Airlines по пути из SFO в STL
    // в настоящее время на высоте 12 км чуть дальше Денвера
    ac3 = new Aircraft(3L, "SAL002", "sqwk", "N54321", "SAL002",
            "SFO-STL", "LJ", "ct",
            40000, 65, 440, 0, 0,
            39.8412964, -105.0048267, 0D, 0D, 0D,
            true, false,
            Instant.now(), Instant.now(), Instant.now());

    Mockito.when(service.getAllAircraft()).thenReturn(Flux.just(ac1, ac2, ac3));
    Mockito.when(service.getAircraftById(1L)).thenReturn(Mono.just(ac1));
    Mockito.when(service.getAircraftById(2L)).thenReturn(Mono.just(ac2));
    Mockito.when(service.getAircraftById(3L)).thenReturn(Mono.just(ac3));
    Mockito.when(service.getAircraftByReg("N12345"))
        .thenReturn(Flux.just(ac1));
    Mockito.when(service.getAircraftByReg("N54321"))
        .thenReturn(Flux.just(ac2, ac3));
}
```

Я присваиваю данные о местоположении воздушного судна с регистрационным номером N12345 переменной-члену `ac1`. Переменным-членам `ac2` и `ac3` присваиваю местоположения воздушного судна N54321, расположенные очень близко друг к другу, имитируя таким образом распространенный случай — часто поступающие от приложения PlaneFinder отчеты о последовательных местоположениях воздушного судна.

Несколько последних строк метода `setUp()` задают поведение, демонстрируемое имитационным компонентом `PositionService` при вызове его методов различными способами. Он похож на макеты методов из посвященной тестированию главы этой книги и значительно отличается лишь типами возвращаемых значений: поскольку фактические методы `PositionService` возвращают типы `Reactor Publisher` — `Flux` или `Mono`, то и фиктивные методы должны вести себя так же.

Тестирование извлечения всех данных о местоположении воздушных судов

Наконец, я создаю метод для тестирования метода `getCurrentACPositions()` класса `PositionController`:

```
@Test
void getCurrentACPositions() {
    StepVerifier.create(client.get()
```

```
            .uri("/acpos")
            .exchange()
            .expectStatus().isOk()
            .expectHeader().contentType(MediaType.APPLICATION_JSON)
            .returnResult(Aircraft.class)
            .getResponseBody())
        .expectNext(ac1)
        .expectNext(ac2)
        .expectNext(ac3)
        .verifyComplete();
}
```

При тестировании приложений, работающих с реактивными потоками данных, возникает множество проблем с обычно рутинным, а то и вообще игнорируемым заданием ожидаемых результатов, получением фактических результатов и определением успешности теста путем сравнения этих двух результатов. Хотя *можно* получать несколько результатов практически одновременно, например при использовании блокирующего типа `Iterable`, реактивные потоки данных `Publisher` не ждут полного результата, чтобы вернуть его как единое целое. С точки зрения компьютера разница состоит в получении, например, одной группы из пяти результатов целиком или пяти результатов очень быстро, но по отдельности.

Основу инструментов тестирования Reactor составляет класс `StepVerifier` и его вспомогательные методы. `StepVerifier` подписывается на `Publisher` и, как ясно из его названия, позволяет разработчику рассматривать результаты, полученные в виде отдельных значений, проверяя каждое из них. При тестировании `getCurrentACPositions` я сделал следующее:

- создал объект `StepVerifier`;
- передал ему объект `Flux`, полученный в результате выполнения следующих шагов:
  - использования компонента `WebTestClient`;
  - вызова метода `PositionController::getCurrentACPositions`, связанного с конечной точкой `/acpos`;
  - вызова `exchange()`;
  - проверки того, что состояние ответа — 200 OK;
  - проверки того, что тип содержимого в заголовке ответа — `application/json`;
  - возврата полученных элементов в виде экземпляров класса `Aircraft`;
  - получения ответа с помощью HTTP-метода `GET`;

- сравнил первое из фактических значений с ожидаемым первым значением `ac1`;
- сравнил второе из фактических значений с ожидаемым вторым значением `ac2`;
- сравнил третье из фактических значений с ожидаемым третьим значением `ac3`;
- проверил все действия и получение сигнала завершения `Publisher`.

Это довольно исчерпывающая проверка ожидаемого поведения, включая условия и возвращаемые значения. В результате выполнения теста выводится следующее (сокращено, чтобы поместилось на странице):

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v2.4.0)

: Starting PositionControllerTest on mheckler-a01.vmware.com with PID 21211
: No active profile set, falling back to default profiles: default
: Started PositionControllerTest in 2.19 seconds (JVM running for 2.879)

Process finished with exit code 0
```

При запуске из IDE результат выглядит примерно так, как показано на рис. 12.1.

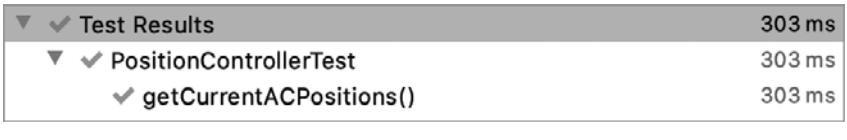


Рис. 12.1. Успешно выполненный тест

Тестирование возможностей поиска приложения Aircraft Positions

Тестирование функциональности поиска из `PositionController::searchForACPosition` требует как минимум двух отдельных тестов, поскольку искать местоположение воздушных судов можно как по идентификатору документа базы данных, так и по регистрационному номеру самолета.

Для тестирования поиска по идентификатору документа базы данных я создал следующий модульный тест:

```
@Test
void searchForACPositionById() {
    StepVerifier.create(client.get()
            .uri("/acpos/search?id=1")
            .exchange()
            .expectStatus().isOk()
            .expectHeader().contentType(MediaType.APPLICATION_JSON)
            .returnResult(Aircraft.class)
            .getResponseBody())
        .expectNext(ac1)
        .verifyComplete();
}
```

Он почти аналогичен модульному тесту для получения всех местоположений воздушных судов, но имеет два заметных различия.

- Указанный URI ссылается на конечную точку поиска и включает параметр поиска `id=1`, чтобы извлечь `ac1`.
- Ожидается только результат `ac1`, как указано в связанной операции `expectNext(ac1)`.

Для тестирования поиска местоположения воздушного судна по его регистрационному номеру я, воспользовавшись моделью регистрации, создал следующий модульный тест с включением двух соответствующих документов местоположений:

```
@Test
void searchForACPositionByReg() {
    StepVerifier.create(client.get()
            .uri("/acpos/search?reg=N54321")
            .exchange()
            .expectStatus().isOk()
            .expectHeader().contentType(MediaType.APPLICATION_JSON)
            .returnResult(Aircraft.class)
            .getResponseBody())
        .expectNext(ac2)
        .expectNext(ac3)
        .verifyComplete();
}
```

Различия этого и предыдущего тестов минимальны.

- URI включает поисковый параметр `reg=N54321`, и в результате должны быть возвращены `ac2` и `ac3`, содержащие оба местоположения воздушного судна с регистрационным номером N54321.
- Проверяется, что получены результаты `ac2` и `ac3`, при помощи связанных операций `expectNext(ac2)` и `expectNext(ac3)`.

Окончательный вариант класса `PositionControllerTest` приведен в следующем листинге:

```
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.reactive.WebFluxTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.messaging.rsocket.RSocketRequester;
import org.springframework.test.web.reactive.server.WebTestClient;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
import reactor.test.StepVerifier;

import java.time.Instant;

@WebFluxTest(controllers = {PositionController.class})
class PositionControllerTest {
    @Autowired
    private WebTestClient client;

    @MockBean
    private PositionService service;
    @MockBean
    private RSocketRequester requester;

    private Aircraft ac1, ac2, ac3;

    @BeforeEach
    void setUp() {
        // Рейс 001 компании Spring Airlines по пути из STL в SFO
        // в настоящее время на высоте 9 км над Канзас-сити
        ac1 = new Aircraft(1L, "SAL001", "sqwk", "N12345", "SAL001",
                "STL-SFO", "LJ", "ct",
                30000, 280, 440, 0, 0,
                39.2979849, -94.71921, 0D, 0D, 0D,
                true, false,
                Instant.now(), Instant.now(), Instant.now());

        // Рейс 002 компании Spring Airlines по пути из SFO в STL
        // в настоящее время на высоте 12 км над Денвером
        ac2 = new Aircraft(2L, "SAL002", "sqwk", "N54321", "SAL002",
                "SFO-STL", "LJ", "ct",
                40000, 65, 440, 0, 0,
                39.8560963, -104.6759263, 0D, 0D, 0D,
                true, false,
                Instant.now(), Instant.now(), Instant.now());
```

```
        // Рейс 002 компании Spring Airlines по пути из SFO в STL
        // в настоящее время на высоте 12 км чуть дальше Денвера
        ac3 = new Aircraft(3L, "SAL002", "sqwk", "N54321", "SAL002",
                "SFO-STL", "LJ", "ct",
                40000, 65, 440, 0, 0,
                39.8412964, -105.0048267, 0D, 0D, 0D,
                true, false,
                Instant.now(), Instant.now(), Instant.now());

        Mockito.when(service.getAllAircraft())
                .thenReturn(Flux.just(ac1, ac2, ac3));
        Mockito.when(service.getAircraftById(1L))
                .thenReturn(Mono.just(ac1));
        Mockito.when(service.getAircraftById(2L))
                .thenReturn(Mono.just(ac2));
        Mockito.when(service.getAircraftById(3L))
                .thenReturn(Mono.just(ac3));
        Mockito.when(service.getAircraftByReg("N12345"))
                .thenReturn(Flux.just(ac1));
        Mockito.when(service.getAircraftByReg("N54321"))
                .thenReturn(Flux.just(ac2, ac3));
    }

    @AfterEach
    void tearDown() {
    }

    @Test
    void getCurrentACPositions() {
        StepVerifier.create(client.get()
                .uri("/acpos")
                .exchange()
                .expectStatus().isOk()
                .expectHeader().contentType(MediaType.APPLICATION_JSON)
                .returnResult(Aircraft.class)
                .getResponseBody())
            .expectNext(ac1)
            .expectNext(ac2)
            .expectNext(ac3)
            .verifyComplete();
    }

    @Test
    void searchForACPositionById() {
        StepVerifier.create(client.get()
                .uri("/acpos/search?id=1")
                .exchange()
                .expectStatus().isOk()
                .expectHeader().contentType(MediaType.APPLICATION_JSON)
                .returnResult(Aircraft.class)
```

```
                .getResponseBody())
            .expectNext(ac1)
            .verifyComplete();
    }

    @Test
    void searchForACPositionByReg() {
        StepVerifier.create(client.get()
                .uri("/acpos/search?reg=N54321")
                .exchange()
                .expectStatus().isOk()
                .expectHeader().contentType(MediaType.APPLICATION_JSON)
                .returnResult(Aircraft.class)
                .getResponseBody())
            .expectNext(ac2)
            .expectNext(ac3)
            .verifyComplete();
    }
}
```

Все тесты из класса `PositionControllerTest` выполняются успешно, как видно на рис. 12.2.

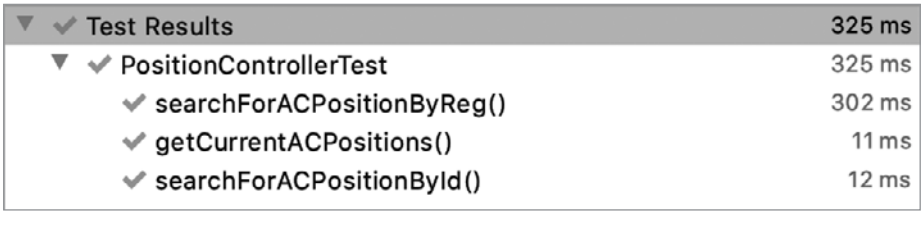


Рис. 12.2. Успешное выполнение всех модульных тестов



Класс StepVerifier открывает широкие возможности тестирования, часть которых мы упомянули в этом разделе. Особенно интересен метод StepVerifier::withVirtualTime, позволяющий тестировать типы Publisher, время от времени генерирующие значения, которые приходится сжимать, поскольку генерируемые одномоментно результаты вполне могут относиться к длительному периоду времени. StepVerifier::withVirtualTime принимает объект Supplier<Publisher> вместо просто объекта Publisher, в остальном принципы его использования такие же.

Это все основные аспекты тестирования реактивных приложений Spring Boot. Но что делать, когда начинаются проблемы в ходе продакшена? Какие инструменты предоставляет Reactor для выявления и разрешения проблем во время реальной работы приложения?

Диагностика и отладка реактивных приложений

При возникновении проблем в типичных приложениях Java обычно генерируется трасса стека вызовов. Императивный код может генерировать полезную, хотя иногда и слишком объемную трассу стека вызовов по нескольким причинам, но в целом сбор и отображение этой полезной информации возможен благодаря двум факторам:

- последовательному выполнению кода, который обычно указывает, как сделать что-либо (императивно);
- выполнению этого последовательного кода в рамках одного потока.

Из всякого правила есть исключения, но в целом это распространенное сочетание, позволяющее зафиксировать последовательность выполняемых шагов вплоть до момента возникновения ошибки: все происходит пошагово на одной «дорожке». Возможно, при этом полные ресурсы системы могут использоваться не очень эффективно, обычно так и происходит, но изоляция и разрешение проблем намного упрощаются.

А теперь обратимся к реактивным потокам данных. В Project Reactor и прочих реализациях реактивных потоков данных для управления и использования дополнительных потоков применяются планировщики. Ресурсы, в противном случае простаивающие или используемые недостаточно эффективно, позволяют масштабировать реактивные приложения намного шире, чем их блокирующие аналоги. Вы можете заглянуть в документацию по ядру Reactor (https://projectreactor.io/docs/core/release/reference), чтобы получить более подробную информацию о планировщиках и управлении их использованием и настройкой, но пока что достаточно сказать, что Reactor прекрасно справляется с автоматическим планированием в абсолютном большинстве случаев.

Впрочем, сразу бросается в глаза одна проблема с генерацией осмысленной трассировки выполнения для реактивных приложений Spring Boot (или любых других реактивных), а именно: не получится просто следить за действиями из одного потока выполнения и получить осмысленный последовательный список выполняемого кода.

И без того непростое из-за переключения из потока в поток отслеживание выполнения осложняется тем, что реактивное программирование разделяет *сборку* и *выполнение* кода. Как упоминалось в главе 8, чаще всего для большинства типов `Publisher` ничего не происходит без *подписки*.

Проще говоря, маловероятно, что вам встретится сбой в продакшене, указывающий на проблему с кодом, в котором декларативно производится сборка конвейера операций `Publisher` (неважно, `Flux` или `Mono`). Сбои практически всегда происходят во время активной работы конвейера — при генерации, обработке и передаче значений `Subscriber`.

Подобное разделение сборки объектов и выполнения кода, а также способность Reactor применять несколько потоков выполнения для завершения цепочки операций требует более совершенного инструментария для диагностики и устранения неполадок, возникающих на этапе выполнения. К счастью, Reactor предоставляет несколько прекрасных инструментов для этого.

Hooks.onOperatorDebug()

Все сказанное ранее не означает невозможность диагностики и устранения неполадок в реактивных приложениях с помощью уже имеющихся результатов трассировки вызовов, а только то, что ее можно значительно усовершенствовать. Как и в большинстве случаев, это видно из кода — или в данном случае из того, что выводится в журнал в результате сбоя.

Для имитации сбоя в цепочке операторов реактивного `Publisher` я снова обращусь к классу `PositionControllerTest` и поменяю одну строку кода в запускаемом перед выполнением каждого теста методе `setUp()`:

```
Mockito.when(service.getAllAircraft()).thenReturn(Flux.just(ac1, ac2, ac3));
```

Я заменил нормально работающий `Flux`, сгенерированный фиктивным методом `getAllAircraft()`, кодом с ошибкой в итоговом потоке значений:

```
Mockito.when(service.getAllAircraft()).thenReturn(
        Flux.just(ac1, ac2, ac3)
                .concatWith(Flux.error(new Throwable("Bad position report")))
);
```

Далее я выполнил тест `getCurrentACPositions()`, чтобы посмотреть на результат умышленного саботажа `Flux` (разбито по строкам, чтобы помещалось на странице):

```
500 Server Error for HTTP GET "/acpos"

java.lang.Throwable: Bad position report
  at com.thehecklers.aircraftpositions.PositionControllerTest
        .setUp(PositionControllerTest.java:59) ~[test-classes/:na]
  Suppressed: reactor.core.publisher.FluxOnAssembly$OnAssemblyException:
```

```
Error has been observed at the following site(s):
  |_ checkpoint ⇢ Handler com.thehecklers.aircraftpositions
       .PositionController
       #getCurrentACPositions() [DispatcherHandler]
  |_ checkpoint ⇢ HTTP GET "/acpos" [ExceptionHandlingWebHandler]
Stack trace:
    at com.thehecklers.aircraftpositions.PositionControllerTest
        .setUp(PositionControllerTest.java:59) ~[test-classes/:na]
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl
        .invoke0(Native Method) ~[na:na]
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl
        .invoke(NativeMethodAccessorImpl.java:62) ~[na:na]
    at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl
        .invoke(DelegatingMethodAccessorImpl.java:43) ~[na:na]
    at java.base/java.lang.reflect.Method
        .invoke(Method.java:564) ~[na:na]
    at org.junit.platform.commons.util.ReflectionUtils
        .invokeMethod(ReflectionUtils.java:686)
        ~[junit-platform-commons-1.6.2.jar:1.6.2]
    at org.junit.jupiter.engine.execution.MethodInvocation
        .proceed(MethodInvocation.java:60)
                ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
    at org.junit.jupiter.engine.execution.InvocationInterceptorChain
        $ValidatingInvocation.proceed(InvocationInterceptorChain.java:131)
        ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
    at org.junit.jupiter.engine.extension.TimeoutExtension
        .intercept(TimeoutExtension.java:149)
        ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
    at org.junit.jupiter.engine.extension.TimeoutExtension
        .interceptLifecycleMethod(TimeoutExtension.java:126)
        ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
    at org.junit.jupiter.engine.extension.TimeoutExtension
        .interceptBeforeEachMethod(TimeoutExtension.java:76)
        ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
    at org.junit.jupiter.engine.execution
        .ExecutableInvoker$ReflectiveInterceptorCall.lambda$ofVoidMethod
          $0(ExecutableInvoker.java:115)
          ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
    at org.junit.jupiter.engine.execution.ExecutableInvoker
        .lambda$invoke$0(ExecutableInvoker.java:105)
          ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
    at org.junit.jupiter.engine.execution.InvocationInterceptorChain
        $InterceptedInvocation.proceed(InvocationInterceptorChain.java:106)
          ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
    at org.junit.jupiter.engine.execution.InvocationInterceptorChain
        .proceed(InvocationInterceptorChain.java:64)
          ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
    at org.junit.jupiter.engine.execution.InvocationInterceptorChain
        .chainAndInvoke(InvocationInterceptorChain.java:45)
          ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
```

```
at org.junit.jupiter.engine.execution.InvocationInterceptorChain
    .invoke(InvocationInterceptorChain.java:37)
      ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
at org.junit.jupiter.engine.execution.ExecutableInvoker
    .invoke(ExecutableInvoker.java:104)
      ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
at org.junit.jupiter.engine.execution.ExecutableInvoker
    .invoke(ExecutableInvoker.java:98)
      ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
at org.junit.jupiter.engine.descriptor.ClassBasedTestDescriptor
    .invokeMethodInExtensionContext(ClassBasedTestDescriptor.java:481)
      ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
at org.junit.jupiter.engine.descriptor.ClassBasedTestDescriptor
    .lambda$synthesizeBeforeEachMethodAdapter
      $18(ClassBasedTestDescriptor.java:466)
      ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor
    .lambda$invokeBeforeEachMethods$2(TestMethodTestDescriptor.java:169)
      ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor
    .lambda$invokeBeforeMethodsOrCallbacksUntilExceptionOccurs
        $5(TestMethodTestDescriptor.java:197)
        ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
at org.junit.platform.engine.support.hierarchical.ThrowableCollector
    .execute(ThrowableCollector.java:73)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor
    .invokeBeforeMethodsOrCallbacksUntilExceptionOccurs
        (TestMethodTestDescriptor.java:197)
        ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor
    .invokeBeforeEachMethods(TestMethodTestDescriptor.java:166)
        ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor
    .execute(TestMethodTestDescriptor.java:133)
        ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor
    .execute(TestMethodTestDescriptor.java:71)
        ~[junit-jupiter-engine-5.6.2.jar:5.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .lambda$executeRecursively$5(NodeTestTask.java:135)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.ThrowableCollector
    .execute(ThrowableCollector.java:73)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .lambda$executeRecursively$7(NodeTestTask.java:125)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.Node
    .around(Node.java:135) ~[junit-platform-engine-1.6.2.jar:1.6.2]
```

```
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .lambda$executeRecursively$8(NodeTestTask.java:123)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.ThrowableCollector
    .execute(ThrowableCollector.java:73)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .executeRecursively(NodeTestTask.java:122)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .execute(NodeTestTask.java:80)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at java.base/java.util.ArrayList.forEach(ArrayList.java:1510) ~[na:na]
at org.junit.platform.engine.support.hierarchical
    .SameThreadHierarchicalTestExecutorService
        .invokeAll(SameThreadHierarchicalTestExecutorService.java:38)
            ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .lambda$executeRecursively$5(NodeTestTask.java:139)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.ThrowableCollector
    .execute(ThrowableCollector.java:73)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .lambda$executeRecursively$7(NodeTestTask.java:125)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.Node
    .around(Node.java:135) ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .lambda$executeRecursively$8(NodeTestTask.java:123)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.ThrowableCollector
    .execute(ThrowableCollector.java:73)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .executeRecursively(NodeTestTask.java:122)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .execute(NodeTestTask.java:80)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at java.base/java.util.ArrayList.forEach(ArrayList.java:1510) ~[na:na]
at org.junit.platform.engine.support.hierarchical
    .SameThreadHierarchicalTestExecutorService
        .invokeAll(SameThreadHierarchicalTestExecutorService.java:38)
            ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .lambda$executeRecursively$5(NodeTestTask.java:139)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.ThrowableCollector
    .execute(ThrowableCollector.java:73)
```

```
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .lambda$executeRecursively$7(NodeTestTask.java:125)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.Node
    .around(Node.java:135) ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .lambda$executeRecursively$8(NodeTestTask.java:123)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.ThrowableCollector
    .execute(ThrowableCollector.java:73)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .executeRecursively(NodeTestTask.java:122)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .execute(NodeTestTask.java:80)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical
    .SameThreadHierarchicalTestExecutorService
        .submit(SameThreadHierarchicalTestExecutorService.java:32)
            ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical
    .HierarchicalTestExecutor.execute(HierarchicalTestExecutor.java:57)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.engine.support.hierarchical
    .HierarchicalTestEngine.execute(HierarchicalTestEngine.java:51)
        ~[junit-platform-engine-1.6.2.jar:1.6.2]
at org.junit.platform.launcher.core.DefaultLauncher
    .execute(DefaultLauncher.java:248)
        ~[junit-platform-launcher-1.6.2.jar:1.6.2]
at org.junit.platform.launcher.core.DefaultLauncher
    .lambda$execute$5(DefaultLauncher.java:211)
        ~[junit-platform-launcher-1.6.2.jar:1.6.2]
at org.junit.platform.launcher.core.DefaultLauncher
    .withInterceptedStreams(DefaultLauncher.java:226)
        ~[junit-platform-launcher-1.6.2.jar:1.6.2]
at org.junit.platform.launcher.core.DefaultLauncher
    .execute(DefaultLauncher.java:199)
        ~[junit-platform-launcher-1.6.2.jar:1.6.2]
at org.junit.platform.launcher.core.DefaultLauncher
    .execute(DefaultLauncher.java:132)
        ~[junit-platform-launcher-1.6.2.jar:1.6.2]
at com.intellij.junit5.JUnit5IdeaTestRunner
    .startRunnerWithArgs(JUnit5IdeaTestRunner.java:69)
        ~[junit5-rt.jar:na]
at com.intellij.rt.junit.IdeaTestRunner$Repeater
    .startRunnerWithArgs(IdeaTestRunner.java:33)
        ~[junit-rt.jar:na]
at com.intellij.rt.junit.JUnitStarter
```

```
        .prepareStreamsAndStart(JUnitStarter.java:230)
            ~[junit-rt.jar:na]
    at com.intellij.rt.junit.JUnitStarter
        .main(JUnitStarter.java:58) ~[junit-rt.jar:na]
java.lang.AssertionError: Status expected:<200 OK>
    but was:<500 INTERNAL_SERVER_ERROR>

> GET /acpos
> WebTestClient-Request-Id: [1]

No content

< 500 INTERNAL_SERVER_ERROR Internal Server Error
< Content-Type: [application/json]
< Content-Length: [142]

{"timestamp":"2020-11-09T15:41:12.516+00:00","path":"/acpos","status":500,
        "error":"Internal Server Error","message":"","requestId":"699a523c"}

  at org.springframework.test.web.reactive.server.ExchangeResult
    .assertWithDiagnostics(ExchangeResult.java:209)
  at org.springframework.test.web.reactive.server.StatusAssertions
    .assertStatusAndReturn(StatusAssertions.java:227)
  at org.springframework.test.web.reactive.server.StatusAssertions
    .isOk(StatusAssertions.java:67)
  at com.thehecklers.aircraftpositions.PositionControllerTest
    .getCurrentACPositions(PositionControllerTest.java:90)
  at java.base/jdk.internal.reflect.NativeMethodAccessorImpl
    .invoke0(Native Method)
  at java.base/jdk.internal.reflect.NativeMethodAccessorImpl
    .invoke(NativeMethodAccessorImpl.java:62)
  at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl
    .invoke(DelegatingMethodAccessorImpl.java:43)
  at java.base/java.lang.reflect.Method.invoke(Method.java:564)
  at org.junit.platform.commons.util.ReflectionUtils
    .invokeMethod(ReflectionUtils.java:686)
  at org.junit.jupiter.engine.execution.MethodInvocation
    .proceed(MethodInvocation.java:60)
  at org.junit.jupiter.engine.execution.InvocationInterceptorChain
    $ValidatingInvocation.proceed(InvocationInterceptorChain.java:131)
  at org.junit.jupiter.engine.extension.TimeoutExtension
    .intercept(TimeoutExtension.java:149)
  at org.junit.jupiter.engine.extension.TimeoutExtension
    .interceptTestableMethod(TimeoutExtension.java:140)
  at org.junit.jupiter.engine.extension.TimeoutExtension
    .interceptTestMethod(TimeoutExtension.java:84)
  at org.junit.jupiter.engine.execution.ExecutableInvoker
    $ReflectiveInterceptorCall
        .lambda$ofVoidMethod$0(ExecutableInvoker.java:115)
  at org.junit.jupiter.engine.execution.ExecutableInvoker
    .lambda$invoke$0(ExecutableInvoker.java:105)
```

```
at org.junit.jupiter.engine.execution.InvocationInterceptorChain
  $InterceptedInvocation.proceed(InvocationInterceptorChain.java:106)
at org.junit.jupiter.engine.execution.InvocationInterceptorChain
  .proceed(InvocationInterceptorChain.java:64)
at org.junit.jupiter.engine.execution.InvocationInterceptorChain
  .chainAndInvoke(InvocationInterceptorChain.java:45)
at org.junit.jupiter.engine.execution.InvocationInterceptorChain
  .invoke(InvocationInterceptorChain.java:37)
at org.junit.jupiter.engine.execution.ExecutableInvoker
  .invoke(ExecutableInvoker.java:104)
at org.junit.jupiter.engine.execution.ExecutableInvoker
  .invoke(ExecutableInvoker.java:98)
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor
  .lambda$invokeTestMethod$6(TestMethodTestDescriptor.java:212)
at org.junit.platform.engine.support.hierarchical.ThrowableCollector
  .execute(ThrowableCollector.java:73)
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor
  .invokeTestMethod(TestMethodTestDescriptor.java:208)
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor
  .execute(TestMethodTestDescriptor.java:137)
at org.junit.jupiter.engine.descriptor.TestMethodTestDescriptor
  .execute(TestMethodTestDescriptor.java:71)
at org.junit.platform.engine.support.hierarchical.NodeTestTask
  .lambda$executeRecursively$5(NodeTestTask.java:135)
at org.junit.platform.engine.support.hierarchical.ThrowableCollector
  .execute(ThrowableCollector.java:73)
at org.junit.platform.engine.support.hierarchical.NodeTestTask
  .lambda$executeRecursively$7(NodeTestTask.java:125)
at org.junit.platform.engine.support.hierarchical.Node.around(Node.java:135)
at org.junit.platform.engine.support.hierarchical.NodeTestTask
  .lambda$executeRecursively$8(NodeTestTask.java:123)
at org.junit.platform.engine.support.hierarchical.ThrowableCollector
  .execute(ThrowableCollector.java:73)
at org.junit.platform.engine.support.hierarchical.NodeTestTask
  .executeRecursively(NodeTestTask.java:122)
at org.junit.platform.engine.support.hierarchical.NodeTestTask
  .execute(NodeTestTask.java:80)
at java.base/java.util.ArrayList.forEach(ArrayList.java:1510)
at org.junit.platform.engine.support.hierarchical
  .SameThreadHierarchicalTestExecutorService
      .invokeAll(SameThreadHierarchicalTestExecutorService.java:38)
at org.junit.platform.engine.support.hierarchical.NodeTestTask
  .lambda$executeRecursively$5(NodeTestTask.java:139)
at org.junit.platform.engine.support.hierarchical.ThrowableCollector
  .execute(ThrowableCollector.java:73)
at org.junit.platform.engine.support.hierarchical.NodeTestTask
  .lambda$executeRecursively$7(NodeTestTask.java:125)
at org.junit.platform.engine.support.hierarchical.Node.around(Node.java:135)
at org.junit.platform.engine.support.hierarchical.NodeTestTask
  .lambda$executeRecursively$8(NodeTestTask.java:123)
at org.junit.platform.engine.support.hierarchical.ThrowableCollector
```

```
    .execute(ThrowableCollector.java:73)
  at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .executeRecursively(NodeTestTask.java:122)
  at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .execute(NodeTestTask.java:80)
  at java.base/java.util.ArrayList.forEach(ArrayList.java:1510)
  at org.junit.platform.engine.support.hierarchical
    .SameThreadHierarchicalTestExecutorService
        .invokeAll(SameThreadHierarchicalTestExecutorService.java:38)
  at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .lambda$executeRecursively$5(NodeTestTask.java:139)
  at org.junit.platform.engine.support.hierarchical.ThrowableCollector
    .execute(ThrowableCollector.java:73)
  at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .lambda$executeRecursively$7(NodeTestTask.java:125)
  at org.junit.platform.engine.support.hierarchical.Node.around(Node.java:135)
  at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .lambda$executeRecursively$8(NodeTestTask.java:123)
  at org.junit.platform.engine.support.hierarchical.ThrowableCollector
    .execute(ThrowableCollector.java:73)
  at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .executeRecursively(NodeTestTask.java:122)
  at org.junit.platform.engine.support.hierarchical.NodeTestTask
    .execute(NodeTestTask.java:80)
  at org.junit.platform.engine.support.hierarchical
    .SameThreadHierarchicalTestExecutorService
        .submit(SameThreadHierarchicalTestExecutorService.java:32)
  at org.junit.platform.engine.support.hierarchical.HierarchicalTestExecutor
    .execute(HierarchicalTestExecutor.java:57)
  at org.junit.platform.engine.support.hierarchical.HierarchicalTestEngine
    .execute(HierarchicalTestEngine.java:51)
  at org.junit.platform.launcher.core.DefaultLauncher
    .execute(DefaultLauncher.java:248)
  at org.junit.platform.launcher.core.DefaultLauncher
    .lambda$execute$5(DefaultLauncher.java:211)
  at org.junit.platform.launcher.core.DefaultLauncher
    .withInterceptedStreams(DefaultLauncher.java:226)
  at org.junit.platform.launcher.core.DefaultLauncher
    .execute(DefaultLauncher.java:199)
  at org.junit.platform.launcher.core.DefaultLauncher
    .execute(DefaultLauncher.java:132)
  at com.intellij.junit5.JUnit5IdeaTestRunner
    .startRunnerWithArgs(JUnit5IdeaTestRunner.java:69)
  at com.intellij.rt.junit.IdeaTestRunner$Repeater
    .startRunnerWithArgs(IdeaTestRunner.java:33)
  at com.intellij.rt.junit.JUnitStarter
    .prepareStreamsAndStart(JUnitStarter.java:230)
  at com.intellij.rt.junit.JUnitStarter
    .main(JUnitStarter.java:58)
Caused by: java.lang.AssertionError: Status expected:<200 OK>
        but was:<500 INTERNAL_SERVER_ERROR>
```

```
  at org.springframework.test.util.AssertionErrors
    .fail(AssertionErrors.java:59)
  at org.springframework.test.util.AssertionErrors
    .assertEquals(AssertionErrors.java:122)
  at org.springframework.test.web.reactive.server.StatusAssertions
    .lambda$assertStatusAndReturn$4(StatusAssertions.java:227)
  at org.springframework.test.web.reactive.server.ExchangeResult
    .assertWithDiagnostics(ExchangeResult.java:206)
  ... 66 more
```

Как видите, подобный объем информации для одного некорректного значения воспринять весьма тяжело. Здесь есть и полезная информация, но она перегружена избыточными, менее полезными данными.



Я с большой неохотой, но вполне намеренно включил полную информацию, выводимую в результате предшествующей ошибки Flux, чтобы продемонстрировать сложности анализа данных, обычно выводимых, когда Publisher сталкивается с ошибкой, и показать, насколько имеющиеся инструменты сокращают шум и усиливают сигнал ключевой информации. Возможность сразу добраться до сути проблемы не только уменьшает разочарование при разработке, но и абсолютно необходима при устранении неполадок в тех приложениях, что критически важны для бизнеса.

Project Reactor включает настраиваемые функции обратного вызова жизненного цикла — так называемые *точки подключения* (hooks), доступные через класс `Hooks`. Особенно полезен для повышения соотношения «сигнал/шум» при возникновении проблем оператор `onOperatorDebug()`.

Вызов `Hooks.onOperatorDebug()` перед созданием экземпляра сбойного `Publisher` позволяет использовать инструмент во время сборки всех последующих экземпляров типа `Publisher` и его подтипов. Чтобы обеспечить захват всей необходимой информации в нужный(-ые) момент(-ы) времени, этот вызов обычно помещается в основной метод приложения, вот так:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import reactor.core.publisher.Hooks;

@SpringBootApplication
public class AircraftPositionsApplication {

  public static void main(String[] args) {
    Hooks.onOperatorDebug();
    SpringApplication.run(AircraftPositionsApplication.class, args);
  }

}
```

Поскольку я демонстрирую данную возможность из тестового класса, то вместо этого вставил `Hooks.onOperatorDebug();` в строку, непосредственно предшествующую сборке нашего умышленно сбойного `Publisher`:

```
Hooks.onOperatorDebug();
Mockito.when(service.getAllAircraft()).thenReturn(
        Flux.just(ac1, ac2, ac3)
                .concatWith(Flux.error(new Throwable("Bad position report")))
);
```

Одно это добавление не делает трассировку вызовов намного менее объемной — в редких случаях любая дополнительная информация может оказаться полезной, — но в абсолютном большинстве случаев добавляемая в журнал методом `onOperatorDebug()` древовидная сводка обратной трассировки вызовов позволяет быстрее выявить причину проблемы и решить ее. Сводка обратной трассировки вызовов для той же ошибки, которую я умышленно внес в тест `getCurrentACPositions()`, приведена на рис. 12.3, чтобы сохранить все подробности и форматирование.



Рис. 12.3. Отладочная обратная трасса вызовов

Вверху дерева — главная улика: ошибка `Flux` была внесена с помощью оператора `concatWith` в строке 68 файла `PositionControllerTest.java`. Благодаря `Hooks.onOperatorDebug()` время выявления ошибки и ее местоположения сократилось от нескольких минут до считаных секунд.

Впрочем, телеметрия всех инструкций сборки для всех последующих вхождений `Publisher` требует определенных расходов ресурсов. Телеметрия кода с помощью точек подключения во время выполнения обходится недешево, ведь при включении режима отладки он применяется ко всему приложению и влияет на все цепочки операций всех реактивных потоков данных `Publisher`. Давайте рассмотрим альтернативный вариант.

Контрольные точки

Вместо того чтобы выводить все возможные обратные трассировки вызовов всех возможных `Publisher`, можно просто установить контрольные точки возле ключевых операторов для упрощения диагностики. Вставка оператора `checkpoint()` в цепочку операторов аналогична созданию точки подключения, но только для данного сегмента этой цепочки операторов.

Существует три вида контрольных точек:

- стандартные контрольные точки с обратной трассировкой вызовов;
- упрощенные контрольные точки, принимающие описательный параметр типа `String`, без обратной трассировки вызовов;
- стандартные контрольные точки с обратной трассировкой вызовов, но также принимающие описательный параметр типа `String`.

Взглянем на них в действии.

Прежде всего удаляем оператор `Hooks.onOperatorDebug()` перед фиктивным методом для `PositionService::getAllAircraft` в методе `setUp()` класса `PositionControllerTest`:

```
//Hooks.onOperatorDebug();      Комментируем или удаляем
Mockito.when(service.getAllAircraft()).thenReturn(
    Flux.just(ac1, ac2, ac3)
        .checkpoint()
        .concatWith(Flux.error(new Throwable("Bad position report")))
        .checkpoint()
);
```

При перезапуске теста для `getCurrentACPositions()` получаем результаты, приведенные на рис. 12.4.



Рис. 12.4. Результаты применения стандартной контрольной точки

Контрольная точка вверху списка указывает, что проблема возникает из-за оператора, непосредственно предшествующего сработавшей контрольной точке. Обратите внимание на то, что информация об обратной трассировке вызовов все же собирается, ведь эта контрольная точка отражает фактический исходный код и конкретный номер строки контрольной точки, вставленной на строке 64 в классе `PositionControllerTest`.

При переключении на упрощенную контрольную точку набор информации обратной трассы вызовов заменяется удобным строковым описанием, указанным разработчиком. И хотя сбор обратной трассировки вызовов для стандартных контрольных точек ограничен определенными рамками, все равно ресурсов для него требуется намного больше, чем для хранения простого объекта `String`. При довольно частом их размещении упрощенные контрольные точки позволяют выявлять проблемные операторы ничуть не хуже. А воспользоваться в коде упрощенными контрольными точками очень легко:

```
// Hooks.onOperatorDebug();      Комментируем или удаляем
Mockito.when(service.getAllAircraft()).thenReturn(
    Flux.just(ac1, ac2, ac3)
        .checkpoint("All Aircraft: after all good positions reported")
```

```
        .concatWith(Flux.error(new Throwable("Bad position report")))
        .checkpoint("All Aircraft: after appending bad position report")
);
```

При перезапуске теста для `getCurrentACPositions()` получаем результаты, приведенные на рис. 12.5.

```
java.lang.Throwable: Bad position report
    at com.thehecklers.aircraftpositions.PositionControllerTest.setUp(PositionControllerTest.java:68) ~[test-classes/:na]
    Suppressed: reactor.core.publisher.FluxOnAssembly$OnAssemblyException:
Error has been observed at the following site(s):
    |_         checkpoint ⇢ All Aircraft: after appending bad position report
    |_                    ⇢ at com.thehecklers.aircraftpositions.PositionService$MockitoMock$1153167644.getAllAircraft(null:-1)
    |_          Flux.from ⇢ at org.springframework.http.codec.json.AbstractJackson2Encoder.encode(AbstractJackson2Encoder.java:178)
    |_   Flux.collectList ⇢ at org.springframework.http.codec.json.AbstractJackson2Encoder.encode(AbstractJackson2Encoder.java:179)
    |_           Mono.map ⇢ at org.springframework.http.codec.json.AbstractJackson2Encoder.encode(AbstractJackson2Encoder.java:180)
    |_          Mono.flux ⇢ at org.springframework.http.codec.json.AbstractJackson2Encoder.encode(AbstractJackson2Encoder.java:181)
    |_          Flux.from ⇢ at org.springframework.http.server.reactive.ChannelSendOperator.<init>(ChannelSendOperator.java:57)
    |_                    ⇢ at org.springframework.http.codec.EncoderHttpMessageWriter.write(EncoderHttpMessageWriter.java:201)
    |_                    ⇢ at org.springframework.web.reactive.result.method.annotation.AbstractMessageWriterResultHandler.writeBody
 (AbstractMessageWriterResultHandler.java:184)
    |_                    ⇢ at org.springframework.web.reactive.result.method.annotation.ResponseBodyResultHandler.handleResult(ResponseBodyResultHandler.java:86)
    |_         checkpoint ⇢ Handler com.thehecklers.aircraftpositions.PositionController#getCurrentACPositions() [DispatcherHandler]
    |_                    ⇢ at org.springframework.web.server.handler.DefaultWebFilterChain.lambda$filter$0(DefaultWebFilterChain.java:128)
    |_         Mono.defer ⇢ at org.springframework.web.server.handler.DefaultWebFilterChain.filter(DefaultWebFilterChain.java:119)
    |_                    ⇢ at org.springframework.web.server.handler.FilteringWebHandler.handle(FilteringWebHandler.java:59)
    |_                    ⇢ at org.springframework.web.server.handler.WebHandlerDecorator.handle(WebHandlerDecorator.java:56)
    |_         Mono.error ⇢ at org.springframework.web.server.handler.ExceptionHandlingWebHandler$CheckpointInsertingHandler.handle(ExceptionHandlingWebHandler
.java:98)
    |_         checkpoint ⇢ HTTP GET "/acpos" [ExceptionHandlingWebHandler]
    |_                    ⇢ at org.springframework.web.server.handler.ExceptionHandlingWebHandler.lambda$handle$0(ExceptionHandlingWebHandler.java:77)
    |_ Mono.onErrorResume ⇢ at org.springframework.web.server.handler.ExceptionHandlingWebHandler.handle(ExceptionHandlingWebHandler.java:77)
```

Рис. 12.5. Результаты применения упрощенной контрольной точки

И хотя файл и номер строки больше не отображаются в верхней контрольной точке, ее ясное описание позволяет легко найти проблемный оператор в сборке `Flux`.

Иногда для создания `Publisher` приходится использовать исключительно сложную цепочку операторов. В подобных случаях имеет смысл включить как описание, так и полную информацию об обратной трассе вызовов для диагностики и поиска проблемы. Для демонстрации очень небольшого примера я еще раз переделал фиктивный метод для `PositionService::getAllAircraft`:

```
//Hooks.onOperatorDebug();      Комментируем или удаляем
Mockito.when(service.getAllAircraft()).thenReturn(
    Flux.just(ac1, ac2, ac3)
        .checkpoint("All Aircraft: after all good positions reported", true)
        .concatWith(Flux.error(new Throwable("Bad position report")))
        .checkpoint("All Aircraft: after appending bad position report", true)
);
```

Повторный запуск теста для `getCurrentACPositions()` дает результаты, приведенные на рис. 12.6.



Рис. 12.6. Результаты применения стандартной контрольной точки с описанием

ReactorDebugAgent.init()

Существует способ получить все выгоды от полной обратной трассировки вызовов для всех `Publisher` в приложении — аналогично генерируемой при использовании точек подключения, — но без снижения быстродействия, как при активировании режима отладки с теми же точками подключения.

Проект Reactor содержит библиотеку `reactor-tools`, включающую отдельный Java-агент для телеметрии кода приложения. `reactor-tools` добавляет в приложение отладочную информацию и связывается с работающим приложением, зависимостью которого является, для отслеживания и трассировки выполнения всех последовательных `Publisher`. В результате выводится столь же подробная обратная трасса вызовов, что и при использовании точек подключения, практически не влияющая на производительность. А поэтому положительных сторон выполнения реактивных приложений при продакшене с включенным `ReactorDebugAgent` много, а отрицательных — мало, а то и вовсе нет.

`reactor-tools` — отдельная библиотека, так что ее необходимо вручную добавить в файл сборки приложения. Я добавил в файл сборки Maven приложения Aircraft Positions следующую запись:

```
<dependency>
    <groupId>io.projectreactor</groupId>
    <artifactId>reactor-tools</artifactId>
</dependency>
```

После сохранения модифицированного файла `pom.xml` я обновляю либо заново импортирую зависимости, чтобы можно было обращаться к `ReactorDebugAgent` изнутри проекта.

Подобно `Hooks.onOperatorDebug()`, `ReactorDebugAgent` обычно инициализируется в основном методе приложения перед его запуском. А поскольку я собираюсь демонстрировать его работу из теста, не загружающего полный контекст приложения, то вставляю вызов метода инициализации непосредственно перед созданием объекта `Flux`, на котором буду демонстрировать ошибку времени выполнения, а также удаляю теперь уже ненужные вызовы `checkpoint()`:

```
//Hooks.onOperatorDebug();
ReactorDebugAgent.init();       // Добавьте эту строку
Mockito.when(service.getAllAircraft()).thenReturn(
        Flux.just(ac1, ac2, ac3)
                .concatWith(Flux.error(new Throwable("Bad position report")))
);
```

Возвращаясь к тесту `getCurrentACPositions()`, я запускаю его и получаю древовидную сводку (рис. 12.7), аналогичную получаемой при использовании `Hooks.onOperatorDebug()`, но без снижения быстродействия во время выполнения.



Рис. 12.7. Результат применения ReactorDebugAgent при ошибке Flux

Существуют и другие инструменты, непосредственно не помогающие тестировать или отлаживать реактивные приложения, однако способствующие повышению их качества. Один из примеров — BlockHound (https://github.com/reactor/BlockHound), который хотя и выходит за рамки данной главы, может оказаться полезным инструментом для выявления блокирующих вызовов, скрывающихся в коде вашего приложения или его зависимостей. И конечно, существуют и другие быстро развивающиеся утилиты, помогающие совершенствовать реактивные приложения и системы самыми разнообразными способами.



НЕ ЗАБУДЬТЕ ИЗВЛЕЧЬ КОД ИЗ РЕПОЗИТОРИЯ

Полный код для этой главы находится в ветке chapter12end в репозитории кода.

Резюме

Реактивное программирование позволяет разработчикам лучше использовать ресурсы распределенных систем и даже распространять мощные механизмы масштабирования за границы приложений и на каналы связи. Разработчики, сталкивавшиеся только с господствующими практиками разработки Java, часто называемыми *императивным* языком Java из-за явной последовательной логики, в отличие от более декларативного подхода, применяемого обычно в реактивном программировании, хотя этот ярлык, как и большинство ярлыков, не идеален, могут обнаружить, что подобные реактивные возможности влекут за собой дополнительные нежелательные затраты. Помимо ожидаемой кривой обучения, которую Spring делает существенно более покатой благодаря параллельным взаимодополняющим реализациям WebMVC и WebFlux, есть и определенные ограничения в смысле инструментария, его зрелости и установившихся практик для основных операций, например тестирования, поиска проблем и отладки.

И хотя реактивная разработка на Java делает лишь первые шаги по сравнению со своим императивным кузеном, их родство привело к намного более быстрой разработке и наработке полезных утилит и процессов. Как уже упоминалось, Spring в разработке основывается на накопленных императивных знаниях, сокращая десятилетия эволюции в готовые к продакшену компоненты, доступные *прямо сейчас*.

В этой главе я познакомил вас с последними достижениями в сфере тестирования и диагностики/отладки проблем, с которыми вы можете столкнуться при

развертывании реактивных приложений Spring Boot, а также показал, как применить на деле WebFlux/Reactor до перехода к продакшену и во время нее для тестирования и отладки реактивных приложений разнообразными способами, и продемонстрировал достоинства каждого из вариантов. Уже сейчас доступно множество инструментов, а грядущие перспективы еще радужнее.

В этой книге мне пришлось выбирать, какие из бесчисленных «лучших сторон» Spring Boot рассматривать, чтобы продемонстрировать, надеюсь, оптимальный способ разрабатывать приложения с помощью Spring Boot. Но так много осталось неохваченным, и я жалею, что не мог удвоить, а то и утроить объем книги для описания всего этого. Спасибо, что были со мной в этом путешествии, надеюсь, в будущем смогу рассказать вам еще больше. Удачи вам в дальнейших начинаниях со Spring Boot.

Об авторе

Марк Хеклер — разработчик ПО и пропагандист Spring из VMware, обладатель званий Java Champion и Google Developer Expert по языку Kotlin. Он сотрудничал с ключевыми игроками в сферах производства, розничных продаж, медицины, научных исследований, телекоммуникаций и финансов, а также с различными общественными организациями. Помимо того, что Марк вносит вклад в разработку ПО с открытым исходным кодом, он является автором и модератором ориентированного на разработчиков блога www.thehecklers.com и периодически ведет интересный твиттер @mkheck.

Об иллюстрации на обложке

На обложке изображен кулик-дутыш *(Calidris melanotos)*.

Обычно обитает в заросших болотах и на заливных лугах по всей Северной Америке, в основном на Великих равнинах. Восточнее, по мере приближения к Атлантическому океану, размер популяции уменьшается. Своим названием дутыш обязан шейным воздушным мешкам у самцов, при сдутии которых возникает специфический звук (обычно передается как «ду-ду-ду...»).

Опознать этих небольших птиц можно по четкой границе между пестрой грудью и ярко-белым брюшком. Привлекая самку во время токования, самец летает над ней, бегает вокруг нее по земле и исполняет причудливый танец, в конце которого вытягивает крылья к небу. По завершении летнего гнездования в тундре дутыши улетают на зиму в Южную Америку, а некоторые — в Австралию и Новую Зеландию.

Многие из животных на обложках книг, издаваемых O'Reilly, находятся на грани исчезновения, но все они важны для нашего мира.

Иллюстрация на обложке принадлежит кисти Карен Монтгомери (Karen Montgomery) и основана на черно-белых гравюрах из *British Birds*.

O'REILLY®

Spring Boot по-быстрому

Spring Boot, который скачивают более 75 миллионов раз в месяц, — наиболее широко используемый фреймворк Java. Его удобство и возможности совершили революцию в разработке приложений, от монолитных до микросервисов. Тем не менее простота Spring Boot может привести в замешательство. Что именно разработчику нужно изучить, чтобы сразу же выдавать результат? Это практическое руководство научит вас писать успешные приложения для критически важных задач.

Марк Хеклер из VMware, компании, создавшей Spring, проведет вас по всей архитектуре Spring Boot, охватив такие вопросы, как отладка, тестирование и развертывание. Если вы хотите быстро и эффективно разрабатывать нативные облачные приложения Java или Kotlin на базе Spring Boot с помощью реактивного программирования, создания API и доступа к разнообразным базам данных — эта книга для вас.

- Узнайте, как Spring Boot упрощает разработку и развертывание нативных облачных приложений.
- Создавайте реактивные приложения и распределенные системы, обмениваясь информацией сквозь границы сети.
- Выясните, как архитектура и подходы Spring Boot повышают производительность разработчиков и переносимость приложений.
- Быстро и надежно развертывайте приложения Spring Boot для решения реальных задач.
- Отслеживайте состояние приложения и системы, чтобы обеспечить максимальное быстродействие и надежность.
- Без проблем отлаживайте, тестируйте облачные приложения и обеспечивайте их безопасность.

«Эта книга и ее автор — идеальные спутники для знакомства со Spring Boot. Надеюсь, вы насладитесь их компанией так же, как и я».

— **Дэвид Сайр,** VMware

«У Марка свежий взгляд на создание приложений с помощью Spring Boot. Не пропустите эту книгу».

— **Грег Л. Тёрнквист,** участник команды Spring, ведущий канала Spring Boot Learning на YouTube

Марк Хеклер — разработчик и пропагандист Spring из VMware, обладатель званий Java Champion и Google Developer Expert по языку Kotlin. Кроме того, он пишет и модерирует блог для разработчиков Hecklers in Development, brewing stronger Java (www.thehecklers.com).

ПИТЕР®

ЗАКАЗ КНИГ:
тел.: (812)703-73-74
books@piter.com

WWW.PITER.COM
каталог книг и интернет-магазин

vk.com/PiterBooks
t.me/PiterForPeople
youtube.com/ThePiterBooks
habr.com/ru/company/piter

ISBN:978-5-4461-3942-2